

O'REILLY®
オライリー・ジャパン

TURING

图灵程序设计丛书

深度学习入门 4

强化学习

Reinforcement Learning



[日] 斋藤康毅 著
郑明智 译



中国工信出版集团



人民邮电出版社
POSTS & TELECOM PRESS

作者介绍

斋藤康毅

1984年出生于日本长崎县，东京工业大学毕业，并完成东京大学研究生院课程。目前在某企业从事人工智能相关的研究和开发工作。著有“鱼书”系列

《深度学习入门：基于Python的理论与实现》《深度学习进阶：自然语言处理》

《深度学习入门2：自制框架》，同时也是*Python in Practice*、*The Elements of Computing Systems*、*Building Machine Learning Systems with Python*的日文版译者。

译者介绍

郑明智

智慧医疗工程师。主要研究方向为医疗与前沿ICT技术的结合及其应用。译有

《深度学习基础与实践》《详解深度学习》《白话机器学习的数学》等书。

数字版权声明

图灵社区的电子书没有采用专有客户端，您可以在任意设备上，用自己喜欢的浏览器和PDF阅读器进行阅读。

但您购买的电子书仅供您个人使用，未经授权，不得进行传播。

我们愿意相信读者具有这样的良知和觉悟，与我们共同保护知识产权。

如果购买者有侵权行为，我们可能对该用户实施包括但不限于关闭该帐号等维权措施，并可能追究法律责任。



图灵程序设计丛书

深度学习入门④：强化学习

Reinforcement Learning

[日] 斋藤康毅 著
郑明智 译

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

O'Reilly Japan, Inc. 授权人民邮电出版社有限公司出版

人民邮电出版社
北 京

图书在版编目 (CIP) 数据

深度学习入门. 4, 强化学习 / (日) 斋藤康毅著 ;
郑明智译. -- 北京 : 人民邮电出版社, 2024. -- (图
灵程序设计丛书). -- ISBN 978-7-115-64917-1

I. TP181

中国国家版本馆 CIP 数据核字第 20242SF099 号

内 容 提 要

本书前半部分介绍强化学习的重要思想和基础知识, 后半部分介绍如何将深度学习应用于强化学习, 遴选讲解了深度强化学习的最新技术。全书从最适合入门的多臂老虎机问题切入, 依次介绍了定义一般强化学习问题的马尔可夫决策过程、用于寻找最佳答案的贝尔曼方程, 以及解决贝尔曼方程的动态规划法、蒙特卡洛方法和 TD 方法。随后, 神经网络和 Q 学习、DQN、策略梯度法等几章则分别讨论了深度学习在强化学习领域的应用。本书延续“鱼书”系列的风格, 搭配丰富的图、表、代码示例, 加上轻松、简明的讲解, 让人循序渐进地理解强化学习中各种方法之间的关系, 于不知不觉中登堂入室。

本书既适合深度学习的初学者, 也适合对人工智能感兴趣的教师、学生和相关从业者学习参考。

◆ 著 [日] 斋藤康毅
译 郑明智
责任编辑 张海艳
责任印制 胡 南

◆ 人民邮电出版社出版发行 北京市丰台区成寿寺路11号
邮编 100164 电子邮件 315@ptpress.com.cn
网址 <https://www.ptpress.com.cn>
北京 印刷

◆ 开本: 880 × 1230 1/32
印张: 10.375 2024 年 8 月第 1 版
字数: 298千字 2024 年 8 月北京第 1 次印刷

著作权合同登记号 图字: 01-2024-0083号

定价: 109.80元

读者服务热线: (010)84084456-6009 印装质量热线: (010)81055316

反盗版热线: (010)81055315

广告经营许可证: 京东市监广登字 20170147 号

版权声明

©Posts and Telecom Press Co., Ltd. 2024.

Authorized translation of the Japanese edition of ゼロから作る Deep Learning
④—強化学習編 ©2022 Koki Saitoh. All Rights Reserved. This translation is published and sold by permission of O'Reilly Japan, Inc., the owner of all rights to publish and sell the same.

简体中文版由人民邮电出版社有限公司出版，2024。

日文原版由 O'Reilly Japan, Inc. 出版，2022。

日文原版的翻译得到 Koki Saitoh 的授权。

此简体中文版的出版和销售得到出版权和销售权的所有者——O'Reilly Japan, Inc. 的许可。

版权所有，未得书面许可，本书的任何部分和全部不得以任何形式重制。

O'Reilly Media, Inc. 介绍

O'Reilly 以“分享创新知识、改变世界”为己任。40 多年来我们一直向企业、个人提供成功所必需之技能及思想，激励他们创新并做得更好。

O'Reilly 业务的核心是独特的专家及创新者网络，众多专家及创新者通过我们分享知识。我们的在线学习（Online Learning）平台提供独家的直播培训、互动学习、认证体验、图书、视频等，使客户更容易获取业务成功所需的专业知识。几十年来 O'Reilly 图书一直被视为学习开创未来之技术的权威资料。我们所做的一切是为了帮助各领域的专业人士学习最佳实践，发现并塑造科技行业未来的新趋势。

我们的客户渴望做出推动世界前进的创新之举，我们希望能助他们一臂之力。

业界评论

“O'Reilly Radar 博客有口皆碑。”

——*Wired*

“O'Reilly 凭借一系列非凡想法（真希望当初我也想到了）建立了数百万美元的业务。”

——*Business 2.0*

“O'Reilly Conference 是聚集关键思想领袖的绝对典范。”

——*CRN*

“一本 O'Reilly 的书就代表一个有用、有前途、需要学习的主题。”

——*Irish Times*

“Tim 是位特立独行的商人，他不光放眼于最长远、最广阔的领域，并且切实地按照 Yogi Berra 的建议去做了：‘如果你在路上遇到岔路口，那就走小路。’回顾过去，Tim 似乎每一次都选择了小路，而且有几次都是一闪即逝的机会，尽管大路也不错。”

——*Linux Journal*

目录

前言

第 1 章 老虎机问题 1

1.1	机器学习的分类与强化学习	1
1.1.1	监督学习	2
1.1.2	无监督学习	2
1.1.3	强化学习	3
1.2	老虎机问题	5
1.2.1	什么是老虎机问题	5
1.2.2	什么是好的老虎机	7
1.2.3	使用数学式表示	8
1.3	老虎机算法	9
1.3.1	价值的估计方法	10
1.3.2	求平均值的实现	12
1.3.3	玩家的策略	15
1.4	老虎机算法的实现	17
1.4.1	老虎机的实现	17
1.4.2	智能代理的实现	19
1.4.3	尝试运行	20
1.4.4	算法平均的特性	23
1.5	非稳态问题	28
1.5.1	解决非稳态问题前的准备工作	29

1.5.2 解决非稳态问题	32
1.6 小结	34
第 2 章 马尔可夫决策过程	36
2.1 什么是 MDP	37
2.1.1 MDP 的具体例子	37
2.1.2 智能代理与环境的互动	39
2.2 环境和智能代理的数学表示	40
2.2.1 状态迁移	40
2.2.2 奖励函数	42
2.2.3 智能代理的策略	43
2.3 MDP 的目标	45
2.3.1 回合制任务和连续性任务	45
2.3.2 收益	46
2.3.3 状态价值函数	47
2.3.4 最优策略和最优价值函数	48
2.4 MDP 的例子	50
2.4.1 回溯线形图	51
2.4.2 找出最优策略	52
2.5 小结	54
第 3 章 贝尔曼方程	56
3.1 贝尔曼方程的推导	57
3.1.1 概率和期望值 (推导贝尔曼方程的准备)	57
3.1.2 贝尔曼方程的推导	60
3.2 贝尔曼方程的例子	64
3.2.1 有两个方格的网格世界	64
3.2.2 贝尔曼方程的意义	68
3.3 行动价值函数与贝尔曼方程	68
3.3.1 行动价值函数	69
3.3.2 使用行动价值函数的贝尔曼方程	70

3.4 贝尔曼最优方程	71
3.4.1 状态价值函数的贝尔曼最优方程	71
3.4.2 Q 函数的贝尔曼最优方程	73
3.5 贝尔曼最优方程的示例	74
3.5.1 应用贝尔曼最优方程	74
3.5.2 得到最优策略	76
3.6 小结	78
第 4 章 动态规划法	79
4.1 动态规划法和策略评估	80
4.1.1 动态规划法简介	80
4.1.2 尝试迭代策略评估	81
4.1.3 迭代策略评估的其他实现方式	86
4.2 解决更大的问题	87
4.2.1 GridWorld 类的实现	88
4.2.2 defaultdict 的用法	94
4.2.3 迭代策略评估的实现	95
4.3 策略迭代法	99
4.3.1 策略的改进	99
4.3.2 重复评估和改进	101
4.4 实施策略迭代法	102
4.4.1 改进策略	103
4.4.2 重复评估和改进	105
4.5 价值迭代法	107
4.5.1 价值迭代法的推导	109
4.5.2 价值迭代法的实现	113
4.6 小结	116
第 5 章 蒙特卡洛方法	117
5.1 蒙特卡洛方法的基础知识	117
5.1.1 骰子的点数和	118

5.1.2	分布模型和样本模型	119
5.1.3	蒙特卡洛方法的实现	121
5.2	使用蒙特卡洛方法评估策略	123
5.2.1	使用蒙特卡洛方法计算价值函数	124
5.2.2	求所有状态的价值函数	126
5.2.3	蒙特卡洛方法的高效实现	129
5.3	蒙特卡洛方法的实现	130
5.3.1	step 方法	130
5.3.2	智能代理类的实现	132
5.3.3	运行蒙特卡洛方法	134
5.4	使用蒙特卡洛方法的策略控制	136
5.4.1	评估和改进	136
5.4.2	使用蒙特卡洛方法实现策略控制	137
5.4.3	ϵ -greedy 算法 (第 1 个修改)	139
5.4.4	修改为固定值 α 的方式 (第 2 个修改)	141
5.4.5	[修改版] 使用蒙特卡洛方法实现策略迭代法	142
5.5	异策略型和重要性采样	145
5.5.1	同策略型和异策略型	145
5.5.2	重要性采样	146
5.5.3	如何减小方差	150
5.6	小结	152

第 6 章 TD 方法 153

6.1	使用 TD 方法评估策略	153
6.1.1	TD 方法的推导	154
6.1.2	MC 方法和 TD 方法的比较	157
6.1.3	TD 方法的实现	158
6.2	SARSA	161
6.2.1	同策略型的 SARSA	161
6.2.2	SARSA 的实现	162
6.3	异策略型的 SARSA	165

6.3.1	异策略型和重要性采样	166
6.3.2	异策略型的 SARSA 的实现	167
6.4	Q 学习	169
6.4.1	贝尔曼方程与 SARSA	170
6.4.2	贝尔曼最优方程与 Q 学习	171
6.4.3	Q 学习的实现	173
6.5	分布模型与样本模型	175
6.5.1	分布模型与样本模型	175
6.5.2	样本模型版的 Q 学习	176
6.6	小结	179

第 7 章 神经网络和 Q 学习 181

7.1	DeZero 简介	182
7.1.1	使用 DeZero	183
7.1.2	多维数组（张量）和函数	184
7.1.3	最优化	186
7.2	线性回归	189
7.2.1	玩具数据集	189
7.2.2	线性回归的理论知识	190
7.2.3	线性回归的实现	191
7.3	神经网络	195
7.3.1	非线性数据集	195
7.3.2	线性变换和激活函数	196
7.3.3	神经网络的实现	197
7.3.4	层与模型	199
7.3.5	优化器（最优化方法）	202
7.4	Q 学习与神经网络	204
7.4.1	神经网络的预处理	204
7.4.2	表示 Q 函数的神经网络	205
7.4.3	神经网络和 Q 学习	208
7.5	小结	212

第 8 章 DQN213

8.1	OpenAI Gym	213
8.1.1	OpenAI Gym 的基础知识	214
8.1.2	随机智能代理	216
8.2	DQN 的核心技术	218
8.2.1	经验回放	218
8.2.2	经验回放的实现	220
8.2.3	目标网络	223
8.2.4	目标网络的实现	224
8.2.5	运行 DQN	227
8.3	DQN 与 Atari	230
8.3.1	Atari 的游戏环境	231
8.3.2	预处理	232
8.3.3	CNN	232
8.3.4	其他技巧	233
8.4	DQN 的扩展	234
8.4.1	Double DQN	234
8.4.2	优先级经验回放	235
8.4.3	Dueling DQN	236
8.5	小结	238

第 9 章 策略梯度法239

9.1	最简单的策略梯度法	239
9.1.1	策略梯度法的推导	240
9.1.2	策略梯度法的算法	241
9.1.3	策略梯度法的实现	243
9.2	REINFORCE	248
9.2.1	REINFORCE 算法	249
9.2.2	REINFORCE 的实现	250
9.3	基线	251

9.3.1	基线的思路	251
9.3.2	带基线的策略梯度法	253
9.4	Actor-Critic	254
9.4.1	Actor-Critic 的推导	255
9.4.2	Actor-Critic 的实现	257
9.5	基于策略的方法的优点	260
9.6	小结	262

第 10 章 进一步学习263

10.1	深度强化学习算法的分类	263
10.2	策略梯度法的改进算法	265
10.2.1	A3C 和 A2C	265
10.2.2	DDPG	268
10.2.3	TRPO 和 PPO	271
10.3	DQN 的改进算法	272
10.3.1	分类 DQN	272
10.3.2	Noisy Network	274
10.3.3	Rainbow	274
10.3.4	在 Rainbow 以后提出的改进算法	275
10.4	案例研究	276
10.4.1	棋盘游戏	277
10.4.2	机器人控制	279
10.4.3	NAS	280
10.4.4	其他案例	282
10.5	深度强化学习的挑战和可能性	283
10.5.1	应用于实际系统	283
10.5.2	将问题表示为 MDP 形式时的建议	286
10.5.3	通用人工智能系统	288
10.6	小结	288

附录 A 异策略型的蒙特卡洛方法	291
附录 B n -step TD 方法	298
附录 C Double DQN 的理解	300
附录 D 策略梯度法的证明	304
后记	308
参考文献	310

前言

自从深度学习面世以来，已经过去了一些年月。如今，深度学习正被应用于许多不同的场景，并取得了巨大的成功。到目前为止，深度学习的成功案例数不胜数，在其包含的深度强化学习领域，也取得了一些惊人的成果。下面就是其中的几个例子：

- 电子游戏玩得比人还好；
- 在围棋领域，击败了世界冠军；
- 在电子竞技领域击败了世界冠军。

这些成功案例是因强化学习和深度学习的结合才实现的。最近，在现实世界（游戏等虚拟世界之外）也出现了成功的案例。例如，在工业领域，根据报道，机器人的操作、半导体芯片设计等方面也取得了重大成果。

许多研究人员还将强化学习视为实现“通用人工智能”的关键技术（至少是与之更接近的技术）。通用人工智能是能够像我们人类一样灵活处理各种问题的智能，它的目标是在一个系统中解决各种问题。强化学习和深度学习将是实现通用人工智能的关键技术。因此，强化学习和深度学习有着巨大的潜力。

本书是“深度学习入门 & 进阶”系列的第四本，主题是强化学习。本书从强化学习的基础知识讲起，带你了解深度学习使用的最先进技术。无须了解这一系列前作中的任何知识，只要掌握了 Python 和数学的基础知识就可以开始阅读本书。

本书的编写理念

强化学习和深度学习相结合的领域叫作深度强化学习（deep reinforcement learning）。深度强化学习是非常热门的领域。近年来，几乎每天都会发布新的算法和应用程序。因此，深度强化学习领域的发展非常快。不过强化学习的基本思想和技术多年来并未改变。即使是最先进的算法也是基于已经存在了多年的思想。因此，理解现代的深度强化学习的捷径是花时间学习已经发展了多年的强化学习的基础知识。

例如，有一个著名的深度强化学习算法叫 DQN（Deep Q-Network 的缩写，DQN 因能够熟练地玩“*Atari 2600*”中的电视游戏而声名鹊起）。DQN 源于一个历史悠久的强化学习理论——Q 学习。如果用数学式来表示 Q 学习，那么只需几行即可。不过，就算记住这些公式也不能让你更接近强化学习的本质（而且死记硬背也没什么意思）。本书从强化学习的基础知识讲起，通过有“连接”的故事的形式来介绍关键思想和技术。通过这种方式积累知识，并将其有机地结合到一起，你就能更接近强化学习的本质。在这个过程中，你也就自然而然能够理解 Q 学习的数学式了。

本书的特点是“从零开始”。我们从零开始实现强化学习算法，不依赖像“黑盒”一样的外部库。代码的实现尽可能简单，这样强化学习的关键思想就会很清晰。另外，本书会以循序渐进的方式来设置要用强化学习来解决的问题。这样你就能体会到强化学习是多么困难，又多么有趣。

整体流程

本书大体上可分为两个部分：前半部分（第 1~6 章）介绍强化学习的基础知识。虽然这部分没有涉及深度学习，但我们将逐一了解强化学习中的重要思想和技术；后半部分（第 7~10 章）介绍如何将深度学习应用于强化学习问题，并会解读最先进的深度强化学习算法。

深度学习将在本书第 7 章中“闪亮登场”。要读到这一章需要一些时间。不过在你掌握了强化学习的基础知识之后，将深度学习的工具应用到

其中并不难。但如果跳过强化学习的基础知识部分，直接进入深度强化学习的章节，那么你对深度强化学习的理解就只能停留在肤浅的层面。

下面简单介绍一下各章的主要内容。首先，第 1 章将解决老虎机问题，也就是在多个候选者中逐个找到最佳者的问题。老虎机问题是强化学习中最简单的问题，最适合用于入门。然后，第 2 章将借助“马尔可夫决策过程”的框架定义一般的强化学习问题。第 3 章将推导“贝尔曼方程”，它是在马尔可夫决策过程中找到最佳答案的关键。接下来，我们将学习解决“贝尔曼方程”的各种方法——动态规划法（第 4 章）、蒙特卡洛方法（第 5 章）和 TD 方法（第 6 章）。学到这里，你应该对强化学习的基本原理有了更深的理解，进而也能更容易学习后续的内容。

我们将在第 7 章学习深度学习，以及将深度学习应用于强化学习的方法。第 8 章将实现 DQN 以及 DQN 的扩展算法。第 9 章将研究有别于 DQN 的另一种算法，即“策略梯度法”（策略梯度法是一类算法的总称，我们在实践中实现的算法是 REINFORCE 和 Actor-Critic）。最后，第 10 章将简要介绍现代深度强化学习算法，具体包括 A3C、DDPG、TRPO 和 Rainbow 算法。另外，这一章还将介绍深度强化学习的案例研究，并会讨论深度强化学习的潜力和存在的问题。

必要的软件

本书中使用的 Python 版本和外部库如下所示。

- Python 3 系列
- NumPy
- Matplotlib
- DeZero（或 PyTorch）
- OpenAI Gym

本书将使用 DeZero 作为深度学习的框架，这个框架是我们在“深度学习入门 & 进阶”系列的第三本书中创建的。DeZero 是一个简单的框架，你

应该很快就能掌握它。第 7 章对如何使用 DeZero 做了介绍。虽然本书使用的是 DeZero，但使用其他框架（如 PyTorch 或 TensorFlow）也不影响阅读本书。



DeZero 和 PyTorch 的许多 API 是相同的。因此，你可以很容易地将 DeZero 版的代码移植到 PyTorch 版的代码中。本书配套的 GitHub 仓库（参见下文）也提供了 PyTorch 版的代码。

OpenAI Gym 是强化学习的模拟环境，第 8 章会介绍如何安装和使用它。

文件构成

本书中使用的代码可以从以下网址获取。¹

<https://github.com/oreilly-japan/deep-learning-from-scratch-4>

该资源库的文件夹结构如表 1 所示。

表 1 文件夹结构

文件夹名称	说明
ch01	第 1 章中使用的源代码
⋮	⋮
ch09	第 9 章中使用的源代码
common	公用的源代码
pytorch	移植到 PyTorch 中的源代码

文件夹 ch01 到 ch09 是本书各章所使用的文件。要运行这些文件，请按以下方法执行 Python 命令（可以从任何目录中执行 Python 命令）。

```
$ python ch01/avg.py
$ python ch08/dqn.py

$ cd ch09
$ python actor_critic.py
```

注 1：也可到图灵社区本书主页 ituring.com.cn/book/3185 下载示例代码。——编者注

变化的东西和不变的东西

如今，人工智能正以惊人的速度向前发展。与人工智能有关的技术和应用程序以令人眼花缭乱的不断在进化，一个又一个流行的技术和应用程序相继出现。不过其中许多东西注定要消亡。但有些东西不会改变，会代代相传。你在本书中学到的就是不变的东西。笔者坚信，强化学习、马尔可夫决策过程、贝尔曼方程、Q 学习、神经网络的基本原则等技术将一如既往地重要。希望读者通过本书熟练掌握强化学习领域的基础知识，并体会到“不变的东西”的魅力。

排版规则

本书在排版上采用如下规则。

黑体字

用来表示新引入的术语、强调的要点以及关键短语。

等宽字 (Constant Width)

用来表示下面这些信息：程序代码、命令、序列、组成元素、语句选项、分支、变量、属性、键值、函数、类型、类、命名空间、方法、模块、属性、参数、值、对象、事件、事件处理器、XML 标签、HTML 标签、宏、文件的内容、来自命令行的输出等。若在其他地方引用了以上这些内容（如变量、函数、关键字等），也会使用该格式标记。

等宽粗体字 (Constant Width Bold)

用来表示用户输入的命令或文本信息。在强调代码的作用时也会使用该格式标记。

等宽斜体字 (Constant Width Italic)

用来表示必须根据用户环境替换的字符串。



用来表示提示、启发以及某些值得深究的内容的补充信息。



表示程序库中存在的 **bug** 或时常会发生的问题等警告信息，以引起读者对该处内容的注意。

读者意见与咨询

虽然笔者已经尽最大努力对本书的内容进行了验证与确认，但仍不免在某些地方出现错误或者容易引起误解的表达等，给读者的理解带来困扰。如果读者遇到这些问题，请及时告知，我们在本书重印时会将其改正，在此先表示不胜感激。与此同时，也希望读者能够为本书将来的修订提出中肯的建议。本书编辑部的联系方式如下。

株式会社 O'Reilly Japan

电子邮件 japan@oreilly.co.jp

本书的主页地址如下。

<http://www.ituring.com.cn/book/3185>

<https://www.oreilly.co.jp/books/9784873119755>（日语）

<https://github.com/oreilly-japan/deep-learning-from-scratch-4>

关于 O'Reilly 的其他信息，可以访问下面的 O'Reilly 主页查看。

<http://www.oreilly.com>（英语）

<http://www.oreilly.co.jp>（日语）

第 1 章

老虎机问题

人类可以在没有老师或教练指导的情况下学习。例如，幼儿通过不断试错，自然地学习抓取物体、走路和跑步（当然有父母的一些支持）。强化学习同样不需要老师的帮助，而是自发地从与环境的互动中学习更好的解决方案。这种能力是人类以及生命体所拥有的重要品质。学习强化学习领域的知识也有助于学习更通用的智能。

在本章中，我们将学习强化学习。首先，我们会介绍强化学习在机器学习中的定位。然后，在简单了解强化学习的特点后，我们会立即解决一个具体问题——老虎机问题，这是强化学习领域最基本的问题。通过解决这个问题，你将体会到强化学习是多么困难和有趣。

1.1 机器学习的分类与强化学习

顾名思义，机器学习是一种向机器提供数据，让它进行学习的技术。我们可以向机器（也就是计算机）提供数据，让它自己找到某种规则或模式。无须程序员编写规则，计算机自身可以根据数据来学习规则。

我们可以根据问题的结构来对机器学习中使用的方法进行划分。通常有以下 3 种方法：监督学习、无监督学习和强化学习。本节将分别对这 3 种方法进行简单的介绍，首先从监督学习开始。

1.1.1 监督学习

机器学习中最传统的方法是监督学习（supervised learning）。在监督学习中，需要提供输入和输出的数据对。例如，在识别手写数字的问题上，需要提供“手写数字的图像”和“正确答案标签”数据对。利用这些数据，计算机学会了如何从输入转换到输出，如图 1-1 所示。

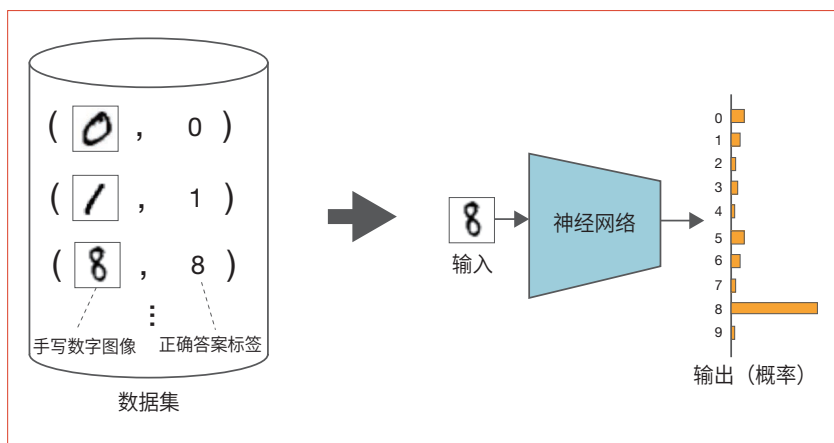


图 1-1 监督学习的例子（当输入手写数字“8”的图像时，计算机判断它是正确答案标签“8”的概率会增加）

监督学习的一个特点是“正确答案标签”的存在。输入的数据由“老师”打标签，这个标签就是正确答案。大多数情况下，“老师”这个角色由人来担任。在图 1-1 的例子中，由人来为每张图片打上正确的标签。

1.1.2 无监督学习

在无监督学习（unsupervised learning）中，没有“老师”的存在，因此我们获取的数据中也没有老师给出的“正确答案标签”。无监督学习的主要目标是找到隐藏在数据中的结构和模式。无监督学习的例子有分组（聚类）、特征提取和降维。图 1-2 是通过 t-SNE^[1] 算法将数据的特征减少到两个维度的例子。

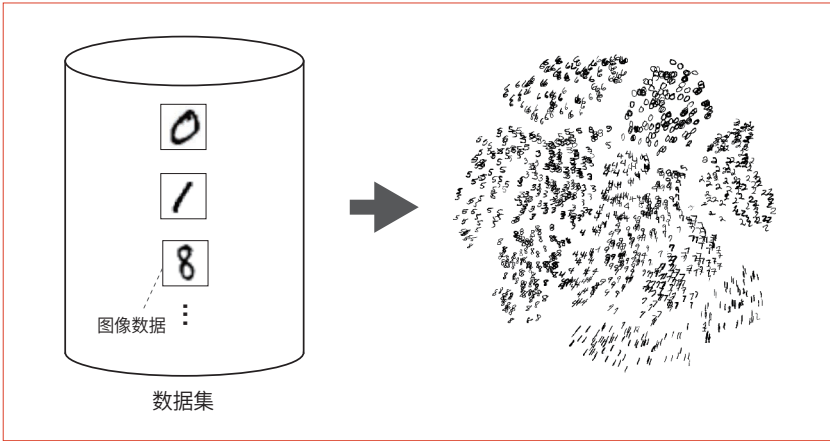


图 1-2 无监督学习的例子（通过 t-SNE 降维）

无监督学习不需要“正确答案标签”。因此，即使是大数据（大量的数据）的**准备也比较容易**。与之相比，监督学习的正确答案标签往往需要由人一个个地手动附加。**手动附加“正确答案标签”的过程叫作“打标签”或“标注”。**



“打标签”的过程需要花费大量的时间。例如，对于 ImageNet 这个用于图像识别的巨大数据集（超过 1400 万张图像），一个人要花费 20 年左右的时间才能完成打标签的工作。

1.1.3 强化学习

接下来是对**强化学习**（reinforcement learning）的简单介绍，这是本书的主题。强化学习是一种与传统的监督学习和无监督学习不同的问题设置（框架）。在强化学习中，**智能代理和环境相互作用**，如图 1-3 所示。



图 1-3 强化学习框架

智能代理是**行动的主体**。作为一个例子，我们很容易想象会**采取行动的东西**，比如机器人。智能代理被置于某种环境中，观察环境的“状态”，并在此基础上采取“行动”。结果，环境的状态发生了变化，智能代理从环境中获得了“奖励”，同时观察到了“新的状态”。强化学习的**目标是掌握使智能代理获得的奖励之和最大化的行动模式**。

作为强化学习的一个例子，我们来思考一下机器人行走的问题。该机器人被放置在真实空间（或模拟器上），并学习如何在其中行走。机器人的目标是学习如何高效地向前行走。对于这个问题，机器人采取了移动四肢的行动。这个行动会改变周围的环境（状态），获得的奖励是它前进的距离，如图 1-4 所示。

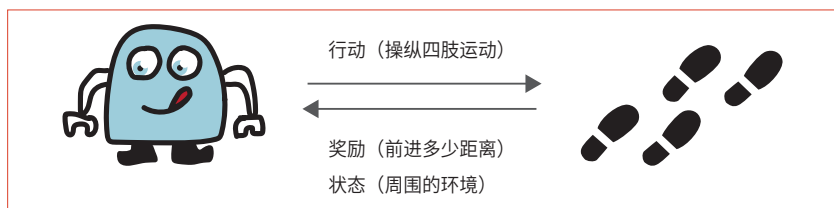


图 1-4 机器人行走问题中的相互作用

在机器人行走问题中，机器人不会直接学习好的行走方式（例如，当四肢以某个角度移动时，更容易行走）。要准备一个“老师”来教这种正确的肢体使用是非常困难的。机器人能做的是**采取一些行动，得到反馈（奖励）**，获得经验，并**从这些经验中学习**。它从经验中学习看起来好的行动，比如当它的手移动到某个角度时，就会失去平衡，而在另一个角度，就不会失去平衡。换句话说，它通过反复试验收集数据，并从收集到的数据中学习良好的行为习惯。



强化学习接受“奖励”**作为来自环境的反馈**。这种“奖励”在本质上与监督学习中的“正确答案标签”**不同**。奖励是对一个行动的反馈，根据奖励不能判断实际采取的行动是不是最佳行动。监督学习则是提供“正确答案标签”。在强化学习的语境下，这意味着无论采取什么行动，都有一个“老师”会提示什么是最佳行动。

以上就是机器学习领域的典型分类。我们观察所使用的数据，将每个

领域的特点总结如下。

- 监督学习：使用带有输入和输出（正确答案标签）的成对数据，学习如何将输入转化为输出。
- 无监督学习：使用没有“正确答案标签”的数据来学习数据中的基本结构。
- 强化学习：学习如何使用智能代理在与环境互动时收集的数据获得更高的奖励。

因此，强化学习与其他类别有很大的不同。在与环境的互动中学习，从反复试错中学习，这才是强化学习。

1.2 老虎机问题

接下来我们将继续研究强化学习的具体问题。这里我们将探讨强化学习中最简单的问题之一——老虎机问题（bandit problem）。老虎机问题虽然简单，但它具有强化学习所处理的问题的基本特征。通过解决老虎机问题，强化学习的特征将变得更加清晰。

1.2.1 什么是老虎机问题

下面介绍一下老虎机问题。首先，老虎机还有一个别称，叫作“强盗机”（bandit）。大家可能知道，每台老虎机都有一个拉杆，当拉动拉杆时，老虎机上的图案会随之发生改变。这些图案的排列将影响你能赢得的硬币的数量（如果没出现特定的图案，获得的硬币就是 0 枚）。



bandit 的本意是“强盗”或“山贼”。一开始它并没有“老虎机”的意思，但是后来赌徒们讽刺地将老虎机称为“单臂强盗”（one-armed bandit）。之所以称之为“单臂强盗”，是因为赌场里的老虎机只有一个拉杆，当你投入硬币后拉动杠杆，大多数情况下硬币就会被“抢走”。

更准确地说，我们探讨的老虎机问题是多臂老虎机问题（multi-armed bandit problem）。多臂本来指的是老虎机有一个以上的臂（拉杆），不过这

个“多臂”问题也可以指图 1-5 所示的有多台单臂老虎机的情况。本书探讨的是后一种情形——多台单臂老虎机的问题。



图 1-5 多臂老虎机问题也就是多台单臂老虎机的问题

老虎机问题涉及的多台老虎机中的每一台都有不同的特点。不同的特点意味着有些机器赢得多，有些机器输得多。在这个前提下，玩家玩老虎机的次数是固定的（如 1000 次）。玩家一开始并不知道关于老虎机的任何信息，比如哪些机器更容易赢。他们需要实际操作机器，根据结果寻找看起来不错的机器。他们的目标是在给定的次数内获得尽可能多的硬币。

我们现在来看一下解决老虎机问题的算法。不过在这之前，我们将用强化学习的术语来描述刚刚介绍的老虎机问题。在强化学习的框架中，出现在老虎机问题中的老虎机叫作环境（environment），老虎机的玩家叫作智能代理（agent）。这两个角色（环境和智能代理）会在强化学习的框架内进行“互动”。

那么，环境和智能代理之间的“互动”是什么样的呢？在老虎机问题中，玩家从多台老虎机中选择一台来玩。这是由智能代理采取的行动（action）。然后，作为玩的结果，玩家从老虎机中获得硬币。硬币就是奖励（reward）。上述概念之间的关系如图 1-6 所示。

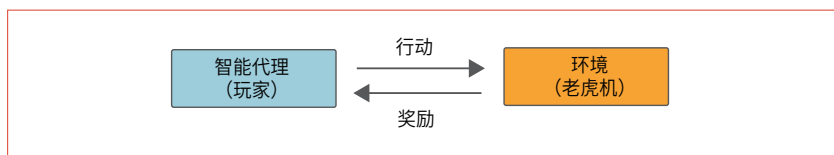


图 1-6 老虎机问题的框架

如图 1-6 所示，智能代理对环境采取行动，结果获得了奖励。智能代理与环境之间的这种互动就是强化学习的框架，老虎机问题也包含在其中。



在一般的强化学习问题中，环境内部拥有“状态”（state）。当智能代理采取某些行动时，环境的状态会发生变化，智能代理会根据状态采取适当的行动。在老虎机问题中，玩家每次都面对同一组老虎机。环境的状态没有改变，因此不需要考虑状态问题。状态发生变化的问题将在第 2 章中处理。

现在我们来思考一下解决老虎机问题的算法。我们的目标是获得尽可能多的硬币。当然，为了实现这个目标，需要选择“好的老虎机”。那什么样的机器是“好的老虎机”呢？下面我们来探讨这个问题。

1.2.2 什么是好的老虎机

在探讨老虎机问题时，首先要记住的重要的一点是老虎机具有随机性。随机性意味着玩老虎机得到的硬币数量（奖励）每次都会改变。我们用“概率”来表示这种随机性，也就是用概率来定量地描述有多随机。

现在我们通过具体的例子来思考一下。下面来看一个有两台老虎机的例子，它们的奖励和概率如图 1-7 所示。

老虎机 a					老虎机 b				
能获得的硬币数量	0	1	5	10	能获得的硬币数量	0	1	5	10
概率	0.70	0.15	0.12	0.03	概率	0.50	0.40	0.09	0.01

图 1-7 老虎机的概率分布表

图 1-7 中的表格叫作概率分布表。通常情况下，玩家并不知道这样的概率分布表。这里探讨的是假定图 1-7 所示的概率分布表是已知的情况。在这种情况下，玩家应该选择哪台老虎机呢？



图 1-7 中的表是一个离散的概率分布表。离散意味着随机变量取的是 0、1、5、10 这样不连续的数值。反之，如果一个随机变量取的是连续值（如所有的实数），那么它就被称为“连续型随机变量”。

那么，在图 1-7 的两台机器中，哪台更好呢？我们应该根据什么标准来判断老虎机的好坏呢？如前所述，玩老虎机的结果是概率性变化的。不过，如果你玩得次数多——理论上是无限次，那么得到的硬币的平均数量就是固定的“单个值”。这个值就是**期望值** (expectation value)。以这个期望值为判断指标，值越大，老虎机就越好。

对于给定的离散概率分布表，可以通过将“值”（得到的硬币数量）和“概率”分别相乘，然后再相加来计算期望值。如果对图 1-7 中的概率分布表进行计算，那么期望值的计算结果如下所示。

$$\text{老虎机 a: } 0 \times 0.70 + 1 \times 0.15 + 5 \times 0.12 + 10 \times 0.03 = 1.05$$

$$\text{老虎机 b: } 0 \times 0.50 + 1 \times 0.40 + 5 \times 0.09 + 10 \times 0.01 = 0.95$$

这两台老虎机各自的期望值可按上述方法计算。这个期望值是指当玩两台老虎机时，平均可以得到的硬币数量。比较这两个期望值，可以发现老虎机 a 的期望值更大。这也就是说，如果把期望值作为指标，那么老虎机 a 就是更好的机器。因此，如果有 a 和 b 两台老虎机，并且要玩 1000 次，那么选择老虎机 a 总是更好的策略。

需要记住的是，像玩老虎机这样的概率事件可以使用“期望值”来评估。换句话说，**为了不被随机性所迷惑，需要把“期望值”作为指标。**



在老虎机问题中，**奖励的期望值**有时会被称为**价值** (value) 这个特殊的名称。特别是“**采取某个行动所得到的奖励的期望值**”被称为“**行动价值**” (action value)。因此，玩老虎机 a 时可获得的硬币数量的期望值也被称为“老虎机 a 的价值”或“a 的行动价值”。

1.2.3 使用数学式表示

为了熟悉数学表述（符号），本节将使用数学式回顾前面介绍过的内容。首先，我们使用 Reward 的首字母 **R** 表示**奖励**。在老虎机问题中，**R** 是老虎机**返回的硬币数量**。在前面的例子中，**R** 的取值为 {0, 1, 5, 10} 中的一个，每个值的“取值难易程度”是概率值。像这种取值由概率决定的变量叫作**随机变量** (random variable)。



在老虎机问题以及强化学习问题中，智能代理一次、两次……连续多次采取行动。我们用 R_t 来表示在第 t 次获得的奖励。

然后，我们用 Action 的首字母 A 表示智能代理采取的行动。在这个例子中，如果分别用 a 和 b 表示选择老虎机 a 和老虎机 b 的行动，那么变量 A 的取值就是 $\{a, b\}$ 其中之一。

接下来是由随机变量定义的“期望值”。我们用 Expectation 的首字母表示期望值，符号为 $\mathbb{E}$ 。例如，奖励 R 的期望值可以写为 $\mathbb{E}[R]$ ，选择行动 A 时的奖励的期望值可以写为 $\mathbb{E}[R|A]$ 。这里的 $|$ 代表“条件概率（条件期望值）”。 $|$ 的右侧是条件。例如，选择行动 a 时的奖励的期望值可以写为 $\mathbb{E}[R|A=a]$ ，或者简单地写为 $\mathbb{E}[R|a]$ 。

奖励的期望值叫作“行动价值”。在强化学习领域，人们习惯于用符号 Q 或 q 来表示行动价值（ Q 是 Quality 的首字母）。这里有以下等式成立。

$$q(A) = \mathbb{E}[R|A]$$

其中的符号 $q(A)$ 表示行动 A 的价值。



行动价值既可以用 $q(A)$ 表示，也可以用 $Q(A)$ 表示。 $q(A)$ 代表“真正的行动价值”，而 $Q(A)$ 表示这是一个估计值。由于智能代理无法知道真正的行动价值 $q(A)$ ，因此它需要“估计”这个值，后面会详细解释。在这种情况下，行动价值的估计值就用 $Q(A)$ 表示。

以上是对数学式的符号的说明。下面我们将继续思考解决老虎机问题的算法。

1.3 老虎机算法

在老虎机问题中，玩家无法知道老虎机的“价值”（奖励的期望值）。而问题的设定需要玩家选择价值最大的老虎机。所以玩家需要实际去玩老虎机，并根据结果判断自己的选择有多好或多差。总结以上内容，可以得

出以下需求。

- 如果每台老虎机的价值（奖励的期望值）是已知的，那么玩家就可以选择最好的老虎机。
- 但玩家不知道每台老虎机的价值。
- 因此，玩家需要尽可能准确地估计每台老虎机的价值。

我们要找到满足上述要求并增加获得的硬币总数的方法。接下来，让我们结合具体的例子来思考如何估计老虎机的价值。



在监督学习中，数据集中存在与问题相对应的正确答案标签。因此，即使正在学习的模型做出了错误的预测，它仍然知道正确答案标签。用强化学习的术语来讲，即使采取了错误的行动，模型仍然会得到正确的答案（正确的行动是什么样）。而在强化学习中，玩家自己采取行动，只能得到该行动的结果（奖励）。例如，我们思考一个玩家玩老虎机，并获得了 1 枚硬币作为结果的场景。在这种情况下，这个值 1 只是一条线索。玩家必须以奖励为线索，自己估计和判断什么是正确答案以及什么是最佳行动。

1.3.1 价值的估计方法

让我们思考这样一种情况：有 a 和 b 两台老虎机，玩家各玩一次，产生的结果如图 1-8 所示。

老虎机	结果
	第 1 次
a	0
b	1

图 1-8 两台老虎机各玩一次的结果

如图 1-8 所示，玩家玩老虎机 a，得到 0 枚硬币，然后玩老虎机 b，得到 1 枚硬币。此时，我们认为“实际获得的奖励的平均值”是对老虎机价值的一种估计。由于只玩了一次老虎机 a，因此我们只能估计老虎机 a 的值为 0、老虎机 b 的值为 1。

当然，由于只玩了一次，想必对老虎机价值的估计不会很可靠。但随着游戏次数的增加，对老虎机价值的估计会变得越来越可靠。在得到图 1-8 的结果后，我们继续思考另一种情况，如图 1-9 所示。

老虎机	结果		
	第 1 次	第 2 次	第 3 次
a	0	1	5
b	1	0	0

图 1-9 又玩了几次老虎机的结果

如图 1-9 所示，玩家分别玩了 3 次老虎机 a 和老虎机 b。玩家玩老虎机 a 分别获得了 0、1 和 5 枚硬币。根据这个结果，可以计算出对老虎机 a 的价值的估计值，我们用 $Q(a)$ 来表示这个值，计算式如下所示。

$$Q(a) = \frac{0+1+5}{3} = 2$$

从上面的计算结果来看，这意味着平均每次获得 2 枚硬币。这个平均值就是对老虎机 a 的价值的估计值。另外，玩家玩老虎机 b 分别获得了 1、0 和 0 枚硬币。根据这个结果，可以计算出对老虎机 b 的价值的估计值，我们用 $Q(b)$ 来表示这个值，计算式如下所示。

$$Q(b) = \frac{1+0+0}{3} = 0.33\dots$$

从结果来看，老虎机 a 似乎更好。当然，此时的估计值并不等于老虎机的“真实价值”。不过我们可以认为它是比各只玩一次老虎机 a 和老虎机 b 时更可靠的估计值。



实际地玩老虎机获得的奖励是基于某个概率分布产生的“样本”（sample）。因此，实际获得的奖励的平均值可以称为样本均值。随着采样次数的增加，样本均值越来越接近真实值（奖励的期望值）。根据“大数定律”这个概率论的定理，如果采样的次数无限多，那么样本均值将与真实值相等。

下面我们尝试编写一下估计老虎机的价值的代码。

1.3.2 求平均值的实现

现在我们只关注一台老虎机，思考玩家玩该老虎机 n 次的情况。换句话说，只考虑一个行动被执行 n 次的情况，并估计该行动的价值。我们用 $R_1, R_2, \dots, R_n$ 来表示在这种情况下实际获得的奖励，用 Q_n 表示执行了 n 次该行动后的行动价值的估计值。那么 Q_n 就可以写成下面的数学式。

$$Q_n = \frac{R_1 + R_2 + \dots + R_n}{n} \quad (1.1)$$

如式 1.1 所示，第 n 次的行动价值的估计值 Q_n 是通过计算 n 个奖励的样本均值而求得的。现在让我们根据式 1.1 来实现样本均值的计算。这里假定连续获得了 10 次奖励，并且每次获得奖励后都要计算估计值。代码如下所示。

ch01/avg.py

```
import numpy as np

np.random.seed(0) # 固定随机数种子
rewards = []

for n in range(1, 11): # 从 1 到 10
    reward = np.random.rand() # 虚拟的奖励
    rewards.append(reward)
    Q = sum(rewards) / n
    print(Q)
```

输出结果

```
0.5488135039273248
0.6320014351498722
...
0.6157662833145425
```

代码中生成一个实数随机数（0.0 和 1.0 之间的随机数）作为虚拟的奖励。代码将获得的奖励添加到 `rewards` 列表中。这样就可以直接实现式 1.1 中的计算。当然，用这段代码可以正确地求出样本均值。不过，这个实现

方式中还有待改进之处。

上面的代码实现中存在的问题是，随着游戏次数（ n ）的增加，rewards 中的元素数量也会增加。此外，求 n 个奖励的总和，也就是代码中的 $\text{sum}(\text{rewards})$ 的计算量也会增加。因此，随着游戏次数的增加，占用的内存和计算量也会增加。

下面介绍样本均值更好的实现方法。首先做一些准备，关注第 $n-1$ 次行动价值的估计值 Q_{n-1} 。计算 Q_{n-1} 的数学式如下所示。

$$Q_{n-1} = \frac{R_1 + R_2 + \cdots + R_{n-1}}{n-1}$$

式子两边乘以 $n-1$ ，得到下面的式子。

$$R_1 + R_2 + \cdots + R_{n-1} = (n-1)Q_{n-1} \quad (1.2)$$

基于式 1.2，将 Q_n 表示如下。

$$\begin{aligned} Q_n &= \frac{R_1 + R_2 + \cdots + R_n}{n} \\ &= \frac{1}{n} \underbrace{(R_1 + \cdots + R_{n-1})}_{(n-1)Q_{n-1}} + R_n \end{aligned} \quad (1.3)$$

$$\begin{aligned} &= \frac{1}{n} \{(n-1)Q_{n-1} + R_n\} \\ &= \left(1 - \frac{1}{n}\right)Q_{n-1} + \frac{1}{n}R_n \end{aligned} \quad (1.4)$$

这里的重点是将式 1.2 代入式 1.3。这样就推导出了式 1.4 所示的 Q_n 和 Q_{n-1} 之间的关系。最后，从式 1.4 可知，要想求出 Q_n ，只需 Q_{n-1} 、 R_n 和 n 这 3 个值即可。重点是每次计算时无须使用之前获得的所有奖励（ $R_1, R_2, \cdots, R_{n-1}$ ）。

之后进一步将式 1.4 变形。

$$Q_n = \left(1 - \frac{1}{n}\right)Q_{n-1} + \frac{1}{n}R_n \quad (1.4)$$

$$= Q_{n-1} + \frac{1}{n}(R_n - Q_{n-1}) \quad (1.5)$$

式 1.5 中需要注意的一点是它呈现了 $Q_n = Q_{n-1} + \dots$ 的形式。由此可以看出, Q_n 是以 Q_{n-1} 为基准发生变化的, 变化的部分在于基准 Q_{n-1} 的右边的第二项, 即 $\frac{1}{n}(R_n - Q_{n-1})$ 。式子中 Q_{n-1} 、 Q_n 、 R_n 的位置关系如图 1-10 所示。

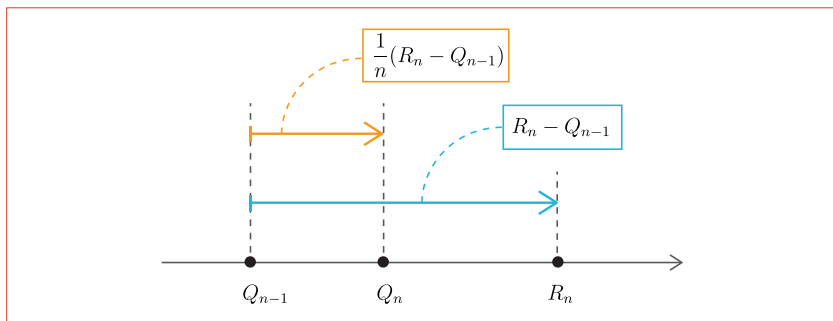


图 1-10 Q_{n-1} 、 Q_n 、 R_n 的位置关系

如图 1-10 所示, 用 $R_n - Q_{n-1}$ 的长度乘以 $\frac{1}{n}$ 得到的值就是要更新的量。当 Q_{n-1} 被更新为 Q_n 时, 它向 R_n 的方向移动, 但移动量受 $\frac{1}{n}$ 调整。由于可以调整更新量, 因此这个 $\frac{1}{n}$ 起到了“学习率”的作用。



随着试验次数 n 的增加, $\frac{1}{n}$ 越来越小。因此, 随着试验次数的增加, Q_n 的更新量会变小。例如, 当 $n=1$ 时, 由于 $\frac{1}{n}=1$, 因此 $Q_n=R_n$ 。此时, Q_n 被更新为 R_n 。另一个极端的例子是 $n \rightarrow +\infty$ 的情况, 由于 $\frac{1}{n}=0$, 因此 $Q_n=Q_{n-1}$ 。此时, Q_n 根本就没有被更新。

基于以上讨论结果, 我们来尝试一个求样本均值的实现方法。如果问题的设置与之前相同, 那么将可以实现如下代码。

ch01/avg.py

```
Q = 0

for n in range(1, 11):
    reward = np.random.rand()
    Q = Q + (reward - Q) / n
    print(Q)
```

实现了式 1.5 的是上面的 $Q = Q + (\text{reward} - Q) / n$ 这行代码。这里要注意的是，对应式 1.5 中的 Q_n 变量和 Q_{n-1} 变量的是同一个变量 Q 。为什么只用一个变量也没有问题呢？查看一下图 1-11 你就会明白了。

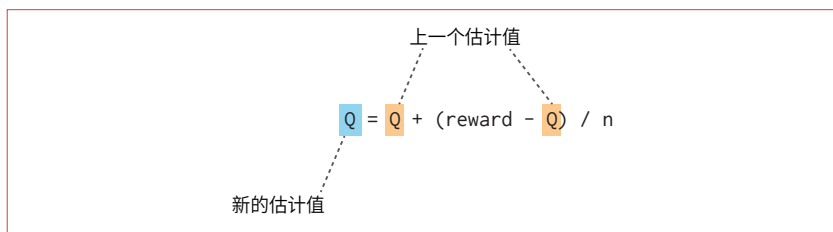


图 1-11 等号左侧和右侧的 Q 的实际状态

如图 1-11 所示，等号右侧的代码中的 Q 是对上一个估计值的引用。等号右侧计算的结果被重新赋值到左侧的 Q 。因此，只用一个变量 Q ，就可以将旧的估计值更新为新的估计值。另外，这种对 Q 的更新也可以用 $+=$ 运算符来写。这种写法的代码如下所示。

```
# Q = Q + (reward - Q) / n
Q += (reward - Q) / n
```

$Q = Q + \dots$ 这种赋值的代码可以写成上述 $Q += \dots$ 的形式。以上就是求样本均值的一个高效实现方法。由于能够按照 $Q_1, Q_2, Q_3, \dots$ 的顺序依次求得这些值，因此这种实现方法也可以称为“增量式实现”。

1.3.3 玩家的策略

现在让我们继续思考智能代理（玩家）应该采取什么策略。换句话说，我们将思考如何找到具有最大价值的老虎机。玩家的第一个可能的策略是根据到目前为止的实际游戏结果，选择最好的老虎机。具体来说，玩家可以考虑选择估计值（实际获得的奖励的平均值）最大的老虎机。这种做法可以称为“贪婪行动”。



“贪婪”（greedy）意味着“只从手头的信息中选择最好的方法，而不考虑将来”。老虎机问题中的贪婪行动也就相当于完全根据之前的游戏经验（根据每台老虎机价值的估计值）来选择最佳机器。

虽然贪婪行动看起来不错，但也有问题。例如，思考一下图 1-8 中所示的各玩一次老虎机 a 和老虎机 b 的情况。假定老虎机 a 的价值的估计值为 0，老虎机 b 的价值的估计值为 1。如果采取贪婪行动，那么从现在起玩家将只会选择老虎机 b。然而，实际上老虎机 a 可能更好。

出现这样的问题是由于老虎机价值的估计值中存在“不确定性”。如果玩家完全相信一个包含“不确定性”的估计值，那么就可能会错过最佳的行动方案。玩家需要减少“不确定性”，即要增强估计值的可靠性。考虑到上述情况，玩家可以尝试以下两种做法。

- 利用目前实际的游戏结果，玩那些看起来最好的老虎机（贪婪行动）。
- 为了对老虎机的价值做出更准确的估计，尝试不同的老虎机。

上述第一种做法是根据以前的经验选择最佳行动。这就是“贪婪行动”，另一种说法叫作利用（exploitation）。正如前面所解释的那样，如果采取贪婪行动的做法，那么玩家将根据以前的经验选择最佳行动。但这样做也有可能错过更好的选择。因此，玩家需要尝试第二种做法，即“非贪婪行动”。这种做法也叫作探索（exploration）。通过探索，玩家可以对每台老虎机的价值做出更准确的估计。



“利用”和“探索”二者之间是权衡关系。我们目前所做的权衡是只可能在“利用”和“探索”之间二选一，即必须牺牲二者之一。

在老虎机问题中，如果想在下一次尝试中得到一个好的结果，那么玩家就应该采取“利用”的做法。但从长期来看，如果想获得好的结果，那么玩家需要的是“探索”。这是因为通过“探索”，玩家找到更好的老虎机的概率更大。如果能找到更好的机器，那么此后就可以一直选择该机器，长期来看会获得更好的结果。

强化学习算法最终归结为如何在“利用”和“探索”之间取得平衡。为了取得这种平衡，人们提出了各种算法。这些算法既有简单的，也有复杂的，其中最基本、最常用的是 ϵ -greedy 算法。

ϵ -greedy 算法是一种简单的算法。它是一种以 ϵ 的概率 [如以 $\epsilon=0.1$ (10%) 的概率] 进行“探索”，以 $1-\epsilon$ 的概率进行“利用”的方法。“探索”过程会随机地选择行动，并尝试各种各样的行动。这使得对每个行动价值的估计值变得更加可靠。然后再以 $1-\epsilon$ 的概率执行贪婪行动（利用）。



ϵ -greedy 算法可以“利用”之前获得的经验。不过，通过偶尔尝试非贪婪行动，它也在“探索”是否有更好的行动。这个 ϵ -greedy 算法有望高效地解决老虎机问题。

以上是对老虎机问题及典型算法 ϵ -greedy 的介绍。下面我们将着手用 Python 实现前面介绍的内容。

1.4 老虎机算法的实现

现在我们来实现解决老虎机问题的算法。为了便于理解，这里只考虑老虎机返回的硬币数量为 0 或 1 的情况。换句话说，假定在玩老虎机时，得到的奖励要么是赢（1），要么是输（0）。然后，假定每台老虎机都有一个设置好的胜率（返回 1 枚硬币的概率）。如果胜率被设置为 0.6，那么就有 60% 的机会得到 1 枚硬币，40% 的机会得到 0 枚硬币。在这种情况下，老虎机的价值（老虎机返回的硬币的期望值）是 0.6。换句话说，胜率就是老虎机的价值。

我们思考 10 台老虎机被设置了不同胜率的情况。玩家不知道这些老虎机上设置的胜率，因此需要根据实际的游戏经验，找到胜率最大的老虎机。

1.4.1 老虎机的实现

下面实现有 10 台老虎机的情况（不过每台都被设置了随机的胜率）。这里我们将老虎机实现为 Bandit 类。这个 Bandit 类中包含 10 台老虎机。代码如下所示。

ch01/bandit.py

```
import numpy as np

class Bandit:
    def __init__(self, arms=10):
        self.rates = np.random.rand(arms) # 每台机器的胜率

    def play(self, arm):
        rate = self.rates[arm]
        if rate > np.random.rand():
            return 1
        else:
            return 0
```

首先是初始化参数 `arms`。`arms` 的意思是“臂的数量”，在这个问题上，它相当于“老虎机的台数”。老虎机的台数默认被设置为 10。另外，在 `__init__` 方法中，每台机器的胜率被设置为随机值。



`np.random.rand()` 可以生成一个介于 0.0 和 1.0 之间的随机数。这个随机数是基于均匀分布生成的。均匀分布意味着随机数的生成是均匀且无偏置的，随机数均匀分布在 0.0 和 1.0 之间（不存在诸如 0.5 左右的数值更有可能被生成的偏置情况）。另外，`np.random.rand(10)` 将生成 10 个介于 0.0 和 1.0 之间的随机数。这就把 10 台老虎机中每台的胜率随机地设置在了 0 和 1 之间。

接下来的方法是 `play(self, arm)`。这个方法参数 `arm` 指定了要玩第几个臂（老虎机）。方法内部的代码会在得到第 `arm` 台老虎机的胜率后，通过 `np.random.rand()` 生成一个介于 0.0 和 1.0 之间的随机数。将这个随机数与 `arm` 指定的老虎机的胜率进行比较，如果胜率大于随机数，就返回 1 作为奖励，否则返回 0。

实现了 `Bandit` 类之后，可以像下面这样编写实际玩老虎机的代码。

```
bandit = Bandit()

for i in range(3):
    print(bandit.play(0))
```

输出结果

```
1
0
0
```

这段代码连续玩了 3 次第 0 台老虎机，并输出了获得的硬币数量（每次运行时，这个输出的结果都不同）。以上就是老虎机的实现。下面进行智能代理（玩家）的实现。

1.4.2 智能代理的实现

在 1.3 节中，我们学习了求样本均值的高效实现方法——增量式实现方法。现在，兼做复习，让我们只关注第 0 台老虎机，并估计该老虎机的价值。代码如下所示。

```
bandit = Bandit()
Q = 0

for n in range(1, 11):
    reward = bandit.play(0) # 玩第 0 台老虎机
    Q += (reward - Q) / n
    print(Q)
```

这段代码连续玩了 10 次第 0 台老虎机，并且每次获得奖励后都会更新老虎机的估计值。到这里为止都是对 1.3 节内容的回顾。接下来尝试计算 10 台老虎机中每台老虎机价值的估计值。实现的代码如下所示。

```
bandit = Bandit()
Qs = np.zeros(10)
ns = np.zeros(10)

for n in range(10):
    action = np.random.randint(0, 10) # 随机行动
    reward = bandit.play(action)

    ns[action] += 1
    Qs[action] += (reward - Qs[action]) / ns[action]
    print(Qs)
```

代码中准备了新的拥有 10 个元素的一维数组 Qs 和 ns（Qs 和 ns 的所有

元素都通过 `np.zeros()` 被初始化为 0)。Qs 的每个元素存储的是对应的老虎机价值的估计值，ns 的每个元素存储的是对应的老虎机被玩过的次数。这样就能估计每台老虎机的价值了。

基于上述知识，我们实现一个名为 Agent 的类。在这个 Agent 类中，我们实现了一种基于 ϵ -greedy 算法选择行动的算法。代码如下所示。

ch01/bandit.py

```
class Agent:
    def __init__(self, epsilon, action_size=10):
        self.epsilon = epsilon
        self.Qs = np.zeros(action_size)
        self.ns = np.zeros(action_size)

    def update(self, action, reward):
        self.ns[action] += 1
        self.Qs[action] += (reward - self.Qs[action]) / self.ns[action]

    def get_action(self):
        if np.random.rand() < self.epsilon:
            return np.random.randint(0, len(self.Qs))
        return np.argmax(self.Qs)
```

初始化参数 epsilon 是 ϵ -greedy 算法中采取随机行动的概率。如果 epsilon=0.1，那么就有 10% 的概率采取随机行动。初始化参数 action_size 是智能代理可以选择的行動的数量。在老虎机问题中，它相当于老虎机的数量。

在上面的代码中，用于估计老虎机价值的是 update 方法。它的代码与之前的基本相同。最后，基于 ϵ -greedy 算法选择行动的方法是 get_action。方法中的代码会以 self.epsilon（如 0.1）的概率选择随机的行动，否则会选择行动价值的估计值最大的行动。另外，通过 `np.argmax(self.Qs)` 可以得到数组 self.Qs 中最大元素的索引。以上就是 Agent 类的实现。

1.4.3 尝试运行

下面让我们试着运行 Bandit 类和 Agent 类，来看看让智能代理执行 1000 次行动能得到多少奖励。代码如下所示。

ch01/bandit.py

```
import matplotlib.pyplot as plt # 导入 matplotlib

steps = 1000
epsilon = 0.1

bandit = Bandit()
agent = Agent(epsilon)
total_reward = 0
total_rewards = []
rates = []

for step in range(steps):
    action = agent.get_action() # ①选择行动
    reward = bandit.play(action) # ②实际去玩，获得奖励
    agent.update(action, reward) # ③从行动和奖励中学习
    total_reward += reward

    total_rewards.append(total_reward)
    rates.append(total_reward / (step+1))

print(total_reward)

# 图形的绘制 (1)
plt.ylabel('Total reward')
plt.xlabel('Steps')
plt.plot(total_rewards)
plt.show()

# 图形的绘制 (2)
plt.ylabel('Rates')
plt.xlabel('Steps')
plt.plot(rates)
plt.show()
```

输出结果

859

来看一下 for 语句中的代码。在这里智能代理和环境进行“交互”。首先，在①处智能代理选择行动。然后，在②处对环境实际地执行在①处选择的行动，获得奖励。最后，在③处让智能代理学习行动和奖励之间的关联性。这个流程会连续执行 1000 次，先把每一次迭代得到的奖励之和添加到 total_rewards 列表中，然后将之前的胜率（获胜所占的百分比）添加到 rates 列表中。

运行上面的代码，最终得到的奖励的总和为 859（每次运行的结果都不同）。这意味着在 1000 次游戏中，有 859 次中奖了。作为参考，图 1-12 展示了 total 的变化情况。

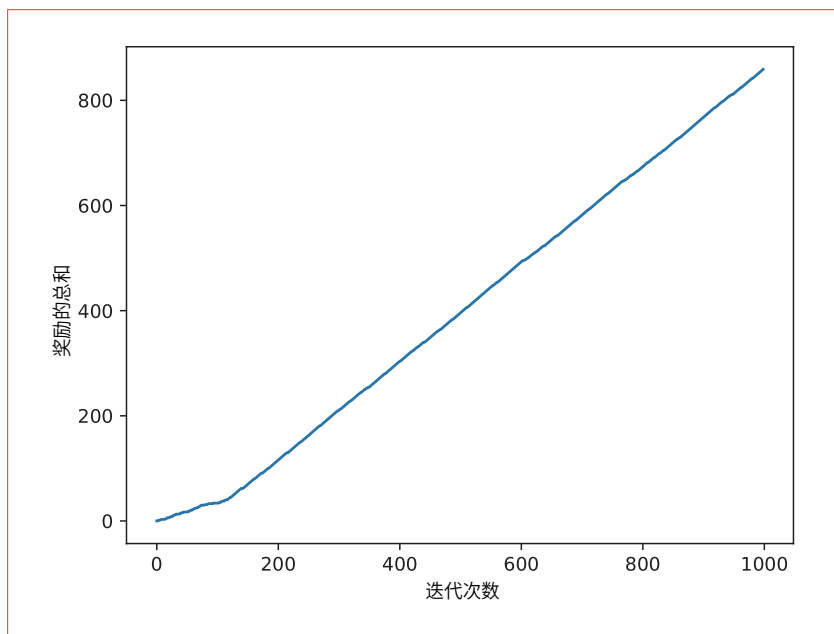


图 1-12 迭代次数与奖励的总和的变化图

如图 1-12 所示，随着每次迭代的进行，奖励的总和稳步增加。但是从这张图上很难看出增长的特征（比如奖励是如何随着迭代次数的增加而增加等）。因此让我们看第二张图。这张图的纵轴是胜率（获胜所占的百分比），如图 1-13 所示。

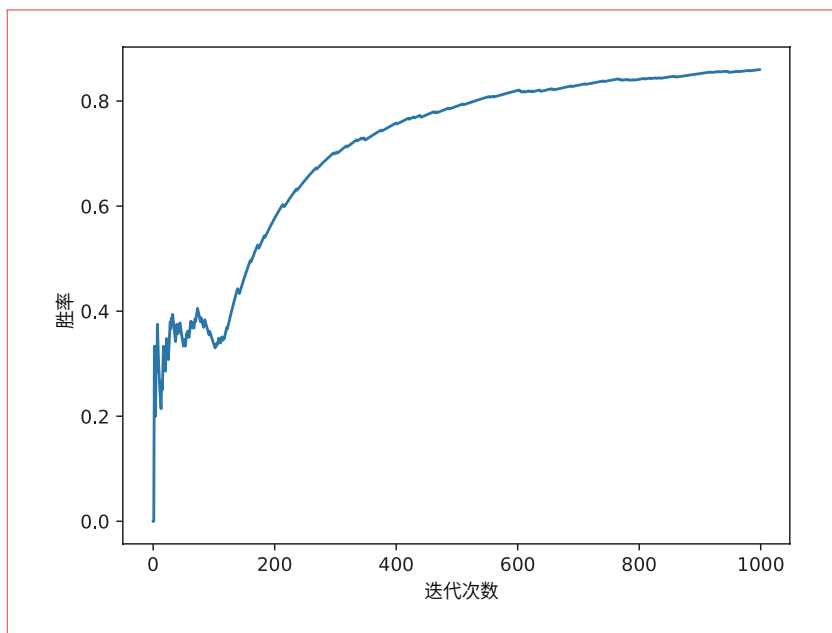


图 1-13 迭代次数与胜率的变化图

如图 1-13 所示，在 100 次迭代后的值大约为 0.4。这意味着在 100 次迭代后的胜率大约为 40%。从这里可以看出，随着迭代的进行，胜率也在提高。胜率在开始阶段迅速增加，经过 500 次迭代后仍然在缓慢增加。最终达到了超过 0.8 的胜率。看起来我们对 ϵ -greedy 算法的实现是正确的。

1.4.4 算法平均的特性

我们刚刚实现的代码在每次运行时结果都不一样。例如，图 1-13 的图形在每次运行时的形状都会发生明显的变化。我们做一次尝试，将同样的代码运行 10 次，并将结果绘制在一张图上，最终得到的图形如图 1-14 所示。

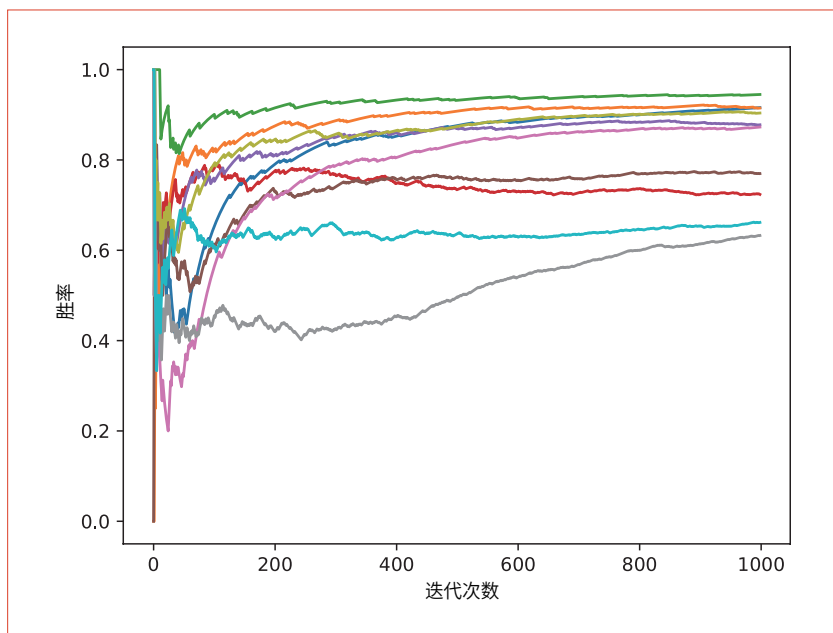


图 1-14 将 10 次运行的结果绘制在一起

如图 1-14 所示，每次运行实验得到的结果都不同。之所以会这样，是因为代码中存在随机性。具体来说，原因在于为 10 台老虎机设置胜率的地方。在那里，胜率是随机设置的。另外，在 ϵ -greedy 算法中，智能代理也是随机选择它们的行动的。由于这些随机性的存在，导致每次实验的结果都不同。



这里的随机性是由在代码层面发生的随机数种子的变化而导致的。如果固定了随机数种子（比如用 `np.random.seed(0)` 来固定），那么每次得到的结果都是相同的。

在比较强化学习算法时，由于随机性的存在，因此在大多数情况下得知单次实验的结果是没什么用的。与之相比，评估算法的“平均好坏”更为有用。一种可行的评估方法是进行多次相同的实验，并对结果进行平均。这样一来，就能看出算法的平均水平有多好。

这里沿用之前做过的玩 1000 次老虎机的实验，重复 200 次，然后对结果进行平均。代码如下所示。

ch01/bandit_avg.py

```
runs = 200
steps = 1000
epsilon = 0.1
all_rates = np.zeros((runs, steps)) # 形状为 (200, 1000) 的数组

for run in range(runs): # 200 次的实验
    bandit = Bandit()
    agent = Agent(epsilon)
    total_reward = 0
    rates = []

    for step in range(steps):
        action = agent.get_action()
        reward = bandit.play(action)
        agent.update(action, reward)
        total_reward += reward
        rates.append(total_reward / (step+1))

    all_rates[run] = rates # ①记录奖励的结果

avg_rates = np.average(all_rates, axis=0) # ②求每次迭代的平均值

# 绘制图形
plt.ylabel('Rates')
plt.xlabel('Steps')
plt.plot(avg_rates)
plt.show()
```

上面的代码进行了 200 次相同的实验，并将每次实验得到的结果存储在 `all_rates` 中。具体来说，在代码中的①处，`rates`（一个有 1000 个元素的数组）被存储在 `all_rates` 的相应位置。然后，通过在代码的②处指定 `axis=0`，可以计算出每次迭代的平均值，如图 1-15 所示。

	1	2	3	...	1000
第 1 次实验	1.0	0.5	0.333	⋮	0.913
第 2 次实验	0.0	0.0	0.0	⋮	0.821
⋮	⋮	⋮	⋮	⋮	⋮
第 200 次实验	1.0	1.0	1.0	⋮	0.615
平均值	0.493	0.497	0.504	⋮	0.838

图 1-15 求每次迭代的平均值

运行上面的代码可以得到图 1-16。

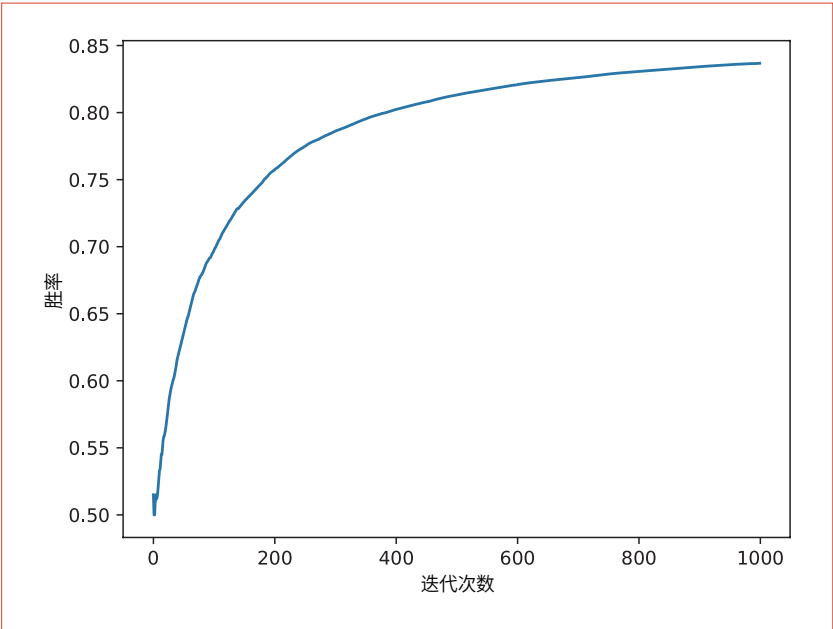


图 1-16 迭代次数与胜率的变化图（200 次实验的平均结果）

图 1-16 显示的是 200 次实验的平均结果。在评估算法时，使用这种平均结果是很重要的。通过观察图 1-16，可以看出这次使用的算法 (ϵ -greedy 算法) 的特点更加鲜明了。首先从 0.5 左右的胜率开始，随着迭代的进行，胜率迅速提高。从大约第 600 次迭代开始胜率逐渐趋于稳定，最终胜率达到 0.83 左右。

顺带一提，上述结果是 ϵ -greedy 算法中的 $\epsilon = 0.1$ 时的结果。改变这个 ϵ 的值，结果也会改变。现在，让我们试着改变几次 ϵ 的值。这里尝试了 3 个 ϵ 值：0.01、0.1 和 0.3。结果如图 1-17 所示。

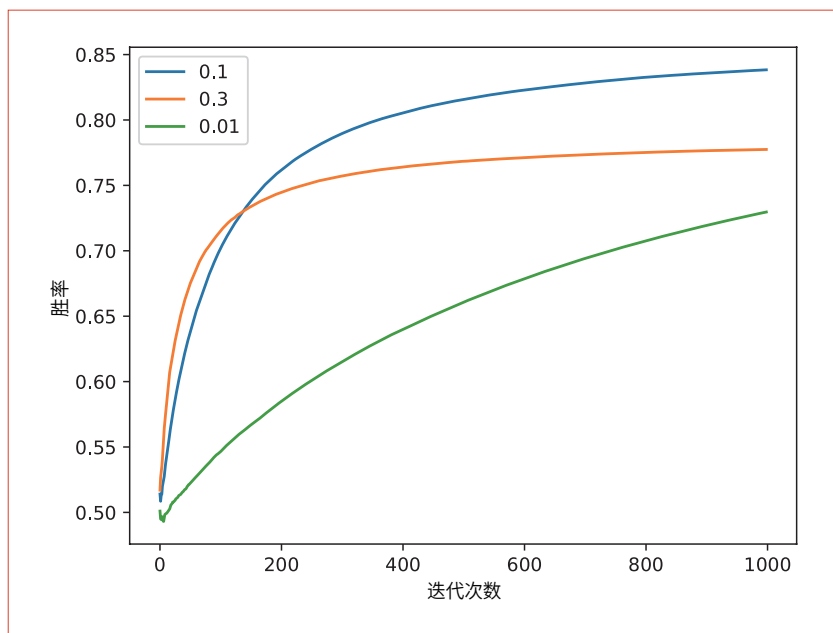


图 1-17 改变 ϵ -greedy 算法的 ϵ 后的结果

如图 1-17 所示，不同的 ϵ 值使得胜率发生了改变。结果显示，对于 $\epsilon = 0.3$ 的情况，胜率会迅速提高，但从大约第 400 次迭代开始，“上升”停止。这是因为探索是以 30% 的概率进行的，选择最佳机器的概率是有限制的。换句话说，我们已经做了过多的探索。

接下来是 $\varepsilon = 0.01$ 的情况（以 1% 的概率进行探索）。这种情况下的奖励增长得很慢，其速度是 3 种情况中最慢的。这应该是探索比例太小的缘故，所以找到最佳机器的概率很低。

最后是 $\varepsilon = 0.1$ 的情况。可以看出，这是 3 种情况中最好的结果。因此，我们可以说 $\varepsilon = 0.1$ ，即以 10% 的概率进行探索是这 3 种情况中取得了“利用和探索”之间最佳平衡的一种。从实验中可知，“利用和探索”之间的平衡可以根据 ε 值来调整。当然，最佳的 ε 值取决于实际的问题。例如，在迭代次数是 100 的情况下， $\varepsilon = 0.3$ 是所试的 3 个 ε 值中最好的结果。就像我们在这个例子中所做的那样，通过尝试几个 ε 值，就能够找到 ε 的最佳值。以上就是对 ε -greedy 算法的实现。

1.5 非稳态问题

前面探讨的老虎机问题都属于稳态问题（stationary problem）。稳态问题是指奖励的概率分布是稳定（不变）的。就我们的老虎机问题而言，为老虎机设定的胜率（老虎机的价值）保持固定。换句话说，一旦设定，老虎机的属性在智能代理游戏期间（比如在 1000 次游戏期间）就不会改变。从以下 Bandit 类的代码中就可以看出前面几节描述的老虎机问题都是稳态问题。

```
class Bandit:
    def __init__(self, arms=10):
        self.rates = np.random.rand(arms) # 一旦设置，就不再改变

    def play(self, arm):
        rate = self.rates[arm]
        if rate > np.random.rand():
            return 1
        else:
            return 0
```

如上述代码所示，`self.rates` 在初始化后没有变化。这种设置使得 Bandit 类成为稳态问题。我们要考虑的下一个问题是让这个 `self.rates` 在每次游戏中都发生轻微的变化。以下面的代码为例。

ch01/non_stationary.py

```

class NonStatBandit:
    def __init__(self, arms=10):
        self.arms = arms
        self.rates = np.random.rand(arms)

    def play(self, arm):
        rate = self.rates[arm]
        self.rates += 0.1 * np.random.randn(self.arms) # 添加噪声
        if rate > np.random.rand():
            return 1
        else:
            return 0

```

上面的代码实现了 `NonStatBandit` 类。这个类实际上只是在 `Bandit` 类的基础上添加了一行代码。添加的代码是在每次游戏时向 `self.rates` 添加一个小随机数的处理。顺带一提，`np.random.randn()` 基于均值为 0、标准差为 1 的正态分布生成随机数。这个处理将导致老虎机的价值（胜率）在每次游戏时都会出现变动。这样的问题设置叫作非稳态问题（non-stationary problem）。接下来我们来思考一下应该如何解决非稳态问题。

1.5.1 解决非稳态问题前的准备工作

首先从复习开始。为了估计老虎机的价值，在 1.4 节中我们计算了样本均值。如果用 $R_1, R_2, \dots, R_n$ 表示实际获得的奖励，那么样本均值的数学式就是下面这样的。

$$\begin{aligned}
 Q_n &= \frac{R_1 + R_2 + \dots + R_n}{n} \\
 &= \frac{1}{n} R_1 + \frac{1}{n} R_2 + \dots + \frac{1}{n} R_n
 \end{aligned}$$

从上面的数学式可以看出，样本均值可以通过求获得的奖励的平均值来计算。值得注意的是，所有的奖励都要乘以 $\frac{1}{n}$ 。这个 $\frac{1}{n}$ 可以被看作每个奖励的“权重”。这种权重可以用图 1-18 来表示。

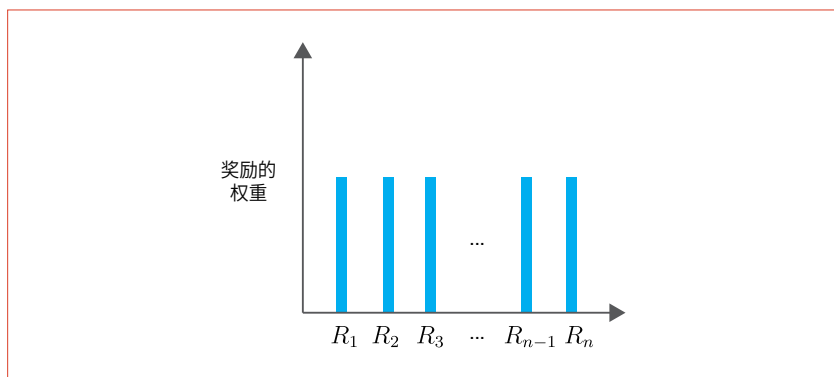


图 1-18 每个奖励的权重（使用样本均值的情况）

如图 1-18 所示，所有奖励的权重都相同。也就是说，所有的奖励（包括新获得的和很久以前获得的）都被平等对待。当然，这种权重对于非稳态问题是不合适的。这是因为在非稳态问题中，环境（老虎机）随时间变化，所以过去获得的数据（奖励）的重要性应该随着时间的推移而下降。反过来说，就是要给新获得的奖励以更大的权重。

1.4 节介绍过计算样本均值的高效实现方法。这里再重复一下，它可以通过以下更新的计算式求出。

$$Q_n = Q_{n-1} + \frac{1}{n}(R_n - Q_{n-1}) \quad (1.5)$$

如式 1.5 所示，行动价值的估计值 Q_n 能够增量式（依次）地更新。下面进入正题。将式 1.5 中的步长 $\frac{1}{n}$ 变为固定值 α ($0 < \alpha < 1$)。变更后的式子如下所示。

$$Q_n = Q_{n-1} + \alpha(R_n - Q_{n-1}) \quad (1.6)$$

如果基于式 1.6 更新行动价值的估计值，那么每个奖励的权重会是多少？我们先来看答案，如图 1-19 所示。

当按固定值 α 更新时，过去获得的奖励越久远，奖励的权重就越小，如图 1-19 所示。权重不仅仅是单纯地变小，而是指数级减少。举一个指数

级减少的例子： R_n 的权重乘以 0.9 就是 R_{n-1} 的权重， R_{n-1} 的权重乘以 0.9 就是 R_{n-2} 的权重，以此类推。图 1-19 所示的权重可用于非稳态问题。这是因为可以给新获得的奖励以更大的权重。

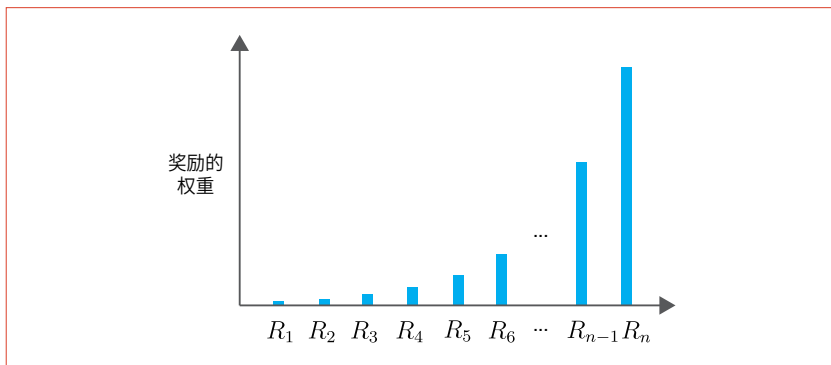


图 1-19 每个奖励的权重（按固定值 α 进行更新的情况）

为什么像式 1.6 中那样用固定值 α 进行更新的做法会产生像图 1-19 中那样的指数权重呢？这里用数学式来解释原因。将数学式展开如下。

$$\begin{aligned}
 Q_n &= Q_{n-1} + \alpha(R_n - Q_{n-1}) \\
 &= \alpha R_n + Q_{n-1} - \alpha Q_{n-1} \\
 &= \alpha R_n + (1 - \alpha)Q_{n-1}
 \end{aligned} \tag{1.7}$$

如变形后的式 1.7 所示， Q_n 和 Q_{n-1} 之间的关系已经很清楚了。如果现在将式 1.7 中的 n 再向下回溯 1 个值，即用 $n-1$ 代替 n ，那么我们将得到以下式子。

$$Q_{n-1} = \alpha R_{n-1} + (1 - \alpha)Q_{n-2} \tag{1.8}$$

式 1.8 显示了 Q_{n-1} 和 Q_{n-2} 之间的关系。现在将式 1.8 中的 Q_{n-1} 代入式 1.7。

$$\begin{aligned}
 Q_n &= \alpha R_n + (1 - \alpha)Q_{n-1} \\
 &= \alpha R_n + (1 - \alpha)\{\alpha R_{n-1} + (1 - \alpha)Q_{n-2}\} \\
 &= \alpha R_n + \alpha(1 - \alpha)R_{n-1} + (1 - \alpha)^2 Q_{n-2}
 \end{aligned} \tag{1.9}$$

观察式 1.9，可以看到 R_n 的权重是 α ， R_{n-1} 的权重是 $\alpha(1-\alpha)$ 。下面沿用前面的做法，展开式 1.9 中的 Q_{n-2} 。

$$Q_n = \alpha R_n + \alpha(1-\alpha)R_{n-1} + \alpha(1-\alpha)^2 R_{n-2} + (1-\alpha)^3 Q_{n-3}$$

对于后面的数，可以重复同样的做法予以展开。如果重复 n 次这样的操作，那么可以得到以下数学式。

$$Q_n = \alpha R_n + \alpha(1-\alpha)R_{n-1} + \cdots + \alpha(1-\alpha)^{n-1} R_1 + (1-\alpha)^n Q_0 \quad (1.10)$$

式 1.10 包含了对每个奖励的权重。它们是指数级减少的，具体如下所示。

- R_n 的权重是 α
- R_{n-1} 的权重是 $\alpha(1-\alpha)$
- R_{n-2} 的权重是 $\alpha(1-\alpha)^2$
- R_{n-3} 的权重是 $\alpha(1-\alpha)^3$
-

由于这个权重指数级减少的特点，式 1.10 中的计算也被称为**指数移动平均**（exponential moving average）或**指数加权移动平均**（exponential weighted moving average）。



式 1.10 中需要注意的一点是，为了求 Q_n 的值，要用到 Q_0 的值。 Q_0 是行动价值的初始值，也就是我们设定的值。因此， Q_n 的值受到我们设定的初始值的影响。换句话说，**我们设定的值会使学习结果出现偏置（bias）**。而在使用样本均值的情况下，不会出现这种偏置。准确地说，**在使用样本均值的情况下，在获得第一次奖励后，用户设定的初始值就会“消失”**。

1.5.2 解决非稳态问题

现在让我们尝试实际地解决非稳态的老虎机问题。我们所要做的就是**用一个固定值 α 来更新估计值**。我们在 AlphaAgent 类中实现了一个以固定值 α 更新估计值的智能代理，代码如下所示。

ch01/non_stationary.py

```
class AlphaAgent:
    def __init__(self, epsilon, alpha, actions=10):
        self.epsilon = epsilon
        self.Qs = np.zeros(actions)
        self.alpha = alpha

    def update(self, action, reward):
        # 用 alpha 更新
        self.Qs[action] += (reward - self.Qs[action]) * self.alpha

    def get_action(self):
        if np.random.rand() < self.epsilon:
            return np.random.randint(0, len(self.Qs))
        return np.argmax(self.Qs)
```

代码很简单，只需用步长 `self.alpha` 来更新估计值。现在我们使用这个 AlphaAgent 类来实现非稳态问题的 NonStatBandit 类。结果如图 1-20 所示。

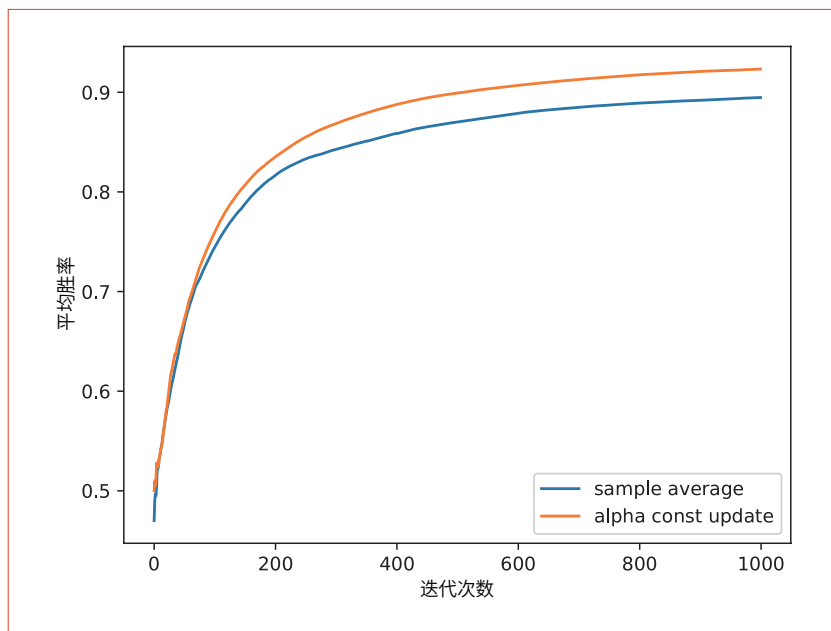


图 1-20 按样本均值以及按固定值 α 进行更新的比较

图 1-20 中的 `alpha const update` 是按固定值 α 进行更新的结果。`alpha` 参数被设定为 `alpha=0.8`。为了便于比较,按样本均值更新估计值的情况被绘制成了标签为 `sample average` 的图形。对比两者可以看出:随着时间的推移,按固定值 α 进行更新的结果会更好。对于使用样本均值的情况,虽然起初表现良好,但随着时间的推移,它与按固定值 α 进行更新的情况产生了差距。样本均值不能很好地应对时间的变化。由此可以看出,按固定值 α 进行更新的方法适合本次的非稳态问题。

1.6 小结

本章介绍了强化学习的基础知识。强化学习是机器学习的一个分支,但它与监督学习和无监督学习存在着明显的不同。区别在于,在强化学习中,环境和智能代理之间会进行交互。智能代理因其采取的行动而获得奖励,它的目标是掌握能够使奖励之和最大化的行动模式。

在本章中,我们解决了老虎机问题。解决老虎机问题的算法可用于从多个备选方案中选择最佳方案的问题。虽然本章以老虎机为例,但该算法也可用于解决其他各种问题。例如,在选择有助于提高销售额的网页设计或选择最有效的药物等问题中也可以应用老虎机算法。

在老虎机问题以及强化学习中,在“利用和探索”之间取得平衡很重要。在本章中,我们学习了 ϵ -greedy 算法作为实现这一目标的算法。 ϵ -greedy 算法虽然“利用”之前获得的经验,但有时也会通过尝试非贪婪行动来“探索”是否有其他更好的行动。这种方法高效地解决了老虎机问题。除了 ϵ -greedy 算法,人们还提出了解决老虎机问题的许多其他方法,典型的方法包括 UCB (Upper Confidence Bound) 算法^[2]和梯度老虎机算法^[3]。

在本章中,我们还学习了“平均”的相关知识。本章中出现了两个“平均”:一个是“样本均值”,它使用同样的权重;另一个是“指数移动平均”,它为新获得的数据赋予更大的权重。我们可以用样本均值或指数移动平均来计算行动价值的估计值。使用哪种方法取决于问题的性质。对于

稳态问题，可以使用样本均值；对于非稳态问题，可以使用指数移动平均。这两种平均也可以按以下方式进行增量式计算。

$$\text{样本均值: } Q_n = Q_{n-1} + \frac{1}{n}(R_n - Q_{n-1})$$

$$\text{指数移动平均: } Q_n = Q_{n-1} + \alpha(R_n - Q_{n-1})$$

如上面的数学式所示，样本均值按 $\frac{1}{n}$ 进行更新，指数移动平均按固定值 α 进行更新。

第 2 章

马尔可夫决策过程

在老虎机问题中，无论智能代理采取什么行动，之后要解决的问题都是一样的。智能代理每次都会玩老虎机，并寻找最好的老虎机。但现实生活中的问题是不同的。例如，我们来思考一下下围棋的情景。在围棋游戏中，智能代理落子后，棋盘上的棋子排列会发生变化（对手落子会进一步改变棋盘上的棋子排列）。智能代理采取的不同行动导致棋局每时每刻都在变化。智能代理需要考虑到棋局的转变，下出最佳的一手。



第 1 章还介绍了“非稳态问题”，即老虎机的奖励设置（奖励的概率分布）会随时间发生变化。在非稳态问题中，奖励的概率分布会随时间发生变化，与智能代理的行动无关。我们现在探讨的则是环境状态根据智能代理的行动而变化的问题。

本章探讨的是情况随智能代理的行动而变化的问题。一些这样的问题被定义为马尔可夫决策过程（MDP）。本章首先介绍在 MDP 中出现的术语，并用数学式表示它们。然后，在阐明 MDP 的目标的基础上，本章通过实际介绍一个简单的 MDP 问题来带你了解实现这些目标的过程。

2.1 什么是 MDP

MDP 是 Markov Decision Process 的缩写，意为“马尔可夫决策过程”（“马尔可夫性”的含义参见 2.2.1 节）。决策过程是智能代理通过与环境互动决定其行动的过程。本节会通过具体的例子来研究 MDP 的特点。

2.1.1 MDP 的具体例子

首先来看一下图 2-1。世界被网格分割，中间是一个机器人。机器人可以在网格上向右或向左移动。本书把图 2-1 这样的世界称为“网格世界”。

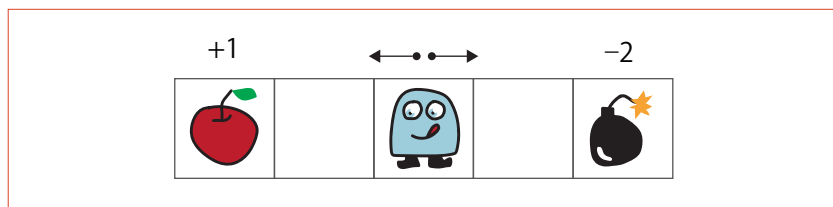


图 2-1 MDP 问题设置的实例：网格世界

如果用强化学习的术语来描述，那么机器人就是“智能代理”，它的周围是“环境”。智能代理可以采取两种行动：向右走或向左走。在图 2-1 中，网格的最左端有一个苹果，最右端有一个炸弹。这些是对机器人的“奖励”。在这个问题中，拿苹果的奖励是 +1，拿炸弹的奖励是 -2。什么都没有的网格（空白网格）的奖励被设置为 0。

在图 2-1 的问题中，智能代理每次行动时情况都会发生变化。例如，让我们思考智能代理向左移动，然后再向左移动的情况。在这种情况下，智能代理得到的奖励是 +1。如果智能代理向右移动，然后再向右移动，那么它得到的奖励就是 -2。智能代理的这些状态迁移如图 2-2 所示。

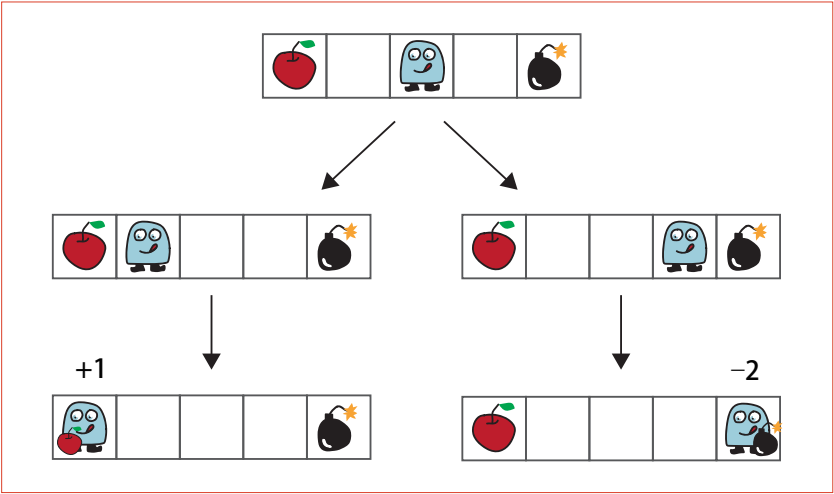


图 2-2 MDP 状态的迁移

如图 2-2 所示，智能代理所处的情况根据其行动而发生改变。在强化学习中，这种情况被称为“状态” (state)。在 MDP 中，状态的变化取决于智能代理的行动，智能代理在状态迁移后执行新的行动。顺带一提，从图 2-2 可以看出，对于这个问题，智能代理的最佳行动是向左移动两次。这样做会得到最好的奖励。



在 MDP 中，我们需要“时间”的概念。在某一时刻，智能代理会采取行动并因此迁移到一个新的状态。此时的时间单位叫作“时间步”。由于时间步是智能代理做出决定的间隔时间，因此它的实际单位取决于问题。

下面让我们思考一下图 2-3 中的问题。

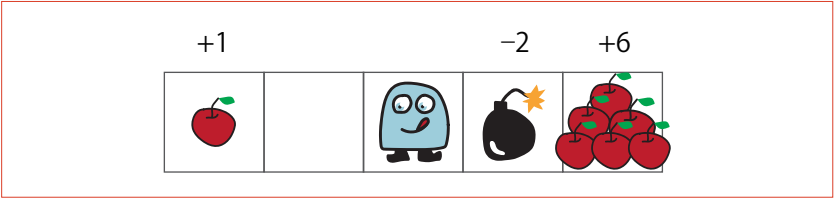


图 2-3 另一个世界（环境）

在图 2-3 中，网格的左端有一个奖励为 +1 的苹果，右边紧邻智能代理的是一个奖励为 -2 的炸弹。而在网格的右端有奖励为 +6 的苹果。在这个问题中，智能代理向右移动马上获得的奖励是负的，但如果再向右移动，就会得到奖励为 +6 的苹果。因此，这个问题的最佳行动应该是向右移动两次。

如图 2-3 中的例子所示，智能代理要考虑的是将来获得的奖励总和，而不是眼前的奖励。换句话说，智能代理的目标是实现奖励总和最大化。

2.1.2 智能代理与环境的互动

在 MDP 中，智能代理与环境之间会进行互动。要点在于当智能代理采取行动时，状态会发生迁移，随之获得的奖励也会相应改变。这样的关系如图 2-4 所示。

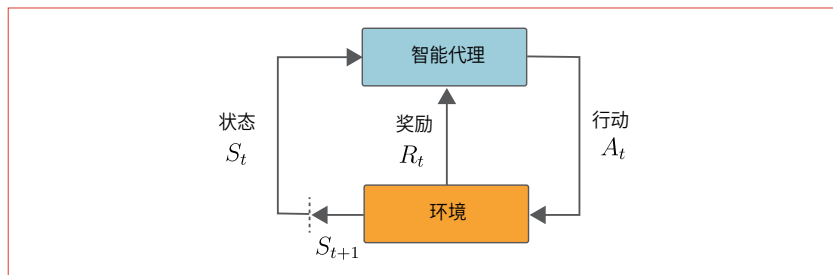


图 2-4 MDP 的循环

在图 2-4 中，假设在时刻 t 的状态是 S_t 。基于这个状态 S_t ，智能代理执行行动 A_t ，获得奖励 R_t 并迁移到下一个状态 S_{t+1} 。智能代理与环境之间的这种实际的互动产生了以下迁移。

$$S_0, A_0, R_0, S_1, A_1, R_1, S_2, A_2, R_2, \dots$$

这个时间序列数据从第一个状态 S_0 开始。在状态 S_0 ，智能代理执行行动 A_0 并获得奖励 R_0 ，时刻变为 1，状态变为 S_1 。接下来，基于状态 S_1 ，智能代理执行行动 A_1 并获得奖励 R_1 ，然后进入下一个状态 S_2 ……这个流程不断持续下去。



在强化学习领域，对于奖励 R_t 的时机，有两种习惯做法。二者的区别在于奖励是 R_t 还是 R_{t+1} 。更精确的描述如下所示。

- 在状态 S_t 中执行行动 A_t ，获得奖励 R_t 并迁移到下一个状态 S_{t+1} 。
- 在状态 S_t 中执行行动 A_t ，获得奖励 R_{t+1} 并迁移到下一个状态 S_{t+1} 。

如上所述，对奖励的处理方式有两种：一种是将奖励视为 R_t ，另一种是将奖励视为 R_{t+1} 。为了代码的易读性，本书采用前一种做法（ R_t 方式）。

以上就是对 MDP 的简单介绍。接下来我们将用数学式来描述 MDP。

2.2 环境和智能代理的数学表示

MDP 通过数学式来表示智能代理、环境以及二者之间的互动。要做到这一点，需要用数学式来表达以下 3 个要素。

- **状态迁移**：状态如何迁移。
- **奖励**：如何给予奖励。
- **策略**：智能代理如何决定行动。

如果这 3 个要素可以用数学式来表示，那么它们就可以作为 MDP 被定义。首先来看一下“状态迁移”。

2.2.1 状态迁移

先来看一下图 2-5 所示的状态迁移的例子。

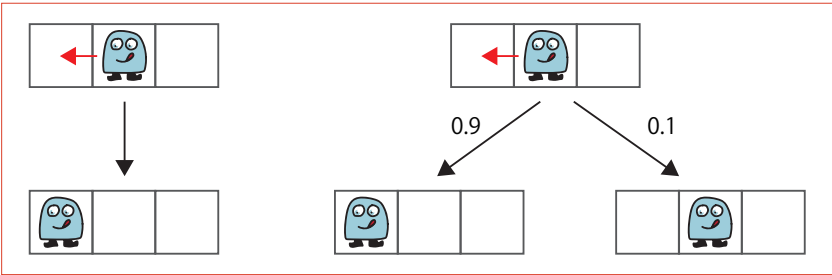


图 2-5 智能代理的行动和状态迁移（左图是确定性的，右图是随机性的）

图 2-5 中的左图描述了智能代理选择向左移动的行动，结果导致它“总是”向左移动的例子。这样的特性叫作**确定性** (deterministic)。在确定性状态迁移的情况下，**下一个状态 s' 是由当前状态 s 和行动 a 唯一决定的**。因此，它可以被表示为如下所示的函数。

$$s' = f(s, a)$$

$f(s, a)$ 是一个给定输入状态 s 和行动 a ，输出下一个状态 s' 的函数。这个 $f(s, a)$ 被称为**状态迁移函数** (state transition function)。

另外，图 2-5 中的右图描述了**随机性** (stochastic) 行动。如果智能代理选择了向左移动的行动，那么它将以 0.9 的概率向左移动，以 0.1 的概率留在原地。这种随机行动可能是地板打滑或智能代理的内部机制（如电机）不能正常工作所导致的。



即使状态迁移是确定性的，它们也可以用概率来描述。以图 2-5 中的左图为例，如果智能代理选择向左移动的行动，那么向左移动的概率可以描述为 1.0，迁移到任何其他状态的概率为 0。

下面介绍随机性状态迁移的表示方法。假设智能代理现在**处于状态 s 并执行了行动 a** ，那么迁移到下一个状态 s' 的概率可以用如下方式表示。

$$p(s' | s, a)$$

| 的右侧是表示“条件”的概率变量。对于当前问题，条件对应于在状态 s 选择了行动 a 。在给定这两个条件的情况下，迁移到 s' 的概率可以表示为 $p(s' | s, a)$ 。像 $p(s' | s, a)$ 这样的概率叫作**状态迁移概率** (state transition probability)。

图 2-6 展示了 $p(s' | s, a)$ 的一个具体例子。

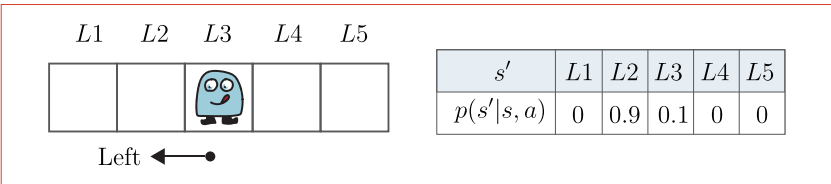


图 2-6 状态迁移概率的例子

如图 2-6 中的左图所示，从左到右的 5 个方块的编号依次为 $L1$ 、 $L2$ 、 $L3$ 、 $L4$ 、 $L5$ 。智能代理向左移动的行动表示为 Left，向右移动的行动表示为 Right。假定智能代理现在处于 $L3$ （状态 $L3$ ），并选择了 Left，那么在图 2-6 的右表中其数值就是这种情况下的状态迁移概率 $p(s' | s = L3, a = \text{Left})$ 。



准确地说，图 2-6 中的右表是状态迁移的“概率分布”。这是因为图中这些值是用概率表示的随机变量 s' 的“所有”可能值。换句话说，这些值是概率的“分布”。

$p(s' | s, a)$ 决定了下一个状态 s' 只取决于当前状态 s 和行动 a 。

换句话说，状态迁移不需要过去的信息——此前处于什么状态以及执行了哪些行动。这个特性被称为马尔可夫性（Markov property）。

MDP 通过假设马尔可夫性的存在来模拟状态迁移和奖励。引入马尔可夫性主要是为了使问题更容易解决。如果不假定马尔可夫性，那么就必须要考虑之前的所有状态和行动，而且组合的数量会呈指数级增长。

2.2.2 奖励函数

接下来要探讨的是奖励。为了简单起见，本书假设奖励的发放是“确定性”的。具体来说，当智能代理处于状态 s ，执行行动 a ，下一个状态是 s' 时，得到的奖励由函数 $r(s, a, s')$ 定义。 $r(s, a, s')$ 叫作奖励函数（reward function）。让我们来看一下图 2-7 所示的一个奖励函数的例子。

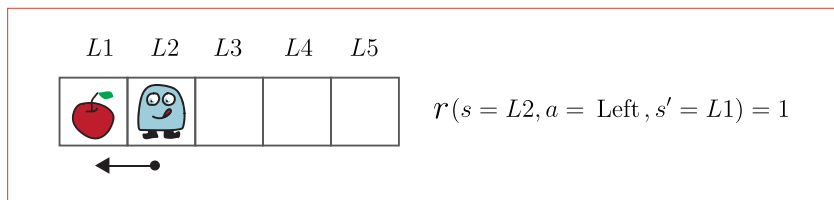


图 2-7 奖励函数的例子

图 2-7 描述了一个智能代理处于 $L2$ （当前状态为 $s=L2$ ），选择了 Left，并迁移到下一个状态 $L1$ 的例子。根据奖励函数 $r(s, a, s')$ ，可以得到这种情

况下的奖励是 1。

另外，在图 2-7 所示的例子中，只需知道下一个状态 s' 就能确定奖励。在当前问题中，奖励只由目的地的苹果决定。因此，奖励函数也可以实现为 $r(s')$ 。



奖励也可以按“随机性”发放。假设有这样一种情况：当智能代理去某个地方时，有 0.8 的概率会被敌人攻击，得到的奖励是 -10（有 0.2 的概率不会被敌人攻击）。这种以随机的形式获得奖励的情况也是可能的。不过，通过设置奖励函数 $r(s, a, s')$ 来返回“奖励的期望值”，我们将在后面章节中推导的关于 MDP 的理论（贝尔曼方程等）在奖励是随机性的情况下同样成立。本书探讨的是奖励按“确定性”发放的情况。

2.2.3 智能代理的策略

接下来探讨智能代理的策略（policy）。策略表示智能代理如何决定其行动。策略的关键在于它使得智能代理仅根据当前状态来决定其行动。所以说只基于当前状态就足够了，是因为环境的迁移是符合马尔可夫性的。

环境的状态迁移只以当前状态 s 和行动 a 为条件来决定下一个状态 s' ，而不需要先前的信息。同样，奖励也是基于当前状态 s 、行动 a 和迁移后的状态 s' 来决定的。这意味着关于环境的所有必要信息都在当前状态中。因此，智能代理只需基于当前状态即可决定其行动。



MDP 的马尔可夫性可以被看作是对环境而不是对智能代理的约束。这意味着为了满足马尔可夫性，环境需要保持某个“状态”。从智能代理的角度来看，在当前状态下有足够的信息来做出最佳选择，所以它可以在此基础上采取行动。

智能代理的策略可以根据其当前状态来决定。智能代理的行动可以是确定性的，也可以是随机性的。确定性的策略在某个状态下总是采取固定的行动，如图 2-8 中的左图所示。

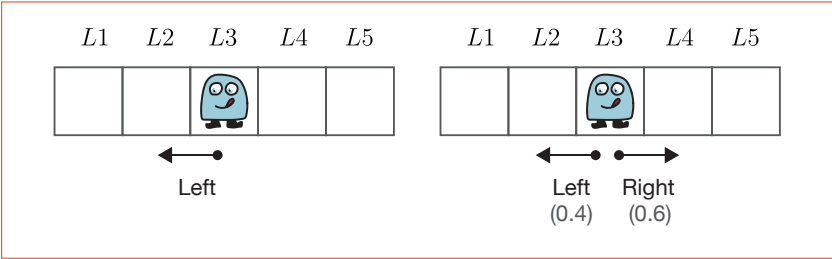


图 2-8 确定性策略和随机性策略

在图 2-8 的左图中，智能代理在某一位置时总是采取向左移动的行动。这样的确定性策略可以定义为如下所示的函数。

$$a = \mu(s)$$

$\mu(s)$ 是以给定的状态作为参数并返回行动 a 的函数。在这个例子中， $\mu(s = L3)$ 会返回 Left。

图 2-8 中的右图描述了一个随机性策略的例子。在这个例子中，智能代理向左移动的概率是 0.4，向右移动的概率是 0.6。这个智能代理的行动是由随机性策略决定的，数学式如下所示。

$$\pi(a | s)$$

$\pi(a | s)$ 表示在状态 s 下采取行动 a 的概率。在图 2-8 的右图的例子中，有以下数学式成立。

$$\begin{aligned}\pi(a = \text{Left} | s = L3) &= 0.4 \\ \pi(a = \text{Right} | s = L3) &= 0.6\end{aligned}$$



确定性策略也可以用随机性策略来表示。例如，确定性策略 $a = \mu(s)$ 可以用一个在状态 s 下采取行动 a 的概率是 1.0、采取任何其他行动的概率是 0 的概率分布来表示。

我们已经成功地用数学式来表示状态迁移、奖励函数和策略。接下来让我们使用这三者来定义 MDP 的目标。

2.3 MDP 的目标

到目前为止，我们已经用数学式描述了环境和智能代理的行动。简要回顾一下，智能代理根据策略 $\pi(a|s)$ 采取行动。首先，它根据它的行动和状态迁移概率 $p(s'|s,a)$ 迁移到下一个状态。然后，它根据奖励函数 $r(s,a,s')$ 获得奖励。在这个框架内，MDP 的目标是找到**最优策略**（optimal policy）。最优策略是使收益最大化的策略（2.3.2 节会介绍收益）。



如果智能代理采取的是确定性策略，那么策略可以用函数 $\mu(s)$ 来表示。不过，由于确定性策略可以用随机性策略来表示（**涵盖**），因此我们将基于随机性策略 $\pi(a|s)$ 进行探讨。出于同样的原因，环境的状态迁移也被假定为随机的。

为了进行最优策略的严密数学式表达，我们开始准备工作。首先要了解 MDP 问题大致可以分为两种，分别为“**回合制任务**”和“**连续性任务**”。

2.3.1 回合制任务和连续性任务

根据问题的不同，MDP 可分为回合制任务和连续性任务。回合制任务是有“**结束**”的问题。例如，围棋被归类为回合制任务。一局围棋的最终结果是“赢 / 输 / 平”三者之一。而且在下一盘棋中，比赛从没有放置任何棋子的状态开始（初始状态）。回合制任务中“回合”的意思是从开始到结束的一系列操作。

回合制任务的另一个例子是图 2-9 所示的网格世界问题。

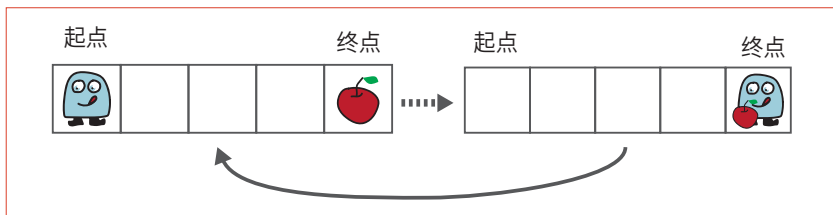


图 2-9 有终点的问题示例

在图 2-9 的例子中，在某个地点有终点，当达到该终点时回合“结束”。然后回合再从初始状态重新开始。

连续性任务则是没有“结束”的问题。例如，库存管理问题就可以被认为是连续性任务。在库存管理问题中，智能代理需要决定进多少货。它们需要根据销售情况和库存量来判断最佳的进货数量（在理想情况下，仓库中不应该有太多的库存，但也应该储存足够的货物以尽可能多地销售）。这样的问题没有设置“结束”，可以被定义为永远持续下去的问题。

2.3.2 收益

下面引入一个新术语——收益 (return)。智能代理的目标是使收益最大化。

为了定义收益，我们思考这样一个场景：设时刻为 t ，状态为 S_t （其中 t 是任意值）。然后，智能代理根据策略 π 执行行动 A_t ，获得奖励 R_t ，之后迁移到新状态 S_{t+1} ，这个过程不断重复进行。在这种情况下，收益 G_t 的定义如下所示。

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots \quad (2.1)$$

如式 2.1 所示，收益被表示为智能代理获得的奖励之和。不过，随着时间的推移，奖励会被 γ 指数级削弱。这个 γ 被称为折现率 (discount rate)，其被设定为 0.0 和 1.0 之间的实数。如果折现率是 0.9，那么有以下式子成立。

$$G_t = R_t + 0.9R_{t+1} + 0.81R_{t+2} + \dots$$

引入折现率主要是为了防止连续性任务的收益变得无穷大。在连续性任务中，如果没有折现率（或 $\gamma=1$ ），那么收益就会发散到无穷大。因此，设置折现率可以防止收益的发散。

折现率也使近期的奖励显得更加重要。这解释了人类乃至生物的许多行动原理。例如，你会选择今天拿到 10 000 元还是一年后拿到 20 000 元？如果折现率使未来的回报呈指数级下降，那么眼前的回报就会更有吸引力。

2.3.3 状态价值函数

我们已经重新定义了“收益”。智能代理的目标是使这种收益最大化。这里有一点需要注意，那就是智能代理和环境的行动可能是“随机性”的。智能代理可能随机地决定行动，状态也可能随机迁移。在这种情况下，获得的收益将呈现随机的特点。即使从相同的状态开始，不同回合的收益也随机变化。例如，某个回合的收益为 10.4，另一个回合的收益为 8.7。

为了处理这种随机行动，需要使用期望值或“收益的期望值”作为衡量标准。收益的期望值的数学式如下所示。

$$v_{\pi}(s) = \mathbb{E}[G_t | S_t = s, \pi] \quad (2.2)$$

在式 2.2 中，我们指定的条件是状态 S_t 为 s 、智能代理的策略为 π （其中时刻 t 是任意值）。在这些条件下，智能代理获得的收益的期望值为式 2.2。在这里，我们用特殊符号 $v_{\pi}(s)$ 来表示收益的期望值。 $v_{\pi}(s)$ 就是状态价值函数（state-value function）。

在式 2.2 的右侧，智能代理的策略 π 被作为条件给出。这是因为如果策略 π 发生变化，那么智能代理获得的奖励也会发生变化，而这些奖励的总和，即收益也会发生变化。为了明确这一点，状态价值函数通常会写作 $v_{\pi}(s)$ ，将 π 写在 v 的右下角。此外，式 2.2 也可以写成下面的形式。

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s] \quad (2.3)$$

在式 2.3 中，记载 π 的位置是 $\mathbb{E}_{\pi}$ 。与式 2.2 一样，这么写的含义是策略 π 是作为条件给出的。从现在起，本书将采取式 2.3 的风格来书写式子。



状态价值函数可以表示为 v_{π} 和 V_{π} ，其中 v_{π} 是真正的状态价值函数， V_{π} 表示作为估计的状态价值函数。

2.3.4 最优策略和最优价值函数

在强化学习中，我们的目标是获得最优策略。这里让我们思考什么是最优策略，以及如何表达“最优”。

首先考虑有两个策略的情况，其中一个叫作 π ，另一个叫作 π' 。在这种情况下，状态价值函数 $v_\pi(s)$ 和 $v_{\pi'}(s)$ 分别由这两个策略决定。假定所有状态下的价值函数都已确定，如图 2-10 所示。

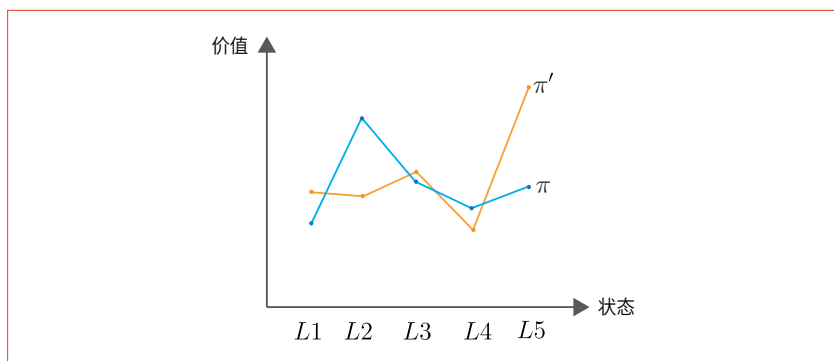


图 2-10 在策略 π 和策略 π' 的各状态下的状态价值函数的图形 (状态从 $L1$ 到 $L5$, 共 5 个)

图 2-10 中值得注意的是，在某个状态下 $v_{\pi'}(s) > v_\pi(s)$ ，另一个状态下 $v_{\pi'}(s) < v_\pi(s)$ 。例如，当状态为 $L1$ 时，如果遵循策略 π' 采取行动，那么收益的期望值就会更高。但当状态为 $L2$ 时，策略 π 则更好。对于状态价值函数的大小会因状态的不同而发生变化的情况，两个策略没有优劣之分。

那么，在什么情况下，可以区分出这两个策略的好坏呢？这就是图 2-11 所示的情况。

在图 2-11 的例子中，在所有的状态下， $v_{\pi'}(s) \geq v_\pi(s)$ 都成立。在这种情况下，可以说策略 π' 比策略 π 更好。这是因为无论从哪个状态开始，使用策略 π' 获得的奖励之和的期望值都比使用策略 π 获得的大（或相等）。

如果想区分出两个策略的好坏，那么它们必须满足在**所有**状态下 $v_{\pi'}(s) \geq v_\pi(s)$ 的条件。当满足这个条件时，就可以说 π' 是比 π 更好的策略（如果在所有状态下 $v_{\pi'}(s) = v_\pi(s)$ ，那么可以说它们是同样好的策略）。

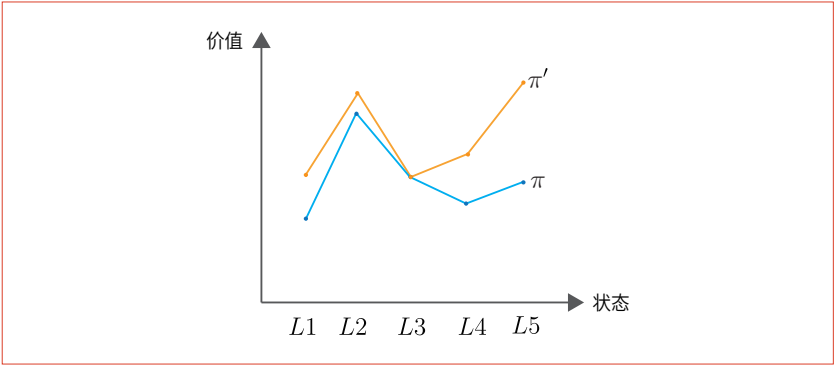


图 2-11 在所有状态下 $v_{\pi'}(s) \geq v_{\pi}(s)$ 的图形

现在可以区分两个策略的好坏了。基于这个思路，我们可以定义什么是**最优策略**。如果用 π_* 表示最优策略，那么策略 π_* 就是与其他策略相比，在所有状态下的状态价值函数 $v_{\pi_*}(s)$ 的值最大的策略，如图 2-12 所示。

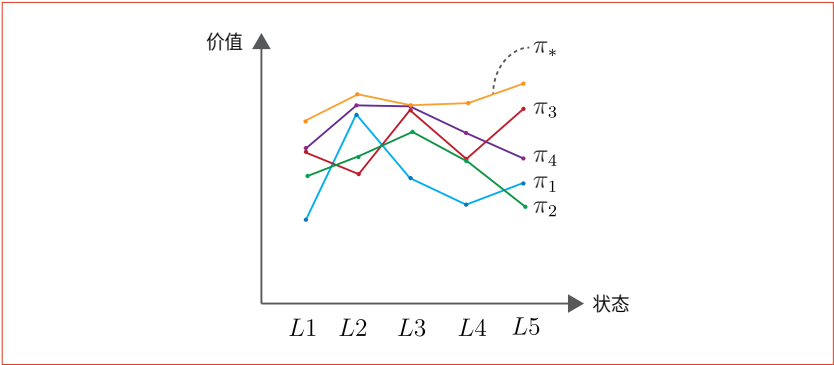


图 2-12 最优策略 π_* 和其他策略

如图 2-12 所示，当所有状态下的某个策略的状态价值函数的值都大于其他任何策略时，这个策略就是最优策略。这里我们需要了解一个重要事实：在 MDP 中，至少存在一个最优策略。此外，最优策略是确定性策略。在确定性策略中，每个状态下的行动是唯一确定的。用数学式可以表示为函数 μ_* ，有 $a = \mu_*(s)$ 成立，当输入状态 s 时，会输出行动 a 。



可以用数学公式来证明在 MDP 中至少有一个最优的确定性策略。本书省略了证明过程。如果你对此感兴趣，可以阅读一下参考文献 [4]。3.4.1 节用具体的例子解释了最优策略是确定性的原因。

最优策略的状态价值函数叫作**最优状态价值函数** (optimal state-value function)。本书使用 v_* 来表示最优状态价值函数。

2.4 MDP 的例子

下面我们来看一个 MDP 的具体问题。这次的问题是一个有两个方格的网格世界，如图 2-13 所示。

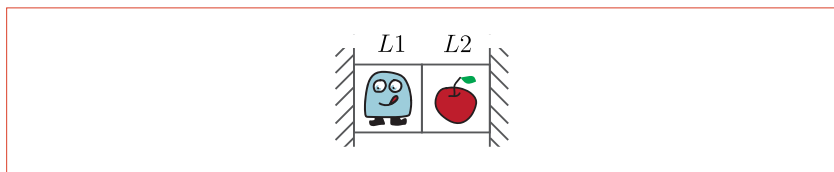


图 2-13 有两个方格的网格世界

图 2-13 中有两个方格，左右两边都有墙。这个问题的设定如下。

- 智能代理可以向右或向左移动。
- 状态迁移是确定性的（如果智能代理采取向右移动的行动，那么它的下一个状态将总是向右移动）。
- 当智能代理从 $L1$ 移动到 $L2$ 时，它将获得一个苹果，奖励为 +1。
- 当智能代理从 $L2$ 移动到 $L1$ 时，苹果会重新出现。
- 如果智能代理撞到了墙，那么它会获得 -1 的奖励（惩罚）：如果智能代理在 $L1$ 并向左移动，那么它会获得 -1 的奖励；如果智能代理在 $L2$ 并向右移动，那么它同样会获得 -1 的奖励（此时苹果不会出现）。
- 这次迭代问题被设定为没有“结束”的连续性任务。

现在让我们来解决图 2-13 中的问题。

2.4.1 回溯线形图

为了整理图 2-13 中的问题，我们先来画“回溯线形图”。回溯线形图是有向图（由节点和箭头组成的图），表示状态、行动和奖励之间的迁移。图 2-14 就是一个回溯线形图的例子。

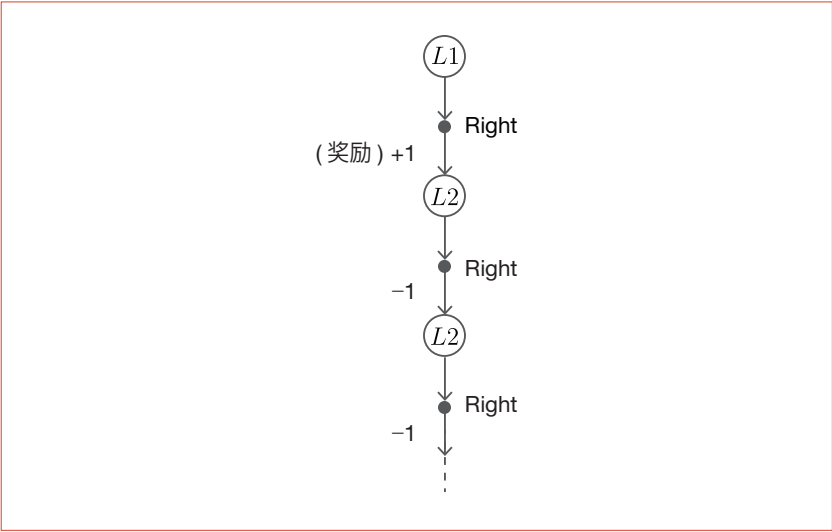


图 2-14 回溯线形图的例子

这里我们思考一下智能代理在任何状态下总是向右移动的情况。图 2-14 是这种情况下的智能代理的行动和状态迁移图。在图 2-14 中，时间从上往下推进。第一个状态是 $L1$ ，图中画出的是从这个状态开始的迁移。

图 2-14 中智能代理的策略是确定性的，也就是说，它总是采取固定的行动。此外，环境中的状态迁移也是确定性的，所以回溯线形图中的迁移以直线延伸。作为参考，我们来看一个智能代理以 50% 的概率向右移动、以 50% 的概率向左移动的回溯线形图的例子，如图 2-15 所示。

在图 2-15 中，由于在每个状态下都有向右和向左这两个可能的行动，因此回溯线形图会扩展和延伸。不过这次我们探讨的是环境中的状态迁移

以及智能代理的策略都是确定性的情况。换句话说，这是图 2-14 所示的那种回溯线形图以直线方式延伸的情况。

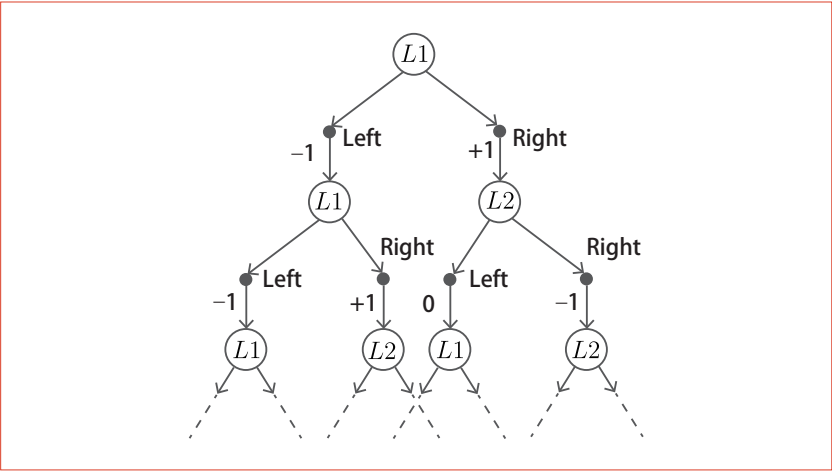


图 2-15 回溯线形图的例子（智能代理的行动是确定性的情况）

2.4.2 找出最优策略

那么，在这个有两个方格的网格世界里，最优策略是什么呢？我们已经知道最优策略是以确定性策略的形式存在的。确定性策略是像 $a = \mu(s)$ 这样的函数。在当前的问题中，状态和行动的数量很少，因此可以列出所有候选的确定性策略。具体来说，由于状态有 2 个，行动有 2 个，因此总共有 $2^2 = 4$ 个确定性策略。这 4 个策略如图 2-16 所示。

	$s = L1$	$s = L2$
$\mu_1(s)$	Right	Right
$\mu_2(s)$	Right	Left
$\mu_3(s)$	Left	Right
$\mu_4(s)$	Left	Left

图 2-16 确定性策略的模式

图 2-16 中列出了在每个状态下要采取的行动（由于是确定性策略，因此在每个状态下采取的行动都是 1 个）。例如，策略 μ_1 是在状态 $L1$ 时向右移动，在状态 $L2$ 时也向右移动。在图 2-16 的 4 个策略中，有一个是最优策略。

下面计算策略 μ_1 的状态价值函数。虽然第 3 章才会更详细地介绍计算方法，但由于在当前问题中，环境和策略的状态迁移都是确定性的，因此可以简单地计算出来。如果在状态 $L1$ 采取策略 μ_1 向右移动，那么智能代理将会获得 +1 的奖励。此后，每当智能代理向右移动并撞到墙，都会获得 -1 的奖励。假设折现率为 0.9，则状态价值函数可以像下面这样计算。

$$\begin{aligned} v_{\mu_1}(s=L1) &= 1 + 0.9 \times (-1) + 0.9^2 \times (-1) + \dots \\ &= 1 - 0.9(1 + 0.9 + 0.9^2 + \dots) \\ &= 1 - \frac{0.9}{1 - 0.9} \\ &= -8 \end{aligned}$$

这里我们使用了无限等比数列的公式，即 $1 + r + r^2 + \dots = \frac{1}{1-r}$ （当 $-1 < r < 1$ 时）。另外，在代码中，可以像下面这样近似地计算。

```
V = 1
for i in range(1, 100):
    V += -1 * (0.9 ** i)
print(V)
```

输出结果

```
-7.999734386011124
```

这段代码没有进行无穷无尽的计算，而是使用 for 语句进行了近似计算。结果大约是 -8，与理论值几乎相同。

接下来研究状态 $L2$ 下的价值函数（使用策略 μ_1 时）。这次智能代理一直撞到右边的墙，所以总是得到 -1 的奖励。此时的状态价值函数可以像下面这样计算。

$$\begin{aligned} v_{\mu_1}(s=L2) &= -1 + 0.9 \times (-1) + 0.9^2 \times (-1) + \dots \\ &= -1 - 0.9(1 + 0.9 + 0.9^2 + \dots) \\ &= -1 - \frac{0.9}{1 - 0.9} \\ &= -10 \end{aligned}$$

这样我们就求出了策略 μ_1 的价值函数。对所有其他策略都执行上述操作，这就得到了图 2-17 所示的结果。

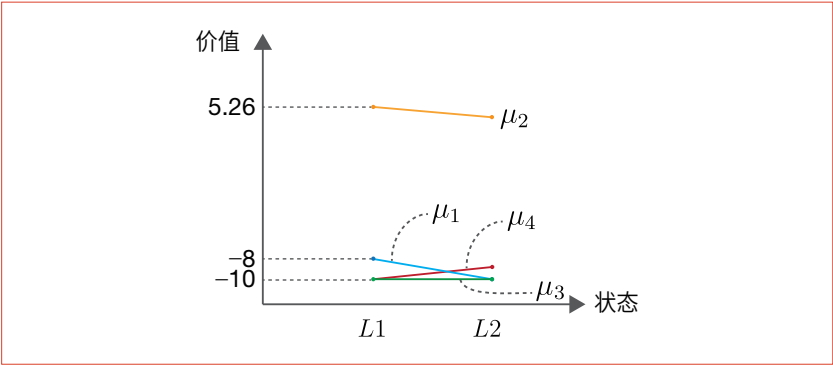


图 2-17 各策略的状态价值函数的图形

从图 2-17 中可以看出，在所有状态下，策略 μ_2 的状态价值函数的值都比其他策略高。因此，策略 μ_2 就是我们期望的策略。策略 μ_2 重复了左右移动的行动，没有撞到墙。在这个过程中，它重复地获得了苹果。由于已经得到了最优策略，因此我们实现了 MDP 的目标。

2.5 小结

在本章中，我们学习了 MDP，它是智能代理与环境之间互动的一种形式。环境有状态迁移概率（或状态迁移函数）和奖励函数，智能代理有策略。环境和智能代理之间会互动。MDP 的目标是在这样一个框架内找到最优策略。最优策略是指所有状态下其状态价值函数的值大于或等于任何其他策略。

在本章中，我们解决了“有两个方格的网格世界”的强化学习问题，并实际地找到了最优策略。具体来说，我们列出了所有可能的策略，并手动计算了每个策略的状态价值函数。最后我们找到了其中最好的策略——最优策略。不过本章中使用的方法只对有两个方格的网格世界这种设定简单的强化学习问题有效。因此，对于接下来的第3章，你需要做好处理更难问题的准备。

第 3 章

贝尔曼方程

在第 2 章中，我们探讨了有两个方格的网格世界问题。在那个问题中，我们假设环境是确定性的，而且智能代理的行动也是确定性的。因此，回溯线形图会延伸为一条直线，如图 3-1 中的左图所示。

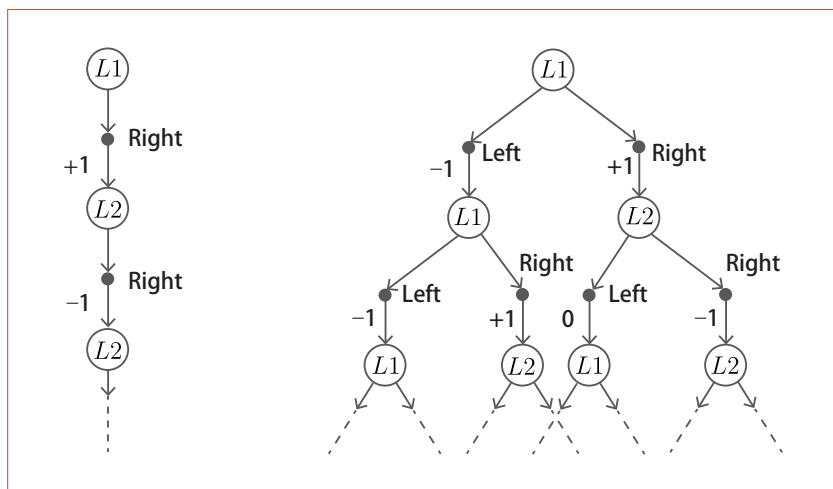


图 3-1 确定性的回溯线形图（左）和随机性的回溯线形图（右）

如果回溯线形图是图 3-1 中左图那样的在一条直线上延伸的图形，那么状态价值函数就可以用第 2 章中的方法求出（通过数学式计算）。但可能还有这样的情况存在：MDP 的行动是随机性的。这种情况下的回溯线形图

将以分支的形式延伸，如图 3-1 中的右图所示。很遗憾，这样的状态价值函数不能用第 2 章中的手动计算方式求出。

本章的目标是求出类似图 3-1 中右图所示情况下的状态价值函数。因此贝尔曼方程（Bellman equation）是关键。贝尔曼方程是在 MDP 中成立的最重要的方程，为许多强化学习算法提供了重要基础。

3.1 贝尔曼方程的推导

本节将推导贝尔曼方程。首先是准备工作，我们将通过一个简单的例子来回顾概率和期望值，之后再行贝尔曼方程的推导。如果你对概率和期望值已经很熟悉，那么可以跳过 3.1.1 节，直接学习 3.1.2 节。

3.1.1 概率和期望值（推导贝尔曼方程的准备）

这里使用骰子作为例子。我们使用的骰子是理想的六面体，每一面的数字出现的概率都是 $\frac{1}{6}$ 。在数学式中，我们用随机变量 x 表示掷骰子出现的数字， x 是 1 和 6 之间的整数。那么每个数字出现的概率就是 $\frac{1}{6}$ 。我们用 $p(x) = \frac{1}{6}$ 表示骰子的数字出现的概率。现在来计算掷骰子的期望值。计算式如下所示。

$$\begin{aligned}\mathbb{E}[x] &= 1 \times \frac{1}{6} + 2 \times \frac{1}{6} + 3 \times \frac{1}{6} + 4 \times \frac{1}{6} + 5 \times \frac{1}{6} + 6 \times \frac{1}{6} \\ &= 3.5\end{aligned}$$

如上所述，期望值是在所有情况下“出现的数字”和“概率”相乘并加在一起得到的和。顺带一提，如果使用 Σ 符号，那么期望值的数学式如下所示。

$$\mathbb{E}[x] = \sum_x xp(x)$$

此时的骰子试验也可以画作回溯线形图，如图 3-2 所示。

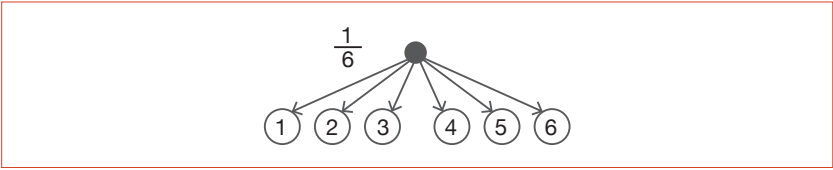


图 3-2 骰子的回溯线形图

如图 3-2 所示，骰子的数字（有可能出现的数字）呈现向下延伸的形状。

现在思考图 3-3 所示的问题。

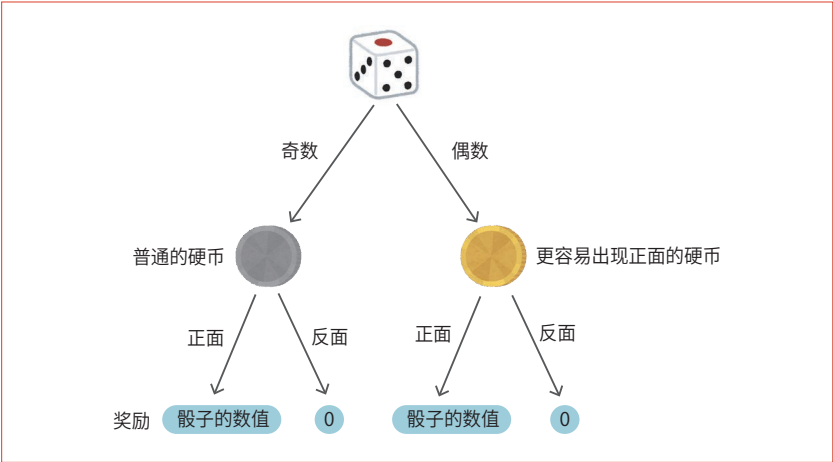


图 3-3 依次抛掷骰子和硬币的问题

这次是先掷骰子，然后再抛硬币的试验。如果先掷骰子得到的是偶数，那么玩家就会得到一枚更容易出现正面的硬币，正面出现的概率为 0.8；如果得到的是奇数，那么玩家就会得到一枚普通的硬币，正面出现的概率为 0.5。然后抛出得到的硬币，如果出现的是正面，那么玩家就会得到掷骰子的数值作为奖励。如果出现的是反面，则奖励为 0。

- 如果骰子掷出 4，而硬币（容易出现正面的硬币）抛出的是正面，那么奖励就是 4。
- 如果骰子掷出 5，而硬币（普通的硬币）抛出的是反面，则奖励为 0。

现在让我们试着求出这个问题的“奖励的期望值”。首先画回溯线形图，如图 3-4 所示。

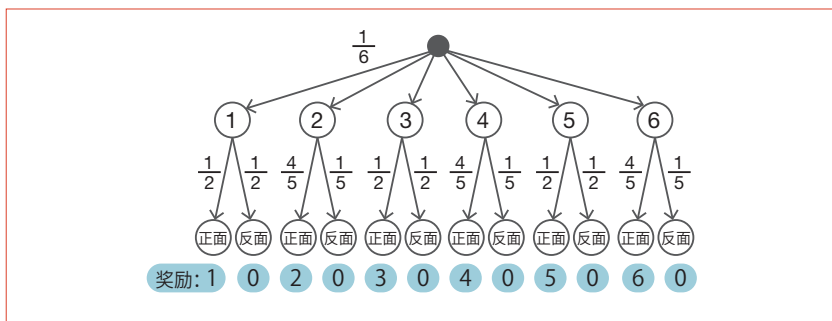


图 3-4 依次抛掷骰子和硬币问题的回溯线形图

从图 3-4 中可以看到，骰子掷出 1 的概率是 $\frac{1}{6}$ ，而接下来硬币出现正面的概率是 $\frac{1}{2}$ 。那么此时的奖励就是 1。换句话说，有 $\frac{1}{6} \times \frac{1}{2} = \frac{1}{12}$ 的概率获得奖励值 1。为了求出奖励的期望值，在所有情况下都要进行这样的计算。实际的计算如下所示。

$$\begin{aligned} & \frac{1}{6} \times \frac{1}{2} \times 1 + \frac{1}{6} \times \frac{1}{2} \times 0 + \frac{1}{6} \times \frac{4}{5} \times 2 + \frac{1}{6} \times \frac{1}{5} \times 0 + \frac{1}{6} \times \frac{1}{2} \times 3 + \frac{1}{6} \times \frac{1}{2} \times 0 + \\ & \frac{1}{6} \times \frac{4}{5} \times 4 + \frac{1}{6} \times \frac{1}{5} \times 0 + \frac{1}{6} \times \frac{1}{2} \times 5 + \frac{1}{6} \times \frac{1}{2} \times 0 + \frac{1}{6} \times \frac{4}{5} \times 6 + \frac{1}{6} \times \frac{1}{5} \times 0 \\ & = 2.35 \end{aligned}$$

现在我们求出了奖励的期望值。过程如下：将图 3-4 的末端节点（黑色圆点）发生的概率和此时的奖励相乘，然后对所有节点进行同样的计算，再将结果相加，最终得到的就是奖励的期望值。

下面把这里的计算结果以带文字的数学式表示出来。设 x 是掷骰子的结果， y 是抛硬币的结果（正面或反面）。在本次试验中，硬币正面出现的概率是可变的，这取决于掷骰子的情况。这可以用条件概率 $p(y|x)$ 表示。具体来说，它的取值如下所示。

$$p(y = \text{正面} | x = 4) = 0.8$$

$$p(y = \text{反面} | x = 3) = 0.5$$

另外， x 和 y 同时发生的概率（这叫作“联合概率”）如下所示。

$$p(x, y) = p(x)p(y|x)$$

至于奖励，在这个问题上， x 和 y 的值决定了能获得的奖励。因此，奖励可以用函数 $r(x, y)$ 来表示，如下面的式子所示。

$$r(x=4, y=\text{正面})=4$$

$$r(x=3, y=\text{反面})=0$$

期望值是“值” $\times$ “该值出现的概率”之和。因此，奖励的期望值可以表示为下面的形式。

$$\begin{aligned}\mathbb{E}[r(x, y)] &= \sum_x \sum_y p(x, y) r(x, y) \\ &= \sum_x \sum_y p(x) p(y|x) r(x, y)\end{aligned}$$

这个形式的数学式也会出现在接下来要介绍的贝尔曼方程的推导中。这样我们就完成了对贝尔曼方程的推导的准备工作。如果你已经理解了上面的内容，那么就能理解贝尔曼方程的推导过程。

3.1.2 贝尔曼方程的推导

下面开始推导贝尔曼方程。首先回顾一下对“收益（回报）”的定义，如下所示。

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots \quad (3.1)$$

我们来思考一下连续型的任务，并假设有无限多的奖励。收益 G_t 是在时刻 t 之后获得的奖励的总和。不过由于折现率 γ 的存在，将来获得的奖励呈指数级下降。在研究式 3.1 的结构时，首先将 $t+1$ 代入式 3.1 中的 t 中。

$$G_{t+1} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \quad (3.2)$$

式 3.2 是在时刻 $t+1$ 之后获得的奖励的总和。考虑到这一点，可以将式 3.1 变为如下形式。

$$\begin{aligned} G_t &= R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots \\ &= R_t + \gamma(R_{t+1} + \gamma R_{t+2} + \cdots) \\ &= R_t + \gamma G_{t+1} \end{aligned} \quad (3.3)$$

如式 3.3 所示，我们已推导出收益 G_t 和 G_{t+1} 之间的关系。这种关系被用于强化学习的许多理论和算法中。

下面将式 3.3 代入状态价值函数的定义式中。状态价值函数是收益的期望值，其数学式的定义如下所示。

$$v_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s] \quad (3.4)$$

如式 3.4 所示，状态 s 的价值函数被表示为 $v_\pi(s)$ 。将式 3.3 代入这个式子的 G_t 中，得到下面的式子。

$$\begin{aligned} v_\pi(s) &= \mathbb{E}_\pi[G_t | S_t = s] \\ &= \mathbb{E}_\pi[R_t + \gamma G_{t+1} | S_t = s] \\ &= \mathbb{E}_\pi[R_t | S_t = s] + \gamma \mathbb{E}_\pi[G_{t+1} | S_t = s] \end{aligned} \quad (3.5)$$

由于期望值是线性的，因此式子中最后一步的展开才能成立。具体来说，如果随机变量是 X 和 Y ，那么 $\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$ 就成立。



这里的探讨是在假设智能代理的策略为随机性策略 $\pi(a|s)$ 的前提下进行的。这是因为确定性策略也可以用随机性策略来表达（涵盖）。同样，我们假设环境的状态迁移也是随机的，数学式为 $p(s'|s, a)$ 。

下面我们逐一推导式 3.5 中的项，首先从 $\mathbb{E}_\pi[R_t | S_t = s]$ 开始。为了计算这个项，先来看一下图 3-5。

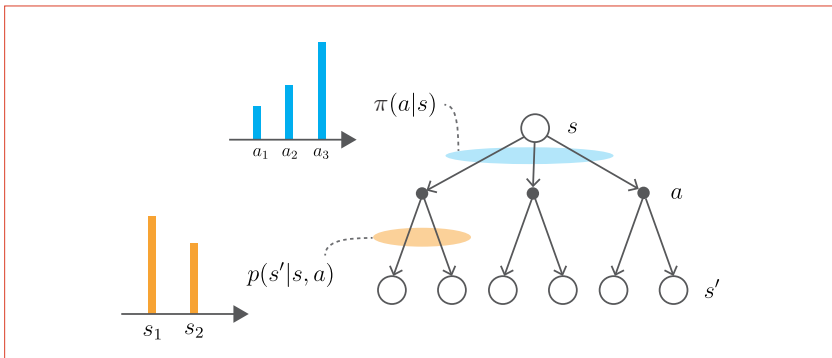


图 3-5 状态和行动的关联性

下面梳理一下目前所处的情况：智能代理现在处于状态 s ，根据策略 $\pi(a|s)$ 采取行动。假设智能代理要采取 3 个行动，那么采取其中每个行动的概率值如下所示。

$$\pi(a = a_1 | s) = 0.2$$

$$\pi(a = a_2 | s) = 0.3$$

$$\pi(a = a_3 | s) = 0.5$$

假设智能代理会基于这个概率分布选择行动，然后会基于状态迁移概率 $p(s'|s, a)$ 迁移到新的状态 s' 。假设智能代理采取了行动 a_1 ，并且有两个候选迁移状态，那么这个情况下的值如下所示。

$$p(s' = s_1 | s, a = a_1) = 0.6$$

$$p(s' = s_2 | s, a = a_2) = 0.4$$

最后，奖励由函数 $r(s, a, s')$ 决定。这就是智能代理所处的情况。

我们来计算一个具体的例子。假设智能代理以 0.2 的概率选择行动 a_1 ，以 0.6 的概率迁移到 s_1 。下面用数学式来描述这个情况。

- 以 $\pi(a = a_1 | s)p(s' = s_1 | s, a = a_1) = 0.2 \times 0.6 = 0.12$ 的概率
- 获得奖励 $r(s, a = a_1, s' = s_1)$

为了求出期望值，对所有候选项都进行这种计算，然后求和。这可以用以下数学式来表示。

$$\mathbb{E}_{\pi}[R_t | S_t = s] = \sum_a \sum_{s'} \pi(a | s) p(s' | s, a) r(s, a, s')$$

如上面的数学式所示，将智能代理行动的概率 $\pi(a | s)$ 、要迁移的状态的概率 $p(s' | s, a)$ 和奖励函数 $r(s, a, s')$ 相乘。对所有候选项都进行上述计算，得到它们的总和。这与 3.1.1 节的“骰子和硬币”的例子中的数学式结构相同。



从现在开始，本书将把 $\sum_a \sum_{s'} \dots$ 符号统一写为 $\sum_{a, s'} \dots$ 的形式。

这样式 3.5 的第一项就推导完毕了，如图 3-6 所示。

$$v_{\pi}(s) = \mathbb{E}_{\pi}[R_t | S_t = s] + \gamma \mathbb{E}_{\pi}[G_{t+1} | S_t = s]$$

$$\sum_{a, s'} \pi(a | s) p(s' | s, a) r(s, a, s')$$

接下来是这个

图 3-6 正在展开的式子

剩下的项是 $\gamma \mathbb{E}_{\pi}[G_{t+1} | S_t = s]$ 。由于 γ 是常数，因此我们要看的是 $\mathbb{E}_{\pi}[G_{t+1} | S_t = s]$ 。这个式子虽然与状态价值函数的定义式相似，但在 G_{t+1} 的部分有所不同。状态价值函数的式子如下所示，式子中是 G_t ，而不是 G_{t+1} 。

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s] \quad (3.4)$$

因此，我们首先要将 $t+1$ 代入式 3.4 的 t 中。式子变化如下。

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_{t+1} | S_{t+1} = s]$$

这就是状态 $S_{t+1} = s$ 时的价值函数。接下来要关注的是 $\mathbb{E}_{\pi}[G_{t+1} | S_t = s]$ 。这是当前时刻为 t 时，下一个时刻 ($t+1$) 的收益期望值。解决的关键在于将条件 $S_t = s$ 变为 $S_{t+1} = s$ 的形式。换句话说，就是要进入下一个时刻。

和以前一样，我们结合具体的例子来说明。智能代理现在处于条件 $S_t = s_0$ 。假设智能代理以 0.2 的概率选择行动 a_1 ，并以 0.6 的概率迁移到状态 s_1 。下面

用数学式来描述这个情况。

- 以 $\pi(a = a_1 | s)p(s' = s_1 | s, a = a_1) = 0.2 \times 0.6 = 0.12$ 的概率
- 迁移到 $\mathbb{E}_\pi[G_{t+1} | S_{t+1} = s_1] = v_\pi(s_1)$

这样通过观察下一个时刻，就可以让下一个状态的价值函数出场了。为了求期望值 $\mathbb{E}_\pi[G_{t+1} | S_t = s]$ ，对所有候选项都进行上述计算，然后求和。计算的数学式如下所示。

$$\begin{aligned}\mathbb{E}_\pi[G_{t+1} | S_t = s] &= \sum_{a, s'} \pi(a | s) p(s' | s, a) \mathbb{E}_\pi[G_{t+1} | S_{t+1} = s'] \\ &= \sum_{a, s'} \pi(a | s) p(s' | s, a) v_\pi(s')\end{aligned}$$

这样就完成了第二项的展开。汇总推导出的式子如下所示。

$$v_\pi(s) = \mathbb{E}_\pi[R_t | S_t = s] + \gamma \mathbb{E}_\pi[G_{t+1} | S_t = s] \quad (3.5)$$

$$\begin{aligned}&= \sum_{a, s'} \pi(a | s) p(s' | s, a) r(s, a, s') + \gamma \sum_{a, s'} \pi(a | s) p(s' | s, a) v_\pi(s') \\ &= \sum_{a, s'} \pi(a | s) p(s' | s, a) \{r(s, a, s') + \gamma v_\pi(s')\} \quad (3.6)\end{aligned}$$

式 3.6 就是贝尔曼方程。贝尔曼方程是表示状态 s 的价值函数和下一个可能的状态 s' 的价值函数之间关系的式子。这个贝尔曼方程对所有状态 s 和所有策略 π 都成立。

3.2 贝尔曼方程的例子

在 3.1 节中，我们推导出了贝尔曼方程。这个贝尔曼方程为解决强化学习问题提供了重要的基础。在实践中，贝尔曼方程可以用来求状态价值函数。为了展示贝尔曼方程的“威力”，我们将使用贝尔曼方程来解决实际的问题。

3.2.1 有两个方格的网格世界

这里我们要解决的问题是图 3-7 中的有两个方格的网格世界。这次假

设智能代理是**随机移动**的。也就是说，它有 50% 的机会向右移动，50% 的机会向左移动。

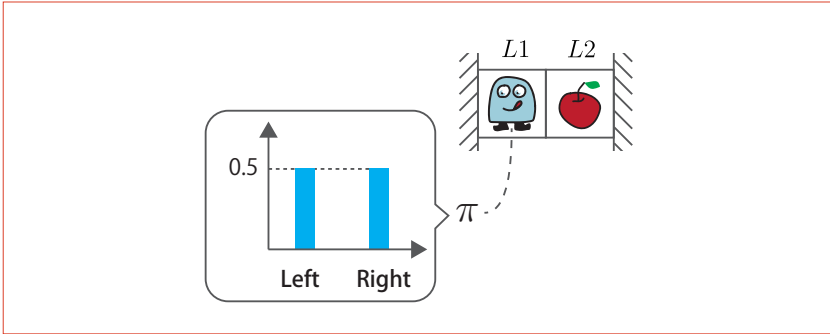


图 3-7 有两个方格的网格世界（撞墙奖励 -1，拿苹果奖励 +1。苹果多次出现）

$v_{\pi}(L_1)$ 是在状态 L_1 中按照随机性策略 π 行动时获得的收益的期望值。它是将来无限期地持续下去的奖励的总和。回溯线形图如图 3-8 所示。

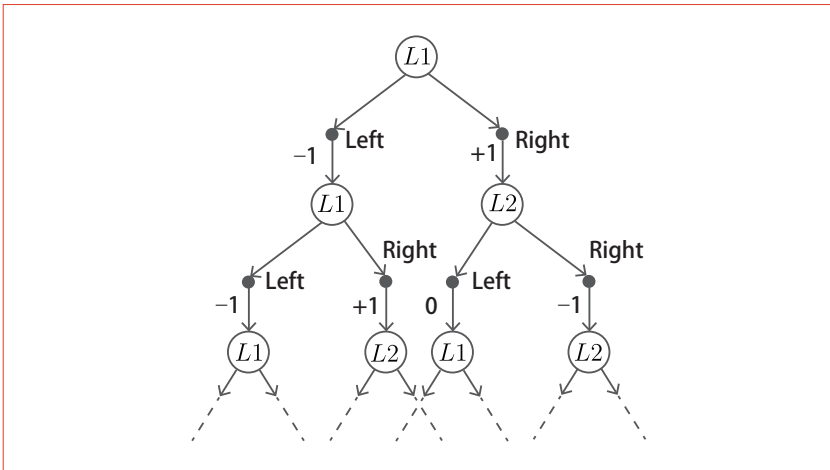


图 3-8 有分支的延伸下去的回溯线形图（在这个问题中，状态的迁移是确定性的）

从图 3-8 中可以看出，这是一个分支无限延伸的计算。对于这种**无限**分支的计算，可以使用**贝尔曼方程**。下面使用贝尔曼方程表示 $v_{\pi}(L_1)$ 。基于 3.1.2 节中的式 3.6，可以将贝尔曼方程表示如下。

$$v_{\pi}(s) = \sum_{a,s'} \pi(a|s) p(s'|s,a) \{r(s,a,s') + \gamma v_{\pi}(s')\} \quad (3.6)$$

$$= \sum_a \pi(a|s) \sum_{s'} p(s'|s,a) \{r(s,a,s') + \gamma v_{\pi}(s')\} \quad (3.7)$$

这里将 $\sum_{a,s'}$ 中的 $\sum_a$ 与 $\sum_{s'}$ 分开写，以便将来式子的展开。另外，这次的问题中的状态迁移是确定性的。也就是说，状态的迁移是由函数 $f(s,a)$ 而不是概率 $p(s'|s,a)$ 决定的。套用式 3.7，有以下结论成立。

- 如果 $s' = f(s,a)$ ，则 $p(s'|s,a) = 1$ 。
- 如果 $s' \neq f(s,a)$ ，则 $p(s'|s,a) = 0$ 。

这意味着在式 3.7 中只剩下与满足 $s' = f(s,a)$ 的 s' 相对应的项。因此，式 3.7 可以简化表示为以下形式。

当 $s' = f(s,a)$ 时

$$v_{\pi}(s) = \sum_a \pi(a|s) \{r(s,a,s') + \gamma v_{\pi}(s')\} \quad (3.8)$$

下面将本问题应用于式 3.8。我们将结合图 3-9 所示的回溯线形图进行探讨。

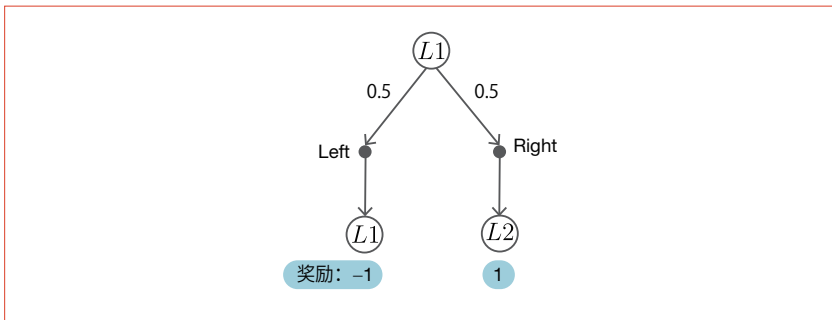


图 3-9 求 $v_{\pi}(L1)$ 的回溯线形图

观察图 3-9，回溯线形图有两个分支，其中一个分支是以 0.5 的概率选择行动 Left，并且总是迁移到状态 L1。在这种情况下得到的奖励是 -1。折现率 γ 被设定为 0.9。因此，式 3.8 中选择 Left 的项可以表示为下面的式子。

$$0.5\{-1 + 0.9v_{\pi}(L1)\}$$

图 3-9 中的另一个分支是以 0.5 的概率选择行动 Right，之后会迁移到状态 $L2$ ，奖励为 1。用数学式可以表示为如下形式。

$$0.5\{1 + 0.9v_{\pi}(L2)\}$$

汇总上述式子，可以将贝尔曼方程表示为如下形式。

$$v_{\pi}(L1) = 0.5\{-1 + 0.9v_{\pi}(L1)\} + 0.5\{1 + 0.9v_{\pi}(L2)\}$$

这个式子是状态 $L1$ 的贝尔曼方程。这个式子整理后如下所示。

$$-0.55v_{\pi}(L1) + 0.45v_{\pi}(L2) = 0 \quad (3.9)$$

下面求状态 $L2$ 下的贝尔曼方程。像刚才一样，我们将结合图 3-10 所示的回溯线形图进行探讨。

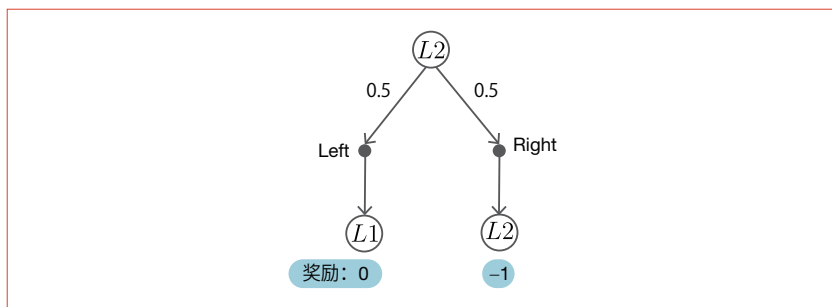


图 3-10 求 $v_{\pi}(L2)$ 的回溯线形图

$$v_{\pi}(L2) = 0.5\{0 + 0.9v_{\pi}(L1)\} + 0.5\{-1 + 0.9v_{\pi}(L2)\} \quad (3.10)$$

$$0.45v_{\pi}(L1) - 0.55v_{\pi}(L2) = 0.5$$

这样就求出了所有状态下的贝尔曼方程。这次我们想知道的两个变量是 $v_{\pi}(L1)$ 和 $v_{\pi}(L2)$ 。而且我们有以下两个方程（式 3.9 和式 3.10）。

$$\begin{cases} -0.55v_{\pi}(L1) + 0.45v_{\pi}(L2) = 0 \\ 0.45v_{\pi}(L1) - 0.55v_{\pi}(L2) = 0.5 \end{cases}$$

二者是一组联立方程，对于这次的问题，可以简单地计算出答案，如下所示。

$$\begin{cases} v_{\pi}(L1) = -2.25 \\ v_{\pi}(L2) = -2.75 \end{cases}$$

这就是随机性策略的状态价值函数。这意味着，当采取随机性策略时，在处于状态 $L1$ 的情况下，将来有望获得 -2.25 的收益。随机移动的确会导致撞墙，在未来会获得负的奖励，这是合理的。 $v_{\pi}(L1)$ 的值比 $v_{\pi}(L2)$ 大也是可以理解的，因为 $L1$ 的旁边就是苹果，第一次行动就有 50% 的概率拿到这个苹果。

3.2.2 贝尔曼方程的意义

正如刚才我们所看到的，贝尔曼方程能够**将无限的计算转换为有限的联立方程**。尽管这次的问题中存在随机性行动，我们还是使用贝尔曼方程求出了状态价值函数。



状态价值函数是收益的期望值，其被定义为数量无限多的奖励的总和。但从式 3.6 可以看出，贝尔曼方程的项并不是“无限”的。通过贝尔曼方程，我们得以从无限中解脱出来。

此外，尽管目前探讨的问题是个小问题，但即使是大问题，也可以使用贝尔曼方程写出联立方程。然后再使用能够解联立方程的算法，就可以自动求状态价值函数。

3.3 行动价值函数与贝尔曼方程

本节将介绍一个新的函数——**行动价值函数** (action-value function)。与状态价值函数一样，行动价值函数也是出现在许多强化学习理论中的重要函数。我们之前已经用状态价值函数推导出了贝尔曼方程。本节会先给出行动价值函数的定义式，然后再推导行动价值函数的贝尔曼方程。



为了简单起见，本书中有时会将状态价值函数简称为“价值函数”。另外，行动价值函数也被习惯性地称为“**Q 函数**”（Q-function）。因此，在本书中，行动价值函数有时会被称为“Q 函数”。

3.3.1 行动价值函数

我们先来回顾一下状态价值函数的定义。

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

状态价值函数有两个条件，分别是状态 s 和策略 π 。在这两个条件的基础上，可以考虑增加一个**额外**的条件：行动 a 。这就是行动价值函数（Q 函数）。它的数学式如下所示。

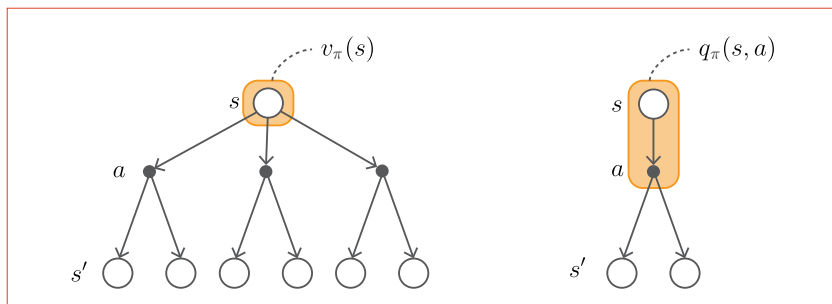
$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a]$$

Q 函数在时刻 t 的状态 s 下采取行动 a ，并从时刻 $t+1$ 开始根据策略 π 采取行动。此时得到的收益的期望值是 $q_{\pi}(s, a)$ 。



需要注意的是， $q_{\pi}(s, a)$ 的行动 a 与策略 π 无关。 $q_{\pi}(s, a)$ 的行动 a 可以自由确定，在采取该行动后，按照策略 π 行动。

Q 函数是在**状态价值函数的基础上将行动 a 增加为条件的函数**。比较二者的回溯线形图，就能更清晰地看到这一点，如图 3-11 所示。



如图 3-11 所示，在状态价值函数的回溯线形图中，行动 a 是根据策略 π 选择的。而在 Q 函数的回溯线形图中，行动 a 可以自由选择。这是状态价值函数和 Q 函数之间的唯一区别。因此，如果想办法使得在 Q 函数中根据策略 π 来选择行动 a ，那么 Q 函数和状态价值函数就一样了。假设在状态 s 中有 $\{a_1, a_2, a_3\}$ 3 个候选行动，那么此时将根据策略 π 选择行动。具体情况如下。

- 选择行动 a_1 的概率为 $\pi(a_1|s)$ （这种情况下的 Q 函数为 $q_\pi(s, a_1)$ ）。
- 选择行动 a_2 的概率为 $\pi(a_2|s)$ （这种情况下的 Q 函数为 $q_\pi(s, a_2)$ ）。
- 选择行动 a_3 的概率为 $\pi(a_3|s)$ （这种情况下的 Q 函数为 $q_\pi(s, a_3)$ ）。

在这种情况下，收益的期望值可以作为 Q 函数的加权和来计算。数学式如下所示。

$$\begin{aligned} & \pi(a_1|s)q_\pi(s, a_1) + \pi(a_2|s)q_\pi(s, a_2) + \pi(a_3|s)q_\pi(s, a_3) \\ &= \sum_{a \in \{a_1, a_2, a_3\}} \pi(a|s)q_\pi(s, a) \end{aligned}$$

上面的数学式是与状态价值函数相同条件下的收益的期望值。因此，有以下式子成立。

$$v_\pi(s) = \sum_a \pi(a|s)q_\pi(s, a) \quad (3.11)$$

3.3.2 使用行动价值函数的贝尔曼方程

接下来推导行动价值函数（Q 函数）的贝尔曼方程。首先将 Q 函数展开为如下形式。

$$\begin{aligned} q_\pi(s, a) &= \mathbb{E}_\pi[G_t | S_t = s, A_t = a] \\ &= \mathbb{E}_\pi[R_t + \gamma G_{t+1} | S_t = s, A_t = a] \end{aligned} \quad (3.12)$$

在这里，状态 s 和行动 a 是固定的。在此情况下，以 $p(s'|s, a)$ 的概率迁移到下一个状态 s' 。然后奖励由函数 $r(s, a, s')$ 给出。基于这些信息，将式 3.12 进一步展开为如下形式。

$$\begin{aligned}
q_{\pi}(s,a) &= \mathbb{E}_{\pi}[R_t + \gamma G_{t+1} \mid S_t = s, A_t = a] \\
&= \mathbb{E}_{\pi}[R_t \mid S_t = s, A_t = a] + \gamma \mathbb{E}_{\pi}[G_{t+1} \mid S_t = s, A_t = a] \\
&= \sum_{s'} p(s' \mid s, a) r(s, a, s') + \gamma \sum_{s'} p(s' \mid s, a) \mathbb{E}_{\pi}[G_{t+1} \mid S_{t+1} = s'] \\
&= \sum_{s'} p(s' \mid s, a) \{r(s, a, s') + \gamma \mathbb{E}_{\pi}[G_{t+1} \mid S_{t+1} = s']\} \\
&= \sum_{s'} p(s' \mid s, a) \{r(s, a, s') + \gamma v_{\pi}(s')\}
\end{aligned} \tag{3.13}$$

这次我们一次性地完成了式子的变形，这和在前3.1节中所做的工作相同（如果对上述式子变形有不理解的地方，请再次阅读3.1节）。进一步根据式3.11，用Q函数来表示状态价值函数 $v_{\pi}(s')$ ，将式3.13展开为如下形式。

$$q_{\pi}(s,a) = \sum_{s'} p(s' \mid s, a) \left\{ r(s, a, s') + \gamma \sum_{a'} \pi(a' \mid s') q_{\pi}(s', a') \right\} \tag{3.14}$$

在式3.14中， a' 表示在时刻 $t+1$ 采取的行动。式3.14就是使用Q函数的贝尔曼方程。

3.4 贝尔曼最优方程

贝尔曼方程是对策略 π 成立的方程。但我们最终想要求出的是最优策略。最优策略是使所有状态下的状态价值函数最大化的策略。最优策略当然也满足贝尔曼方程。此外，利用策略是“最优”的特性，我们可以用一种更简单的形式来表示贝尔曼方程。本节将介绍对最优策略成立的方程，即贝尔曼最优方程（Bellman optimality equation）。

3.4.1 状态价值函数的贝尔曼最优方程

首先从贝尔曼方程开始。下面来回顾一下贝尔曼方程的数学式。

$$\begin{aligned}
v_{\pi}(s) &= \sum_{a, s'} \pi(a \mid s) p(s' \mid s, a) \{r(s, a, s') + \gamma v_{\pi}(s')\} \\
&= \sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s, a) \{r(s, a, s') + \gamma v_{\pi}(s')\}
\end{aligned} \tag{3.7}$$

这里将 $\sum_{a,s'}$ 分开写为 $\sum_a$ 和 $\sum_{s'}$ ，以便将来式子的展开。贝尔曼方程对任何策略都成立。因此，如果最优策略是 $\pi_*(a|s)$ ，那么有以下贝尔曼方程成立。

$$v_*(s) = \sum_a \pi_*(a|s) \sum_{s'} p(s'|s, a) \{r(s, a, s') + \gamma v_*(s')\} \quad (3.15)$$

上面的数学式中的 $v_*(s)$ 表示最优策略的价值函数。下面要思考的问题是：根据最优策略 $\pi_*(a|s)$ 选择行动 a 。最优策略 $\pi_*(a|s)$ 会选择什么样的行动呢？让我们思考一下图 3-12 所示的例子。

$$v_*(s) = \sum_a \pi_*(a|s) \sum_{s'} p(s'|s, a) \{r(s, a, s') + \gamma v_*(s')\}$$

= \begin{cases} -2 & (a_1) \\ 0 & (a_2) \\ 4 & (a_3) \end{cases}

图 3-12 应该选择 3 个行动中的哪一个

图 3-12 中有 $\{a_1, a_2, a_3\}$ 3 个候选行动，假定 $\sum_{s'} p(s'|s, a) \{r(s, a, s') + \gamma v_*(s')\}$ 的值分别为 -2、0 和 4。在这种情况下，我们应基于什么样的概率分布来选择行动呢？

当然，由于要找的是最优策略，因此应该以 100% 的概率选择数值最大的行动 a_3 。换句话说，这是一个确定性策略。因此可以用确定性策略 $\mu_*(s)$ 来表示随机性策略 $\pi_*(a|s)$ 。而如果每次选择的都是行动 a_3 ，那么 $v_*(s)$ 的值就是 4。

从这个例子可以看出，最优策略是选择使 $\sum_{s'} p(s'|s, a) \{r(s, a, s') + \gamma v_*(s')\}$ 的**值最大**的行动，而这个最大值就是 **$v_*(s)$** 。可以像下面这样用数学式表示。

$$v_*(s) = \max_a \sum_{s'} p(s'|s, a) \{r(s, a, s') + \gamma v_*(s')\} \quad (3.16)$$

式 3.16 是用 $\max$ 算子来表示的，这就是贝尔曼最优方程。



$\max$ 算子是在多个元素中只选择一个具有最大值的元素的运算。假设有一个由 4 个值组成的组合（在数学上称为“集合”） $x = \{1, 2, 3, 4\}$ 。在这种情况下，为了得到 x 在这 4 个值中的函数 $g(x) = x_2$ 的最大值，可以写出下面的式子。

$$\max_x g(x) = 16$$

函数 $g(x)$ 在 $x=4$ 时取最大值 $g(x) = x_2 = 16$ 。这个计算可以用 $\max$ 算子来表示。

3.4.2 Q 函数的贝尔曼最优方程

同样，对于行动价值函数（Q 函数），我们也可以求出贝尔曼最优方程。下面使用同样的做法来表示 Q 函数的贝尔曼最优方程。



基于最优策略的行动价值函数叫作最优行动价值函数（optimal action-value function）。本书使用 q_* 表示最优行动价值函数。

下面是 Q 函数的贝尔曼方程。

$$q_\pi(s, a) = \sum_{s'} p(s' | s, a) \left\{ r(s, a, s') + \gamma \sum_{a'} \pi(a' | s') q_\pi(s', a') \right\}$$

这个贝尔曼方程对所有的策略 π 都成立。当然，它对最优策略 π_* 也是成立的，所以代入最优策略 π_* 。

$$q_*(s, a) = \sum_{s'} p(s' | s, a) \left\{ r(s, a, s') + \gamma \sum_{a'} \pi_*(a' | s') q_*(s', a') \right\} \quad (3.17)$$

下面的展开与 3.4.1 节的做法相同。这里 π_* 是最优策略，所以可以用 $\max$ 算子来简化。具体来说就是将 $\sum_{a'} \pi_*(a' | s')$ 部分替换为 $\max_{a'} \dots$ 。因此有以下式子成立。

$$q_*(s, a) = \sum_{s'} p(s' | s, a) \left\{ r(s, a, s') + \gamma \max_{a'} q_*(s', a') \right\} \quad (3.18)$$

上面的式 3.18 就是 Q 函数的贝尔曼最优方程。



在 MDP 中，至少存在一个确定性的最优策略。确定性策略是指在给定状态下总是选择特定行动的策略。因此，最优策略可以表示为像 $\mu_*(s)$ 这样的函数。根据问题的不同，可能存在一个以上的最优策略，但它们的价值函数的值相同。因此，最优策略的价值函数可以用符号 $v_*(s)$ 来表示。同样，最优策略的 Q 函数也只存在一个，即 $q_*(s, a)$ 。

3.5 贝尔曼最优方程的示例

本节将再次处理有两个方格的网格世界的问题，如图 3-13 所示。智能代理从 $L1$ 移动到 $L2$ 的奖励是 +1，撞到墙的奖励是 -1，并且苹果会多次出现。

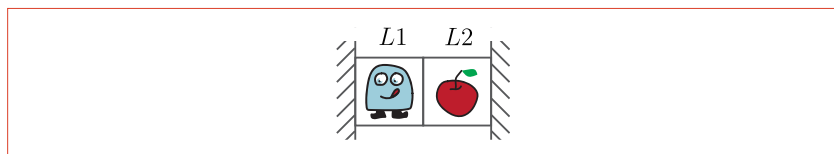


图 3-13 有两个方格的网格世界

3.5.1 应用贝尔曼最优方程

这里的目标是将贝尔曼最优方程应用于有两个方格的网格世界。之前推导出的贝尔曼最优方程的式子如下所示。

$$v_*(s) = \max_a \sum_{s'} p(s' | s, a) \{r(s, a, s') + \gamma v_*(s')\} \quad (3.16)$$

如果状态迁移是确定性而不是随机性的，那么上面的式子就可以进一步做如下简化。

当 $s' = f(s, a)$ 时

$$v_*(s) = \max_a \{r(s, a, s') + \gamma v_*(s')\} \quad (3.19)$$

上面的式子可以简化的原因如下。

- 如果 $s' = f(s, a)$ ，则 $p(s' | s, a) = 1$ 。

- 如果 $s' \neq f(s,a)$, 则 $p(s'|s,a) = 0$ 。

因此式 3.16 中只剩下了满足 $s' = f(s,a)$ 的项。

现在将式 3.19 中的贝尔曼最优方程应用于有两个方格的网格世界。作为参考, 绘制以状态 $L1$ 和状态 $L2$ 为起点的回溯线形图 (前一个时间步的回溯线形图), 如图 3-14 所示。

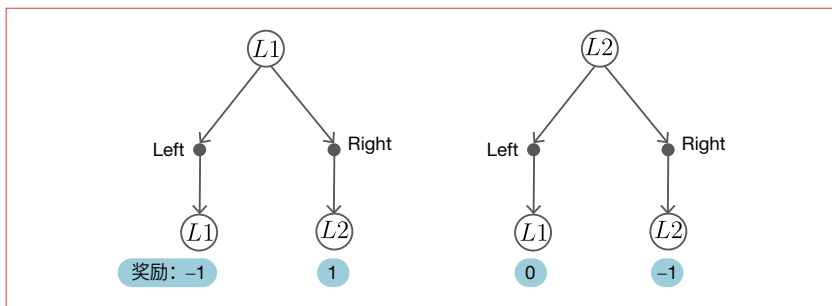


图 3-14 以状态 $L1$ 和状态 $L2$ 为起点的回溯线形图

结合图 3-14, 可以得到折现率为 0.9 时的贝尔曼最优方程, 如下所示。

$$v_*(L1) = \max \begin{Bmatrix} -1 + 0.9v_*(L1), \\ 1 + 0.9v_*(L2) \end{Bmatrix}$$

$$v_*(L2) = \max \begin{Bmatrix} 0.9v_*(L1), \\ -1 + 0.9v_*(L2) \end{Bmatrix}$$

上面式子中的 $\max\{\dots\}$ 表示 $\max$ 算子。例如, $\max\{a,b\}$ 表示对 a 和 b 进行比较, 并返回二者中的最大值。如上述式子所示, 我们有了两个联立方程。这次的变量是两个, 即 $v_*(L1)$ 和 $v_*(L2)$, 方程也是两个。因此, 我们可以从这个方程组中求出 $v_*(L1)$ 和 $v_*(L2)$ 。这个联立方程的解如下所示。

$$v_*(L1) = 5.26$$

$$v_*(L2) = 4.73$$



上面的联立方程中进行了 $\max$ (取最大值) 运算, $\max$ 是一个非线性运算。因此, 对求解联立方程的算法而言, 不能使用“线性求解器”来求解, 但可以使用“非线性求解器”。另外, 对于与这次探讨的小问题类似的问题, 也可以通过手动计算求出。

对于有两个方格的网格世界这样的小问题，可以利用贝尔曼最优方程直接求解。但我们最终想知道的是最优策略。接下来思考一下最优策略。

3.5.2 得到最优策略

这里我们假设最优行动价值函数 $q_*(s,a)$ 是已知的。在这种情况下，状态 s 下的最优行动可以通过以下式子求出。

$$\mu_*(s) = \operatorname{argmax}_a q_*(s,a) \quad (3.20)$$

argmax 返回的不是最大值，而是最大值的参数（在本例中是行动 a ）。像上面的等式一样，如果知道了最优的行动价值函数，那么只需选择使其成为最大值的行动即可。这个行动的选择就是最优策略。

在 3.3.2 节中，有以下式子成立。

$$q_\pi(s,a) = \sum_{s'} p(s'|s,a) \{r(s,a,s') + \gamma v_\pi(s')\} \quad (3.13)$$

在该式子中， q_π 和 v_π 中表示策略的下标 π 可以被替换为表示最优策略的下标*。然后，代入式 3.20，得到以下式子。

$$\mu_*(s) = \operatorname{argmax}_a \sum_{s'} p(s'|s,a) \{r(s,a,s') + \gamma v_*(s')\} \quad (3.21)$$

式 3.21 表明我们可以用最优状态价值函数 $v_*(s)$ 来得到最优策略 $\mu_*(s)$ 。



式 3.20 和式 3.21 也可以称为“贪婪的策略”。贪婪的策略指的是在局部的候选者中寻求最优行动。拿现在的例子来说，贝尔曼最优方程只与当前状态 (s) 和下一个状态 (s') 相关，只需考虑下一个状态就能选择价值最大的行动。

下面使用式 3.21 来求解有两个方格的网格世界问题的最优策略。我们已经求出了有两个方格的网格世界的最优状态价值函数 $v_*(L1)$ 和 $v_*(L2)$ 。结合图 3-15，求出在状态 $L1$ 下的最优行动。

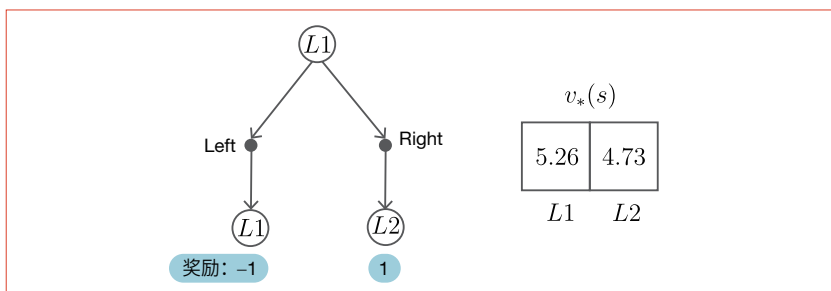


图 3-15 回溯线形图和最优状态价值函数

如图 3-15 所示，可以采取的行动有 Left 和 Right 这两个。此时如果采取行动 Left，那么智能代理就会进入状态 $L1$ ，并获得奖励 -1 。此时式 3.21 中的 $\sum_{s'} p(s'|s, a) \{r(s, a, s') + \gamma v_*(s')\}$ 的值如下所示（0.9 是折现率）。

$$-1 + 0.9v_*(L1) = -1 + 0.9 \times 5.26 = 3.734$$

另外，如果采取了行动 Right，那么智能代理就会进入状态 $L2$ ，并获得奖励 1。此时的值如下所示。

$$1 + 0.9v_*(L2) = 1 + 0.9 \times 4.73 = 5.257$$

如果在这两个值之间选择带来最大值的行动，那就是行动 Right。这意味着在状态 $L1$ 下的最佳行动是 Right。

通过同样的计算可知在状态 $L2$ 下的最佳行动是 Left。这样就求出了最优策略。如图 3-16 所示，最优策略是在状态 $L1$ 时采取向右移动的行動，在状态 $L2$ 时采取向左移动的行動的策略。

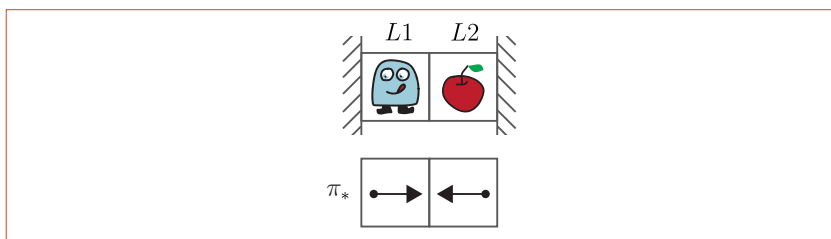


图 3-16 有两个方格的网格世界的最优策略

如上所述，一旦知道了最优状态价值函数，就可以得到最优策略。

3.6 小结

在本章中，我们学习了贝尔曼方程。我们推导了贝尔曼方程，并使用贝尔曼方程求解了一个小问题。具体来说，我们通过贝尔曼方程得到了联立方程，进而求出了价值函数。遗憾的是，这种使用联立方程的方法不能用于实际问题，因为它们的计算量太大。不过贝尔曼方程为许多强化学习算法提供了一个重要的基础。

强化学习的最终目标是得到最优策略。我们在本章中还学习了贝尔曼最优方程。贝尔曼最优方程是对最优策略成立的特殊的贝尔曼方程。只要知道最优策略的价值函数，就很容易得到最优策略。最后，将本章所学的重要数学式列举如下。

【贝尔曼方程】

$$\begin{aligned} v_{\pi}(s) &= \sum_{a, s'} \pi(a | s) p(s' | s, a) \{r(s, a, s') + \gamma v_{\pi}(s')\} \\ q_{\pi}(s, a) &= \sum_{s'} p(s' | s, a) \left\{ r(s, a, s') + \gamma \sum_{a'} \pi(a' | s') q_{\pi}(s', a') \right\} \end{aligned}$$

【贝尔曼最优方程】

$$\begin{aligned} v_*(s) &= \max_a \sum_{s'} p(s' | s, a) \{r(s, a, s') + \gamma v_*(s')\} \\ q_*(s, a) &= \sum_{s'} p(s' | s, a) \left\{ r(s, a, s') + \gamma \max_{a'} q_*(s', a') \right\} \end{aligned}$$

【最优策略】

$$\begin{aligned} \mu_*(s) &= \operatorname{argmax}_a q_*(s, a) \\ &= \operatorname{argmax}_a \sum_{s'} p(s' | s, a) \{r(s, a, s') + \gamma v_*(s')\} \end{aligned}$$

第 4 章

动态规划法

在第 3 章中，我们学习了贝尔曼方程的知识。使用贝尔曼方程，我们可以得到联立方程。如果能解开这个联立方程，那么就能求出价值函数。这个流程如图 4-1 所示。

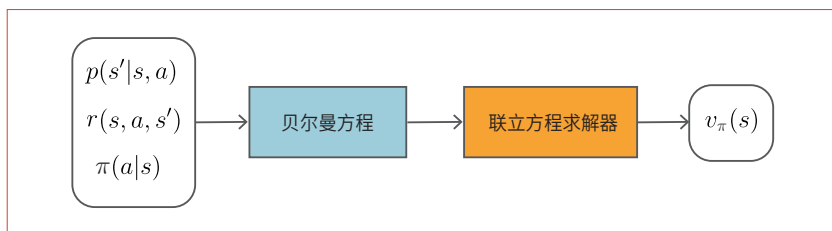


图 4-1 使用贝尔曼方程求价值函数的流程

如图 4-1 所示，使用贝尔曼方程，从状态迁移概率 $p(s'|s, a)$ 、奖励函数 $r(s, a, s')$ 和策略 $\pi(a|s)$ 这 3 条信息中可以得到联立方程，然后使用求解联立方程的程序（联立方程求解器）可以求出价值函数。不过这种显式地列出联立方程并直接求解的方法只对小问题有效。在实践中，当状态和行动模式的数量稍有增加，这种方法就会变得难以使用。对于这样的情况，应运而生了一种方法——动态规划法（dynamic programming, DP）。即使状态和行动的数量增加到一定程度，通过动态规划法，也可以对价值函数进行评估。本章将介绍动态规划法。

4.1 动态规划法和策略评估

强化学习的问题通常涉及两项任务：**策略评估** (policy evaluation) 和 **策略控制** (policy control)。策略评估是求给定策略 π 的价值函数 $v_\pi(s)$ 和 $q_\pi(s, a)$ 。而策略控制是控制策略并将其调整为最优策略。

当然，强化学习的最终目标是策略控制。但要实现这一目标，首先要解决的常常是策略评估。这是因为在大部分情况下，我们很难直接获得最优策略。在此我们使用一种叫作**动态规划法**（以下简称 **DP**）的算法进行策略评估。

4.1.1 动态规划法简介

首先回顾一下已学过的内容。我们对价值函数的定义如下所示。

$$v_\pi(s) = \mathbb{E}_\pi[R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots | S_t = s]$$

这个包含无限多的项的期望值通常是无法计算的，但贝尔曼方程可以解决这个无限多的项的计算问题，如下所示。

$$v_\pi(s) = \sum_{a, s'} \pi(a | s) p(s' | s, a) \{r(s, a, s') + \gamma v_\pi(s')\} \quad (4.1)$$

如式 4.1 所示，贝尔曼方程表示的是“当前状态 s 的价值函数 $v_\pi(s)$ ”与“下一状态 s' 的价值函数 $v_\pi(s')$ ”之间的关系。贝尔曼方程为许多强化学习算法提供了重要的基础。使用 DP 的方法**也是从贝尔曼方程衍生出来的**，其思路是将贝尔曼方程变形为“**更新式**”。数学式如下所示。

$$V_{k+1}(s) = \sum_{a, s'} \pi(a | s) p(s' | s, a) \{r(s, a, s') + \gamma V_k(s')\} \quad (4.2)$$

这里用 $V_{k+1}(s)$ 表示**第 $k+1$ 次进行更新的价值函数**，用 $V_k(s)$ 表示第 k 次进行更新的价值函数。 $V_{k+1}(s)$ 和 $V_k(s)$ 是“**推测值**”，它们与真正的价值函数 $v(s)$ **不同**。因此，我们用大写的 V 来表示它们。

式 4.2 的特点是使用“下一个可能状态的价值函数 $V_k(s')$ ”来更新“目前所处状态的价值函数 $V_{k+1}(s)$ ”。我们在这里所做的是用估计值 $V_k(s')$ 来改进另一个估计值 $V_{k+1}(s)$ 。这种用估计值来改进估计值的过程叫作**自举法** (bootstrapping)。



bootstrapping 本来指的是在穿鞋时用手指挑起来的鞋背上的一个“环”。后来引申为在没有他人帮助的情况下，只靠自己进行改进的过程。

现在介绍使用 DP 的具体算法。首先为 $V_0(s)$ **设置初始值**（比如在所有状态下进行 $V_0(s) = 0$ 的初始化操作）。然后利用式 4.2，基于 $V_0(s)$ 更新 $V_1(s)$ 。接下来根据 $V_1(s)$ 更新 $V_2(s)$ 。通过重复这个过程，我们就接近了最终的目标 $v_\pi(s)$ 。这种算法被称为**迭代策略评估** (iterative policy evaluation)。

当在实际问题中使用迭代策略评估算法时，**迭代更新必须在某一点停止**。更新的次数可以根据每次更新的量来确定。关于这一点，我们马上会看到一个实现的示例。此外，根据证明，基于式 4.2 的重复更新会收敛到 $v_\pi(s)$ 。不过必须满足一些条件才能收敛。更多信息请参见参考文献 [5]。



DP 是一类算法的总称，一般指的是通过将目标问题划分为更小的问题来求出答案的方法。DP 的本质是“不进行两次相同的计算”。有两种方式可以实现这一点：“自上而下的方式”（也叫作“记忆化”）和“自下而上的方式”。之前介绍的按照 $V_0(s)$ 、 $V_1(s)$ ……的顺序一次次地向上更新价值函数的方式是“自下而上的方式”。

4.1.2 尝试迭代策略评估

下面以“有两个方格的网格世界”为例，看看迭代策略评估算法的流程。思考图 4-2 所示的问题。

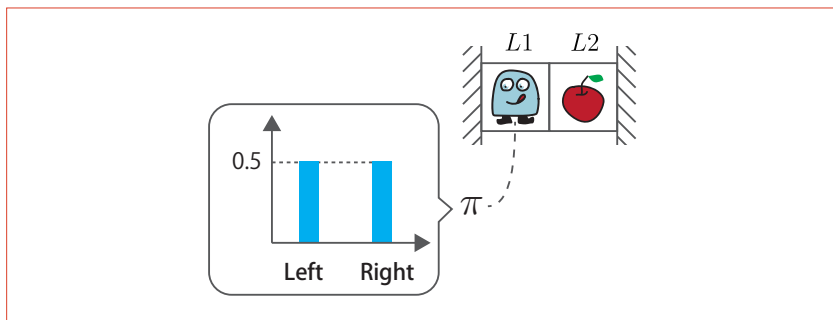


图 4-2 有两个方格的网格世界（撞墙的奖励为 -1，从 L1 移动到 L2 的奖励为 +1）

如图 4-2 所示，智能代理根据随机性策略 π （或以 0.5 的概率向左移动，或以 0.5 的概率向右移动）来采取行动。另外，这个问题中的状态迁移是确定性的。也就是说，如果在状态 s 下执行了行动 a ，那么下一个状态 s' 就只有一个。用数学式来描述就是下一个状态 s' 是由函数 $f(s,a)$ 唯一确定的。在这种情况下，我们可以对价值函数的更新式 4.2 做出简化，如图 4-3 所示。

$$\begin{aligned}
 V_{k+1}(s) &= \sum_{a,s'} \pi(a|s) p(s'|s,a) \{r(s,a,s') + \gamma V_k(s')\} \quad (4.2) \\
 &\quad \downarrow \text{将 } \Sigma \text{ 分开书写} \\
 V_{k+1}(s) &= \sum_a \pi(a|s) \sum_{s'} p(s'|s,a) \{r(s,a,s') + \gamma V_k(s')\} \\
 &\quad \downarrow \text{状态迁移是确定性的} \\
 \text{当 } s' = f(s,a) \text{ 时} \\
 V_{k+1}(s) &= \sum_a \pi(a|s) \{r(s,a,s') + \gamma V_k(s')\} \quad (4.3)
 \end{aligned}$$

图 4-3 状态迁移是确定性的情况的更新式

在图 4-3 所示的推导中，我们暂时将式 4.2 中的 $\sum_{a,s'}$ 拆开写为了 $\sum_a$ 和 $\sum_{s'}$ 。另外，由于状态迁移是确定性的，因此式子可以进一步简化。具

体来说，由于下一个状态 s' 在这个问题中是唯一确定的，因此不需要像 s' 那样对所有的状态求和，而只计算一个 s' 即可，如图 4-3 中的式 4.3 所示。在这个问题中，我们根据式 4.3 更新价值函数。

下面用迭代策略评估算法来求策略 π 的价值函数。首先将 $V_0(s)$ 的初始值设为 0。由于在有两个方格的网格世界里有两个状态，因此有以下初始化的式子成立。

$$V_0(L1) = 0$$

$$V_0(L2) = 0$$

然后根据式 4.3 更新 $V_0(s)$ 。结合图 4-4 的回溯线形图会更容易理解。

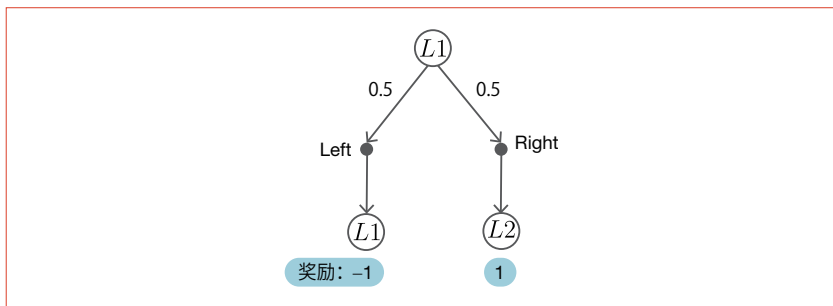


图 4-4 从状态 $L1$ 开始的回溯线形图

图 4-4 中有两个分支。一种可能的情况是以 0.5 的概率选择了向左移动的行动，获得奖励 -1，状态仍然是 $L1$ 。将折现率 γ 定为 0.9，并将其应用于式 4.3，可以得到以下式子。

$$0.5\{-1 + 0.9V_0(L1)\}$$

图 4-4 中另一种可能的情况是在状态 $L1$ 选择了向右移动的行动。在这种情况下，智能代理得到奖励 1，迁移到状态 $L2$ 。将此情况应用于式 4.3，可以得到以下式子。

$$0.5\{1 + 0.9V_0(L2)\}$$

根据上述信息，可以求出 $V_1(L1)$ ，如下所示。

$$\begin{aligned}
 V_1(L1) &= 0.5\{-1 + 0.9V_0(L1)\} + 0.5\{1 + 0.9V_0(L2)\} \\
 &= 0.5(-1 + 0.9 \times 0) + 0.5(1 + 0.9 \times 0) \\
 &= 0
 \end{aligned}$$

使用同样的方法，还可以计算出 $V_1(L2)$ 。

$$\begin{aligned}
 V_1(L2) &= 0.5\{0 + 0.9V_0(L1)\} + 0.5\{-1 + 0.9V_0(L2)\} \\
 &= 0.5(0 + 0.9 \times 0) + 0.5(-1 + 0.9 \times 0) \\
 &= -0.5
 \end{aligned}$$

现在所有状态的价值函数的更新都已完成（这次探讨的问题总共只有两个状态）。结果汇总在图 4-5 中。

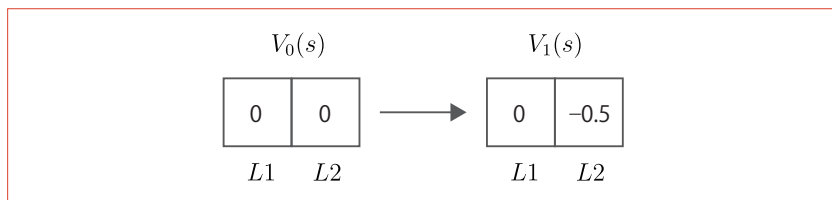


图 4-5 价值函数的第一次更新

如图 4-5 所示， $V_0(s)$ 已被更新为 $V_1(s)$ 。接下来要做的是重复这个过程。具体来说，就是从 $V_1(s)$ 计算 $V_2(s)$ ，然后从 $V_2(s)$ 计算 $V_3(s)$ ，以此类推。下面让我们使用 Python 来实现这个计算。代码如下所示。

```

V = {'L1': 0.0, 'L2': 0.0}
new_V = V.copy() # 复制 V

for _ in range(100):
    new_V['L1'] = 0.5 * (-1 + 0.9 * V['L1']) + 0.5 * (1 + 0.9 * V['L2'])
    new_V['L2'] = 0.5 * (0 + 0.9 * V['L1']) + 0.5 * (-1 + 0.9 * V['L2'])
    V = new_V.copy()
print(V)

```

输出结果

```

'L1': 0.0, 'L2': -0.5
'L1': -0.22499999999999998, 'L2': -0.725
'L1': -0.42749999999999994, 'L2': -0.9274999999999999
...
'L1': -2.2499335965027827, 'L2': -2.7499335965027827

```


由于这次的问题有两个状态，因此我们把这两个状态的价值函数保存在字典数据结构中。然后，将当前的价值函数设为 V ，将待更新的价值函数设为 new_V 。运行上面的代码，我们就会看到 V 的更新过程。顺带一提，真正的价值函数的值是 $[-2.25, -2.75]$ 。上面的结果显示，经过 100 次更新后的值几乎与真正的值相同。



上面的代码进行了 $\text{new_V} = V.\text{copy}()$ 、 $V = \text{new_V}.\text{copy}()$ 等字典的复制操作。这使得 new_V 和 V 成为不同的对象，因此更新 new_V 中的元素不会影响 V 中的元素。

上面的代码中进行了固定次数的更新（具体来说是 100 次）。接下来，我们通过设置阈值来自动确定更新的次数。代码如下所示。

ch04/dp.py

```
V = {'L1': 0.0, 'L2': 0.0}
new_V = V.copy()

cnt = 0 # 记录更新了几次
while True:
    new_V['L1'] = 0.5 * (-1 + 0.9 * V['L1']) + 0.5 * (1 + 0.9 * V['L2'])
    new_V['L2'] = 0.5 * (0 + 0.9 * V['L1']) + 0.5 * (-1 + 0.9 * V['L2'])

    # 更新量的最大值
    delta = abs(new_V['L1'] - V['L1'])
    delta = max(delta, abs(new_V['L2'] - V['L2']))

    V = new_V.copy()

    cnt += 1
    if delta < 0.0001:
        print(V)
        print(cnt)
        break
```

输出结果

```
'L1': -2.249167525908671, 'L2': -2.749167525908671
76
```

代码中的阈值被设置为了 0.0001，循环持续进行，直到更新量的最大值低于该阈值。具体做法是求 new_V 和 V 的相对应元素的差的绝对值。由

于这次有两个元素，因此要从两个差的绝对值中选择较大的那个。这个值就是 δ 。如果 δ 小于阈值，那么更新就会停止。上面的结果表明，经过 76 次更新，价值函数已经与真正的值相差无几。

4.1.3 迭代策略评估的其他实现方式

在实现解决前面问题的迭代策略评估算法时，我们使用了两个字典：一个是保存当前价值函数的字典 V ，另一个是用于更新的字典 $\text{new_}V$ 。下面我们用这两个字典来更新价值函数，如图 4-6 所示。

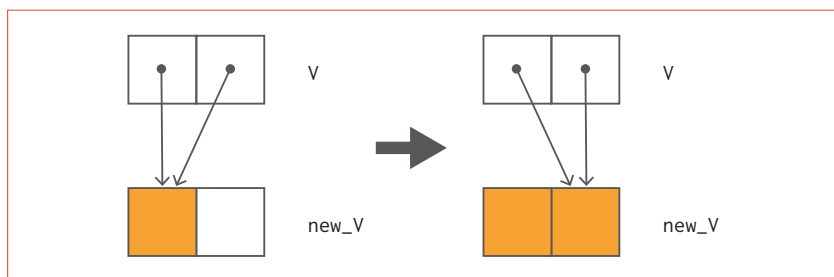


图 4-6 之前的更新方法

图 4-6 中需要注意的是，在计算 $\text{new_}V$ 的每个元素时我们使用了字典 V 的值。这个过程还有另一种实现方式，即覆盖字典中的每个元素，如图 4-7 所示。

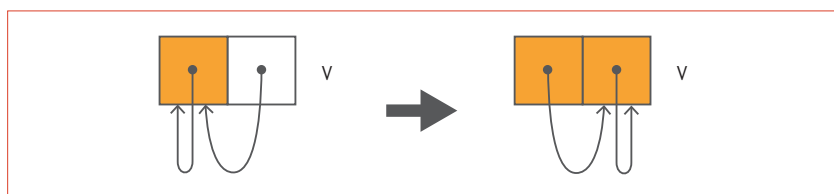


图 4-7 新的更新方法

图 4-7 中的方法只使用了字典 V ，字典中的每个元素都被覆盖了。这里把这种方式叫作“覆盖方式”。



这两种方法在无限次迭代后都会收敛到正确的值。不过覆盖方式的更新速度几乎每次都更快。这是因为在覆盖方式的情况下，更新后的元素可以立即使用。例如，图 4-7 中就使用了左边的更新后的值来更新右边的元素。

现在让我们使用覆盖方式来实现 DP。代码如下所示。

ch04/dp_inplace.py

```
V = {'L1': 0.0, 'L2': 0.0}

cnt = 0
while True:
    t = 0.5 * (-1 + 0.9 * V['L1']) + 0.5 * (1 + 0.9 * V['L2'])
    delta = abs(t - V['L1'])
    V['L1'] = t

    t = 0.5 * (0 + 0.9 * V['L1']) + 0.5 * (-1 + 0.9 * V['L2'])
    delta = max(delta, abs(t - V['L2']))
    V['L2'] = t

    cnt += 1
    if delta < 0.0001:
        print(V)
        print(cnt)
        break
```

输出结果

```
'L1': -2.2493782177156936, 'L2': -2.7494201578106514
60
```

这次我们只使用字典 V，其中的每个元素都会被立即覆盖。上面的结果显示，经过 60 次更新计算就结束了。之前的结果是 76 次更新，所以更新的次数的确降低了。从现在开始，我们将使用覆盖方式来实现迭代策略评估算法。

4.2 解决更大的问题

如果使用迭代策略评估算法，那么即使状态和行动模式的数量大一些，也能高效求解。本节将从两个方格的网格世界问题“毕业”，开始挑战一

个更大的问题（尽管仍然是小问题）。这次我们要处理的是“ 3×4 网格世界”，如图 4-8 所示。

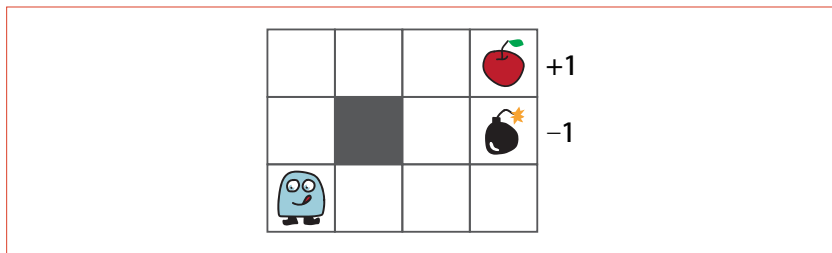


图 4-8 3×4 网格世界

“ 3×4 网格世界”问题的设定如下。

- 智能代理可以向上、下、左、右四个方向移动。
- 图 4-8 中的灰色方块代表墙，智能代理不能进入。
- 网格外也是墙，智能代理无法前进。
- 撞墙的奖励是 0。
- 苹果的奖励是 +1，炸弹的奖励是 -1。
- 环境的状态迁移是唯一确定的。换句话说，如果智能代理选择向右移动，那么（如果没有墙的话）它将总是向右移动。
- 这个任务是回合制任务，当苹果被拿走时，任务就会结束。

下面将这个 3×4 网格世界实现为 GridWorld 类。

4.2.1 GridWorld 类的实现

这里将 GridWorld 类实现在 common/gridworld.py 中。首先来看一下初始化部分的代码。

```
import numpy as np

class GridWorld:
    def __init__(self):
        self.action_space = [0, 1, 2, 3]
```

common/gridworld.py

```

self.action_meaning = {
    0: "UP",
    1: "DOWN",
    2: "LEFT",
    3: "RIGHT",
}

self.reward_map = np.array(
    [[0, 0, 0, 1.0],
     [0, None, 0, -1.0],
     [0, 0, 0, 0]]
)

self.goal_state = (0, 3)
self.wall_state = (1, 1)
self.start_state = (2, 0)
self.agent_state = self.start_state

```

GridWorld 类拥有几个实例变量。`self.action_space` 表示行动空间，也就是“候选行动”。在上面的代码中，行动由 4 个数字 [0、1、2、3] 表示，每个数字的含义在 `self.action_meaning` 中定义。例如，0 表示向上移动 (UP)，1 表示向下移动 (DOWN)。

`self.reward_map` 是奖励映射。这个奖励映射表示移动到每个方格时可以获得的奖励值。奖励映射是以 NumPy 的二维数组 (`np.ndarray`) 的形式创建的，其坐标系如图 4-9 所示。

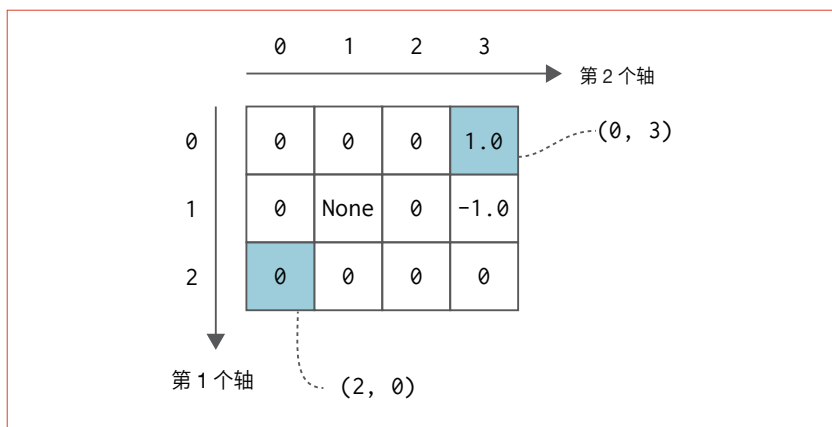


图 4-9 奖励映射的坐标系

这次的问题是**回合制任务**。这意味着问题的设定是这样的：当智能代理到达目的地时，它将再次返回到起点并执行相同的任务。上面代码中有一行是 `self.goal_state = (0, 3)`，意思是目标状态是 $(0, 3)$ 。另外，墙由 `self.wall_state = (1, 1)` 设置，智能代理的初始位置由 `self.agent_state = (2, 0)` 设置。

下面的代码是 `GridWorld` 类的方法。

`common/gridworld.py`

```
class GridWorld:
    ...

    @property
    def height(self):
        return len(self.reward_map)

    @property
    def width(self):
        return len(self.reward_map[0])

    @property
    def shape(self):
        return self.reward_map.shape

    def actions(self):
        return self.action_space # [0, 1, 2, 3]

    def states(self):
        for h in range(self.height):
            for w in range(self.width):
                yield (h, w)
```

上述代码使用 `@property` 装饰器实现了 `GridWorld` 类的一些有用的实例变量。`@property` 装饰器可以放在方法名之前的一行，**它使得目标方法可以作为实例变量来使用**。对于本次的问题，我们需要知道网格世界的大小和形状，使用这些信息的代码如下所示。

```
env = GridWorld()

# 不通过 env.height() 而是通过 env.height 来使用
print(env.height) # 3
print(env.width) # 4
print(env.shape) # (3, 4)
```

上面的代码还为 GridWorld 类实现了 actions 方法和 states 方法。使用这些方法，我们就可以按顺序访问所有行动和所有状态。使用示例如下所示。

```
for action in env.actions():
    print(action)

print('===')

for state in env.states():
    print(state)
```

输出结果

```
0
1
2
3
===
(0, 0)
(0, 1)
...
(2, 3)
```

上面的代码使用了 for 语句来访问所有的行动和状态。



state() 方法中使用了 yield。与 return 一样，yield 不仅可以返回函数的值，还可以暂停函数的处理，转到另一个处理（另一个处理结束后，函数会从暂停的地方恢复处理）。多亏了这个功能，它才可以像上面那样与 for 语句等迭代处理结合使用。

下面让我们来实现表示环境的状态迁移的方法 next_state() 和奖励函数的方法 reward()。代码如下所示。

```
class GridWorld:
    ...
    def next_state(self, state, action):
        # ①移动目的地的计算
        action_move_map = [(-1, 0), (1, 0), (0, -1), (0, 1)]
        move = action_move_map[action]
        next_state = (state[0] + move[0], state[1] + move[1])
        ny, nx = next_state

        # ②移动目的地在网格世界的边界之外，还是移动目的地是墙？
        if nx < 0 or nx >= self.width or ny < 0 or ny >= self.height:
            next_state = state
```

```

elif next_state == self.wall_state:
    next_state = state

return next_state # ③返回下一个状态

def reward(self, state, action, next_state):
    return self.reward_map[next_state]

```

在代码①处，移动目的地可以被计算出来，此时暂时忽略墙和网格世界的边界。然后，代码②处可以判断智能代理是否已经移动到了边界外或墙上。如果是不能移动的目的地，则 $\text{next_state} = \text{state}$ 。另外，在这个任务中，状态迁移是确定性的，所以代码③处会返回下一个状态。

为了与式 $r(s,a,s')$ 中的参数相对应，奖励函数的方法的参数被设计成了以下形式： $\text{reward}(\text{self}, \text{state}, \text{action}, \text{next_state})$ 。不过在这个问题中，只需下一个状态就能确定奖励的值。

另外，GridWorld 类还提供了一个用于可视化的方法 $\text{render_v}(\text{self}, \text{v}=\text{None}, \text{policy}=\text{None})$ ，其中的代码实现不重要，所以这里只介绍如何使用它。代码如下所示。

```

env = GridWorld()
env.render_v()

```

图 4-10 展示了 3×4 网格世界的可视化图形。

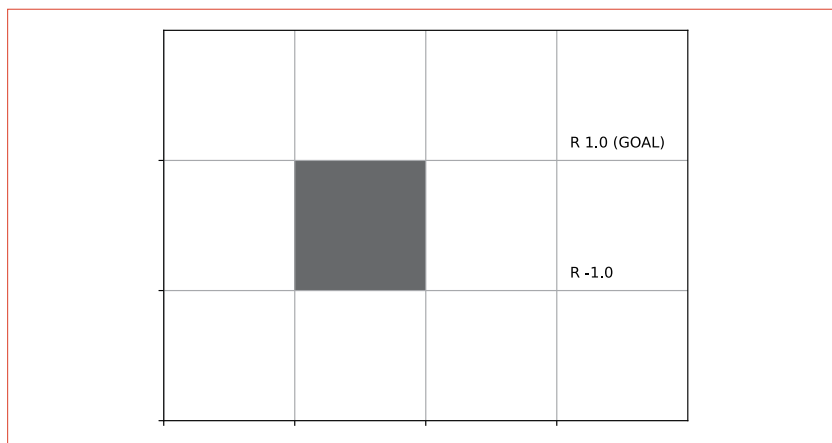


图 4-10 3×4 网格世界的可视化

执行上述代码，网格世界就会显示出来。墙壁会被画成灰色，奖励值会显示在方格的左下角。奖励为 0 的方格显示为空白。这个 `render_v` 方法可以接收状态价值函数作为参数。作为测试，下面给定了一个虚拟的状态价值函数，绘制出的图形如图 4-11 所示。

ch04/gridworld_play.py

```
env = GridWorld()
V = {}
for state in env.states():
    V[state] = np.random.randn() # 虚拟的状态价值函数
env.render_v(V)
```



图 4-11 3×4 网格世界的可视化（给定状态价值函数）

上面代码中的 `V` 是虚拟的状态价值函数。如果把价值函数传给 `render_v` 方法，那么每个方格的价值函数的值会显示在方格的右上角。另外，值的大小会以热力图的形式展示出来：为负的值越小，方格的红色越深；为正值越大，方格的绿色越深。



`render_v` 方法也接收策略作为参数。此时，每个方格中概率最高的行动会用箭头表示。另外，我们还提供了用于行动价值函数（`Q` 函数）的可视化的 `render_q` 方法。关于 `render_q` 方法的具体用法，请参见 5.4.5 节。

以上是对 GridWorld 类的介绍，这个类还实现了 reset() 方法和 step(action) 方法。reset() 是将游戏返回到初始状态的方法，具体来说是将智能代理返回到起始位置。step(action) 方法可以使智能代理执行 action 行动，并将时间前进一步。这两个方法实际上是被用来移动智能代理。迭代策略评估算法不使用这两个方法，因为它并不使智能代理实际采取行动。

后面将实现迭代策略评估算法。在此之前，我们来介绍一下 Python 标准库中的 collections.defaultdict 的用法，使用 defaultdict，就能简单地实现价值函数和策略。

4.2.2 defaultdict 的用法

在前面的代码中，我们用字典实现了价值函数，比如使用了像下面这样的代码。

```
from common.gridworld import GridWorld

env = GridWorld()
V = {}

# 字典元素的初始化
for state in env.states():
    V[state] = 0

state = (1, 2)
print(V[state]) # 输出状态 (1,2) 的价值函数
```

上面的示例代码用字典实现了价值函数，即 $V = \{\}$ 。我们要像 $V[key]$ 这样使用字典，但如果 key 不存在，就会发生错误。因此，我们要像上述代码那样将所有的元素都初始化。Python 标准库中的 defaultdict 可以把我们从初始化这项烦琐工作中解放出来。它的使用方法如下所示。

```
from collections import defaultdict # 导入 defaultdict
from common.gridworld import GridWorld

env = GridWorld()
V = defaultdict(lambda: 0)

state = (1, 2)
print(V[state]) # 0
```

上面的代码通过设置 `V = defaultdict(lambda: 0)`，就能将 `V` 作为普通的字典使用了。此外，当访问不存在于字典中的键时，`defaultdict` 会自动创建带有默认值的键。在上面的例子中，在访问像 `V[state]` 那样的键时，由于键为 `state` 的元素不存在，因此此时值为 `0` 的元素会被创建。后续我们将使用 `defaultdict` 来实现价值函数。另外，我们也会使用 `defaultdict` 来实现策略。



设置 `defaultdict` 的默认值的方法有几种，其中使用 `lambda` 来设置的方法很简单，比如我们可以这样写：`defaultdict(lambda: 0)` 或者 `defaultdict(lambda: "A")`。

下面使用 `defaultdict` 来实现随机性策略。在 3×4 网格世界的问题中，智能代理可以采取 4 个行动，我们用 `[0, 1, 2, 3]` 来表示每个行动。如果采取这 4 个行动的概率是均匀随机的，那么每个行动被采取的概率是 $\frac{1}{4} = 0.25$ 。因此，行动的概率分布可以表示为 `{0:0.25, 1:0.25, 2:0.25, 3:0.25}`。在此基础上，可以按以下方式实现随机性策略。

```
pi = defaultdict(lambda: {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25})

state = (0, 1)
print(pi[state]) # {0:0.25, 1:0.25, 2:0.25, 3:0.25}
```

上面的代码会将随机性策略实现为 `pi`。给定一个状态，`pi` 会返回该状态下行动的概率分布。例如，上面的例子输出的是状态 `(0, 1)` 下行动的概率分布。如上面的结果所示，所有的行动都遵循以 `0.25` 的概率被选择的概率分布。

4.2.3 迭代策略评估的实现

接下来实现迭代策略评估算法。首先实现只进行一个时间步的更新的方法。这里实现的 `eval_onestep` 函数接收以下 4 个参数。

- `pi (defaultdict)`: 策略
- `V (defaultdict)`: 价值函数
- `env (GridWorld)`: 环境

- gamma (float): 折现率

函数代码如下所示。

ch04/policy_eval.py

```
def eval_onestep(pi, V, env, gamma=0.9):
    for state in env.states(): # ①访问各个状态
        if state == env.goal_state: # ②目的地的价值函数总是为 0
            V[state] = 0
            continue

        action_probs = pi[state] # probs 是 probabilities 的缩写
        new_V = 0

        # ③访问各行动
        for action, action_prob in action_probs.items():
            next_state = env.next_state(state, action)
            r = env.reward(state, action, next_state)
            # ④新的价值函数
            new_V += action_prob * (r + gamma * V[next_state])
        V[state] = new_V
    return V
```

eval_onestep 函数使用了两重 for 语句。代码①处依次访问了各个状态。示意图如图 4-12 所示。

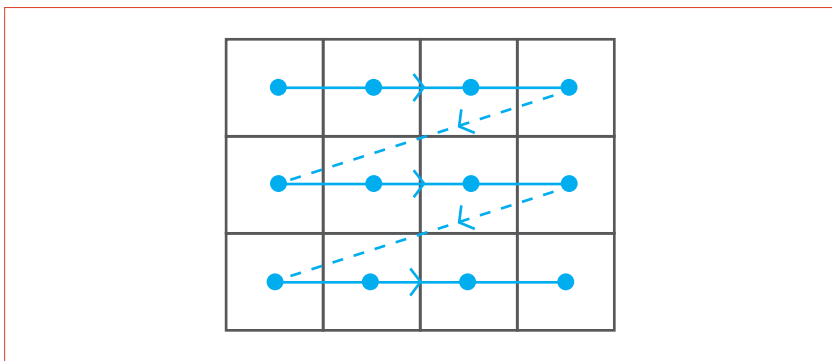


图 4-12 按顺序访问各个状态

如图 4-12 所示，所有的状态都是按顺序访问的。在代码②处，如果状态 state 是目的地，那么价值函数就会被设置为 0。这是因为如果智能代理

已抵达目的地，那么回合就结束了，接下来就什么也不做了。因此，在目的地的价值函数的值总是为 0。

在代码③处，行动的概率分布会被取出。然后会从状态迁移函数 (`env.next_state(state, action)`) 得到下一个状态 (`next_state`)。之后是适用于作为迭代策略评估算法的更新式的式 4.3 (下面再次列出)。

当 $s' = f(s, a)$ 时

$$V_{k+1}(s) = \sum_a \pi(a|s) \{r(s, a, s') + \gamma V_k(s')\} \quad (4.3)$$

将式 4.3 与上述代码中的④处相比较，就能清晰地看出二者间的对应关系。这样 `eval_onestep` 函数就完成了价值函数的一个时间步的更新。后面就是重复这个更新过程。因此方法的实现如下所示。

ch04/policy_eval.py

```
def policy_eval(pi, V, env, gamma, threshold=0.001):
    while True:
        old_V = V.copy() # 更新前的价值函数
        V = eval_onestep(pi, V, env, gamma)

        # 求更新量的最大值
        delta = 0
        for state in V.keys():
            t = abs(V[state] - old_V[state])
            if delta < t:
                delta = t

        # 与阈值比较
        if delta < threshold:
            break
    return V
```

参数 `threshold` 是更新的阈值。如上面的代码所示，重复调用 `eval_onestep` 函数，当更新量的最大值小于阈值时，停止更新。

接下来使用 `GridWorld` 类和已经实现的 `policy_eval` 函数来进行策略的评估。代码如下所示。

ch04/policy_eval.py

```

env = GridWorld()
gamma = 0.9

pi = defaultdict(lambda: {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25})
V = defaultdict(lambda: 0)

V = policy_eval(pi, V, env, gamma)
env.render_v(V, pi)

```

上面的代码初始化了随机性策略 π 和价值函数 V 。随后评估该随机性策略，得到价值函数。运行上述代码得的图形如图 4-13 所示。

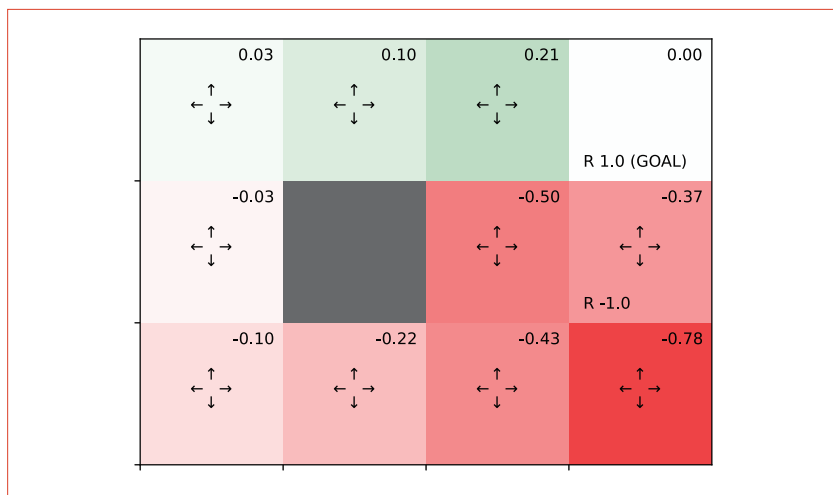


图 4-13 随机性策略的价值函数

图 4-13 显示了随机性策略的价值函数。例如，左下角的起点的价值函数是 -0.10 。这意味着如果智能代理从左下角的起点随机移动，那么获得的收益的期望值将是 -0.10 。智能代理也可能因为随机移动而意外地得到炸弹。根据 -0.10 可知，得到炸弹 (-1 的奖励) 的概率比得到苹果 ($+1$ 的奖励) 的概率略大一些。另外，总的来说，底部和中间的几行都是负数。这说明炸弹在这些地方的影响更大。

运行上面的代码，立即就得到了结果。使用 DP 能够高效地评估这些策

略。这样的话，处理一定程度上更大的网格世界也没问题。不过上面只进行了策略的评估。接下来我们将研究如何找到最优策略。

4.3 策略迭代法

我们的目标是得到最优策略。实现这个目标的一种方法是解出一组满足贝尔曼最优方程的联立方程。不过这种方法的计算量非常大。具体来说，当状态的大小为 S 、行动的大小为 A 时，求解将需要 A^S 这么大的计算量。问题的规模稍作增加，就会使得求解无法在合理的时间内完成。



在大多数情况下，直接求解贝尔曼最优方程是很困难的。因此，我们首先要做的是对当前的策略进行评估。这是因为如果可以正确评估当前的策略，那么就可以对其进行“改进”。

在 4.2 节中，我们使用 DP 评估了策略。那是一种叫作“迭代策略评估”的算法。除了可以对策略进行评估，还可以对策略进行改进。本节将学习改进策略的方法。

4.3.1 策略的改进

改进策略的线索在“最优策略”中。这里我们使用以下符号来介绍如何改进策略。

- 最优策略: $\mu_*(s)$
- 最优策略的状态价值函数: $v_*(s)$
- 最优策略的行动价值函数: $q_*(s,a)$

首先回顾一下已学过的内容。3.5.2 节介绍过最优策略 μ_* 的式子，如下所示。

$$\mu_*(s) = \operatorname{argmax}_a q_*(s,a) \quad (4.4)$$

$$= \operatorname{argmax}_a \sum_{s'} p(s' | s,a) \{r(s,a,s') + \gamma v_*(s')\} \quad (4.5)$$

如式 4.4 和式 4.5 所示，最优策略是由 argmax_a （取最大值的行动 a ）所决定的。这个 argmax_a 操作从局部候选者中选择最佳行动。由于这个原因，由式 4.4 和式 4.5 得到的策略也被称为“贪婪策略”。



根据式 4.5，如果知道最优价值函数 v_* ，那么就可以找到最优策略 μ_* 。不过要想知道最优价值函数 v_* ，又需要先知道最优策略 μ_* 。这是一个先有鸡还是先有蛋的问题。也就是说，要知道 v_* ，需要先知道 π_* ；而要知道 π_* ，需要先知道 v_* 。

式 4.4 是最优策略 μ_* 的式子，在这里我们考虑将式 4.4 应用于一些确定性的策略 μ 。来看一下下面的式子。

$$\mu'(s) = \operatorname{argmax}_a q_\mu(s, a) \quad (4.6)$$

$$= \operatorname{argmax}_a \sum_{s'} p(s' | s, a) \{r(s, a, s') + \gamma v_\mu(s')\} \quad (4.7)$$

式子中各部分的含义如下。

- 当前策略： $\mu(s)$
- 策略 $\mu(s)$ 的状态价值函： $v_\mu(s)$
- 新的策略： $\mu'(s)$

另外，通过式 4.6 或式 4.7 更新策略的做法叫作“贪婪化”。

那么，贪婪化的策略 $\mu'(s)$ 的特点是什么呢？如果在所有状态 s 下 $\mu(s)$ 和 $\mu'(s)$ 都是一样的，那么可以说策略 $\mu(s)$ 就已经是最优策略了。这是因为如果该策略没有被式 4.6 改变，那么它就满足以下式子。

$$\mu(s) = \operatorname{argmax}_a q_\mu(s, a)$$

这正是最优策略所满足的式子。因此，如果 $\mu'(s)$ 在所有状态 s 下都没有被贪婪化所更新，那么 $\mu(s)$ 就已经是最优策略了。

如果策略被贪婪化更新了，也就是说出现了策略 μ' 和策略 μ 不同的情况，该怎么办？我们知道，在这种情况下策略总会被改进。更确切地说，对于所有状态 s ，有 $v_{\mu'}(s) \geq v_\mu(s)$ 成立。



这里只说明了通过式 4.6 或式 4.7 改进策略的事实，而省略了证明。策略改进的数学基础是**策略改进定理** (policy improvement theorem)。关于策略改进定理及其证明，请参考参考文献 [5]。

综上所述，策略的贪婪化意味着：

- 策略总是被改进；
- 如果该策略没有被改进（更新），那么它就是最优策略。

4.3.2 重复评估和改进

通过式 4.6 或式 4.7 对策略进行贪婪化更新，策略总是会得到改进。这里假定基于状态价值函数的式 4.7 进行探讨。我们在 4.2 节中已经实现了评估状态价值函数的算法。现在我们有找出最优策略的关键思路。该方法如图 4-14 所示。

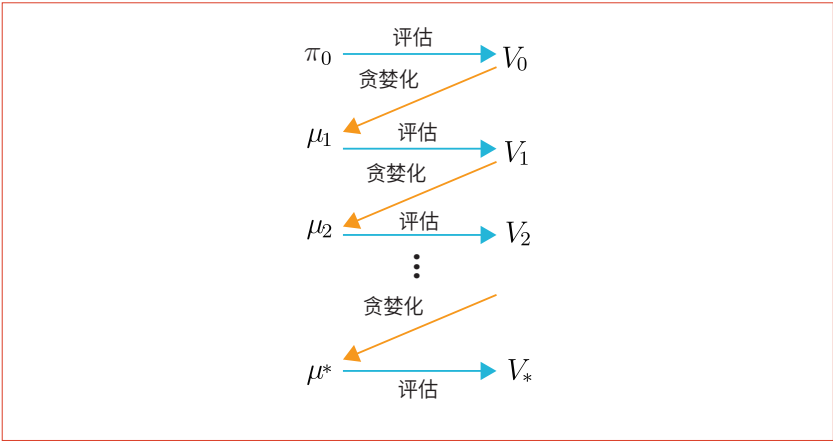


图 4-14 策略改进的流程

图 4-14 中的流程写下来是这样的。

- 首先从策略 π_0 开始。由于策略 π_0 也可能是随机性策略，因此可以用 $\pi_0(s|a)$ 来表示，而不表示为 $\mu_0(s)$ 。

- 然后对策略 π_0 的价值函数进行评估，得到 V_0 。这个过程可以通过迭代策略评估算法来进行。
- 接下来使用价值函数 V_0 进行贪婪化（根据式 4.7 更新策略）。由于贪婪化策略总是选择一个行动，因此得到确定性策略 μ_1 。

之后就是重复这个流程。如果将此过程持续下去，那么就会达到策略不再因贪婪化而变更的一个点。此时的策略就是最优策略。这种重复评估和改进的算法叫作策略迭代法（policy iteration）。



环境由状态迁移概率 $p(s'|s,a)$ 和奖励函数 $r(s,a,s')$ 来表示。在强化学习领域，这两者有时被称为“环境模型”，或者直接被称为“模型”。如果环境模型是已知的，那么智能代理就可以在不采取任何行动的情况下评估价值函数，然后使用策略迭代法来找出最优策略。在智能代理没有实际采取任何行动的情况下找出最优策略的问题叫作规划（planning）问题。本章讨论的问题就是规划问题。另外，强化学习通常在环境模型未知的问题设定下进行，在这种情况下，智能代理会采取实际行动，在获得经验的同时找出最优策略。

4.4 实施策略迭代法

和前面一样，本节会求解 3×4 网格世界问题（参见图 4-15）。目标是使用策略迭代法得到最优策略。

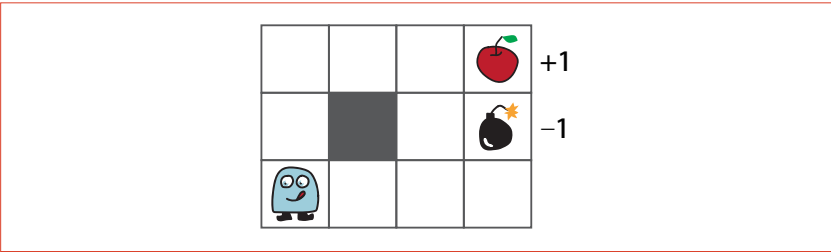


图 4-15 3×4 网格世界

我们已经实现了评估策略的代码，剩下的就是改进这些策略。本节将实现达到这一目标的代码。

4.4.1 改进策略

为了改进策略，我们会基于当前价值函数得到贪婪策略。数学式如下所示。

$$\mu'(s) = \operatorname{argmax}_a \sum_{s'} p(s' | s, a) \{r(s, a, s') + \gamma v_{\mu}(s')\} \quad (4.7)$$

另外，这个问题中的状态迁移是唯一的（确定性的）。因此，策略的贪婪化可以做如下简化。

当 $s' = f(s, a)$ 时

$$\mu'(s) = \operatorname{argmax}_a \{r(s, a, s') + \gamma v_{\mu}(s')\} \quad (4.8)$$

如式 4.8 所示，下一个状态 s' 只有一个。下面根据式 4.8 实现获得贪婪策略的函数。在此之前做一下准备工作，即实现 `argmax` 函数。它会接收字典作为参数，并返回具有最大字典值的键。我们先来了解它的用法，示例代码如下所示。

```
action_values = {0: 0.1, 1: -0.3, 2: 9.9, 3: -1.3} # 9.9 是最大值（键是 2）

max_action = argmax(action_values)
print(max_action)
```

输出结果

2

上面的代码返回了具有最大字典值的键。这个函数的实现如下所示。

ch04/policy_iter.py

```
def argmax(d):
    max_value = max(d.values())
    max_key = 0
    for key, value in d.items():
        if value == max_value:
            max_key = key
    return max_key
```

代码非常简单。对于作为参数给出的字典 d ，函数会取出最大的值并返回 key 。如果有多个最大值，那么为了简单起见，函数会返回处于最后位置的键。因此，函数总是会返回一个键作为返回值。

下面使用 `argmax` 函数实现将价值函数贪婪化的函数。代码如下所示。

ch04/policy_iter.py

```
def greedy_policy(V, env, gamma):
    pi = {}

    for state in env.states():
        action_values = {}

        for action in env.actions():
            next_state = env.next_state(state, action)
            r = env.reward(state, action, next_state)
            value = r + gamma * V[next_state] # ①
            action_values[action] = value
        max_action = argmax(action_values) # ②
        action_probs = {0: 0, 1: 0, 2: 0, 3: 0}
        action_probs[max_action] = 1.0
        pi[state] = action_probs # ③

    return pi
```

`greedy_policy(V, env, gamma)` 函数会将价值函数 V 、环境 env 和折现率 $gamma$ 作为参数。它可以基于参数中的价值函数 V 返回贪婪化的策略。

代码①处会对每个行动计算式 4.8 中的 $r(s,a,s') + \gamma V_{\pi}(s')$ 。然后，在代码②处，使用 `argmax` 取出具有最大价值函数的行动 (`max_action`)。最后，生成使 `max_action` 被选中的概率为 1.0 的概率分布，随后把它设定为在状态 `state` 下行动的概率分布。以上就是对策略进行贪婪化的函数的实现。



贪婪化后的策略是确定性的策略。因此，我们也可以像上面代码③处写的 `pi[state] = action_probs` 那样，只指定一个行动。不过我们已经实现的评估策略的 `policy_eval(pi, ...)` 函数接收的参数是随机性策略。因此，这里按照随机性策略来实现。

4.4.2 重复评估和改进

前面我们已经完成了重复评估和改进的策略迭代法的代码实现的准备工作。这里用函数名 `policy_iter(env, gamma, threshold=0.001, is_render=False)` 来实现它。每个参数的介绍如下所示。

- `env` (`Environment`): 环境。
- `gamma` (`float`): 折现率。
- `threshold` (`float`): 进行策略评估时, 停止更新的阈值。
- `is_render` (`bool`): 表示是否应该绘制评估和改进策略的过程的标志位。

代码实现如下所示。

ch04/policy_iter.py

```
def policy_iter(env, gamma, threshold=0.001, is_render=False):
    pi = defaultdict(lambda: {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25})
    V = defaultdict(lambda: 0)

    while True:
        V = policy_eval(pi, V, env, gamma, threshold) # ①评估
        new_pi = greedy_policy(V, env, gamma) # ②改进

        if is_render:
            env.render_v(V, pi)

        if new_pi == pi: # ③更新检查
            break
        pi = new_pi

    return pi
```

先初始化策略 `pi` 和价值函数 `V`。代码中使用 `defaultdict` 赋予它们初始值。策略 `pi` 的初始值被设定为每个行动以相同的概率被选择。

上述代码的主要部分是①和②两处。首先, ①处代码会对当前的策略进行评估, 得到价值函数 `V`。然后, ②处代码会根据 `V` 得到贪婪化的策略 `new_pi`。接下来, ③处代码会检查策略是否已被更新。如果没有更新, 则意味着贝尔曼最优方程得到满足, 此时的 `pi` 和 `new_pi` 是最优策略。在这种情况下, 代码会退出 `while` 循环, 返回 `pi`。

现在尝试用 `policy_iter` 函数来解决实际的问题。

ch04/policy_iter.py

```
env = GridWorld()
gamma = 0.9
pi = policy_iter(env, gamma)
```

如图 4-16 所示，运行上面的代码可以直观地看到策略迭代法的每一步的结果。

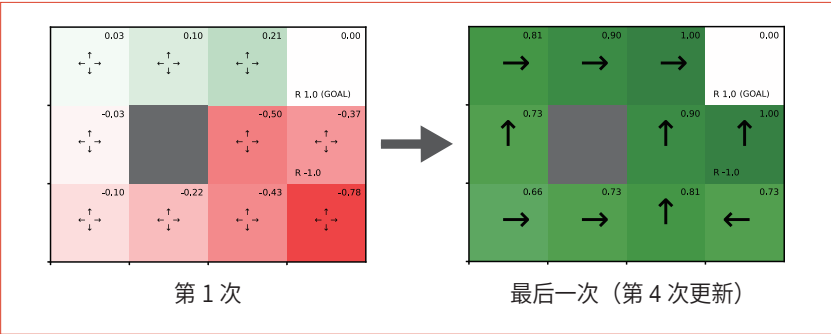


图 4-16 第 1 次以及最后一次价值函数和策略的结果（每个方格中显示了价值函数的值和策略）

如图 4-16 所示，我们从随机性的策略开始，大部分方格的价值函数的值是负的（红色）。不过，随着持续地更新策略，在第 4 次更新后，除了目的地方格，所有方格中的策略都变成了正值（绿色）。另外，观察前进的方向（箭头），可以发现智能代理在每个方格中都朝着避开炸弹和拿走苹果的方向前进。这就是最优策略。

太棒了！我们用策略迭代法得出了最优策略。也就是说，我们已经完全解决了 3×4 网格世界问题。



在 3×4 网格世界问题中，有两个确定性的最优策略：一个是图 4-16 所示的策略；另一个是相对于图 4-16 所示的策略，在左下角的方格中采取“向上”前进的策略。从左下角的方格出发，无论选择“向右”还是“向上”，都能在最短的时间内到达目的地。

4.5 价值迭代法

前面我们使用策略迭代法得到了最优策略。回顾一下，策略迭代法的思路是交替重复“评估”和“改进”这两个阶段，如图 4-17 所示。

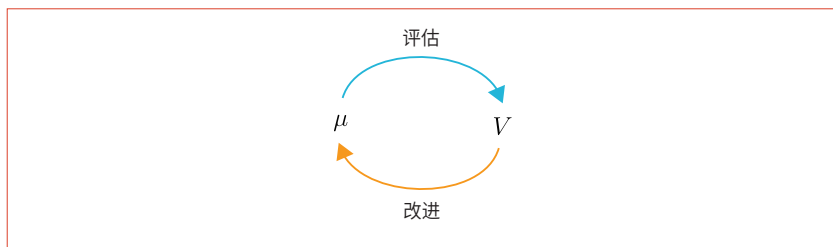


图 4-17 策略迭代法的流程

如前所述，在评估阶段，通过对策略进行评估，可以得到价值函数。随后，在改进阶段，通过对价值函数进行贪婪化，可以得到改进后的策略。通过交替重复这一过程，我们逐渐接近最优策略 μ_* 和最优价值函数 v_* 。图 4-18 展示了这个过程。

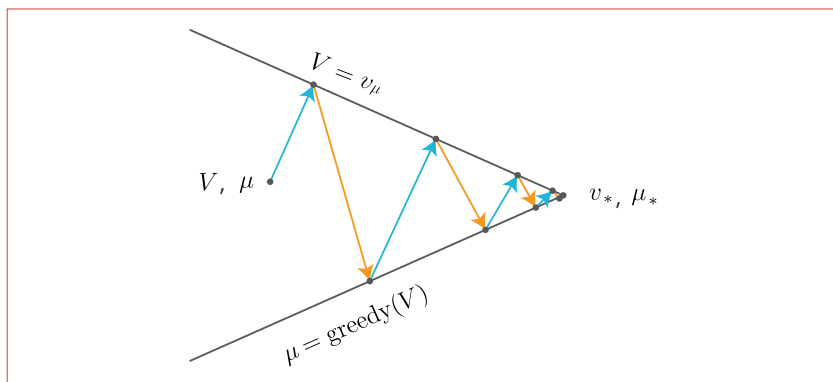


图 4-18 使用策略迭代法时的价值函数和策略的变化（示意图）

在图 4-18 中，价值函数 V 和策略 μ 的取值空间以二维形式表示。尽管该空间本来是多维且复杂的，但为了直观地理解，这里以二维空间来说明。

图 4-18 中有两条直线，其中一条是 $V = v_\mu$ 的直线。这条直线是任意价值函数 V 与策略 μ 的真正的价值函数 v_μ 相等的地方。另一条直线是通过贪婪化价值函数 V 得到的策略与 μ 相等的地方。

策略迭代法在评估和改进之间交替进行。在评估阶段，评估策略 μ ，得到 V_μ ，这相当于移动到图 4-18 中的直线 $V = v_\mu$ 上。在改进阶段，则对 V 进行贪婪化。这相当于移动到图 4-18 中的直线 $\mu = \text{greedy}(V)$ 上。通过这两个任务的交替进行， V 和 μ 被更新，不久就到达了两条直线相交地点的 v_* 和 μ_* 。

 策略在这里被表示为确定性策略 μ ，而不是随机性策略 π 。这是因为对价值函数的贪婪化决定了在每个状态下的行动只有一个。

如图 4-18 所示，策略迭代法在两条直线之间蜿蜒前进以到达目的地。当然，到达目的地的路径有许多可能的变化，例如，图 4-19 所示的轨迹也是可能的。

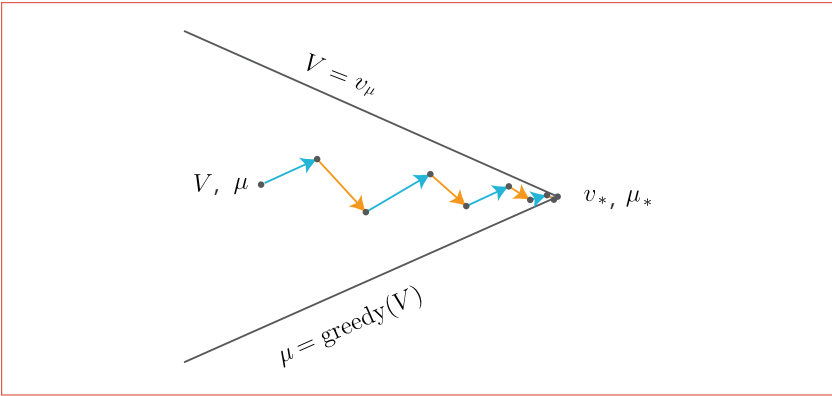


图 4-19 价值函数和策略的变化的另一个例子

图 4-19 中画的轨迹是 Z 字形的，在到达两条直线之前，蜿蜒前进。其实在评估和改进这两个任务之间交替重复进行的算法（框架）中，在完全做完评估之前进入改进阶段，以及在完全做完改进之前进入评估阶段的做法就形成了图 4-19 所示的那种向目的地前进的轨迹。这就是所谓的通用策略迭代（generalized policy iteration）。正如“通用”一词所包含的含义，它

是一种通用的想法，可以在很大范围内应用。



交替重复进行策略评估和策略改进这两个过程的算法在评估和改进的“颗粒度”，即多么精确地评估或改进这一点上有自由裁量的空间。具体来说，在评估阶段，只要 V 越来越接近 v_μ 即可，而不必完全更新到 v_μ 。同样，在改进阶段，只需朝贪婪化的方向推进即可（只有部分进行贪婪化）。

策略迭代法全面地进行了评估和改进。（准确地说，迭代策略评估算法在小于阈值时停止更新，所以没有进行完全正确的评估，但可以说它所做的评估仍然是近似正确的。）这种策略迭代法是广义策略迭代的一个实现。

在策略迭代法中，不仅要最大限度地进行评估和改进，而且要交替切换这两个阶段。那么如果最小限度地进行评估和改进，会发生什么呢？这就引出了接下来我们要介绍的价值迭代法（value iteration）。

4.5.1 价值迭代法的推导

在进入价值迭代法的探讨之前，让我们再次回顾一下策略迭代法。在策略迭代法的评估阶段，价值函数会被重复地更新，如图 4-20 所示。

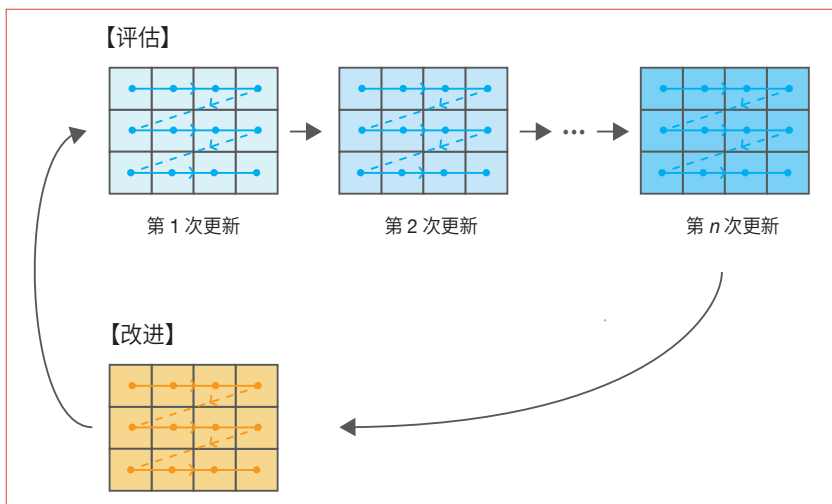


图 4-20 使用迭代策略评估算法进行更新

如图 4-20 所示，所有状态的价值函数被多次进行更新。一旦更新过程收敛，接下来就是“改进”（贪婪化）阶段。与之相反的更新方法是一次只更新一个状态，然后立即进入“改进”阶段。这就是价值迭代法的核心思想。示意图如图 4-21 所示。

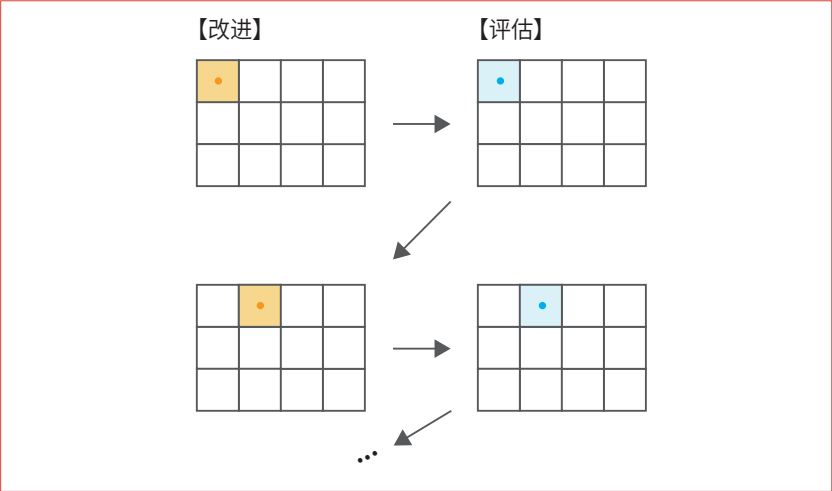


图 4-21 在每个状态下依次重复进行改进和评估的流程

在图 4-21 中，一旦一个状态的改进阶段结束，就会立即转入评估阶段。在评估阶段，一个状态的价值函数只会被更新一次。然后，改进另一个位置（在图 4-21 中是左上角的第二个位置），并对其进行评估，持续进行这个流程。



在图 4-21 中，首先改进的是左上角的位置（状态）。虽然这个例子是从改进阶段开始的，但由于改进和评估这两个阶段是交替进行的，因此不管先进行哪个阶段，都不失普遍性。

下面我们用数学式来表示图 4-21 中的想法。首先从改进阶段开始。在改进阶段进行的贪婪化可以写为如下形式：

$$\mu(s) = \operatorname{argmax}_a \sum_{s'} p(s' | s, a) \{r(s, a, s') + \gamma V(s')\} \tag{4.8}$$

现在，我们用 $V(s)$ 表示当前状态的价值函数。如式 4.8 所示，它基于

当前状态、奖励和下一个状态，通过 $\arg\max$ 进行计算。由于通过 $\arg\max$ 确定了一个行动，因此它可以表示为 $\mu(s)$ 这样的确定性策略。

接下来是评估阶段。如果更新前的价值函数是 $V(s)$ ，更新后的价值函数是 $V'(s)$ ，那么基于 DP 的更新式（迭代策略评估算法）可以用如下形式表示。

$$V'(s) = \sum_{a, s'} \pi(a|s) p(s'|s, a) \{r(s, a, s') + \gamma V(s')\} \quad (4.9)$$

在式 4.9 中，策略 $\pi(a|s)$ 被表示为随机性策略。不过，一旦经过改进阶段之后，策略就可以表示为贪婪策略了。贪婪策略是确定性策略，因为它只选择一个取最大值的行动。因此，式 4.9 中的策略可以被视为确定性策略 $\mu(s)$ 。可以对式 4.9 做如下简化。

当 $a = \mu(s)$ 时

$$V'(s) = \sum_{s'} p(s'|s, a) \{r(s, a, s') + \gamma V(s')\} \quad (4.10)$$

这就是评估阶段的价值函数的更新式。下面继续观察式 4.8 和式 4.10，如图 4-22 所示。

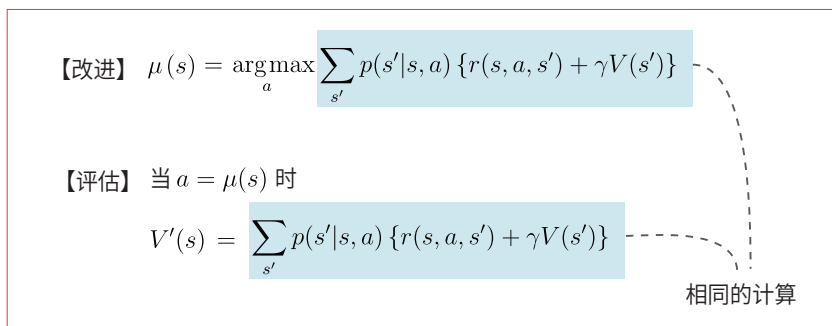


图 4-22 在改进阶段和评估阶段进行的计算

由图 4-22 可知，改进阶段和评估阶段重复进行了相同的计算。更确切地说，在改进阶段进行了找出贪婪行动的计算之后，又使用贪婪行动进行了同样的计算。这个重复的计算当然可以合并为一个。合并后的数学式如下所示。

$$V'(s) = \max_a \sum_{s'} p(s' | s, a) \{r(s, a, s') + \gamma V(s')\} \quad (4.11)$$

如式 4.11 所示，它直接用取最大值的 $\max$ 算子更新价值函数。式 4.11 消除了图 4-22 中的重复计算。另外，在式 4.11 中还值得注意的是策略 μ 没有出现。换句话说，**价值函数的更新没有使用策略**。因此，通过式 4.11 获得最优价值函数的算法叫作“**价值迭代法**”（因为不需要策略，所以名字中不包括“策略”一词）。价值迭代法通过这一个式子进行评估和改进。



贝尔曼最优方程表示为以下式子。

$$v_*(s) = \max_a \sum_{s'} p(s' | s, a) \{r(s, a, s') + \gamma v_*(s')\}$$

将这个贝尔曼最优方程与式 4.11 进行比较，可以看出，式 4.11 是贝尔曼最优方程的“更新式”。

另外，式 4.11 的更新式也可以用如下形式表示。

$$V_{k+1}(s) = \max_a \sum_{s'} p(s' | s, a) \{r(s, a, s') + \gamma V_k(s')\}$$

上面式子中的 V_k 表示更新了 k 次的价值函数， V_{k+1} 表示更新了 $k+1$ 次的价值函数。价值迭代法是从 $k=0$ 开始，按 $k=1, 2, 3, \dots$ 的顺序更新价值函数 V_k 的算法，满足 DP 的特征，即“不进行两次相同的计算”。这意味着 $k=0, 1, 2, 3, \dots$ 时的 V_k 分别只被计算一次（例如， V_3 不会被计算两次）。因此，**价值迭代法是一种基于 DP 的算法**（属于总称为 DP 的算法）。

价值迭代法在**经过无限次更新后会得到最优价值函数**。但在实践中，我们需要在某一点停止更新。实现这个目标的一种方法是**使用阈值**。事先设定阈值，当所有状态的更新量低于阈值时停止更新。

另外，得到 $V_*(s)$ 后，可以通过以下式子得到最优策略 $\mu_*(s)$ 。

$$\mu_*(s) = \operatorname{argmax}_a \sum_{s'} p(s' | s, a) \{r(s, a, s') + \gamma V_*(s')\} \quad (4.12)$$

如式 4.12 所示，找到的贪婪策略就是最优策略。这就是价值迭代法。

4.5.2 价值迭代法的实现

接下来是价值迭代法的实现。这次还是解决 3×4 网格世界的问题。由于这个问题的状态迁移是确定性的，因此价值函数的更新式子可以简化，如图 4-23 所示。

$$V'(s) = \max_a \sum_{s'} p(s'|s, a) \{r(s, a, s') + \gamma V(s')\} \quad (4.11)$$

↓ 状态迁移是确定性的

$$\text{当 } s' = f(s, a) \text{ 时}$$

$$V'(s) = \max_a \{r(s, a, s') + \gamma V(s')\} \quad (4.13)$$

图 4-23 在状态迁移是确定性的情况下，使用价值迭代法的更新式

下面开始实现。首先实现根据式 4.13 只进行一次更新的函数。代码如下所示。

ch04/value_iter.py

```
def value_iter_onestep(V, env, gamma):
    for state in env.states(): # ①访问所有状态
        if state == env.goal_state: # 目的地的价值函数总是为 0
            V[state] = 0
            continue

        action_values = []
        for action in env.actions(): # ②访问所有行动
            next_state = env.next_state(state, action)
            r = env.reward(state, action, next_state)
            value = r + gamma * V[next_state] # ③新的价值函数
            action_values.append(value)

        V[state] = max(action_values) # ④取出最大值
    return V
```

`value_iter_onestep` 函数接收的参数是价值函数 `V`、环境 `env` 和折现率 `gamma`。代码①处会依次访问所有状态，代码②处会依次访问所有行动。代

码③处会计算式 4.13 中 $\max$ 算子的内容，代码④处会取最大值并设置为 $V[\text{state}]$ 。

后面要反复调用这个 `value_iter_onestep` 函数，直到更新收敛。完成这个任务的函数被实现为 `value_iter`，代码如下所示。

ch04/value_iter.py

```
def value_iter(V, env, gamma, threshold=0.001, is_render=True):
    while True:
        if is_render:
            env.render_v(V)

        old_V = V.copy() # 更新前的价值函数
        V = value_iter_onestep(V, env, gamma)

        # 求更新量的最大值
        delta = 0
        for state in V.keys():
            t = abs(V[state] - old_V[state])
            if delta < t:
                delta = t
        # 与阈值比较
        if delta < threshold:
            break
    return V
```

`value_iter` 函数会重复调用前面实现的 `value_iter_onestep` 函数。调用次数是由价值函数的更新量来决定的。具体来说，更新会持续进行，直到价值函数的最大更新量小于阈值（`threshold`）。`value_iter` 函数还需要一个布尔值 `is_render` 作为参数。如果 `is_render=True`，那么在 `while` 循环中每次更新时都会画出价值函数的值。

下面使用 `value_iter` 函数。代码如下所示。

ch04/value_iter.py

```
from common.gridworld import GridWorld
from ch04.policy_iter import greedy_policy

V = defaultdict(lambda: 0)
env = GridWorld()
gamma = 0.9

V = value_iter(V, env, gamma)
```

```
pi = greedy_policy(V, env, gamma)
env.render_v(V, pi)
```

先使用 `value_iter` 函数得到最优价值函数。得到最优价值函数之后，可以对其进行贪婪化，得到最优策略（参见式 4.12）。这部分已经在函数 `greedy_policy` 中实现了。运行上面的代码可以得到图 4-24 所示的结果。

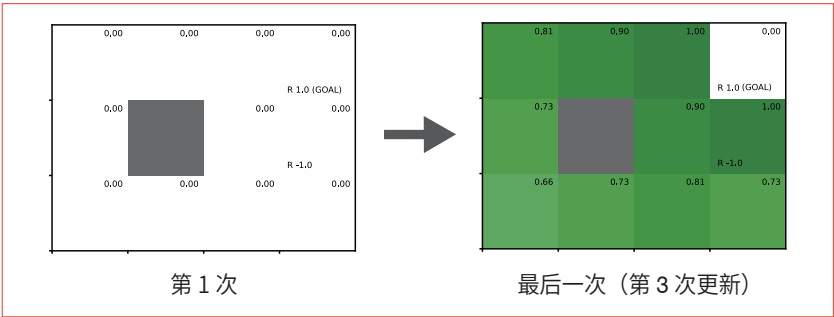


图 4-24 使用价值迭代法的第 1 次和最后一次的价值函数

价值函数的更新从一个最初所有元素都是 0 的字典开始。随后价值函数的值会在 3 次更新后收敛，如图 4-24 所示。这就是最优状态的价值函数。在最优状态价值函数的基础上得到的贪婪策略如图 4-25 所示。

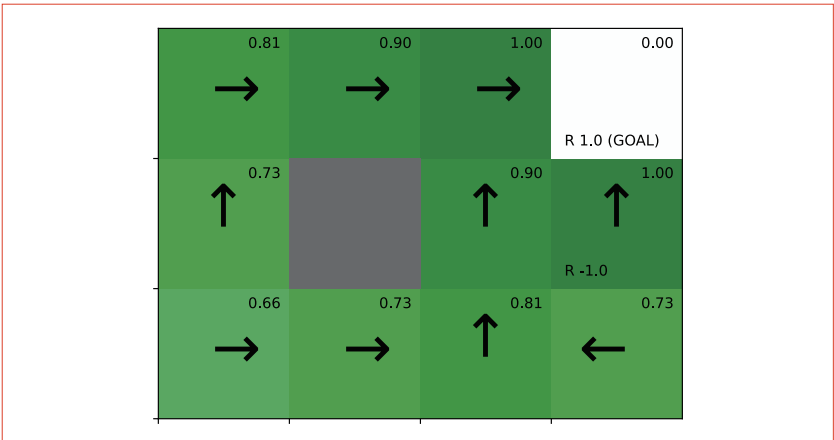


图 4-25 基于最优状态价值函数得到的最优策略

图 4-25 是基于最优状态价值函数得到的贪婪策略，也就是最优策略。我们得到的的确是向右上角奖励为 1.0 的“苹果”移动的策略。通过使用价值迭代法，我们高效地得到了最优策略。

4.6 小结

在本章中，我们学习了如何使用 DP 来获得最优策略。具体的算法是策略迭代法和价值迭代法。策略迭代法在评估和改进这两个阶段之间交替重复进行。在评估阶段，使用 DP 对价值函数进行评估。在评估完价值函数后，就可以通过对价值函数贪婪化来改进策略。如果不能改进，那么说明这个策略就是最优策略。

价值迭代法融合了评估阶段和改进阶段。具体来说，价值函数的更新只需要下面一个式子。通过更新，可以得到最优价值函数（只要知道了最优价值函数，就能得到最优策略）。

$$V_{k+1}(s) = \max_a \sum_{s'} p(s' | s, a) \{r(s, a, s') + \gamma V_k(s')\}$$

本章实现了策略迭代法和价值迭代法这两个算法，在将它们应用于网格世界问题后，我们成功得到了最优策略。

第 5 章

蒙特卡洛方法

在第 4 章中，我们使用 DP 得到了最优价值函数和最优策略。但要使用这个方法，环境模型（状态迁移概率和奖励函数）必须是已知的。遗憾的是，有些问题的环境模型可能是未知的。即使环境模型已知，但由于涉及大量的计算，使用 DP 方法往往也是不切实际的。强化学习领域主要处理的是在环境模型未知的情况下寻找更好的策略的问题。要做到这一点，智能代理需要实际采取行动并从经验中学习。

本章的主题是蒙特卡洛方法（Monte Carlo method）。蒙特卡洛方法是对数据进行反复采样并根据结果进行估计的方法的总称。在强化学习中，蒙特卡洛方法可以用来从经验中估计价值函数。这里的“经验”是指从环境和智能代理之间的实际互动中获得的数据。具体来说，经验是一系列关于状态、行动和奖励的数据。本章的目标是根据智能代理获得的经验来估计价值函数。实现这个目标之后，我们再去了解找到最优策略的方法。

从本章开始，我们将进入真正的强化学习领域。前面我们花时间学习了强化学习的重要基础知识。在学到的这些知识的基础上，我们自然而然就能理解强化学习的问题和使用蒙特卡洛方法解决问题的手法。

5.1 蒙特卡洛方法的基础知识

之前我们处理的都是环境模型已知的问题。例如，在网络世界问题中，对于智能代理的行动，它的下一个迁移目的地和奖励都是已知的。用数学

式来说，就是状态迁移概率 $p(s'|s,a)$ 和奖励函数 $r(s,a,s')$ 是已知的。（如果状态迁移是确定性的，那么它们也可以用函数 $s' = f(s,a)$ 来表示。）对于这种环境模型已知的问题，我们可以在智能代理侧对状态、行动和奖励的迁移进行模拟。

不过对于现实生活中的许多问题，我们并不知道它们的环境模型。以商品库存管理的问题为例，在这个问题中，售出多少商品对应于环境的状态迁移概率。但在现实生活中完全掌握这个数据是不可能的，因为商品的销售是由各种因素的复杂的相互作用决定的。

此外，即使理论上能够知道环境的状态迁移概率，我们往往也需要进行大量的计算或编码。为了感受它的复杂，本节首先以骰子为例来研究一些简单的任务。

5.1.1 骰子的点数和

我们来思考掷两个骰子的问题。假设骰子的每个面出现的概率是 $\frac{1}{6}$ 。我们把两个骰子掷出的点数和表示为概率分布，因此可以画出一张图 5-1 所示的迁移图。

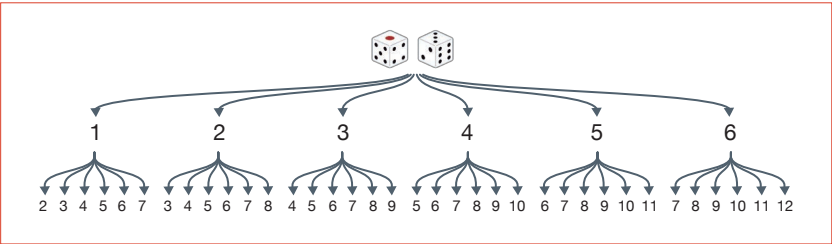


图 5-1 两个骰子掷出的点数和的迁移图（第一个迁移表示第一个骰子掷出的点数，第二个迁移表示两个骰子掷出的点数和）

如果画出的是像图 5-1 那样的迁移图，那么图的底部出现的就是所有的骰子掷出的点数和。这些和一共有 36 个，对它们进行汇总，即可得到概率分布。例如，在图 5-1 中有两种情况的点数和是 3，所以 3 出现的概率是 $\frac{2}{36}$ 。将所有情况汇总，可以得到图 5-2 所示的概率分布。

骰子的点数和	2	3	4	5	6	7	8	9	10	11	12
概率	$\frac{1}{36}$	$\frac{2}{36}$	$\frac{3}{36}$	$\frac{4}{36}$	$\frac{5}{36}$	$\frac{6}{36}$	$\frac{5}{36}$	$\frac{4}{36}$	$\frac{3}{36}$	$\frac{2}{36}$	$\frac{1}{36}$

图 5-2 两个骰子掷出的点数和的概率分布

图 5-2 是两个骰子掷出的点数和的概率分布。下面用这个概率分布来计算期望值。代码实现如下所示。

```
ps = {2: 1/36, 3: 2/36, 4: 3/36, 5: 4/36, 6: 5/36, 7: 6/36,
      8: 5/36, 9: 4/36, 10: 3/36, 11: 2/36, 12: 1/36}

V = 0
for x, p in ps.items():
    V += x*p
print(V) # 6.999999999999999
```

上面的代码根据输入的概率分布（骰子的点数和以及概率）求期望值。结果是 6.999...（这次由于数值计算的误差，结果不是正确的值 7）。只要知道概率分布，就可以像上面那样计算出期望值。



之所以在这里计算期望值，是因为在强化学习中，期望值的计算是主要任务。回顾一下，强化学习的目标是使收益最大化，而收益被定义为奖励总和的期望值。

5.1.2 分布模型和样本模型

我们已经将骰子的点数和表示为概率分布。换句话说，我们已经把掷骰子的试验建模为概率分布。可以将这样一个表示概率分布的模型称为分布模型（distribution model）。分布模型并不是表示模型的唯一方法。

除了分布模型，样本模型（sample model）也是表示模型的方法。样本模型是只需要采样的模型。拿骰子的例子来说，采样的意思是实际掷骰子并观测骰子的点数和。分布模型要求明确地持有概率分布，样本模型则要求能够进行采样，如图 5-3 所示。

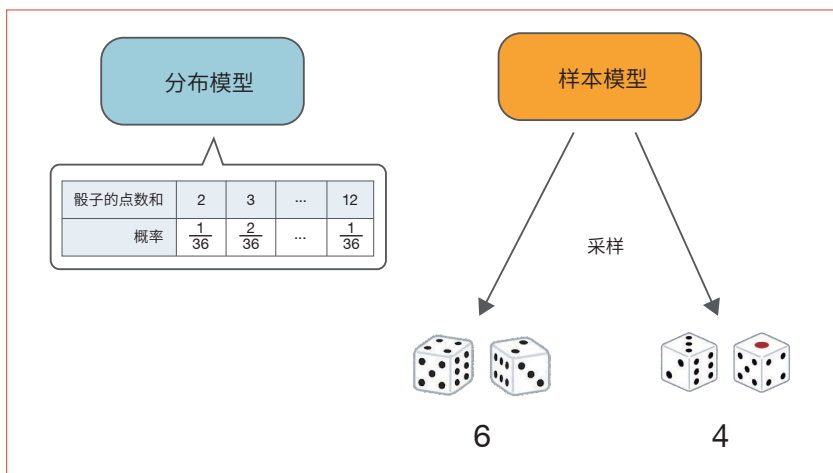


图 5-3 分布模型（左）和样本模型（右）的例子

图 5-3 中的样本模型使用的是具体的样本数据，比如 6 或 4。在样本模型中，不需要显式地提供概率分布，唯一的要求是可以进行采样。但是，如果重复进行采样，那么采样的概率分布就应该像图 5-2 那样。

下面来实际地实现采样模型。和前面一样来实现两个骰子的情况，这次使用的是样本模型。代码如下所示。

ch05/dice.py

```
import numpy as np

def sample(dices=2):
    x = 0
    for _ in range(dices):
        x += np.random.choice([1, 2, 3, 4, 5, 6])
    return x
```

简要地介绍一下代码。首先，`np.random.choice([1, 2, 3, 4, 5, 6])` 以相同的概率从列表 `[1, 2, 3, 4, 5, 6]` 中选择一个元素。这是掷出一个骰子的结果（样本数据）。由于要掷两个骰子，因此这个操作要做两次。下面来试用一下这里实现的 `sample` 方法。

```
print(sample()) # 10
print(sample()) # 4
print(sample()) # 8
```

如上面的代码所示，每次运行的结果都会发生变化。这样我们就能成功地将两个骰子实现为样本模型。实现起来很容易，因为不需要准备概率分布。

另外，有了这个样本模型，我们也可以很容易地实现有 10 个骰子的情况。在这种情况下，只需通过 `sample(10)` 修改参数即可。而在分布模型中实现 10 个骰子并不容易。如果只是单纯地想做到这一点，则需要考虑 $6^{10} = 60\,466\,176$ 种组合。

下面使用样本模型来计算期望值。具体的方法就是做大量的采样并取平均值。这就是蒙特卡洛方法。输入数值，再取平均值，虽然方法很简单，但由于样本的数量是无限的，根据大数法则，它们的平均值会收敛到正确的值。



在第 1 章的老虎机问题中，我们通过实际玩老虎机，根据平均值来估计老虎机的好坏（价值）。当时所用的方法正是蒙特卡洛方法。本章的主题是将已经在老虎机问题中用过的蒙特卡洛方法应用于强化学习问题。

5.1.3 蒙特卡洛方法的实现

下面试着编写一下用蒙特卡洛方法求期望值的代码。

```
trial = 1000

samples = []
for _ in range(trial):
    s = sample()
    samples.append(s)

V = sum(samples) / len(samples) # 求平均值
print(V)
```

输出结果

6.98

上面的代码在采样 1000 次后找到了平均值。代码将每次试验的结果添加到列表 `samples` 中，最后求出了平均值。上面的结果显示为 6.98，与正确答案 7 相比，可以说这是基本上正确的值。虽然每次运行的结果不尽相同，但都是近似于 7 的值。



随着样本数量的增加，蒙特卡洛方法变得更加可信。拿专业术语来说，这就是方差变小。直观地说，方差就是与正确答案的偏差的范围。5.5.2 节会更详细地介绍方差。

下面思考每次获得样本数据时求平均值的情况。这种情况可以简单地通过以下代码来实现。

```
trial = 1000

samples = []
for _ in range(trial):
    s = sample()
    samples.append(s)
V = sum(samples) / len(samples) # 每次都计算平均值
print(V)
```

和之前一样，代码会将样本数据添加到列表中，然后使用列表求平均值。这一次的平均值计算是在 `for` 语句中进行的。这样做虽然是正确的，但我们也可以考虑更高效的实现。这就是我们在 1.3.2 节中学到的“增量式实现”。这里只介绍实现的要点，如图 5-4 所示。

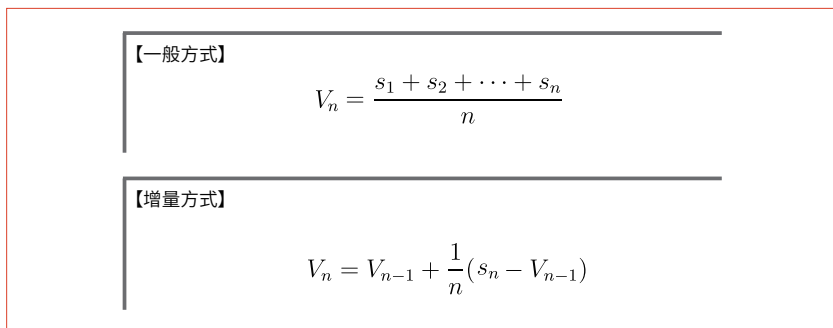


图 5-4 求平均值的计算式比较（获得的第 n 个样本数据用 s_n 表示，获得 n 个样本数据后的价值函数用 V_n 表示）

求平均值的计算既可以用图 5-4 中上面的式子（一般方式）也可以用下面的式子（增量方式），二者的结果是相同的。如果每次获得样本数据时都计算平均值，那么增量方式的效率更高。下面用增量方式来求平均值。代码如下所示。

ch05/dice.py

```
trial = 1000
V, n = 0, 0

for _ in range(trial):
    s = sample()
    n += 1
    V += (s - V) / n # 或者 V = V + (s - V) / n
print(V)
```

输出结果

```
4.0
6.0
5.333333333333333
...
6.959959959959965
6.960000000000005
```

上面的代码在运行过程中，会在每次获得样本数据时计算平均值。结果显示，随着样本数据的增加，平均值逐渐接近正确答案 7。

以上就是对蒙特卡洛方法的基础知识的介绍。接下来，我们将把蒙特卡洛方法应用于强化学习问题。

5.2 使用蒙特卡洛方法评估策略

在 5.1 节中，我们学习了蒙特卡洛方法。蒙特卡洛方法可以进行实际采样，并从样本数据中计算出期望值。当然，这种方法也可以应用于强化学习问题。具体来说，我们可以从智能代理在实际行动中获得的经验（样本数据）估计价值函数。这里通过蒙特卡洛方法计算给定策略 π 的价值函数。本节只进行“策略评估”，而寻找最优策略的“策略控制”将在后续章节中介绍。

5.2.1 使用蒙特卡洛方法计算价值函数

先来回顾一下价值函数。价值函数的式子如下所示。

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G | s] \quad (5.1)$$

式 5.1 中的 G 表示从状态 s 开始获得的收益（收益是带有折现率的奖励的总和）。如式 5.1 所示，价值函数 $v_{\pi}(s)$ 被定义为根据策略 π 行动时获得的收益的期望值。让我们思考这样一个场景：有一个回合制任务，智能代理在某一时刻到达了目的地。

下面使用蒙特卡洛方法计算式 5.1 的价值函数。我们让智能代理根据策略 π 实际采取行动。这样得到的实际收益就是样本数据。蒙特卡洛方法会收集大量这样的样本数据，并计算出它们的平均值。这个计算的数学式如下所示。

$$V_{\pi}(s) = \frac{G^{(1)} + G^{(2)} + \dots + G^{(n)}}{n} \quad (5.2)$$

这里用 G 表示从状态 s 开始获得的收益，用 $G^{(i)}$ 表示在第 i 回合获得的收益。为了用蒙特卡洛方法计算，我们要像式 5.2 所示那样运行 n 个回合，并计算得到的样本数据的平均值。



蒙特卡洛方法只能用于回合制任务。由于连续性任务没有“结束”，因此式 5.2 中的 $G^{(1)}$ 、 $G^{(2)}$ 等收益的样本数据将无法确定。

下面结合具体的例子来看看蒙特卡洛方法。思考智能代理采取图 5-5 所示的行动的情况。

图 5-5 描述了智能代理从状态 s 开始根据策略 π 采取行动的结果。在图 5-5 的例子中，获得的奖励分别为 1、0 和 2。如果这个任务的折现率 γ 是 1，那么在状态 s 下获得的收益如下所示。

$$G^{(1)} = 1 + 0 + 2 = 3$$

这是第一个样本数据。此时估计的价值函数如下所示。

$$V_{\pi}(s) = G^{(1)} = 3$$

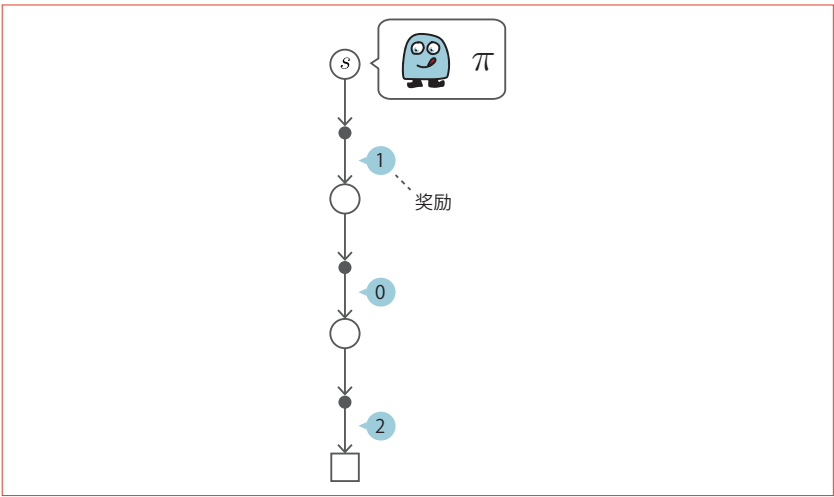


图 5-5 智能代理的试验 (○：状态；●：行动；□：目的地，数值表示实际获得的奖励)

接下来进行第 2 次试验，如图 5-6 中的右图所示。

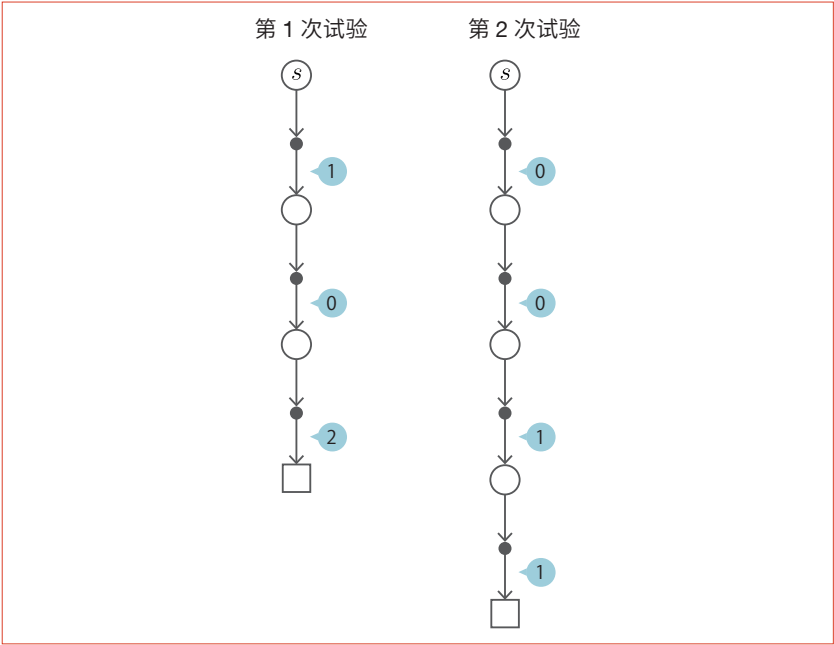


图 5-6 智能代理的第 2 次试验

这次从状态 s 开始获得的奖励分别为 0、0、1 和 1。尽管这次也是从状态 s 开始，但获得的奖励不同。这是因为智能代理的策略可能是随机性的，环境中的状态迁移也可能是随机性的。如果二者中任何一个随机性的，那么每次试验获得的奖励将随机变化。对于这种随机变化的数值（奖励），可以使用蒙特卡洛方法。

根据图 5-6，第 2 次试验的奖励如下所示。

$$G^{(2)} = 0 + 0 + 1 + 1 = 2$$

第 1 次试验的奖励 $G^{(1)}$ 是 3，第 2 次试验的奖励 $G^{(2)}$ 是 2，它们的平均值如下所示：

$$\frac{G^{(1)} + G^{(2)}}{2} = \frac{3 + 2}{2} = 2.5$$

因此，此时的 $V_{\pi}(s)$ 是 2.5。

像这样通过实际行动并计算收益的平均值，就可以近似计算 $V_{\pi}(s)$ 了。近似的精度随着试验次数的增加而增加。

5.2.2 求所有状态的价值函数

到目前为止，我们只是关注一个状态，并使用蒙特卡洛方法求出了该状态的价值函数。接下来我们将尝试求所有状态的价值函数。简单来讲，这可以通过改变起始状态，并重复之前的流程来计算。如果有 A 、 B 、 C 共 3 个状态，那么我们可以像图 5-7 那样求出每个状态的价值函数。

如图 5-7 所示，我们会收集从每个状态开始并采取实际行动的样本数据，然后通过计算各状态的收益的平均值来求价值函数。但这种方法在计算效率上存在问题。更确切地说，在每个状态的价值函数是可以独立求得的这一点上还有待改进。例如，从状态 A 开始获得的收益（样本数据）仅用于计算 $V_{\pi}(A)$ ，而不能计算其他价值函数。

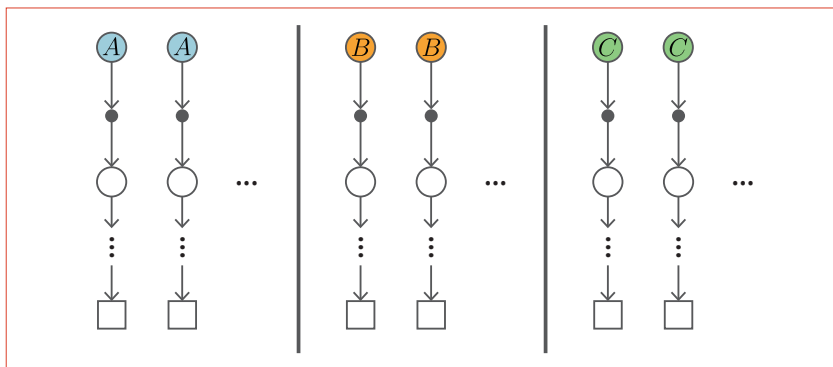


图 5-7 求从每个状态开始的收益的示例



图 5-7 所示的方法只能应用于可以从任意状态开始的问题。例如，在游戏或基于模拟器的问题中，智能代理可能会从任意位置开始。但对于现实生活中的问题，从任意位置开始几乎是不可能的。

下面让我们考虑更高效的方法。先来思考一下图 5-8 所示的例子。

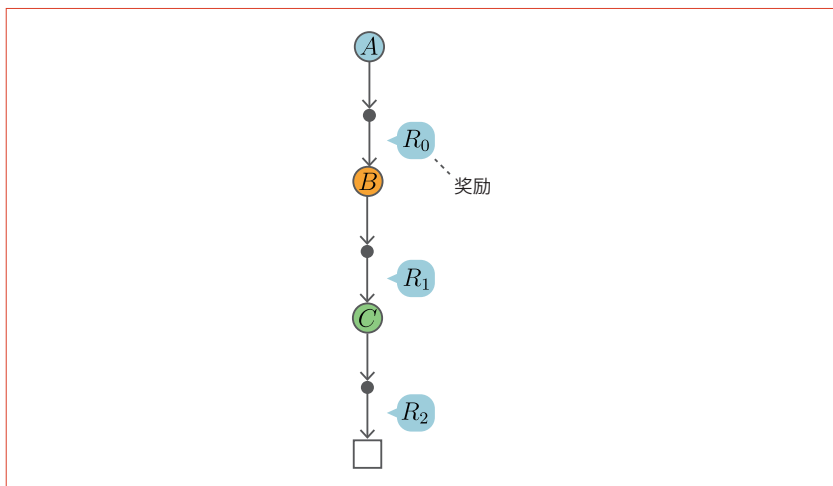


图 5-8 从状态 A 开始并根据策略 π 行动的例子

图 5-8 显示了从状态 A 开始根据策略 π 采取行动的结果。假设智能代理依次经过状态 A 、 B 和 C 并到达了目的地，在这个过程中获得的奖励分

别为 R_0 、 R_1 和 R_2 。如果这个任务的折现率为 γ ，那么从状态 A 开始时获得的收益如下所示。

$$G_A = R_0 + \gamma R_1 + \gamma^2 R_2$$

这是以状态 A 为起始位置时获得的收益。下面把注意力转移到从状态 B 向下的迁移，如图 5-9 所示。

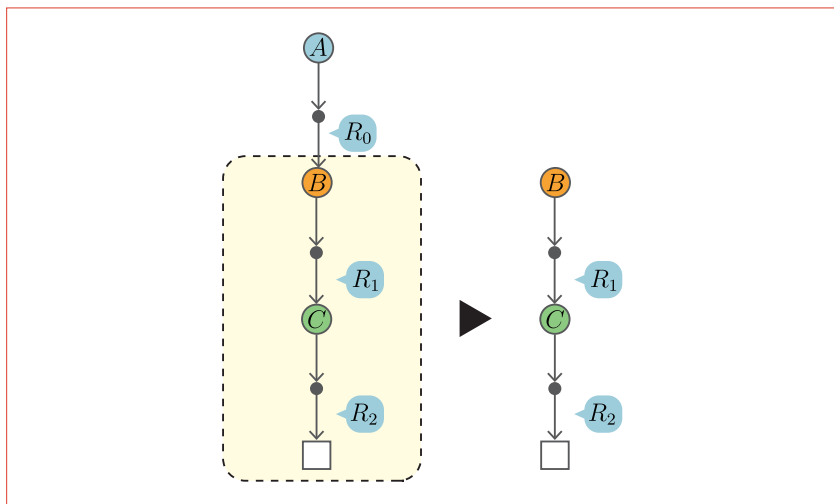


图 5-9 从状态 B 向下的迁移

图 5-9 中的数据可被视为从状态 B 开始时获得的收益的样本数据。在这种情况下，由于获得的奖励是 R_1 和 R_2 ，因此从状态 B 开始时获得的收益如下所示。

$$G_B = R_1 + \gamma R_2$$

同样，从状态 C 开始时获得的收益如下所示。

$$G_C = R_2$$

通过这样的计算，我们仅从图 5-8 中的一次实验就获得了 3 个状态的收益（样本数据）。



即使智能代理的起始位置是固定的，如果在回合重复进行的过程中经历所有的状态，那么我们也能收集所有状态收益的样本数据。如果智能代理采取随机性策略，随着回合的重复进行而迁移到各种状态，那么智能代理就有可能经由所有的状态。在这种情况下，没有必要将智能代理的起始状态设置为任意的位置。

5.2.3 蒙特卡洛方法的高效实现

最后补充说明一下高效计算收益的方法。从图 5-8 的例子来看，我们需要计算以下 3 个收益。

$$G_A = R_0 + \gamma R_1 + \gamma^2 R_2$$

$$G_B = R_1 + \gamma R_2$$

$$G_C = R_2$$

这些计算虽然不难，但仍然有可以改进的空间。要做到这一点，需要先将它们变形为如下形式。

$$G_A = R_0 + \gamma G_B$$

$$G_B = R_1 + \gamma G_C$$

$$G_C = R_2$$

上面式子中值得注意的是，在计算 G_A 的式子中使用了 G_B ，而且在计算 G_B 的式子中使用了 G_C 。利用好这种模式，就可以避免重复的计算。要做到这一点，需要按照从后往前，也就是 G_C 、 G_B 、 G_A 的顺序进行计算。

$$G_C = R_2$$

$$G_B = R_1 + \gamma G_C$$

$$G_A = R_0 + \gamma G_B$$

在上面这些式子中，首先求 G_C ，然后使用 G_C 求 G_B ，最后使用 G_B 求 G_A 。像这样按照从后往前的顺序求收益，就可以省略重复计算。以上是对使用蒙特卡洛方法进行策略评估的介绍。

5.3 蒙特卡洛方法的实现

这次我们尝试使用蒙特卡洛方法解决第4章中解决过的 3×4 网格世界问题（参见图 5-10）。

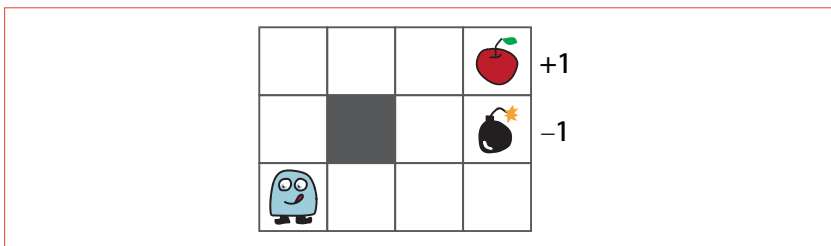


图 5-10 3×4 网格世界

这次的策略评估在不使用环境模型（状态迁移概率和奖励函数）的情况下进行。要做到这一点，需要有使智能代理实际采取行动的方法。本书在 `GridWorld` 类中提供了一个名为 `step` 的方法。下面先来介绍 `step` 方法的使用方法。

5.3.1 step 方法

`GridWorld` 类中有一个 `step` 方法。这个 `step` 方法可以用来使智能代理采取行动。使用示例如下所示。

```
from common.gridworld import GridWorld

env = GridWorld()
action = 0 # 虚拟的行动
next_state, reward, done = env.step(action)

print('next_state:', next_state)
print('reward:', reward)
print('done:', done)
```

输出结果

```
next_state: (1, 0)
reward: 0.0
done: False
```

step 方法会将行动作为参数。env.step(action) 这行代码会对环境执行行动 action，执行结果是得到 next_state、reward 和 done 这 3 个值。next_state 是下一个状态，reward 是奖励，done 是回合是否结束的标志位。结合图 5-11，可以更清晰地了解 state、action、reward 和 next_state 之间的关系。

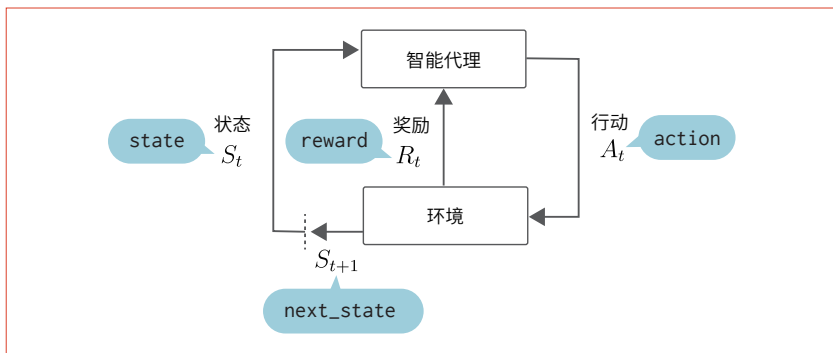


图 5-11 代码和数学式之间的对应关系

如图 5-11 所示，设当前时刻为 t ， S_t 对应于 state， A_t 对应于 action。智能代理在时刻 t 采取行动，获得奖励 R_t ，并进入下一个状态 S_{t+1} 。此时获得的奖励 R_t 对应于 reward，下一个状态 S_{t+1} 对应于 next_state。



在 3×4 网格世界中，状态迁移是确定性的，但也有随机性的情况（例如，当智能代理采取向右前进的行动时，有 80% 的概率会向右移动，20% 的概率会留在原地）。在随机性状态迁移的情况下，即使在同一状态下执行相同的行动，step 方法每次返回的值也不同。

GridWorld 类通过 step 方法使智能代理采取行动，以获得样本数据。GridWorld 类还提供了 reset 方法。这个方法被用来将环境重置为初始状态。下面是 reset 方法的使用示例。

```
env = GridWorld()
state = env.reset()
```

如上述代码所示，reset 方法会返回初始状态 (state)。以上是对 GridWorld 类的补充说明。

5.3.2 智能代理类的实现

接下来实现使用蒙特卡洛方法评估策略的智能代理。假设该智能代理按照随机性策略采取行动。这里将智能代理实现为 `RandomAgent` 类。下面是代码的前半部分。

ch05/mc_eval.py

```
class RandomAgent:
    def __init__(self):
        self.gamma = 0.9
        self.action_size = 4

        random_actions = {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25}
        self.pi = defaultdict(lambda: random_actions)
        self.V = defaultdict(lambda: 0)
        self.cnts = defaultdict(lambda: 0)
        self.memory = []

    def get_action(self, state):
        action_probs = self.pi[state]
        actions = list(action_probs.keys())
        probs = list(action_probs.values())
        return np.random.choice(actions, p=probs)
```

初始化的 `__init__` 方法设置了折现率 `gamma` 和行动的个数 `action_size`。然后创建了随机行动的概率分布 `random_actions`，并将其设置为策略 `self.pi`。`self.V` 表示价值函数，`self.memory` 持有从智能代理的实际行动中获得的经验（状态、行动、奖励这三者的数据列）。`self.cnts` 被用在通过增量式实现求收益的平均值的处理上。

接下来是 `get_action(self, state)` 方法。这个方法取出了在 `state` 下采取的一个行动。重要的部分是 `np.random.choice(actions, p=probs)`。这行代码根据 `probs` 的概率分布对行动进行了一次采样。



允许智能代理选择行动，且行动可以被采样是蒙特卡洛方法的前提条件，所以即使不像 `self.pi` 这样保持行动的概率分布，也有其他方法可以做到这一点。这里使用的智能代理实现方法是基于分布模型的，还可以考虑使用样本模型的方法。6.5 节将介绍通过智能代理的样本模型来实现的方法。

下面是 RandomAgent 类的其余代码（后半部分）。

ch05/mc_eval.py

```
class RandomAgent:
    ...
    def add(self, state, action, reward):
        data = (state, action, reward)
        self.memory.append(data)

    def reset(self):
        self.memory.clear()

    def eval(self):
        G = 0
        for data in reversed(self.memory): # 反向遍历
            state, action, reward = data
            G = self.gamma * G + reward
            self.cnts[state] += 1
            self.V[state] += (G - self.V[state]) / self.cnts[state]
```

先来介绍一下用于记录实际采取的行动和奖励的 add 方法。当这个方法被调用时，“状态、行动、奖励”会被汇总到 (state, action, reward) 元组，之后会被添加到列表 self.memory 中。关于汇总到元组的部分，这里再做一些补充说明。例如，思考获得以下时间序列数据的情况。

$$S_0, A_0, R_0, S_1, A_1, R_1, \dots, S_8, A_8, R_8, S_9$$

在获得以上数据后，代码中像下面这样保存数据。

```
# agent.memory
[(S0, A0, R0), (S1, A1, R1), ..., (S8, A8, R8)]
```

由于数据是以 (state, action, reward) 为单位汇总的，因此在代码中会呈现上面的形式。需要注意的是，最后的状态（本例中的 S_9 ）没有保存在 self.memory 的列表中。不添加最后的状态的原因是最后的状态（目的地中的状态）的价值函数总是为 0。换句话说，由于不必更新最后的状态的价值函数，因此就没有将其添加到 self.memory 中。

RandomAgent 类中执行蒙特卡洛方法的是 eval 方法。这个方法首先将收益 G 初始化为 0，然后反向遍历实际获得的 self.memory，计算在每个状

态下获得的收益。而每个状态下的价值函数是通过计算之前获得的收益的平均值得到的。这里的平均值的计算是通过增量方式实现的。以上就是对 RandomAgent 类的介绍。

5.3.3 运行蒙特卡洛方法

下面将智能代理的 RandomAgent 类与环境的 GridWorld 类联合起来运行。代码如下所示。

ch05/mc_eval.py

```
env = GridWorld()
agent = RandomAgent()

episodes = 1000
for episode in range(episodes):
    state = env.reset()
    agent.reset()

    while True:
        action = agent.get_action(state)
        next_state, reward, done = env.step(action)

        agent.add(state, action, reward)
        if done:
            agent.eval()
            break

        state = next_state

env.render_v(agent.V)
```

上面的代码运行了 1000 个回合。回合开始后，代码重置环境和智能代理，让智能代理在 while 循环中采取行动。代码会记录在这个过程中获得的“状态、行动、奖励”的样本数据。到达目的地后，代码再根据获得的样本数据，用蒙特卡洛方法更新价值函数。最后，代码退出 while 循环并开始下一个回合。在 1000 个回合结束之后，由 `env.render_v(agent.V)` 将价值函数可视化。运行上述代码，可以得到图 5-12 所示的价值函数。

这次评估的是随机性策略的价值函数。虽然智能代理的起始位置被固定在左下角，但由于随机性策略，它要经过所有的位置。这样就能评估所

有位置（状态）的价值函数。

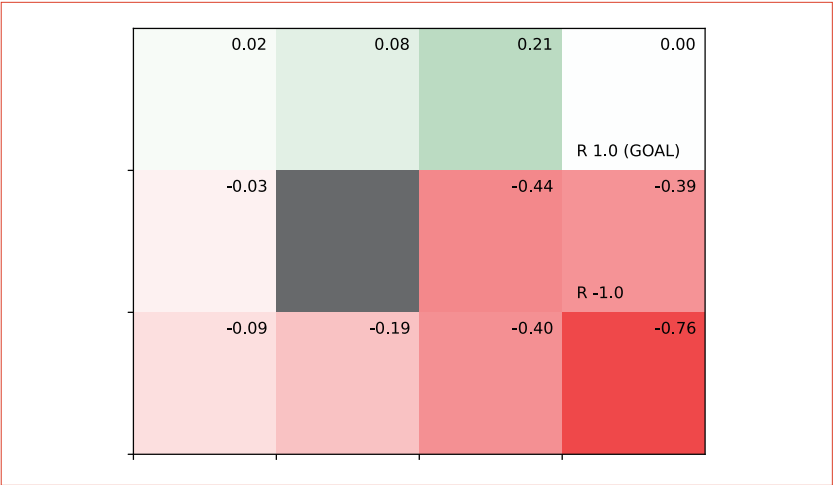


图 5-12 通过蒙特卡洛方法得到的价值函数



如果智能代理的起始位置是固定的，且智能代理的策略是确定性的，那么智能代理将只经过某些确定的状态。在这种情况下，我们不可能收集所有状态的收益的样本数据。

顺便来看一下图 5-12 中的结果与使用动态规划法（DP）评估的结果有什么不同，如图 5-13 所示。

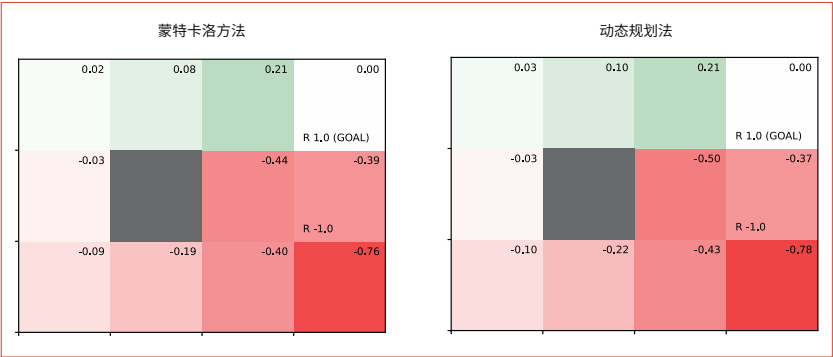


图 5-13 通过蒙特卡洛方法得到的价值函数（左）与通过动态规划法得到的价值函数（右）

图 5-13 中右图的动态规划法的结果是正确的，不过其与使用蒙特卡洛方法的结果基本相同。这意味着即使不知道环境模型，我们依然能够正确地评估这些策略。

5.4 使用蒙特卡洛方法的策略控制

在 5.3 节中，我们使用蒙特卡洛方法进行了策略评估。策略评估之后是寻找最优策略的策略控制。本节将介绍使用蒙特卡洛方法的策略控制。不过并没有多少新的东西需要学习。我们已经在 4.3 节中学习了其关键思想，也就是交替重复进行评估和改进。下面先来回顾一下这些内容。

5.4.1 评估和改进

最优策略是交替重复进行评估和改进而得到的。在评估阶段，对策略进行评估以获得价值函数。在改进阶段，通过价值函数的贪婪化来改进策略。通过交替重复这两个过程，我们逐渐接近最优策略和最优价值函数。

例如，在评估阶段，给定策略 π ，通过蒙特卡洛方法可以得到 $V_\pi(s)$ 。下一步是改进阶段。在改进阶段，我们可以进行贪婪化。这个阶段的数学式如下所示。

$$\mu(s) = \operatorname{argmax}_a Q(s, a) \quad (5.3)$$

$$= \operatorname{argmax}_a \sum_{s'} p(s' | s, a) \{r(s, a, s') + \gamma V(s')\} \quad (5.4)$$

改进阶段会选择价值函数值最大的行动（本书中称之为“贪婪化”）。对于 Q 函数（行动价值函数）的情况，则选择 Q 函数值最大的行动，如式 5.3 所示。由于此时只确定了一个行动，因此可以用函数 $\mu(s)$ 来表示。也可以像式 5.4 那样用价值函数 V 来表示。

到 5.3 节为止，我们一直都在评估价值函数 V 。如果要用价值函数 V 来改进策略，那么就要计算式 5.4。但式 5.4 存在问题。对于一般的强化学习问题，我们不可能知道环境的模型 $p(s' | s, a)$ 和 $r(s, a, s')$ 。而如果没有环境模型，

那么式 5.4 是无法计算的。因此，式 5.3 中的 Q 函数被用来改进策略。式 5.3 只是取出使 $Q(s,a)$ 的值最大的行动 a ，不需要环境模型。

如果要对 Q 函数进行改进，则必须对其进行评估。到 5.3 节为止，我们使用的是蒙特卡洛方法来评估状态价值函数的。我们需要把它切换到 Q 函数上。为此，可以将蒙特卡洛方法的更新式从 $V(s)$ 切换到 $Q(s,a)$ 。数学式如下所示。

【对状态价值函数的评估】

$$\text{一般方式: } V_n(s) = \frac{G^{(1)} + G^{(2)} + \cdots + G^{(n)}}{n}$$

$$\text{增量方式: } V_n(s) = V_{n-1}(s) + \frac{1}{n} \{G^{(n)} - V_{n-1}(s)\}$$

【对 Q 函数的评估】

$$\text{一般方式: } Q_n(s,a) = \frac{G^{(1)} + G^{(2)} + \cdots + G^{(n)}}{n}$$

$$\text{增量方式: } Q_n(s,a) = Q_{n-1}(s,a) + \frac{1}{n} \{G^{(n)} - Q_{n-1}(s,a)\} \quad (5.5)$$

其中 G_n 是在第 n 回合获得的收益， $V_n(s)$ 是第 n 回合结束时状态价值函数的估计值。同样，设 $Q_n(s,a)$ 为第 n 回合结束时 Q 函数的估计值。如上述式子所示，无论是状态价值函数还是 Q 函数，在蒙特卡洛方法中进行的计算本身是不变的，只是对象发生了变化。

5.4.2 使用蒙特卡洛方法实现策略控制

接下来我们将实现使用蒙特卡洛方法进行策略控制的智能代理。这里把它实现为 `McAgent` 类，下面是代码的前半部分。

```
class McAgent:
    def __init__(self):
        self.gamma = 0.9
        self.action_size = 4

        random_actions = {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25}
        self.pi = defaultdict(lambda: random_actions)
```

```

self.Q = defaultdict(lambda: 0) # 使用 Q, 不使用 V
self.cnts = defaultdict(lambda: 0)
self.memory = []

def get_action(self, state):
    action_probs = self.pi[state]
    actions = list(action_probs.keys())
    probs = list(action_probs.values())
    return np.random.choice(actions, p=probs)

def add(self, state, action, reward):
    data = (state, action, reward)
    self.memory.append(data)

def reset(self):
    self.memory.clear()

```

这段代码与 5.3 节中实现的 RandomAgent 类几乎完全相同，唯一的区别是将 self.V 改成了 self.Q。接下来实现主要的策略控制。代码如下所示。

```

def greedy_probs(Q, state, action_size=4):
    qs = [Q[(state, action)] for action in range(action_size)]
    max_action = np.argmax(qs)

    action_probs = {action: 0.0 for action in range(action_size)}
    # 此时的 action_probs 是 {0:0.0, 1:0.0, 2:0.0, 3:0.0}
    action_probs[max_action] = 1 # ①
    return action_probs

class McAgent:
    ...
    def update(self):
        G = 0
        for data in reversed(self.memory):
            state, action, reward = data
            G = self.gamma * G + reward
            key = (state, action)
            self.cnts[key] += 1
            self.Q[key] += (G - self.Q[key]) / self.cnts[key] # ②

        self.pi[state] = greedy_probs(self.Q, state)

```

代码中首先准备了一个名为 greedy_probs 的函数。这个函数不是智能代理的方法，而且会被实现为外部函数。这样实现的原因是为了今后在其他类中使用这个函数。greedy_probs(Q, state) 函数返回的是行动的概率分布。具体

来说，就是采取贪婪行动的概率分布。也就是说，它是只采取在状态 `state` 下 `Q` 函数值最大的行动的概率分布。如果在某个状态下 `Q` 函数的第 0 个行动是最大的，那么 `greedy_probs` 函数就会返回 `{0: 1.0, 1: 0.0, 2: 0.0, 3: 0.0}`。

`update` 方法会更新 `self.Q`。这里要注意，`self.Q` 的键是 `(state, action)` 元组。`update` 方法会基于式 5.5，以增量方式更新 `self.Q`。`self.Q` 的更新结束后，`state` 中的策略就是贪婪化的。

这就是 `McAgent` 类，但事实上这段代码的效果并不好。有以下两点需要改进。

- 代码①处：不要进行完全的贪婪化，而是进行 ϵ -greedy。
- 代码②处：以固定值 α 的方式进行 `Q` 的更新。

下面介绍这两个修改点及修改理由。

5.4.3 ϵ -greedy 算法（第 1 个修改）

智能代理在改进阶段会对策略进行贪婪化。通过贪婪化，可以将智能代理在给定的状态下执行的行动固定为一个。（如果 `Q` 值相同，则智能代理可能会采取多个行动，这种情况也是存在的。）假设策略是贪婪的，智能代理采取了图 5-14 所示的行动。

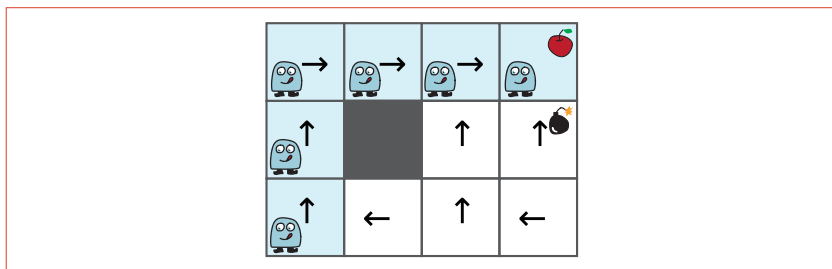


图 5-14 贪婪的策略和智能代理前进的路线

如果像图 5-14 那样只采取贪婪的行动，那么智能代理前进的路线就会被固定为一条。这样就不能收集所有状态和行动组合收益的样本数据了。为了解决这个问题，需要让智能代理进行探索。



正如在老虎机问题中所说明的那样，这里也有利用和探索之间的权衡。基于以往经验的最优行动是“利用”，而尝试行动并获得新经验是“探索”。这种利用和探索之间是一种权衡关系。

让智能代理进行探索的一种方法是 ϵ -greedy 算法。通过在智能代理的行动中增加一点儿随机性，该方法基本上会使智能代理选择 Q 函数值最大的行动，同时以较低的概率选择随机行动。这样做可以防止出现智能代理只选择固定的状态和行动的问题。（在理想情况下，智能代理有望通过所有状态，并覆盖所有行动。）而且在大多数情况下，由于可以采取贪婪的行动，因此会获得近似于最优的策略。

下面让我们来实现 ϵ -greedy 版本的 `greedy_probs` 函数。代码如下所示。

ch05/mc_control.py

```
def greedy_probs(Q, state, epsilon=0, action_size=4):
    qs = [Q[(state, action)] for action in range(action_size)]
    max_action = np.argmax(qs)

    base_prob = epsilon / action_size
    action_probs = {action: base_prob for action in range(action_size)}
    # 此时的 action_probs 是 {0:  $\epsilon/4$ , 1:  $\epsilon/4$ , 2:  $\epsilon/4$ , 3:  $\epsilon/4$ }
    action_probs[max_action] += (1 - epsilon)
    return action_probs
```

上面的代码会将概率分布从完全的贪婪化改为 ϵ -greedy。由于这个问题的行动数量有 4 个，因此为了创建 ϵ -greedy 的概率分布，首先需要将所有行动的概率设为 $\frac{\epsilon}{4}$ ，然后为 Q 函数值最大的行动增加 $1-\epsilon$ 的概率（参见图 5-15）。

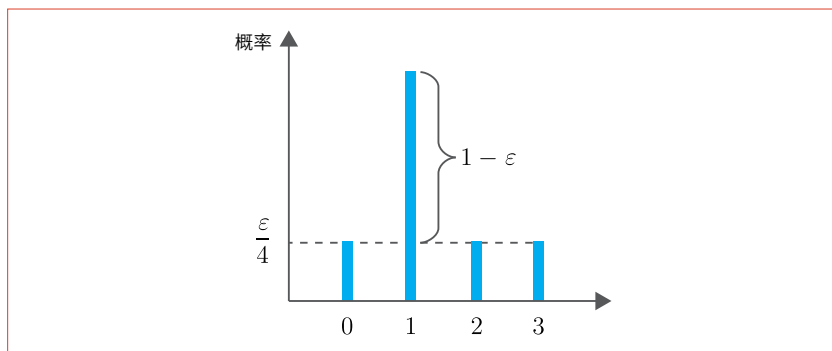


图 5-15 每个行动被 ϵ -greedy 算法选中的概率

这里实现的 `greedy_probs` 函数将来会被反复使用，所以同样的代码也会被放在 `common.utils` 中，今后可以通过 `from common.utils import greedy_probs` 来导入它。

5.4.4 修改为固定值 α 的方式（第 2 个修改）

接下来是第 2 个修改。先来看一下修改后的代码。

```
ch05/mc_control.py

# 修改前
# self.Q[key] += (g - self.Q[key]) / self.cnts[state] # ②

# 修改后
alpha = 0.1
self.Q[key] += (g - self.Q[key]) * alpha # ②
```

如上所述，代码②处使用固定值 `alpha` 进行更新。修改前和修改后的方式的区别如图 5-16 所示。

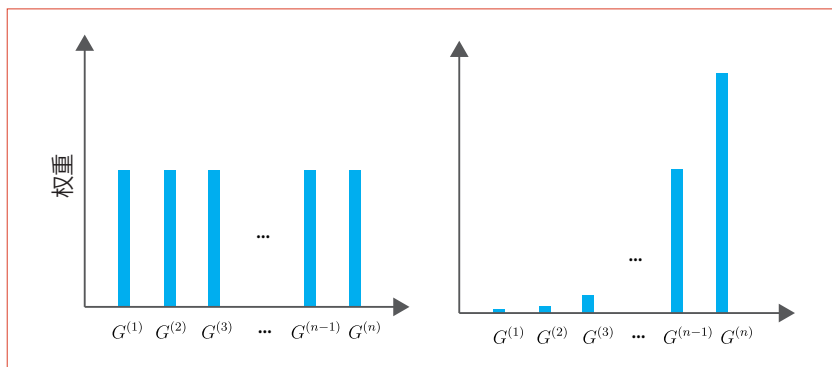


图 5-16 修改前的方式（左）和固定值 α 的方式（右）中为每个数据赋予的权重

修改前的方式是对之前获得的样本数据 ($G^{(1)}, G^{(2)}, \dots, G^{(n)}$) 求均匀权重的平均值。这个值也叫“样本均值”。在使用样本均值的情况下，每个数据的权重都是 $\frac{1}{n}$ 。与之相比，以固定值 α 更新的方式的每个数据的权重都是以指数形式增长的，如图 5-16 中的右图所示。这个值也叫“指数移动平均”。指数移动平均为较新的数据赋予了更大的权重。

指数移动平均适用于蒙特卡洛策略控制。原因是产生收益（样本数据）的概率分布会随时间而变化。更确切地说，随着回合的进行，策略会被更新，导致产生收益的概率分布会有变动。用老虎机问题中的术语来说，这就是“非稳态”。1.5.1 节中解释过，当样本数据（收益）从非稳态概率分布中产生时，使用指数移动平均是合适的。



在产生收益的过程中，我们会重复经历“环境中的状态迁移”和“智能代理的策略”这两个随机性流程。如果“环境中的状态迁移”的概率分布不变，并且“智能代理的策略”的概率分布也不变，那么采样的收益分布就是“稳态”的。但如果其中任何一个发生变化，那么收益的概率分布就是“非稳态”的。这时我们会重复改进策略。因此，每个回合的策略都会发生变化。与此同时，收益的概率分布也会发生变化。

5.4.5 [修改版] 使用蒙特卡洛方法实现策略迭代法

修改后的 McAgent 类如下所示。

ch05/mc_control.py

```
class McAgent:
    def __init__(self):
        self.gamma = 0.9
        self.epsilon = 0.1 #  $\epsilon$ -greedy 的  $\epsilon$ 
        self.alpha = 0.1 # 更新 Q 值时的固定值 alpha
        self.action_size = 4

        random_actions = {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25}
        self.pi = defaultdict(lambda: random_actions)
        self.Q = defaultdict(lambda: 0)
        self.memory = []

    def get_action(self, state):
        action_probs = self.pi[state]
        actions = list(action_probs.keys())
        probs = list(action_probs.values())
        return np.random.choice(actions, p=probs)

    def add(self, state, action, reward):
        data = (state, action, reward)
        self.memory.append(data)
```

```

def reset(self):
    self.memory.clear()

def update(self):
    G = 0
    for data in reversed(self.memory):
        state, action, reward = data
        G = self.gamma * G + reward
        key = (state, action)
        self.Q[key] += (G - self.Q[key]) * self.alpha # ①固定值
        self.pi[state] = greedy_probs(self.Q, state, self.epsilon)

```

代码在初始化过程中添加了参数 `self.epsilon` 和 `self.alpha`。`self.epsilon` 是用于 ϵ -greedy 的参数。如果 `self.epsilon` 是 0.1，那么智能代理将以 10% 的概率采取随机行动，以 90% 的概率采取贪婪行动。另一个参数 `self.alpha` 是在更新 Q 函数时使用的固定值。代码①处使用固定值 `self.alpha` 来更新 Q 函数。

以上就是 McAgent 类的修改版的代码。我们可以将这个 McAgent 类与 GridWorld 类结合起来使用，代码如下所示。

ch05/mc_control.py

```

env = GridWorld()
agent = McAgent()

episodes = 10000
for episode in range(episodes):
    state = env.reset()
    agent.reset()

    while True:
        action = agent.get_action(state)
        next_state, reward, done = env.step(action)

        agent.add(state, action, reward)
        if done:
            agent.update()
            break

        state = next_state

env.render_q(agent.Q)

```

上面的代码训练了 10 000 次 Q 函数，最后通过 `env.render_q(agent.Q)` 进行了可视化。运行上面的代码可以得到图 5-17 所示的图形。

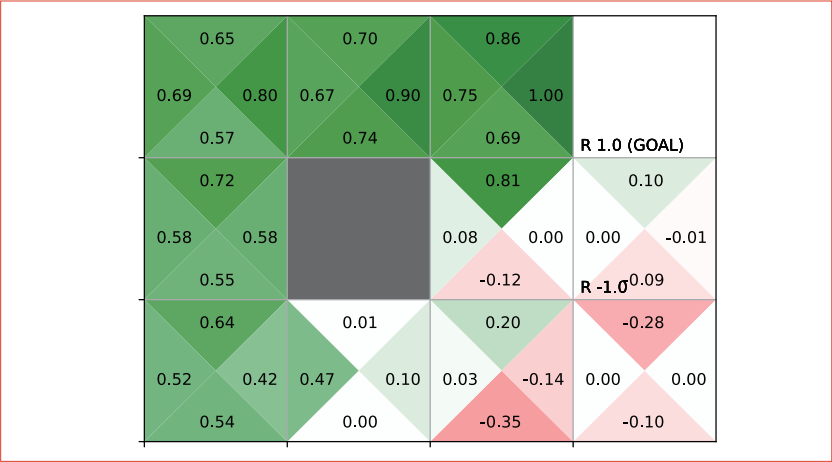


图 5-17 Q 函数的可视化

由于每个方格中都有 4 个行动，因此在进行 Q 函数的可视化时，每个方格被分成了 4 部分，如图 5-17 所示。从图 5-17 中可以看出，负的奖励会被避开，奖励为正数的行动的 Q 函数较大（每次运行的结果都不同）。如果从这个 Q 函数中提取贪婪行动，就会得到图 5-18 所示的结果。

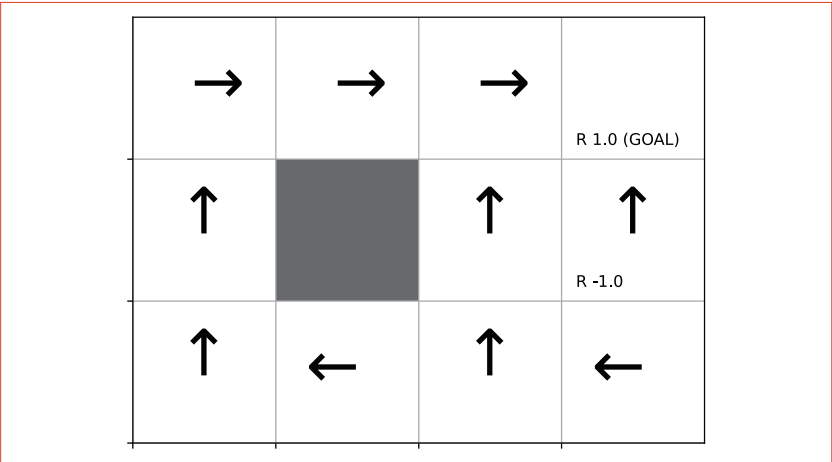


图 5-18 从 Q 函数中得到的贪婪策略

如上所述,从 Q 函数中得到的贪婪策略产生的结果接近于最优策略。其实智能代理也是通过 ε -greedy 在每个方格中采取随机行动的,但由于大多数行动是贪婪的,因此这种策略也基本能得到好的结果。以上就是对蒙特卡洛策略控制的实现。

5.5 异策略型和重要性采样

在 5.4 节中,通过结合蒙特卡洛方法和 ε -greedy 算法,我们得到了接近最优的策略。但这并不是一个完全最优的策略。如果可能,我们只想采取 Q 函数值最大的行动。换句话说,我们想要的是“利用”。但这种做法就不能“探索”了。因此,我们以 ε 这个小的概率进行了探索。也就是说, ε -greedy 算法是一种妥协。本节将介绍如何使用蒙特卡洛方法来学习完全最优的策略。在此之前,先来介绍一下“同策略型”和“异策略型”。

5.5.1 同策略型和异策略型

通过观察他人的行动,我们可以提高自己的技能。例如,我们可以观察网球运动员的挥拍来改善自己的挥拍技术。借用强化学习的用语,可以说我们基于从其他地方获得的经验改进了自己的策略。这种做法在强化学习中叫作异策略型(off-policy)。而基于自己的经验改进自己的策略的做法叫作同策略型(on-policy)。



从作用的角度来看,智能代理的策略可以被看作两种策略:一种是作为评估和改进对象的策略。我们会对这种策略进行评估和改进。这种策略叫作目标策略(target policy);另一种是智能代理实际用来采取行动的策略。智能代理会基于这种策略产生状态、行动和奖励的样本数据。这种策略叫作行为策略(behaviour policy)。

前面在使用策略时,我们没有区分目标策略和行为策略。也就是说,之前的“用于评估和改进的目标策略”与“用于实际行动的行为策略”是一样的。这种目标策略和行为策略相同的情况叫作“同策略型”。与之相比,分开考虑目标策略和行为策略的情况叫作“异策略型”。同策略型中的“on”

和异策略型中的“off”在这里分别是“连接”和“分开”的意思。异策略型意味着目标策略和行为策略是分开的。

这里的主题是“异策略型”。如同网球运动员的例子，人们根据从其他策略（行为策略）中获得的经验来评估和改进自己的策略（目标策略）。通过异策略型，我们可以只让行为策略进行探索，只让目标策略进行利用。但在使用从行为策略获得的样本数据来求目标策略的期望值时，我们需要一些计算上的技巧。此时用到的就是重要性采样（importance sampling）这项技术。下面来介绍一下重要性采样。

5.5.2 重要性采样

重要性采样利用从其他概率分布中采样的数据来计算某个概率分布的期望值。为了说明重要性采样，下面来看一个计算期望值 $\mathbb{E}_\pi[x]$ 的简单例子。设 x 是随机变量， x 的概率用 $\pi(x)$ 表示。期望值的式子如下所示。

$$\mathbb{E}_\pi[x] = \sum x\pi(x)$$

我们先复习一下学过的内容。为了用蒙特卡洛方法近似这个期望值，我们从概率分布 π 对 x 进行采样，并取其平均值。数学式如下所示：

$$\begin{aligned} \text{sampling: } x^{(i)} &\sim \pi \quad (i=1, 2, \dots, n) \\ \mathbb{E}_\pi[x] &= \frac{x^{(1)} + x^{(2)} + \dots + x^{(n)}}{n} \end{aligned}$$

式子中的符号 $x^{(i)} \sim \pi$ 表示第 i 个数据 $x(i)$ 是从概率分布 π 中采样的。

接下来回到正题。我们现在探讨的是从别的概率分布中采样 x 的场景。假设 x 是从概率分布 b （而不是 π ）中采样的。在这种情况下，期望值 $\mathbb{E}_\pi[x]$ 如何被近似呢？解决这个问题的关键在于下面这个式子的变形。

$$\begin{aligned} \mathbb{E}_\pi[x] &= \sum x\pi(x) \\ &= \sum x \frac{b(x)}{b(x)} \pi(x) \\ &= \sum x \frac{\pi(x)}{b(x)} b(x) \end{aligned} \tag{5.6}$$

这里的要点是插入 $\frac{b(x)}{b(x)}$ 。 $\frac{b(x)}{b(x)}$ 总是为 1，所以等式成立。然后，我们将式子变为 $\sum \dots b(x)$ 的形式，如式 5.6 所示，这样它就可以被视为概率分布 $b(x)$ 的期望值了。实际对式 5.6 进行变形，可以得到以下式子。

$$\begin{aligned}\mathbb{E}_{\pi}[x] &= \sum x \frac{\pi(x)}{b(x)} b(x) \\ &= \mathbb{E}_b \left[x \frac{\pi(x)}{b(x)} \right]\end{aligned}\tag{5.7}$$

这里需要注意的是式子中的 $\mathbb{E}_b$ ，它表示的是概率分布 b 的期望值。另外每个 x 都要乘以 $\frac{\pi(x)}{b(x)}$ 这一点也很重要。如果 $\rho(x) = \frac{\pi(x)}{b(x)}$ ，那么我们就可以把式子看作每个 x 都乘以权重 $\rho(x)$ 。根据以上探讨，基于式 5.7 的蒙特卡洛方法如下所示。

$$\begin{aligned}\text{sampling: } x^{(i)} &\sim b \quad (i=1,2,\dots,n) \\ \mathbb{E}_{\pi}[x] &\approx \frac{\rho(x^{(1)})x^{(1)} + \rho(x^{(2)})x^{(2)} + \dots + \rho(x^{(n)})x^{(n)}}{n}\end{aligned}$$

这样我们就使用从不同于 π 的概率分布 b 中采样的数据计算出了 $\mathbb{E}_{\pi}[x]$ 。下面来实现重要性采样的代码。这里我们对图 5-19 所示的概率分布进行重要性采样。

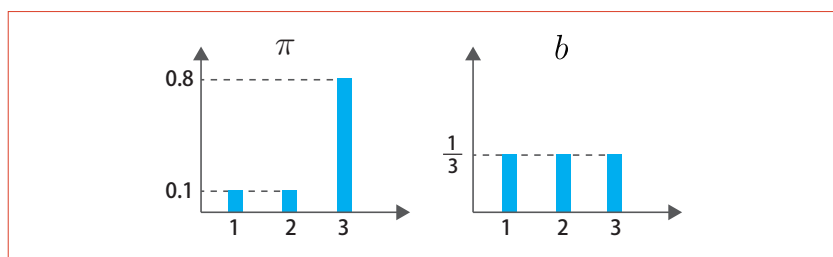


图 5-19 概率分布 π 和 b

这里的目标是求出期望值 $\mathbb{E}_{\pi}[x]$ 。我们先尝试用普通的蒙特卡洛方法求概率分布 π 的期望值。代码如下所示。

ch05/importance_sampling.py

```
import numpy as np

x = np.array([1, 2, 3])
pi = np.array([0.1, 0.1, 0.8])

# 期望值
e = np.sum(x * pi)
print('E_pi[x]', e)

# 蒙特卡洛方法
n = 100
samples = []
for _ in range(n):
    s = np.random.choice(x, p=pi) # 使用 pi 进行采样
    samples.append(s)

mean = np.mean(samples)
var = np.var(samples)
print('MC: {:.2f} (var: {:.2f})'.format(mean, var))
```

输出结果

```
E_pi[x] 2.7
MC: 2.78 (var: 0.27)
```

首先，套用定义式求期望值。求出的结果是 2.7（这是真实的值）。然后，使用蒙特卡洛方法来求这个值。这里只基于概率分布 π 采样 100 个数据，然后求平均值。为此，我们用 NumPy 的 `np.mean` 方法求出了平均值。结果是 2.78，接近于真实值。作为参考，我们还使用 NumPy 的 `np.var` 方法求出了方差。方差的值是 0.27。在与下一次重要性采样的结果进行比较时，这个值将被用作参考。方差用于表示数据的离散程度。期望值和方差之间的关系可以用以下式子表示。

$$\text{Var}[X] = \mathbb{E}[(X - \mathbb{E}[X])^2]$$

方差是数据 X 和 X 的平均值 $\mathbb{E}[X]$ 之间的差的平方的期望值。直观地说，它代表了数据的离散程度，如图 5-20 所示。



图 5-20 将每个数据作为二维平面上的点时的方差的示意图（圆心是平均值）

下面尝试用重要性采样的方法来求期望值。代码如下所示。

ch05/importance_sampling.py

```
b = np.array([1/3, 1/3, 1/3])
n = 100
samples = []

for _ in range(n):
    idx = np.arange(len(b)) # [0, 1, 2]
    i = np.random.choice(idx, p=b) # 使用 b 进行采样
    s = x[i]
    rho = pi[i] / b[i]
    samples.append(rho * s)

mean = np.mean(samples)
var = np.var(samples)
print('IS: {:.2f} (var: {:.2f})'.format(mean, var))
```

输出结果

IS: 2.95 (var: 10.63)

这里使用概率分布 b 进行采样，不过进行采样的目标是 b 的索引 $([0, 1, 2])$ 。这是因为在计算权重 ρ 时，要用到的是采样得到的索引。

接下来看看上面的结果。平均值是 2.95，虽然与真实值 2.7 有一些差距，但也还算接近。此外，方差是 10.63，这说明数据的离散程度大于蒙特卡洛方法的结果（使用蒙特卡洛方法时的方差是 0.27）。

5.5.3 如何减小方差

方差越小，用较少的样本得到的近似值就越准确。反之，方差越大，越需要更多的样本才能得到准确的近似值。下面介绍如何用重要性采样来减小方差。首先，我们结合图 5-21 来了解一下为什么重要性采样方法的方差会比较大。

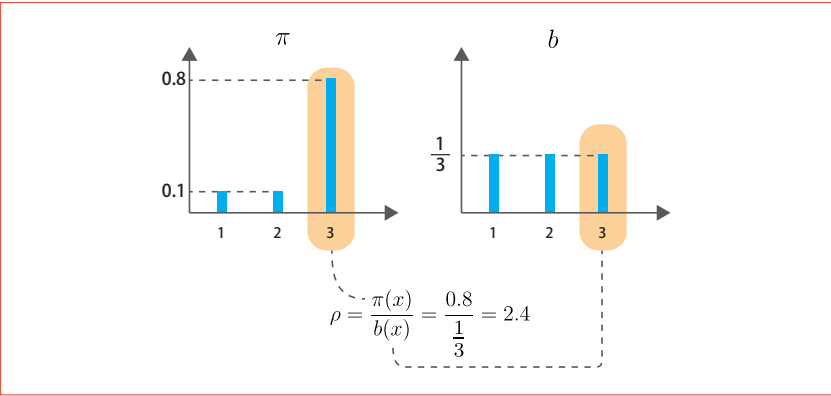


图 5-21 基于概率分布 b 采样 3 的例子

图 5-21 展示了选择 3 作为样本数据的例子。此时的权重 ρ 为 2.4。因此，3 这个值要乘以 2.4。这意味着尽管我们得到的值是 3，但实际得到的值是 $3 \times 2.4 = 7.2$ 。听起来可能有些奇怪，但这是有道理的。原因如下。

- 由于 3 是概率分布 π 的代表值，因此本来它应该被更频繁地采样。
- 但在概率分布 b 中，3 这个值并不经常出现。
- 为了填补这个落差，当 3 被采样时，它会被调整为乘以权重，从而使该值变得更大。

考虑到概率分布 π 和 b 之间的差异，我们将采样值乘以其权重来调整采样值，这么做是合理的。可是，明明采样的值是 3，它却被当作了 7.2，如果这是第一个采样数据，那么此时的估计值就是 7.2。与真正的值 2.7 相比，7.2 的偏离较大。因此，通过权重 ρ 对实际得到的值的填补越大，方差与真正的值的差异就越大。

那么，怎样才能减小方差呢？一种方法是使两个概率分布（ b 和 π ）更加接近。这样可以使权重 ρ 的值更接近于 1。下面来做实验。与之前的代码相比，这次只有概率分布中的 b 的值被改变了。

ch05/importance_sampling.py

```
b = np.array([0.2, 0.2, 0.6]) # 修改概率分布
n = 100
samples = []

for _ in range(n):
    idx = np.arange(len(b)) # [0, 1, 2]
    i = np.random.choice(idx, p=b)
    s = x[i]
    rho = pi[i] / b[i]
    samples.append(rho * s)

mean = np.mean(samples)
var = np.var(samples)
print('IS: {:.2f} (var: {:.2f})'.format(mean, var))
```

输出结果

```
IS: 2.72 (var: 2.48)
```

上面的代码假设 b 的概率分布为 $[0.2, 0.2, 0.6]$ ，其形状更接近于 π 的概率分布。结果是，平均值为 2.72，更接近于正确答案。另外，方差是 2.48，比以前小。

因此，在进行重要性采样时，可以通过使两个概率分布更加接近来减小方差。不过，强化学习的重点是让一个策略（概率分布）进行“探索”，另一个策略进行“利用”。在满足这个条件的基础上，就可以通过使两个概率分布尽可能地接近来减小方差了。

以上就是对重要性采样的介绍。使用重要性采样可以实现异策略型。具体来说，基于行为策略的概率分布采样的数据可以用来计算目标策略的期望值。详细方法见附录 A。这是正在研究中的课题，如果你对此感兴趣，可以参考一下。

5.6 小结

在强化学习中，智能代理会对环境实际采取行动，然后根据获得的经验找到更好的策略。强化学习这个领域的特点是环境和智能代理之间存在相互作用。我们在本章中学到的技术是蒙特卡洛方法。通过蒙特卡洛方法，可以根据实际获得的经验近似求出价值函数。

一旦可以通过蒙特卡洛方法评估 Q 函数，之后就能根据 Q 函数改进策略了。通过交替重复进行这种评估和改进过程，可以得到更好的策略。不过，在策略的改进阶段完全贪婪化之后，就不能“探索”了。本章并未采用完全贪婪的方式，而是通过 ϵ -greedy 实现了利用和探索之间的平衡。在解决强化学习问题时，我们需要在利用和探索之间取得平衡，以找到最优的行动。

另外，我们还根据策略的作用将策略分为了目标策略和行为策略。智能代理可以基于行为策略实际采取行动，并利用获得的经验来更新目标策略。目标策略和行为策略相同的情况是同策略型，而将二者分开考虑的情况是异策略型。在异策略型的情况下，智能代理需要根据行为策略采取行动，并利用结果来计算目标策略的期望值。这是通过重要性采样来实现的。

第 6 章

TD 方法

在第 5 章中，我们学习了蒙特卡洛方法。使用蒙特卡洛方法，我们可以在不使用环境模型的情况下对策略进行评估，而且通过交替重复地评估和改进，可以得到最优策略或接近最优的策略。不过，蒙特卡洛方法只能在达到回合的终点时更新价值函数。这是因为只有在回合结束时，才能确定收益。



对于连续性任务，不能使用蒙特卡洛方法。即使是回合制任务，如果完成一个回合需要很长时间，那么蒙特卡洛方法也将需要大量的时间来更新价值函数。尤其是在回合的开始阶段，智能代理的策略往往是随机性的，因此甚至需要更多的时间。

本章将介绍一种不使用环境模型，而且每次采取行动时都会更新价值函数的方法，即 TD 方法。TD 方法中的 TD 是 Temporal Difference 的缩写，是“时间差分”的意思。该方法不需要等待回合的结束，每过一定的时间就会进行策略的评估和改进。

6.1 使用 TD 方法评估策略

TD 方法是我们学过的蒙特卡洛方法和动态规划法的结合。因此，我们首先复习一下这两种方法，然后再推导出 TD 方法。为了简单起见，我们将蒙特卡洛方法缩写为“MC 方法”，将动态规划法缩写为“DP 方法”（动态

规划法通常被缩写为“DP”，这里为了与 TD 方法和 MC 方法保持统一，我们将其缩写为“DP 方法”）。

6.1.1 TD 方法的推导

先来复习一下收益。我们对收益的定义如下所示。

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots \quad (6.1)$$

$$= R_t + \gamma G_{t+1} \quad (6.2)$$

假设从时刻 t 开始获得的奖励为 $R_t, R_{t+1}, \dots$ 此时的收益可以表示为带有折现率的奖励的总和，如式 6.1 所示。也可以使用 G_{t+1} 递归地表示 G_t ，如式 6.2 所示。使用收益，可以对价值函数作如下定义。

$$v_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s] \quad (6.3)$$

$$= \mathbb{E}_\pi[R_t + \gamma G_{t+1} | S_t = s] \quad (6.4)$$

如上面的式子所示，价值函数可以被定义为期望值。本节接下来将展示以下两点。

- 使用 MC 方法的式子可以从式 6.3 中导出。
- 使用 DP 方法的式子可以从式 6.4 中导出。

下面从 MC 方法开始。MC 方法不进行期望值的计算，而是通过对实际获得的收益的样本数据取平均来近似式 6.3 的期望值。有两种类型的“平均”：样本均值和指数移动平均。为了实现指数移动平均，每当获得新的收益时，都要用固定值 α 来进行更新。这个过程可以用如下数学式表示。

$$V'_\pi(S_t) = V_\pi(S_t) + \alpha \{G_t - V_\pi(S_t)\} \quad (6.5)$$

设 V_π 为当前的价值函数， V'_π 为更新后的价值函数。式 6.5 以 G_t 为目标，对当前的价值函数 V_π 进行更新。我们通过 α 来调整朝着 G_t 方向的更新量。

接下来是 DP 方法，DP 方法根据式 6.4 计算期望值。与 MC 方法不同，DP 方法需要根据数学式计算期望值。数学式如下所示。

$$v_{\pi}(s) = \mathbb{E}_{\pi}[R_t + \gamma G_{t+1} | S_t = s] \quad (6.4)$$

$$= \sum_{a, s'} \pi(a | s) p(s' | s, a) \{r(s, a, s') + \gamma v_{\pi}(s')\} \quad (6.6)$$

上面的式子使用状态迁移概率 $p(s' | s, a)$ 和奖励函数 $r(s, a, s')$ 计算期望值。顺带一提，式 6.6 是贝尔曼方程。DP 方法可以根据式 6.6 依次更新价值函数。更新式如下所示。

$$V'_{\pi}(s) = \sum_{a, s'} \pi(a | s) p(s' | s, a) \{r(s, a, s') + \gamma V_{\pi}(s')\} \quad (6.7)$$

式 6.7 中的要点在于当前状态的价值函数是使用下一状态的价值函数来更新的。这个过程的特点在于需要考虑所有的迁移。与 MC 方法相比，DP 方法的特点更加清晰，如图 6-1 所示。

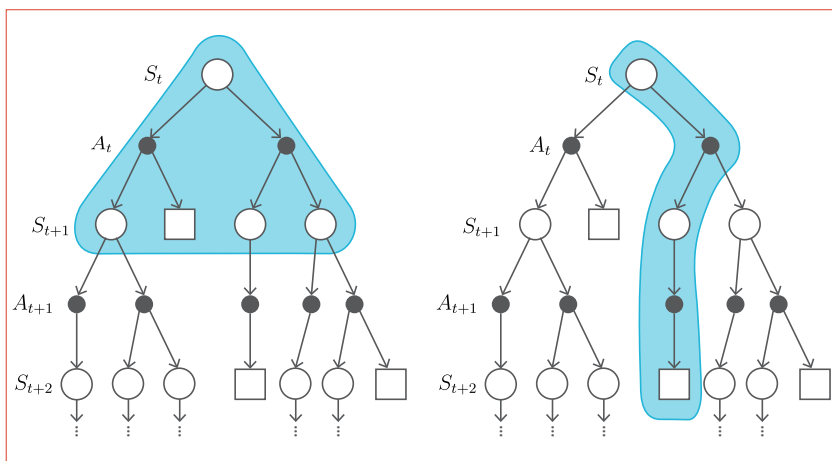


图 6-1 DP 方法（左）与 MC 方法（右）的比较

图 6-1 绘制了从状态 S_t 开始的所有后续的迁移。DP 方法使用价值函数的下一个估计值来更新价值函数的当前估计值。这种做法叫作“自举”。MC 方法则根据实际获得的经验来更新当前的价值函数。将这两种方法融合起来的做法就是图 6-2 所示的 TD 方法。

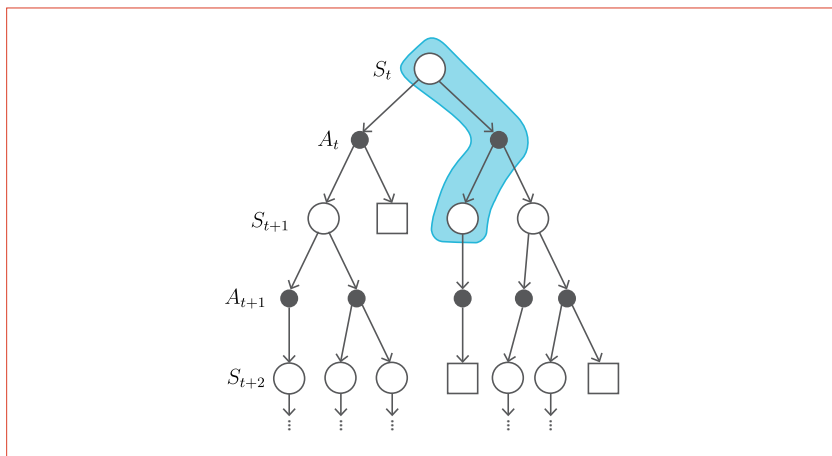


图 6-2 TD 方法的思路

TD 方法只使用下一个行动和价值函数来更新当前的价值函数，如图 6-2 所示。该方法的两个要点如下。

- 像 DP 方法一样，通过“自举”的方式就能依次更新价值函数。
- 像 MC 方法一样，无须了解环境相关的知识，使用采样数据就能对价值函数进行更新。

接下来使用数学式来推导 TD 方法。首先展开下面的式子。

$$v_{\pi}(s) = \sum_{a,s'} \pi(a|s) p(s'|s,a) \{r(s,a,s') + \gamma v_{\pi}(s')\} \quad (6.6)$$

$$= \mathbb{E}_{\pi}[R_t + \gamma v_{\pi}(S_{t+1}) | S_t = s] \quad (6.8)$$

式 6.6 会计算所有候选的 $r(s,a,s') + \gamma v_{\pi}(s')$ ，也就是奖励和下一个价值函数。把式子写为期望值 $\mathbb{E}_{\pi}$ 的形式，得到式 6.8。TD 方法会使用式 6.8 更新价值函数，根据样本数据近似计算 $R_t + \gamma v_{\pi}(S_{t+1})$ 的部分。TD 方法的更新式如下所示。

$$V'_{\pi}(S_t) = V_{\pi}(S_t) + \alpha \{R_t + \gamma V_{\pi}(S_{t+1}) - V_{\pi}(S_t)\} \quad (6.9)$$

式 6.9 中的 V_{π} 是价值函数的估计值，目标是 $R_t + \gamma v_{\pi}(S_{t+1})$ 。 $R_t + \gamma v_{\pi}(S_{t+1})$

叫作“TD 目标”。TD 方法会朝着 TD 目标的方向更新 $V_{\pi}(S_t)$ 。



这里我们使用下一步的信息作为 TD 目标。我们可以拓展这个想法，即使用 2 步、3 步甚至 n 步之后的信息。这种做法叫作“ n -step TD 方法”。附录 B 介绍了 n -step TD 方法。如果你对此感兴趣，可以阅读一下。

6.1.2 MC 方法和 TD 方法的比较

当环境模型未知时，我们可以分别使用 MC 方法和 TD 方法。这就引出了一个问题：到底应该使用 MC 方法还是 TD 方法呢？或者说哪种方法更好呢？由于 MC 方法不能用于连续性任务，因此在遇到这种任务时应该使用 TD 方法。那么对于回合制任务呢？遗憾的是，对于这种任务，人们还未能从理论上证明哪种方法总是更好。但对于现实生活中的许多问题，TD 方法训练起来更快（价值函数更新得更快）。看一下 MC 方法和 TD 方法的目标，你就能清楚其中的原因了，如图 6-3 所示。

$$\text{【MC方法】 } V'_{\pi}(S_t) = V_{\pi}(S_t) + \alpha \{G_t - V_{\pi}(S_t)\}$$

$$\text{【TD方法】 } V'_{\pi}(S_t) = V_{\pi}(S_t) + \alpha \{R_t + \gamma V_{\pi}(S_{t+1}) - V_{\pi}(S_t)\}$$

图 6-3 MC 方法与 TD 方法的比较

在图 6-3 中，MC 方法的目标是 G_t ，朝着这个方向更新 V_{π} 。 G_t 是达到目标后获得收益的样本数据。而 TD 方法的目标是根据下一步的信息进行计算。在 TD 方法中，每当进入下一个时间步，价值函数都能得到更新，这有望带来更高效的学习。

此外，由于 MC 方法的目标是积累大量时间步的数据得到的结果，因此这个值的偏差会变大。换言之，离散程度（方差）会变得更大。而 TD 方法是基于下一步的数据，所以值的变动会比较小。这种偏差的示意图如图 6-4 所示。

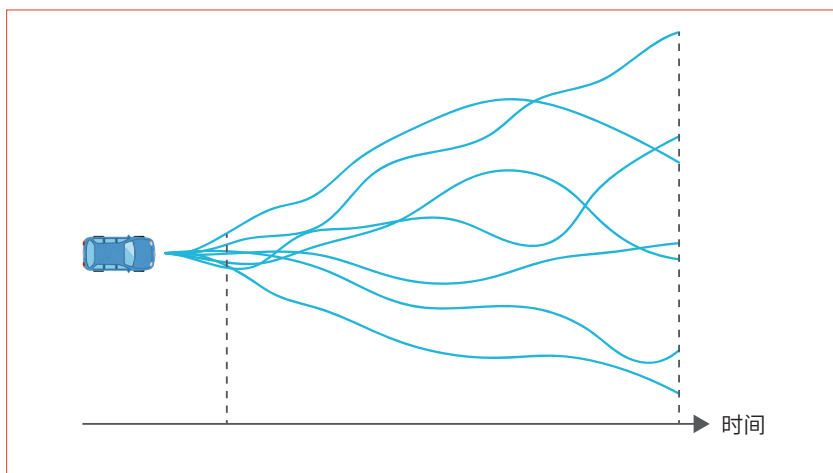


图 6-4 偏差随着时间的推移而增大的示意图

图 6-4 展示的是汽车的运动。汽车向前行驶，但司机随机地向右或向左打方向盘（或什么也不做）。图中延伸出来的几条线表示几种可能的情况。这里需要注意的是，随着时间的推移，移动目标的变化非常大。这张示意图也适用于 MC 方法和 TD 方法的目标。由于 MC 方法会经历许多时间步，所以目标的偏差较大。而 TD 方法的目标（TD 目标）是下一个时间步，所以偏差较小。

TD 目标是 $R_t + \gamma V_{\pi}(S_{t+1})$ ，这个 TD 目标中使用了估计值 V_{π} 。TD 方法会用估计值更新估计值，也就是进行“自举”。由于包含估计值，因此 TD 目标并不是正确的值。有一个专门的术语是“偏置”，它被用来表示 TD 目标与正确的值之间的差值。但偏置会随着不断的更新而减少，最终变为 0。而 MC 方法的目标中不包含估计值。换句话说，MC 方法的目标是“无偏置”的。

6.1.3 TD 方法的实现

现在实现 TD 方法。下面用 TD 方法来评估智能代理的随机性策略。代码如下所示。

ch06/td_eval.py

```

class TdAgent:
    def __init__(self):
        self.gamma = 0.9
        self.alpha = 0.01
        self.action_size = 4

        random_actions = {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25}
        self.pi = defaultdict(lambda: random_actions)
        self.V = defaultdict(lambda: 0)

    def get_action(self, state):
        action_probs = self.pi[state]
        actions = list(action_probs.keys())
        probs = list(action_probs.values())
        return np.random.choice(actions, p=probs)

    def eval(self, state, reward, next_state, done):
        next_V = 0 if done else self.V[next_state] # 目标的价值函数是 0
        target = reward + self.gamma * next_V

        self.V[state] += (target - self.V[state]) * self.alpha

```

TdAgent 类与之前实现的智能代理类（第 5 章中实现的 McAgent 类等）有很多共同之处。因此，这里只介绍使用 TD 方法评估策略的 eval 方法。包括参数在内的 eval 方法是 eval(self, state, reward, next_state, done)。在 state 状态下采取行动 action，获得奖励 reward，在进入下一个状态 next_state 时，这个方法会被调用。此外，这个方法还会把表示回合是否结束（next_state 是否为目标）的标志位 done 作为参数。



价值函数是将来获得的奖励的总和。由于在目标处没有任何后续状态，因此此处的价值函数始终为 0。

下面让智能代理实际采取行动，并进行策略的评估。这次运行 1000 个回合。代码如下所示。

ch06/td_eval.py

```

env = GridWorld()
agent = TdAgent()

```

```
episodes = 1000
for episode in range(episodes):
    state = env.reset()

    while True:
        action = agent.get_action(state)
        next_state, reward, done = env.step(action)

        agent.eval(state, reward, next_state, done) # 每次都要调用
        if done:
            break
        state = next_state

env.render_v(agent.V)
```

这段代码与第5章的MC方法的代码(ch05/mc_eval.py)基本相同。主要区别在于,这段代码每次都会调用智能代理的eval方法。TD方法每次前进一个时间步时都会进行更新,而MC方法只有在到达目标时才会调用eval方法。

运行上面的代码,结果如图6-5所示。

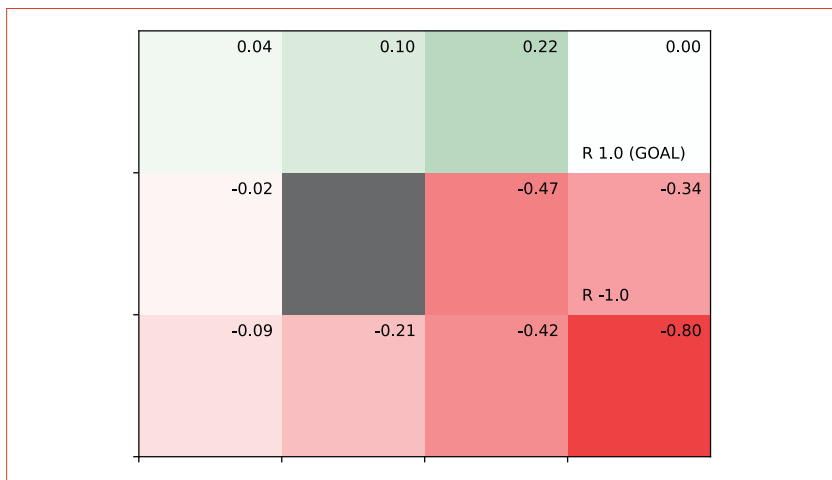


图 6-5 通过 TD 方法得到的价值函数

如图6-5所示,我们完成了对采取随机性策略的智能代理的价值函数的评估。这是一个近似正确的结果。我们成功地使用TD方法进行了策略评估。

6.2 SARSA

策略评估完成后，下一步是策略控制（我们已经多次接触这一过程）。这里也要通过重复评估和改进的过程使策略接近最优策略。我们使用的是同策略型的 SARSA 方法。



5.5 节介绍过，策略控制有两种方法：同策略型和异策略型。本节会介绍同策略型的方法，6.3 节将介绍异策略型的方法。

6.2.1 同策略型的 SARSA

6.1 节对价值函数 $V_\pi(s)$ 进行了评估。在进行策略控制时，我们需要将 $Q_\pi(s, a)$ 而不是 $V_\pi(s)$ 作为评估对象。在改进阶段，策略需要进行贪婪化，并且 $V_\pi(s)$ 需要环境模型。但在 $Q_\pi(s, a)$ 的情况下，可以直接进行以下计算，不需要环境模型，这一点在 5.4.1 节中已有说明。

$$\mu(s) = \operatorname{argmax}_a Q_\pi(s, a)$$

6.1 节使用价值函数 $V_\pi(s)$ 推导了 TD 方法。TD 方法的更新式如式 6.9 所示。

$$V'_\pi(S_t) = V_\pi(S_t) + \alpha \{R_t + \gamma V_\pi(S_{t+1}) - V_\pi(S_t)\} \quad (6.9)$$

下面将 TD 方法式子中的状态价值函数 $V_\pi(S_t)$ 改为行动价值函数 $Q_\pi(S_t, A_t)$ ，也就是把式 6.9 中的 $V_\pi(S_t)$ 和 $V_\pi(S_{t+1})$ 分别替换为 $Q_\pi(S_t, A_t)$ 和 $Q_\pi(S_{t+1}, A_{t+1})$ 。

$$Q'_\pi(S_t, A_t) = Q_\pi(S_t, A_t) + \alpha \{R_t + \gamma Q_\pi(S_{t+1}, A_{t+1}) - Q_\pi(S_t, A_t)\} \quad (6.10)$$

式 6.10 是以 Q 函数为对象的 TD 方法的更新式。下面解释一下同策略型的策略控制方法。在同策略型中，智能代理只有一个策略。更确切地说，实际采取行动的策略（行为策略）与进行评估和改进的策略（目标策略）是一致的。



在同策略型中，行为策略和目标策略是相同的，在改进阶段不能完全贪婪化。这是因为如果这样做，就无法进行探索了。因此，我们采用了 ε -greedy 算法（作为一种折中方法）。这样一来，智能代理既能进行探索，又能在大多数情况下采取贪婪的行动。

这里假设智能代理根据策略 π 采取行动。具体来说，假设在 t 和 $t+1$ 时刻，智能代理会采取图 6-6 所示的行动。

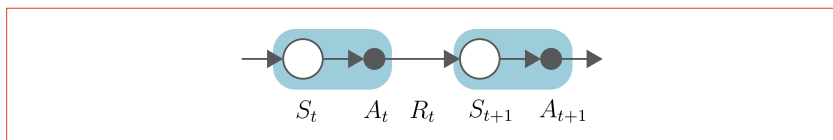


图 6-6 t 和 $t+1$ 时刻的状态以及行动的迁移

Q 函数会将状态和行动的数据对视为一个整体。因此，在图 6-6 中， t 时刻的状态和行动的数据对会被组合为 (S_t, A_t) ，下一个时间步的数据对会被组合为 (S_{t+1}, A_{t+1}) 。当得到图 6-6 所示的数据 $(S_t, A_t, R_t, S_{t+1}, A_{t+1})$ 时，我们就能根据式 6.10 立即更新 $Q_\pi(S_t, A_t)$ 。更新完成后，即可进入改进阶段。在本例中，由于 $Q_\pi(S_t, A_t)$ 已更新，因此状态 S_t 的策略可能会改变。具体来说，状态 S_t 的策略可以按以下式子更新。

$$\pi'(a | S_t) = \begin{cases} \underset{a}{\operatorname{argmax}} Q_\pi(S_t, a) & (1 - \varepsilon \text{ 的概率}) \\ \text{随机行动} & (\varepsilon \text{ 的概率}) \end{cases} \quad (6.11)$$

如式 6.11 所示，智能代理会以 ε 的概率采取随机行动，否则会选择贪婪行动。贪婪行动可以改进“策略”，随机行动则可以改进“探索”。这种 ε -greedy 算法会更新状态 S_t 的行动选择策略。

使用式 6.10 进行的评估和使用式 6.11 进行的更新就像这样交替重复，之后就能得到接近最优的策略。这就是 SARSA 算法。它的名称来源于 TD 方法中使用的数据 $(S_t, A_t, R_t, S_{t+1}, A_{t+1})$ 的首字母。

6.2.2 SARSA 的实现

下面来实现 SARSA。我们将它实现为 `SarsaAgent` 类，代码如下所示。

ch06/sarsa.py

```

from collections import defaultdict, deque
import numpy as np
from common.utils import greedy_probs

class SarsaAgent:
    def __init__(self):
        self.gamma = 0.9
        self.alpha = 0.8
        self.epsilon = 0.1
        self.action_size = 4

        random_actions = {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25}
        self.pi = defaultdict(lambda: random_actions)
        self.Q = defaultdict(lambda: 0)
        self.memory = deque(maxlen=2) # ①使用 deque

    def get_action(self, state):
        action_probs = self.pi[state] # ②从 pi 中选择
        actions = list(action_probs.keys())
        probs = list(action_probs.values())
        return np.random.choice(actions, p=probs)

    def reset(self):
        self.memory.clear()

    def update(self, state, action, reward, done):
        self.memory.append((state, action, reward, done))
        if len(self.memory) < 2:
            return

        state, action, reward, done = self.memory[0]
        next_state, next_action, _, _ = self.memory[1]
        # ③下一个 Q 函数
        next_q = 0 if done else self.Q[next_state, next_action]

        # ④使用 TD 方法进行更新
        target = reward + self.gamma * next_q
        self.Q[state, action] += (target - self.Q[state, action]) * self.alpha

        # ⑤策略的改进
        self.pi[state] = greedy_probs(self.Q, state, self.epsilon)

```

SarsaAgent 类与此前实现的智能代理类有许多共同之处。下面依次介绍一下代码①～⑤。

- ① 这里使用了 Python 标准库的 `collections.deque`。`deque` 的使用方法与列表相同。此外，如果添加的元素多于指定的最大元素数 (`maxlen`)，那么 `deque` 就会根据“First in First out”（先进先出）的规则删除最旧的元素。`deque` 可用于仅保存两个最新的经验数据。
- ② 由于 `SarsaAgent` 类是同策略型，因此这里只有一个策略。`get_action(self, state)` 方法会取出状态 `state` 下的一个行动。这个行动是从策略 `self.pi` 中选择的。
- ③ 如果 `done` 标志位为 `True`，则表示已到达目标。目标的 `Q` 函数总是为 0，因为 `Q` 函数是将来获得的奖励的总和。到达目标意味着后续什么都没有了。
- ④ 根据 SARSA 的式 6.10 更新 `self.Q`。
- ⑤ 使用第 5 章中实现的 `greedy_probs` 函数来改进策略。这样策略 `self.pi` 在状态 `state` 下的行动就是 ϵ -greedy 的。

下面运行 `SarsaAgent` 类。和之前一样，我们将挑战“ 3×4 网格世界”任务。这里进行 10 000 个回合的训练，最后通过 `env.render_q(agent.Q)` 将 `Q` 函数可视化。代码如下所示。

ch06/sarsa.py

```
env = GridWorld()
agent = SarsaAgent()

episodes = 10000
for episode in range(episodes):
    state = env.reset()
    agent.reset()

    while True:
        action = agent.get_action(state)
        next_state, reward, done = env.step(action)

        agent.update(state, action, reward, done) # 每次都要调用

    if done:
        # 到达目标时也要调用
        agent.update(next_state, None, None, None)
        break
```



```
state = next_state

env.render_q(agent.Q)
```

这里值得注意的是调用 `agent.update` 方法的时机。它在 `while` 循环中会被重复调用。此外，`agent.update` 方法会以两次调用作为一个组合，进行策略的更新。因此，当达到目标时，我们会以 `agent.update(next_state, None, None, None)` 的形式进行额外的调用。现在运行上面的代码，结果如图 6-7 所示。

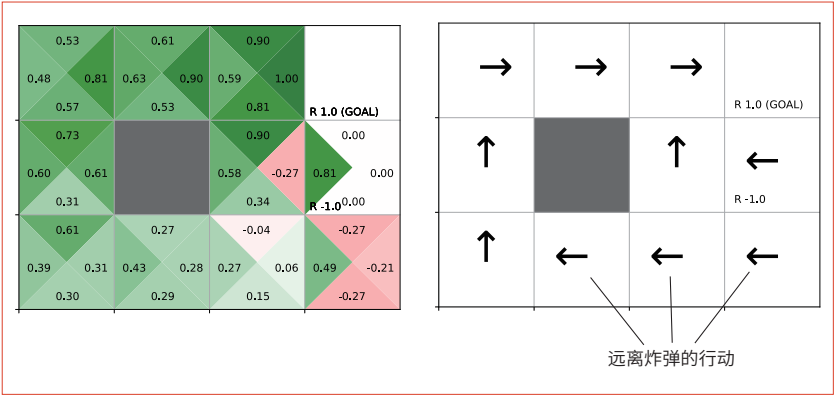


图 6-7 通过 SARSA 得到的结果

图 6-7 中的结果在每次运行时都不同，但总体良好。图 6-7 中只用箭头绘制了贪婪的行动，但策略中包含了 ϵ 的随机行动。由于策略的随机性，因此可以看到行动都尽可能远离炸弹。这样我们就完成了同策略型的 SARSA 的实现。

6.3 异策略型的 SARSA

6.2 节实现了同策略型的 SARSA。接下来是异策略型的 SARSA。一般来说，Q 学习在这里该“登场”了，但本书将首先推导异策略型的 SARSA，之后再介绍 Q 学习。

6.3.1 异策略型和重要性采样

在异策略型的情况下，智能代理拥有两种策略，即行为策略和目标策略。智能代理首先会基于行为策略，通过采取各种行动收集大量样本数据，然后再使用样本数据贪婪地更新目标策略。此时需要注意以下两点。

- 如果行为策略和目标策略的概率分布相似，则结果会更稳定。考虑到这一点，我们对当前的 Q 函数的行为策略进行 ε -greedy 更新，对目标策略进行贪婪更新。
- 由于两种策略不同，因此我们使用重要性采样来校正权重 ρ 。

下面进行具体的说明。考虑更新 $Q_\pi(S_t, A_t)$ 的情况。此时的 SARSA 的更新式如下所示。

$$Q'_\pi(S_t, A_t) = Q_\pi(S_t, A_t) + \alpha \{R_t + \gamma Q_\pi(S_{t+1}, A_{t+1}) - Q_\pi(S_t, A_t)\} \quad (6.10)$$

图 6-8 展示了这个更新式对应的回溯线形图。

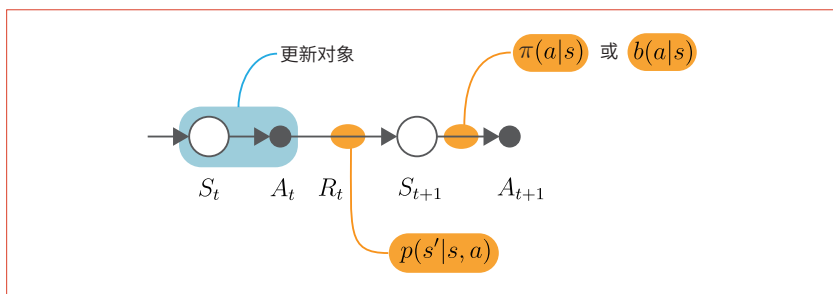


图 6-8 与 SARSA 更新式对应的回溯线形图

如图 6-8 所示，数据对 (S_t, A_t) 是更新的对象。像 (S_t, A_t) 这样的更新对象可以任意选择。下面思考所选数据对在下一个时刻 $t+1$ 的迁移。此时，下一个状态 S_{t+1} 是基于环境的状态迁移概率 $p(s'|s, a)$ 进行采样的。而在状态 S_{t+1} 中选择的行动是基于目标策略 π 或行为策略 b 进行采样的。利用由此获得的采样数据，可以根据式 6.10 更新 $Q_\pi(S_t, A_t)$ 。为了明确地表明基于策略 π 来选择行动，我们将 SARSA 的更新式改写为如下形式。

$$\begin{aligned} \text{sampling: } A_{t+1} &\sim \pi \\ Q'_\pi(S_t, A_t) &= Q_\pi(S_t, A_t) + \alpha \{R_t + \gamma Q_\pi(S_{t+1}, A_{t+1}) - Q_\pi(S_t, A_t)\} \end{aligned} \quad (6.12)$$

式 6.12 表示 $Q_\pi(S_t, A_t)$ 朝着 $R_t + \gamma Q_\pi(S_{t+1}, A_{t+1})$ 方向进行更新。这个 $R_t + \gamma Q_\pi(S_{t+1}, A_{t+1})$ 叫作“TD 目标”。

现在考虑行动 A_{t+1} 由策略 b 进行采样的情况。在这种情况下，我们通过权重 ρ 来修正 TD 目标（重要性采样）。权重 ρ 是策略为 π 时获得 TD 目标的概率与策略为 b 时获得 TD 目标的概率之比。其数学式如下所示。

$$\rho = \frac{\pi(A_{t+1} | S_{t+1})}{b(A_{t+1} | S_{t+1})}$$

因此，异策略型的 SARSA 的更新式如下所示。

$$\begin{aligned} \text{sampling: } A_{t+1} &\sim b \\ Q'_\pi(S_t, A_t) &= Q_\pi(S_t, A_t) + \alpha \{ \rho (R_t + \gamma Q_\pi(S_{t+1}, A_{t+1})) - Q_\pi(S_t, A_t) \} \end{aligned} \quad (6.13)$$

如上面的式子所示，行动基于策略 b 进行采样，TD 目标基于权重 ρ 进行修正。

6.3.2 异策略型的 SARSA 的实现

下面我们来实现异策略型的 SARSA。代码如下所示。

ch06/sarsa_off_policy.py

```
class SarsaOffPolicyAgent:
    def __init__(self):
        self.gamma = 0.9
        self.alpha = 0.8
        self.epsilon = 0.1
        self.action_size = 4

        random_actions = {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25}
        self.pi = defaultdict(lambda: random_actions)
        self.b = defaultdict(lambda: random_actions)
        self.Q = defaultdict(lambda: 0)
        self.memory = deque(maxlen=2)
```

```

def get_action(self, state):
    action_probs = self.b[state] # ①从行为策略获得
    actions = list(action_probs.keys())
    probs = list(action_probs.values())
    return np.random.choice(actions, p=probs)

def reset(self):
    self.memory.clear()

def update(self, state, action, reward, done):
    self.memory.append((state, action, reward, done))
    if len(self.memory) < 2:
        return

    state, action, reward, done = self.memory[0]
    next_state, next_action, _, _ = self.memory[1]

    if done:
        next_q = 0
        rho = 1
    else:
        next_q = self.Q[next_state, next_action]
        # ②求权重 rho
        rho = self.pi[next_state][next_action] / self.b[next_state][next_action]

    # ③基于 rho 进行 TD 目标的修正
    target = rho * (reward + self.gamma * next_q)
    self.Q[state, action] += (target - self.Q[state, action]) * self.alpha
    # ④改进每个策略
    self.pi[state] = greedy_probs(self.Q, state, 0)
    self.b[state] = greedy_probs(self.Q, state, self.epsilon)

```

下面依次介绍一下代码①～④。

- ① 在取出行动的 `get_action` 方法中，基于 `self.b` 的概率分布取出行动。
- ② 通过重要性采样求权重 `rho`。通过目标策略 `self.pi` 和行为策略 `self.b` 的概率之比可以计算出权重 `rho`。
- ③ 将作为函数更新对象的 TD 目标 (`target`) 乘以权重 `rho`。
- ④ 对目标策略进行贪婪改进，对行为策略进行 ϵ -greedy 改进。

下面使用这里实现的 `SarsaOffPolicyAgent` 类来解决网格世界问题。运行智能代理的代码与 6.2 节相同。图 6-9 展示了得到的结果。

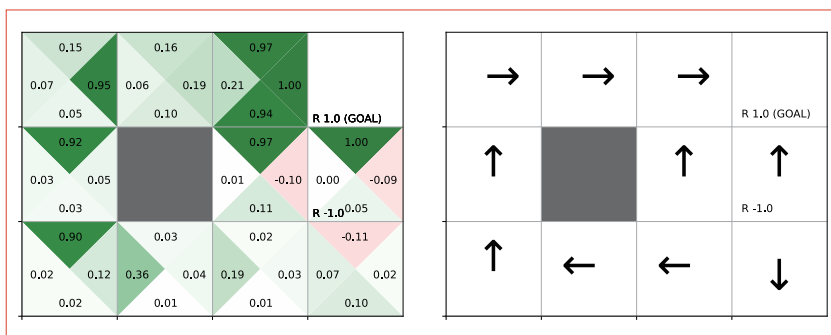


图 6-9 通过异策略型的 SARSA 得到的结果

每次运行的结果各不相同。图 6-9 的结果表明仍有改进的余地。这样我们就完成了异策略型的 SARSA 的实现，下面介绍下一个方法。

6.4 Q 学习

6.3 节实现了异策略型的 SARSA。在异策略型情况下，智能代理拥有两种策略，即行为策略和目标策略。这两种策略起到了不同的作用，行为策略负责探索，目标策略负责利用。这样就有希望得到最优策略。对于异策略型的 SARSA，需要使用重要性采样。不过，如果可以，应该尽可能避免使用重要性采样这种方法。

重要性采样的问题是结果容易变得不稳定。尤其是在两种策略的概率分布不同的情况下，差异越大，重要性采样中使用的权重 ρ 的变化也就越大。这会导致 SARSA 更新式中的目标跟着发生变化，从而使 Q 函数的更新变得不稳定。

解决这个问题的方法就是 Q 学习 (Q-learning)。Q 学习有以下 3 个特点。

- 采用 TD 方法
- 是异策略型
- 不使用重要性采样

在推导 Q 学习之前，先来看一看贝尔曼方程和 SARSA 之间的关系。

之后我们将结合贝尔曼最优方程推导出 Q 学习。总体的流程是由贝尔曼方程推导出 SARSA，由贝尔曼最优方程推导出 Q 学习，如图 6-10 所示。

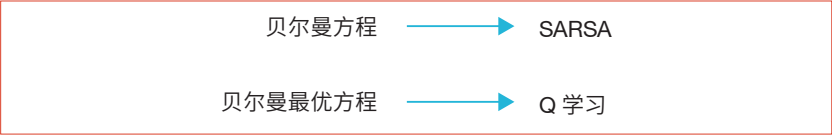


图 6-10 贝尔曼方程与 SARSA 的关系（上）以及贝尔曼最优方程与 Q 学习的关系（下）

6.4.1 贝尔曼方程与 SARSA

先来看看 SARSA 与贝尔曼方程的关系。回顾一下当策略 π 的 Q 函数为 $q_\pi(s,a)$ 时的贝尔曼方程，如下所示。

$$q_\pi(s,a) = \sum_{s'} p(s'|s,a) \left\{ r(s,a,s') + \gamma \sum_{a'} \pi(a'|s') q_\pi(s',a') \right\}$$

在贝尔曼方程中，以下两点需要重点关注。

- 基于环境的状态迁移概率 $p(s'|s,a)$ 来考虑下一步的所有状态迁移。
- 通过智能代理的策略 π 来考虑下一步的所有行动。

从图 6-11 的回溯线形图中，我们能更清晰地看出这两点。

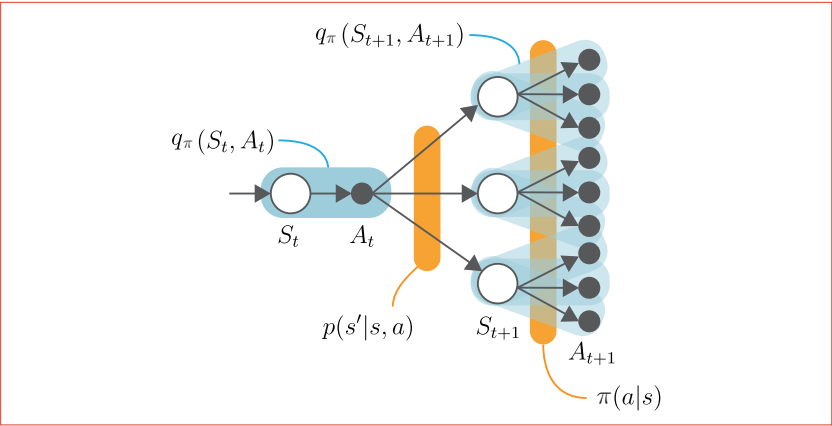


图 6-11 Q 函数的贝尔曼方程的回溯线形图

如图 6-11 所示，贝尔曼方程考虑了下一个状态和下一个行动的所有候选情况。如果从这个角度来看 SARSA，则可以将其实看作贝尔曼方程的“采样版”。“采样版”是指使用采样数据，而不是所有的迁移。SARSA 的回溯线形图如图 6-12 所示。

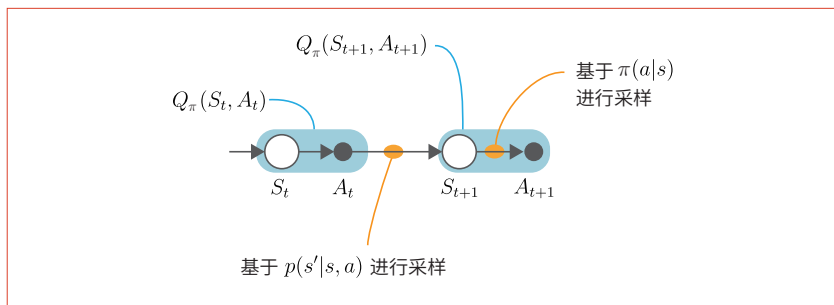


图 6-12 SARSA 的回溯线形图

如图 6-12 所示，SARSA 首先会根据 $p(s'|s, a)$ 对下一个状态 S_{t+1} 进行采样，然后再根据策略 $\pi(a|s)$ 对下一个行动 A_{t+1} 进行采样。此时 SARSA 的 TD 目标就是 $R_t + \gamma Q_\pi(S_{t+1}, A_{t+1})$ 。Q 函数会沿着这个目标的方向缓慢更新。

现在进入正题的探讨。如果贝尔曼方程与 SARSA 相对应，那么我们就就能想到与贝尔曼最优方程相对应的东西。没错，那就是 Q 学习！

6.4.2 贝尔曼最优方程与 Q 学习

在 4.5 节中，我们学习了价值迭代法。价值迭代法是一种将评估和改进两个过程合二为一，以获得最优策略的方法。价值迭代法的要点在于，基于贝尔曼最优方程，只需重复一个更新式，就能得到最优策略。这里我们会探讨一种使用贝尔曼最优方程进行更新的方法，而且是它的“采样版”。

首先来看一下 Q 函数的贝尔曼最优方程，如下所示。

$$q_*(s, a) = \sum_{s'} p(s' | s, a) \left\{ r(s, a, s') + \gamma \max_{a'} q_*(s', a') \right\}$$

这里用 $q_*(s, a)$ 表示最优策略 π_* 中的 Q 函数。与贝尔曼方程不同，贝尔

曼最优方程使用了 $\max$ 算子。这个贝尔曼最优方程的回溯线形图如图 6-13 所示。

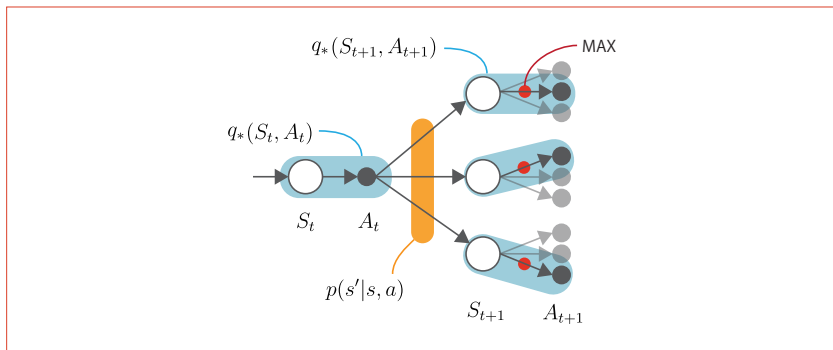


图 6-13 Q 函数的贝尔曼最优方程的回溯线形图

如图 6-13 所示，行动 A_{t+1} 是 Q 函数值最大的行动。下面把图 6-13 改写成“采样版”，如图 6-14 所示。

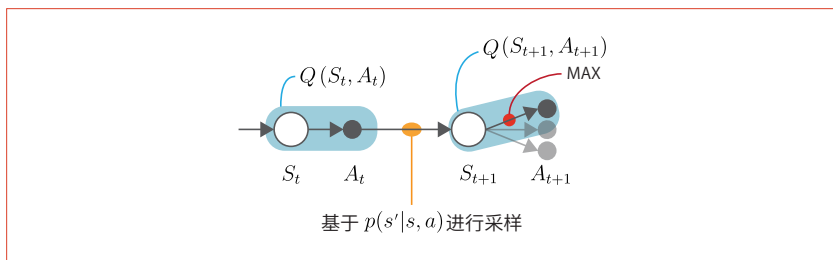


图 6-14 采样版的贝尔曼最优方程的回溯线形图

基于图 6-14 的方法就是 Q 学习。在 Q 学习中，估计值 $Q(S_t, A_t)$ 的目标是 $R_t + \gamma \max_a Q(S_{t+1}, a)$ 。Q 学习会沿着这个目标的方向更新 Q 函数。数学式如下所示。

$$Q'(S_t, A_t) = Q(S_t, A_t) + \alpha \left\{ R_t + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right\} \quad (6.14)$$

基于式 6.14 重复更新 Q 函数，就能接近最优策略的 Q 函数。

图 6-14 的重点是，行动 A_{t+1} 是根据 Q 函数的最大值选择的（老生常谈）。行动 A_{t+1} 并不是根据某个策略采样得到的，而是由 max 算子选择的。因此，无须通过重要性采样进行修正（尽管它是一种异策略型方法）。

下面我们来总结一下 Q 学习。Q 学习是一种异策略型方法。它拥有目标策略和行为策略这两种策略，其中行为策略 b 会进行探索。常用的行为策略是对现在的估计值的 Q 函数进行 ϵ -greedy 化的策略。一旦确定了行为策略，智能代理就会基于该策略选择行动，并收集样本数据。每当智能代理采取行动时，都会使用式 6.14 更新 Q 函数。这就是 Q 学习。

6.4.3 Q 学习的实现

下面来实现 Q 学习。代码如下所示。

ch06/q_learning.py

```
from collections import defaultdict
import numpy as np
from common.gridworld import GridWorld
from common.utils import greedy_probs

class QLearningAgent:
    def __init__(self):
        self.gamma = 0.9
        self.alpha = 0.8
        self.epsilon = 0.1
        self.action_size = 4

        random_actions = {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25}
        self.pi = defaultdict(lambda: random_actions)
        self.b = defaultdict(lambda: random_actions) # 行为策略
        self.Q = defaultdict(lambda: 0)

    def get_action(self, state):
        action_probs = self.b[state] # 从行为策略获取
        actions = list(action_probs.keys())
        probs = list(action_probs.values())
        return np.random.choice(actions, p=probs)

    def update(self, state, action, reward, next_state, done):
        if done:
            next_q_max = 0
```

```

else:
    next_qs = [self.Q[next_state, a] for a in range(self.action_size)]
    next_q_max = max(next_qs)

    target = reward + self.gamma * next_q_max
    self.Q[state, action] += (target - self.Q[state, action]) * self.alpha

    self.pi[state] = greedy_probs(self.Q, state, epsilon=0)
    self.b[state] = greedy_probs(self.Q, state, self.epsilon)

```

这里值得关注的是 `update(self, state, action, reward, next_state, done)` 的参数。在 Q 学习中，只需要 `state`、`action`、`reward`、`next_state` 和 `done` 这 5 个信息就可以更新 Q 函数。`update` 方法会取出下一个状态下的 Q 函数的最大值。然后使用基于贝尔曼最优方程的式 6.14 更新 Q 函数。如果更新 Q 函数，那么就对行为策略 `self.b` 进行 ϵ -greedy 更新，对目标策略 `self.pi` 进行贪婪更新。

下面运行 `QLearningAgent` 类。代码如下所示。

ch06/q_learning.py

```

env = GridWorld()
agent = QLearningAgent()

episodes = 10000
for episode in range(episodes):
    state = env.reset()

    while True:
        action = agent.get_action(state)
        next_state, reward, done = env.step(action)

        agent.update(state, action, reward, next_state, done)
        if done:
            break
        state = next_state

env.render_q(agent.Q)

```

上面的代码将绘制 Q 函数的值和智能代理的目标策略。结果如图 6-15 所示。

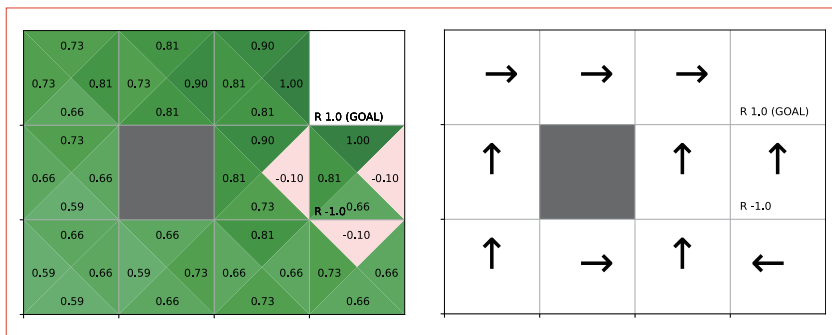


图 6-15 通过 Q 学习获得的 Q 函数和策略

虽然结果每次都会发生变化，但在大多数情况下，我们能获得最优策略。图 6-15 中的结果也是最优策略。这样我们就完成了 Q 学习的实现。

6.5 分布模型与样本模型

在本章中，我们学习了 TD 方法。具体的算法是 SARSA 和 Q 学习。本章要学习的强化学习算法就是这些。本节将补充介绍智能代理的实现方法。首先介绍与智能代理的实现方法相关的“分布模型”和“样本模型”。之后你将了解到本章前面实现的都是分布模型，而样本模型的实现更简单。

6.5.1 分布模型与样本模型

实现随机行动有两种方法，分别是分布模型和样本模型。在 5.1.2 节中，我们已经了解到与环境相关的模型有分布模型和样本模型，它们同样适用于智能代理。确定智能代理行动的方式也可以基于分布模型或样本模型来实现。

分布模型是一种明确保持概率分布的模型。例如，一个随机行动的智能代理可以作为分布模型来实现，如下所示。

```
from collections import defaultdict
import numpy as np
```

```
class RandomAgent:
    def __init__(self):
        random_actions = {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25} # 概率分布
        self.pi = defaultdict(lambda: random_actions)

    def get_action(self, state):
        action_probs = self.pi[state]
        actions = list(action_probs.keys())
        probs = list(action_probs.values())
        return np.random.choice(actions, p=probs) # 采样
```

上面的代码会将每个状态下行动的概率分布保存为 `self.pi`，然后使用该概率分布对实际行动进行采样。这就是使用分布模型的智能代理的实现方式。这种显式地保留概率分布的做法是分布模型的特征。

在分布模型之外，智能代理的另一种实现方式是样本模型。样本模型是只将采样作为条件的模型。由于不需要持有概率分布，因此它比分布模型更容易实现。例如，对于执行随机行动的智能代理，可以将其作为样本模型来实现。

```
class RandomAgent:
    def get_action(self, state):
        return np.random.choice(4)
```

上面的代码不需要概率分布，只需从 4 个行动中随机选取一个即可。即使是如此简单的代码，也能实现采取随机行动的智能代理。由于它不需要显式的概率分布，因此可以用很少的代码来编写。

6.5.2 样本模型版的 Q 学习

下面来看看 Q 学习。首先回顾一下 6.4 节实现的 Q 学习。当时我们是将智能代理作为分布模型来实现的。再来看一下代码。

```
from collections import defaultdict
import numpy as np
from common.utils import greedy_probs

class QLearningAgent:
    def __init__(self):
        self.gamma = 0.9
```

```

self.alpha = 0.8
self.epsilon = 0.1
self.action_size = 4

random_actions = {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25}
self.pi = defaultdict(lambda: random_actions) # 目标策略
self.b = defaultdict(lambda: random_actions) # 行为策略
self.Q = defaultdict(lambda: 0)

def get_action(self, state):
    action_probs = self.b[state]
    actions = list(action_probs.keys())
    probs = list(action_probs.values())
    return np.random.choice(actions, p=probs)

def update(self, state, action, reward, next_state, done):
    if done:
        next_q_max = 0
    else:
        next_qs = [self.Q[next_state, a] for a in range(self.action_size)]
        next_q_max = max(next_qs)

    target = reward + self.gamma * next_q_max
    self.Q[state, action] += (target - self.Q[state, action]) * self.alpha

    # pi 是贪婪的, b 是  $\epsilon$ -greedy 的
    self.pi[state] = greedy_probs(self.Q, state, epsilon=0)
    self.b[state] = greedy_probs(self.Q, state, self.epsilon)

```

在这段代码中，我们关注的两个策略是 `self.pi` 和 `self.b`，它们都是作为概率分布被保存的。因此，这个模型是分布模型。还要注意的，在 `update` 方法中，`self.pi` 和 `self.b` 会被更新。



`update` 方法会更新策略在 `state` 下的概率分布。此时，对 `Q` 函数 (`self.Q`) 进行贪婪更新的策略是 `self.pi`，进行 ϵ -greedy 更新的策略是 `self.b`。

在实现样本模型之前，作为准备工作，我们先简化上面的代码。以下是需要做的两处修改。

- 删除 `self.pi`。
- 在 `get_action` 方法中进行 `self.b` 的更新。

在介绍之前，先来看一下代码。

```
class QLearningAgent:
    def __init__(self):
        self.gamma = 0.9
        self.alpha = 0.8
        self.epsilon = 0.1
        self.action_size = 4

        random_actions = {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25}
        # self.pi = ... # 不使用 self.pi
        self.b = defaultdict(lambda: random_actions)
        self.Q = defaultdict(lambda: 0)

    def get_action(self, state):
        # 此时  $\epsilon$ -greedy 化
        self.b[state] = greedy_probs(self.Q, state, self.epsilon)

        action_probs = self.b[state]
        actions = list(action_probs.keys())
        probs = list(action_probs.values())
        return np.random.choice(actions, p=probs)

    def update(self, state, action, reward, next_state, done):
        if done:
            next_q_max = 0
        else:
            next_qs = [self.Q[next_state, a] for a in range(self.action_size)]
            next_q_max = max(next_qs)

        target = self.gamma * next_q_max + reward
        self.Q[state, action] += (target - self.Q[state, action]) * self.alpha
```

首先删除目标策略 `self.pi`。`self.pi` 是对 `Q` 函数 (`self.Q`) 进行贪婪更新的策略，之前每次调用 `update` 方法时都会更新。但其他代码目前并不需要 `self.pi`。因此，可以删除 `self.pi`。如果需要目标策略，则可以直接通过贪婪化 `Q` 函数来创建。

接下来是行为策略 `self.b`。在前面的代码中，它是在 `update` 方法中更新的。这次把它改为在 `get_action` 方法被调用时进行更新。由于 `self.b` 是对 `Q` 函数进行 ϵ -greedy 的策略，因此只要有 `Q` 函数，就可以随时创建 `self.b`。

下面将上面的代码改为样本模型。代码如下所示。

ch06/q_learning_simple.py

```

class QLearningAgent:
    def __init__(self):
        self.gamma = 0.9
        self.alpha = 0.8
        self.epsilon = 0.1
        self.action_size = 4
        self.Q = defaultdict(lambda: 0)

    def get_action(self, state):
        if np.random.rand() < self.epsilon:
            return np.random.choice(self.action_size)
        else:
            qs = [self.Q[state, a] for a in range(self.action_size)]
            return np.argmax(qs)

    def update(self, state, action, reward, next_state, done):
        if done:
            next_q_max = 0
        else:
            next_qs = [self.Q[next_state, a] for a in range(self.action_size)]
            next_q_max = max(next_qs)

        target = self.gamma * next_q_max + reward
        self.Q[state, action] += (target - self.Q[state, action]) * self.alpha

```

与之前的代码相比，唯一的变化是删除了行为策略 `self.b`。 `get_action` 方法不使用 `self.b`，而是直接使用 `Q` 函数，利用 ϵ -greedy 选择行动。具体来说，它会以 `self.epsilon = 0.1` 的概率选择随机行动，否则就选择 `Q` 函数值最大的行动。这样就实现了通过 ϵ -greedy 选择行动。

可以看出，上面的代码并没有将策略作为概率分布来保存（更确切地说，它根本没有保存策略）。这就是样本模型的实现。由于不需要保存概率分布，因此其实现起来很简单。从第 7 章开始，我们将使用神经网络对 `Q` 学习进行扩展，并以此处展示的样本模型的实现为基础介绍后续内容。

6.6 小结

本章介绍了 TD 方法。与蒙特卡洛方法一样，TD 方法可以根据智能代理的实际行动结果评估价值函数。TD 方法的特点是只根据当前和下一步的

信息更新价值函数。而蒙特卡洛方法只有在智能代理达到目标后才会更新价值函数。这意味着 TD 方法有望更高效地更新价值函数。

使用 TD 方法进行策略控制有两种典型算法，一种是 SARSA，另一种是 Q 学习。一般来说，SARSA 是一种同策略型的方法，它使用 TD 方法对 Q 函数进行评估，并通过 ϵ -greedy 算法对策略进行改进。另外，它还通过 ϵ -greedy 算法进行探索和利用。在本章中，我们还了解到 SARSA 也可以扩展到异策略型。

本章还基于贝尔曼最优方程推导出了 Q 学习。Q 学习是一种异策略型的方法，它可以在不使用重要性采样的情况下高效稳定地更新 Q 函数。在强化学习领域，Q 学习是一种特别重要的算法。我们终于获得了 Q 学习这个重要的“武器”。

第 7 章

神经网络和 Q 学习

到目前为止，我们只处理了状态和行动规模较小的问题。例如，在 3×4 网格世界问题中，有 12 个候选状态和 4 个候选行动，因此候选 Q 函数的总数为 $12 \times 4 = 48$ 个。在这样的规模下，将 Q 函数保存为表格（在 Python 代码中保存为字典）是没有问题的，但实际情况更为复杂。状态和候选行动的数量可能非常庞大。对于这类问题，像以前那样将 Q 函数作为表格来保存是不现实的。

例如，国际象棋的棋局模式有 10 的 123 次方那么多，这意味着状态也有那么多。如此多的状态实际上不可能以表格的形式保存下来。更重要的是，我们还要单独评估和改进表格中的一个要素。遍历如此庞大的数量是不现实的。

为了解决这个问题，可以考虑用简洁紧凑的函数来近似 Q 函数。为此最有力的方法是深度学习。迄今为止，强化学习和深度学习的结合产生了很多创新，而我们也终于要向深度强化学习“进军”了。

本章首先介绍深度学习框架 DeZero 的使用方法，然后使用 DeZero 学习神经网络的基础知识，最后使用神经网络实现第 6 章中已实现的 Q 学习。



具有深“层”的神经网络就是深度学习。本书不区分“深度学习”和“神经网络”这两个术语。

7.1 DeZero 简介

接下来的几节将结合代码对神经网络（深度学习）进行介绍。我们首先学习深度学习的框架，然后解决机器学习的基础问题（线性回归），最后实现神经网络。



本书的主题是“强化学习”。因此，接下来说明的与深度学习相关的内容只限于作为“工具”使用。如果你已经了解深度学习，则可以跳过这部分内容，直接进入 7.4 节。

本书使用的是一个名为 DeZero 的深度学习框架。DeZero 是“深度学习入门 & 进阶”系列的第 3 本书《深度学习入门 2：自制框架》中创建的框架，如图 7-1 所示。



图 7-1 《深度学习入门 2：自制框架》和 DeZero

DeZero 基于 PyTorch 创建，是根据一个重视“易于理解”的理念而设计的框架。当然，如果是第一次使用 DeZero，你也不用担心。如果有使用过 PyTorch 或 TensorFlow 等最新的深度学习框架的经验，你将很快熟悉 DeZero。本节将简要介绍 DeZero 的使用方法。



本书会使用 DeZero 进行讲解。不过由于 DeZero 是 PyTorch 的一种“方言”，因此我们可以轻松地将 DeZero 代码改写为 PyTorch 代码。在本书的支持页面（GitHub）上也有 PyTorch 版的代码。如果你想使用 PyTorch，请参考这些代码。

7.1.1 使用 DeZero

首先安装 DeZero。可以使用 pip 按如下方式安装 DeZero。

```
$ pip install dezero
```

安装结束后，需要试用一下 DeZero。可以先从 Variable 类开始。Variable 是 NumPy 的多维数组 (np.ndarray) 的封装类。这个类的使用方法如下所示。

```
import numpy as np
from dezero import Variable

x_np = np.array(5.0)
x = Variable(x_np)

y = 3 * x ** 2
print(y)
```

输出结果

```
variable(75.0)
```

上面的代码会从 dezero 模块导入 Variable 类。通过 `x = Variable(x_np)` 使 x 成为 Variable 的实例。之后，我们就可以像处理普通的 np.ndarray 那样进行计算了。上面的代码中包含 `y = 3 * x ** 2`，运行后立即可得到结果 75.0。

接下来试着求导数。这里调用 backward 方法。紧接上面的代码，运行以下代码。

```
y.backward()
print(x.grad)
```

输出结果

```
variable(30.0)
```

输出 y 是一个 Variable 实例。通过对该输出 y 调用 backward 方法，执行反向传播 (backpropagation)，可以求出各变量的导数。另外，上面的代码进行了 `y = 3 * x ** 2` 的计算，这个计算的数学式是 $y = 3x^2$ 。它的导数是 $\frac{dy}{dx} = 6x$ ，代入 $x = 5$ ，结果是 30。的确与上面的结果一致。

7.1.2 多维数组（张量）和函数

机器学习一般处理的是多维数组（张量）。多维数组是汇总处理多个数值（元素）的数据结构。元素的排列是有“方向”的，这个方向被称为“维度”或“轴”。图 7-2 是多维数组的示例。



图 7-2 多维数组示例

图 7-2 中从左到右依次是零维数组、一维数组和二维数组，分别叫作“标量”“向量”和“矩阵”。标量表示单个数值。向量会沿着一个轴排列数值，矩阵则会沿着两个轴排列数值。另外，多维数组也叫作“张量”。因此，图 7-2 中的示例从左到右依次也被称为“零阶张量”“一阶张量”和“二阶张量”。

下面介绍向量的内积。这里假设有两个向量，即 $a = (a_1, \dots, a_n)$ 和 $b = (b_1, \dots, b_n)$ 。此时，向量的内积的定义如式 7.1 所示。

$$a \cdot b = a_1b_1 + a_2b_2 + \dots + a_nb_n \tag{7.1}$$

如式 7.1 所示，两个向量之间相应元素的乘积相加就是向量的内积。



在数学式中标记符号时，如果是标量，就用 a 、 b 这样的符号标记它们；如果是向量或矩阵，则用 $\mathbf{a}$ 、 $\mathbf{b}$ 这样的粗斜体字来标记它们。

最后介绍矩阵的乘积。矩阵的乘积是根据图 7-3 中的步骤进行计算的。

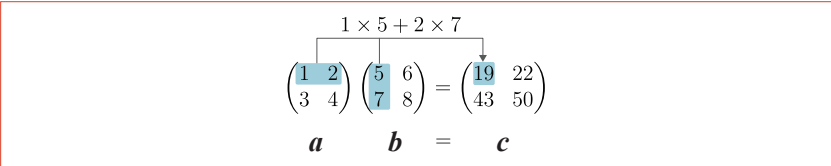


图 7-3 矩阵乘积的计算方法

如图 7-3 所示，矩阵的乘积计算是在左侧矩阵的“横向排列的向量”和右侧矩阵的“纵向排列的向量”之间计算内积，然后将结果存储在新矩阵的相应元素中。例如， a 的第 1 行和 b 的第 1 列的内积是 c 的第 1 行第 1 列的元素， a 的第 2 行和 b 的第 1 列的内积是 c 的第 2 行第 1 列的元素，以此类推。

下面使用 DeZero 计算向量的内积和矩阵的乘积。为此，我们使用 `dezero.functions` 包中的 `matmul` 函数。

ch07/dezero1.py

```
import numpy as np
from dezero import Variable
import dezero.functions as F

# 向量的内积
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
a, b = Variable(a), Variable(b) # 可以省略
c = F.matmul(a, b)
print(c)

# 矩阵的乘积
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])
c = F.matmul(a, b)
print(c)
```

输出结果

```
variable(32)
variable([[19 22]
          [43 50]])
```

这里计算了向量的内积和矩阵的乘积。为此，请像上面的代码那样，首先通过 `import dezero.functions as F` 进行导入，这样就可以将 DeZero 函数作为 `F.matmul` 来使用了。如上面的代码所示，向量的内积和矩阵的乘积的计算都可以使用 `F.matmul` 函数。



DeZero 函数可以直接处理 `np.ndarray` 实例（此时，它将在 DeZero 内部将实例转换为 `Variable` 实例）。因此，上面代码中的 `np.ndarray` 实例 `a` 和 `b` 可直接作为 `F.matmul()` 的输入，而无须显式转换为 `Variable` 实例。

另外，在矩阵和向量的计算中，注意它们的“形状”是非常重要的。例如，在矩阵的乘积的计算中，存在图 7-4 所示的关系。

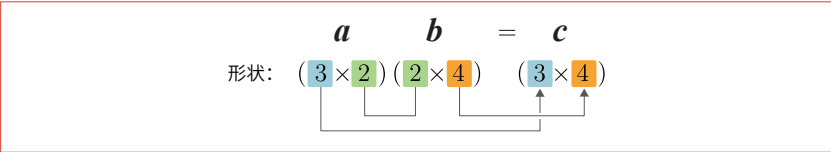


图 7-4 在计算矩阵的乘积时，对应的维度（轴）的元素数量应相同

在图 7-4 的例子中， 3×2 的矩阵 `a` 和 2×4 的矩阵 `b` 的乘积是 3×4 的矩阵 `c`。此时，矩阵 `a` 和 `b` 的对应维度（轴）的元素数量必须一致。作为结果的矩阵 `c` 的大小由矩阵 `a` 的行数和矩阵 `b` 的列数决定。

7.1.3 最优化

下面使用 DeZero 来解决简单的问题：找出以下式子表示的函数的最小值。

$$y = 100(x_1 - x_0^2)^2 + (x_0 - 1)^2$$

这个函数叫作“Rosenbrock 函数”。关于 Rosenbrock 函数真正的最小值的探索是很难的，由于这个函数的特殊形状，因此它被作为最优化的基准问题得到了广泛使用。这里的目标是找到使 Rosenbrock 函数的输出最小的 x_0 和 x_1 。先来揭晓答案，Rosenbrock 函数的最小值为 $(x_0, x_1) = (1, 1)$ 。我们使用 DeZero 来确认是否能够找出它的最小值。



找到使函数取最小值或最大值的函数参数（输入）的操作叫作最优化。这里的目标是使用 DeZero 解决最优化问题。

首先求 Rosenbrock 函数在 $(x_0, x_1) = (0.0, 2.0)$ 处的导数，用数学式来表示是 $\frac{\partial y}{\partial x_0}$ 和 $\frac{\partial y}{\partial x_1}$ 。用 DeZero 则需执行以下操作。

```
import numpy as np
from dezero import Variable

def rosenbrock(x0, x1):
    y = 100 * (x1 - x0 ** 2) ** 2 + (x0 - 1) ** 2
    return y

x0 = Variable(np.array(0.0))
x1 = Variable(np.array(2.0))

y = rosenbrock(x0, x1)
y.backward()
print(x0.grad, x1.grad)
```

输出结果

```
variable(-2.0) variable(400.0)
```

如上面的代码所示，先通过 Variable 封装数值数据（np.ndarray 实例），完全按照公式进行编码即可。之后只需调用 `y.backward()` 即可自动求导。

运行上面的代码，得到在 x_0 和 x_1 处的导数（ $\frac{\partial y}{\partial x_0}$ 和 $\frac{\partial y}{\partial x_1}$ ）分别为 -2.0 和 400.0。我们将这两个导数的集合 $(-2.0, 400.0)$ 及其向量形式分别称为梯度（gradient）和梯度向量。梯度表示在每个点上函数输出增加最快的方向。拿上面的例子来说，在 $(x_0, x_1) = (0.0, 2.0)$ 处， y 值增加最快的方向就是 $(-2.0, 400.0)$ 。反过来说，梯度乘以 -1 得到的 $(-2.0, 400.0)$ 的方向就是 y 值减少最快的方向。



对于形状复杂的函数，在大多数情况下，梯度指示的方向不一定有最大值（或者在与梯度相反的方向不一定有最小值）。但如果只看局部区域，那么梯度指示的就是函数输出增加最大的方向。因此，如果重复进行沿着梯度的方向前进某个距离、在新的地点再次求出梯度，我们就有望慢慢接近目标位置（最大值或最小值）。这就是梯度下降法（gradient descent）。

下面把梯度下降法应用到我们的问题上。这里所说的问题是找到 Rosenbrock 函数的“最小值”。因此，我们要沿着梯度乘以 -1 的方向前进。考虑到这一点，可以执行以下操作。

ch07/dezero2.py

```
x0 = Variable(np.array(0.0))
x1 = Variable(np.array(2.0))

lr = 0.001 # 学习率
iters = 10000 # 重复次数

for i in range(iters):
    print(x0, x1)
    y = rosenbrock(x0, x1)

    x0.cleargrad()
    x1.cleargrad()
    y.backward()

    x0.data -= lr * x0.grad.data
    x1.data -= lr * x1.grad.data

print(x0, x1)
```

上面的代码已将重复更新的次数设置为 `iters`（这个 `iters` 是 `iterations` 的缩写）。另外，还要预先设置与梯度相乘的值。上面的代码中已经设置了 `lr = 0.001`。这个 `lr` 是 `learning rate` 的首字母缩写，表示“学习率”。实际的变量更新的代码是 `x0.data -= lr * x0.grad.data`。

需要注意的是，`x0` 和 `x0.grad` 都是 `Variable` 实例。实际的数据（`np.ndarray`）在 `x0.data` 和 `x0.grad.data` 中。由于这里只更新数据，因此会对 `.data` 属性直接进行计算。如果对 `Variable` 实例进行了计算，则后面要进行额外的反向传播计算。



上面代码的 `for` 语句部分重复使用 `Variable` 实例 `x0` 和 `x1` 进行了求导。此时，因为导数值被一次次地加到了 `x0.grad` 和 `x1.grad` 中，所以在重新求导时，需要将此前已被加过的导数重置一次。因此，在进行反向传播之前，需要调用各变量的 `cleargrad` 方法来重置导数。

运行上面的代码。代码会更新 (x_0 , x_1) 的值，最终得到以下结果。

输出结果

```
variable(0.9944984367782456) variable(0.9890050527419593)
```

由于本次问题的答案为 (1.0, 1.0)，因此上面的结果虽然不完全正确，但也是大致近似的值。

以上是对 DeZero 的基础知识的介绍。下面让我们使用 DeZero 来解决机器学习的问题。

7.2 线性回归

机器学习使用数据解决问题。不需要人来思考问题的解决方法，机器学习是让计算机从收集的数据中发现（学习）问题的解决方法。这种从数据中找到解法的做法是机器学习的核心。这里我们使用 DeZero 解决机器学习的问题。首先来实现机器学习中最基本的线性回归。

7.2.1 玩具数据集

下面使用 DeZero 来解决实践性问题。首先，制作实验用的小数据集。这样的小数据集叫作玩具数据集 (toy dataset)。考虑到复现性，我们固定随机数的种子以生成数据，代码如下所示。

```
import numpy as np

np.random.seed(0) # 固定随机数的种子
x = np.random.rand(100, 1)
y = 5 + 2 * x + np.random.rand(100, 1)
```

上面的代码创建了由 x 和 y 两个变量构成的数据集。此时，点群位于直线上，代码将 y 加上作为噪声的随机数。绘制这些 (x , y) 点，得到的图形如图 7-5 所示。

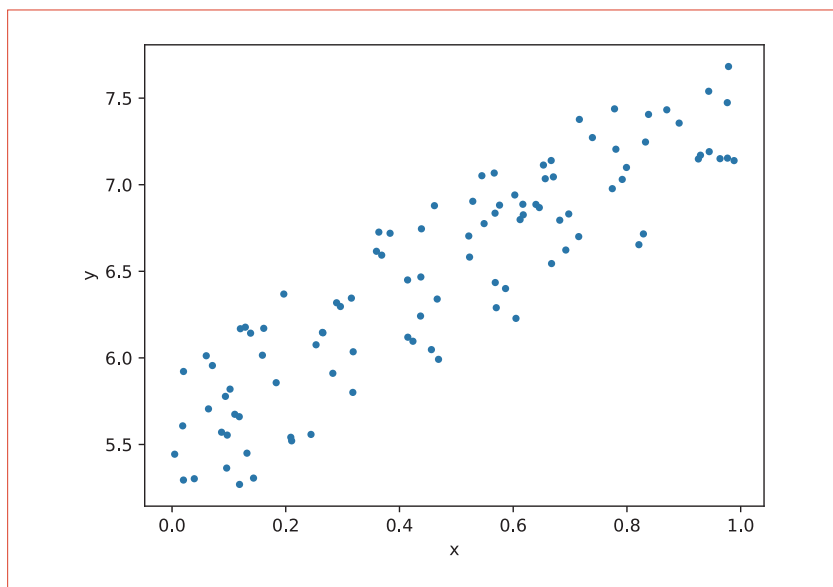


图 7-5 含有噪声的数据集

如图 7-5 所示， x 和 y 呈“线性”关系，但数据中包含噪声。我们的目标是建立能够基于 x 值预测 y 值的模型（数学式）。



根据 x 的值预测实数 y 的过程叫作回归（regression）。当预测的模型为线性（直线）时，这个过程叫作“线性回归”。

7.2.2 线性回归的理论知识

接下来的目标是找到拟合给定数据的函数。这里假设 y 和 x 之间的关系是线性的，所以可以用式子 $y = Wx + b$ 来表示（式子中的 W 是标量）。直线 $y = Wx + b$ 的图形如图 7-6 所示。

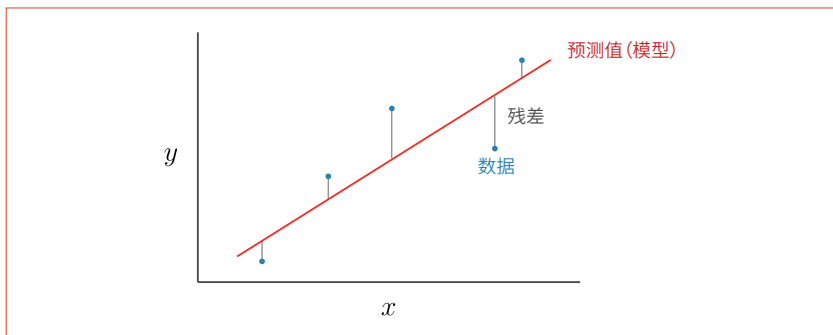


图 7-6 线性回归的示例

如图 7-6 所示，我们的目标是找到拟合数据的直线 $y = Wx + b$ 。为此，需要尽可能减少数据与预测值之间的差，这个差叫作残差（residual）。这里将表示模型预测值和数据的匹配程度的指标定义为以下式子。

$$L = \frac{1}{N} \sum_{i=1}^N (Wx_i + b - y_i)^2 \quad (7.2)$$

式 7.2 假设一共有 N 个点，求出 (x_i, y_i) 各点的误差的平方，并将它们相加。最后为了求平均值，再除以 N 。这个式子叫作均方差（mean squared error）。另外，式 7.2 的定义是 $\frac{1}{N}$ …，但也有定义为 $\frac{1}{2N}$ …的情况。无论是哪一种定义，在用梯度下降法求解时，同样都是通过调整学习率的值来解决问题。



评估模型好坏的函数是损失函数（loss function）。因此，在线性回归问题中，可以使用均方差作为损失函数。

我们的目标是找到使式 7.2 表示的损失函数最小的 W 和 b 。这是函数最优化问题。7.1 节使用梯度下降法解决了这样的问题。这里同样可以使用这种方法。

7.2.3 线性回归的实现

下面使用 DeZero 实现线性回归。本节会将代码分为前半部分和后半部分予以介绍。首先是前半部分的代码。

ch07/dezero3.py

```

import numpy as np
from dezero import Variable
import dezero.functions as F

# 玩具数据集
np.random.seed(0)
x = np.random.rand(100, 1)
y = 5 + 2 * x + np.random.rand(100, 1)
x, y = Variable(x), Variable(y) # 可以省略

W = Variable(np.zeros((1, 1)))
b = Variable(np.zeros(1))

def predict(x):
    y = F.matmul(x, W) + b
    return y

```

上面的代码创建了参数 W 和 b ，二者是 `Variable` 实例 (W 为大写字母)。至于形状， W 为 $(1, 1)$ ， b 为 $(1,)$ 。另外，上面的代码定义了 `predict` 函数。该函数使用求矩阵乘积的 `matmul` 函数进行计算。通过使用矩阵乘积，可以一次性地对多个数据（在上面的示例中是 100 个数据）进行计算。此时形状的变化如图 7-7 所示。

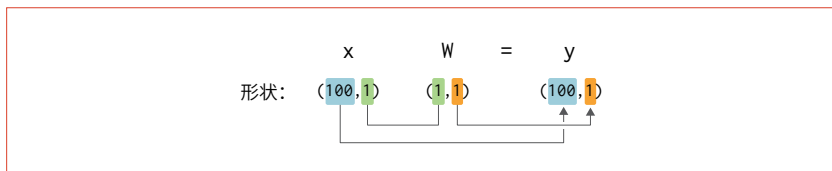


图 7-7 矩阵乘积的形状的变化（省略与 b 的加法计算）

从图 7-7 可以看出，对应维度的元素数相同。作为结果的 y 的形状为 $(100, 1)$ ，也就是对于拥有 100 个数据的 x 分别进行了与 W 的乘积运算。这样就能通过一次性计算求出所有数据的预测值。另外，这里的 x 的数据维度为 1，但当维度为 D 时，只要使 W 的形状为 $(D, 1)$ ，就同样能够对各数据进行计算。如果 $D = 4$ ，那么代码就会像图 7-8 所示那样计算矩阵的乘积。

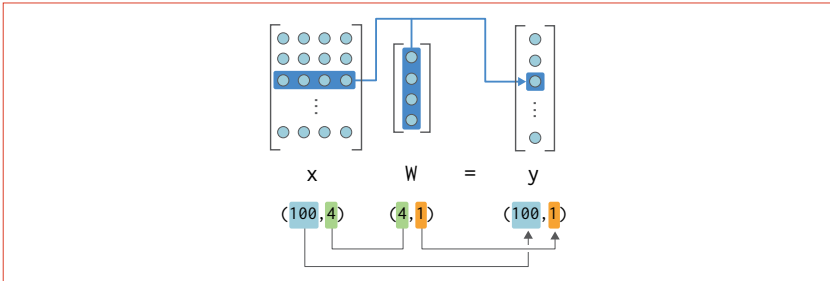


图 7-8 矩阵乘积的形状的变化 (x 的数据维度为 4 的情况)

如图 7-8 所示, 使 $x.shape[1]$ 和 $W.shape[0]$ 相等, 就可以正确地计算矩阵的乘积。此时, 对 100 个数据分别进行与 W 的向量内积计算。下面是后半部分的代码。

ch07/dezero3.py

```
def mean_squared_error(x0, x1):
    diff = x0 - x1
    return F.sum(diff ** 2) / len(diff)

lr = 0.1
iters = 100

for i in range(iters):
    y_pred = predict(x)
    loss = mean_squared_error(y, y_pred)
    # 或者是 loss = F.mean_squared_error(y, y_pred)
    W.cleargrad()
    b.cleargrad()
    loss.backward()

    W.data -= lr * W.grad.data
    b.data -= lr * b.grad.data

    if i % 10 == 0: # 每隔 10 次进行输出
        print(loss.data)

print('====')
print('W =', W.data)
print('b =', b.data)
```

输出结果

```
42.296340129442335
0.24915731977561134
0.10078974954301652
```

```

0.09461859803040694
0.0902667138137311
0.08694585483964615
0.08441084206493275
0.0824757102229121
0.08099850454041051
0.07987086218625004
====
W = [[2.11807369]]
b = [5.46608905]

```

上面的代码将求均方差的函数实现为了 `mean_squared_error(x0, x1)`。只需使用 `DeZero` 函数即可实现式 7.2，然后通过梯度下降法更新参数。另外，`DeZero` 中也准备了求均方差的函数 `F.mean_squared_error`。

运行上面的代码。从结果可以看出，损失函数的输出值逐渐减少。最终得到的值为 `W = [[2.11807369]]` 和 `b = [5.46608905]`。作为参考，我们绘制出了由这些参数确定的直线的图形，如图 7-9 所示。

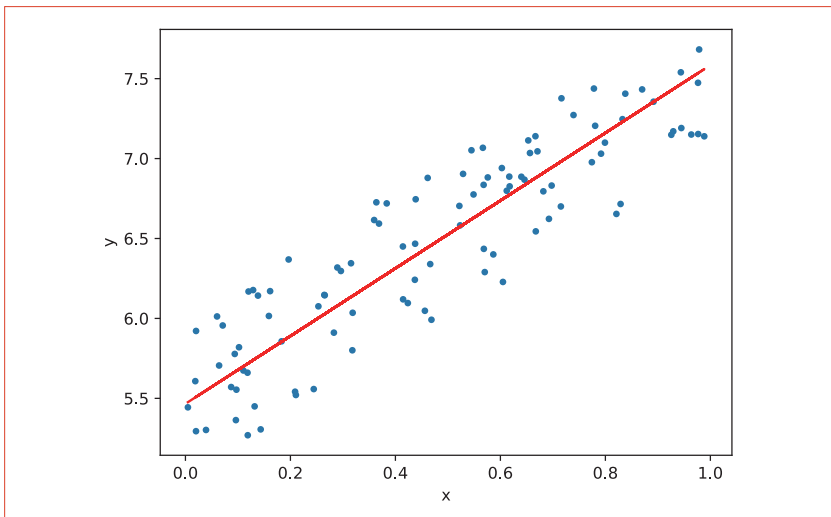


图 7-9 训练后的模型

如图 7-9 所示，我们成功得到了拟合数据的模型。我们使用 `DeZero` 正确地实现了线性回归。

以上就是对线性回归的实现的所有介绍。

7.3 神经网络

在 7.2 节中，我们使用 DeZero 实现了线性回归，并使其能够正常工作。一旦实现了线性回归，就很容易将其扩展到神经网络。本节将通过修改 7.2 节的代码，使用 DeZero 实现神经网络。

7.3.1 非线性数据集

7.2 节使用了在直线上分布的数据集。这里将通过以下代码生成更复杂的数据集。

```
import numpy as np

np.random.seed(0)
x = np.random.rand(100, 1)
y = np.sin(2 * np.pi * x) + np.random.rand(100, 1)
```

上面的代码使用 `sin` 函数生成数据。绘制这些 (x, y) 点，得到的图形如图 7-10 所示。

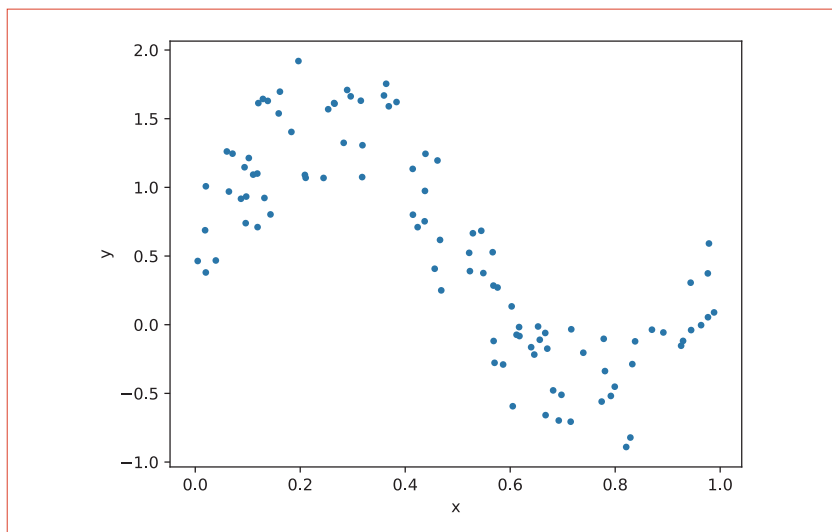


图 7-10 本节使用的数据集

如图 7-10 所示， x 和 y 非线性相关。这种非线性数据集当然无法通过线性回归来处理。此时就该神经网络“登场”了。

7.3.2 线性变换和激活函数

7.2 节以简单的数据集为对象，实现了线性回归。线性回归中执行的计算只有矩阵的乘积和加法运算（损失函数除外）。这部分代码如下所示。

```
y = F.matmul(x, W) + b
```

如上面的代码所示，先求输入 x 和参数 W 的矩阵乘积，然后再加上 b 。这种变换叫作线性变换（linear transformation）或仿射变换（affine transformation）。DeZero 提供了线性变换的函数 `F.linear`。它的具体使用方法如下所示。

```
y = F.linear(x, W, b)
```



严格来说，线性变换是 $y = F.matmul(x, W)$ ，不包括加上 b 的部分。不过，在神经网络领域，通常将包含 b 的加法运算在内的计算称为“线性变换”（本书也沿用了这一做法）。此外，线性变换也用于神经网络的全连接层。参数 W 叫作权重（weight），参数 b 叫作偏置（bias）。

线性变换是对输入数据进行线性的变换，而神经网络是对线性变换的输出进行非线性的变换。进行非线性变换的函数叫作“激活函数”。典型的激活函数有 sigmoid 函数、ReLU 函数等，如图 7-11 所示。

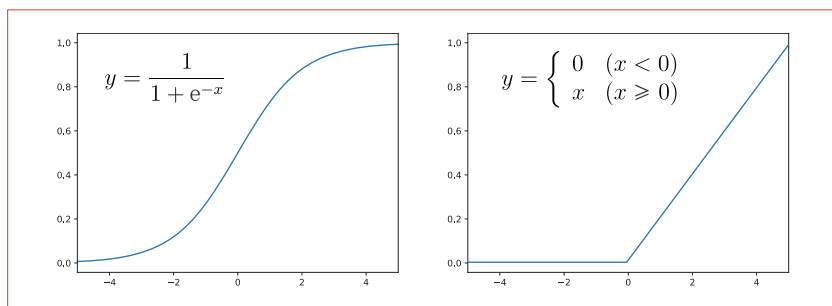


图 7-11 sigmoid 函数（左）和 ReLU 函数（右）

如图 7-11 所示，sigmoid 函数和 ReLU 函数都是非线性函数（非“直线”函数）。在神经网络中，图 7-11 所示的非线性变换适用于张量的每个元素。在 DeZero 中，F.sigmoid 函数可作为 sigmoid 函数使用，F.relu 函数则可作为 ReLU 函数使用。

7.3.3 神经网络的实现

典型的神经网络会交替使用线性变换和激活函数。例如，两层的神经网络的实现如下所示（这里省略了生成参数的代码）。

```
W1, b1 = Variable(...), Variable(...)
W2, b2 = Variable(...), Variable(...)

def predict(x):
    y = F.linear(x, W1, b1)
    y = F.sigmoid(y)
    y = F.linear(y, W2, b2)
    return y
```

上面的代码依次应用了线性变换和激活函数。这就是用于神经网络的推理（predict）的代码。当然，要正确执行推理，还需要训练（train）。在训练神经网络的过程中，我们会在推理过程之后添加损失函数，然后再找出使该损失函数的输出最小的参数。下面我们使用实际的数据集来训练神经网络。整体的代码如下所示。

```
import numpy as np
from dezero import Variable
import dezero.functions as F

# 数据集
np.random.seed(0)
x = np.random.rand(100, 1)
y = np.sin(2 * np.pi * x) + np.random.rand(100, 1)

# ①参数的初始化
I, H, O = 1, 10, 1
W1 = Variable(0.01 * np.random.randn(I, H))
b1 = Variable(np.zeros(H))
W2 = Variable(0.01 * np.random.randn(H, O))
b2 = Variable(np.zeros(O))
```

```

# ②神经网络的推理
def predict(x):
    y = F.linear(x, W1, b1)
    y = F.sigmoid(y)
    y = F.linear(y, W2, b2)
    return y

lr = 0.2
iters = 10000

# ③神经网络的训练
for i in range(iters):
    y_pred = predict(x)
    loss = F.mean_squared_error(y, y_pred)

    W1.cleargrad()
    b1.cleargrad()
    W2.cleargrad()
    b2.cleargrad()

    loss.backward()

    W1.data -= lr * W1.grad.data
    b1.data -= lr * b1.grad.data
    W2.data -= lr * W2.grad.data
    b2.data -= lr * b2.grad.data
    if i % 1000 == 0: # 每隔 1000 次进行输出
        print(loss.data)

```

输出结果

```

0.8165178492839196
0.24990280802148895
...
0.07618764131185574

```

首先在代码①处进行参数的初始化。这里的 I (1) 对应输入层的维数, H (10) 对应隐藏层的维数, O (1) 对应输出层的维数。 I 和 O 的值都是 1, 这是根据当前问题的设定, 即输入数据和输出数据的维数均为 1 而自动确定的。而 H 是超参数, 可以设置为任何大于 1 的整数。另外, 偏置会被初始化为 0 向量 (`np.zeros(...)`), 权重会被初始化为小的随机数 (`0.01 * np.random.randn(...)`)。



神经网络需要将权重初始化为随机数。关于这么做的原因，请参阅“深度学习入门 & 进阶”系列的第一本书《深度学习入门：基于Python的理论与实现》中的6.2.1节。

然后在②处进行神经网络的推理，在③处更新参数，即进行神经网络的训练。除了参数的数量有所增加，③处的代码与7.2节的代码完全相同。执行上述代码后，神经网络开始训练。训练后的神经网络预测出了图7-12中的曲线。

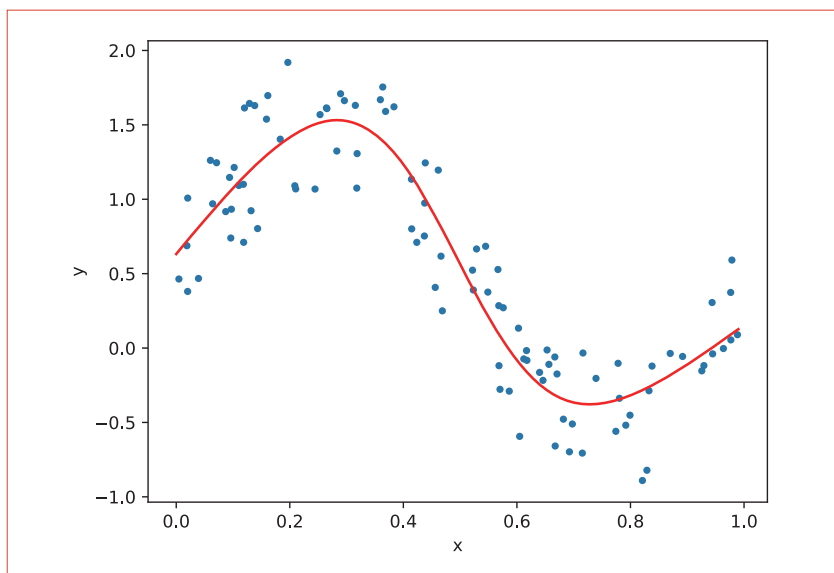


图 7-12 训练后的神经网络

如图7-12所示，sin函数的曲线得到了很好的展现。通过在线性回归的实现上叠加激活函数和线性变换，非线性关系也被正确地训练出来了。这就是神经网络。下面介绍DeZero中提供的模块，以便更轻松地编写此处实现的代码。首先来看看层与模型。

7.3.4 层与模型

DeZero提供了可轻松实现神经网络的工具类。先来看一下`dezero.layers`

包中的层。层这个类具有管理和初始化参数的功能。这里使用进行线性变换的 Linear 类。Linear 类的初始化需要以下几个参数。

```
Linear(out_size, nobias=False, dtype=np.float32, in_size=None)
```

out_size 是输出的大小（输出数据的维数），nobias 是表示是否使用偏置的标志位，dtype 是数据类型，in_size 是输入的大小（输入数据的维数）。



在 Linear 类内部，用于线性变换的权重和偏置会被初始化，并用于实际的线性变换的计算。权重和偏置是根据 Linear 类初始化时传入的 in_size 和 out_size 生成的。如果 in_size 为 None，则在数据流入（给定用于线性变换的输入 x）时会获取输入的大小，并在此时自动初始化权重和偏置。

下面来看看使用 Linear 层的代码。

```
import numpy as np
import theano.tensor as L

linear = L.Linear(10) # 只指定输出的大小

batch_size, input_size = 100, 5
x = np.random.randn(batch_size, input_size)
y = linear(x)

print('y shape:', y.shape)
print('params shape:', linear.W.shape, linear.b.shape)

for param in linear.params():
    print(param.name, param.shape)
```

输出结果

```
y shape: (100, 10)
params shape: (5, 10) (10,)
W (5, 10)
b (10,)
```

如上面的代码所示，如果使用 linear = L.Linear(10) 生成层，那么网络将会根据 y = linear(x) 进行线性变换的计算。权重和偏置在 linear 实例内部，我们可以通过 linear.W 和 linear.b 访问它们。另外，也可以通过 linear.params() 访问所有参数。

通过 DeZero，我们可以像搭乐高积木一样，使用层的组合来构建神经网络。此外，神经网络还可以被定义为一个单独的类，如下面的代码所示（PyTorch 也采用了这种做法）。

```
from dezero import Model
import dezero.layers as L
import dezero.functions as F

class TwoLayerNet(Model):
    def __init__(self, hidden_size, out_size):
        super().__init__()
        self.l1 = L.Linear(hidden_size)
        self.l2 = L.Linear(out_size)

    def forward(self, x):
        y = F.relu(self.l1(x))
        y = self.l2(y)
        return y
```

上面的代码通过继承 `Model` 类来实现模型。代码在初始化时会生成必要的层，而实际的处理（神经网络的前向传播）写在 `forward` 方法中。通过继承 `Model` 类，就可以管理模型持有的所有参数。例如，我们可以实现以下用法。

```
model = TwoLayerNet(10, 1)

# 访问所有参数
for param in model.params():
    print(param)

# 重置所有参数的梯度
model.cleargrads()
```

如上面的代码所示，我们可以通过 `model.params()` 依次访问所有参数。此外，还有一种可以重置所有参数的梯度的方法，即 `model.cleargrads()`。

下面使用 `dezero.Model` 和 `dezero.layers` 对 `sin` 函数的非线性数据进行训练。代码如下所示。

```
import numpy as np
from dezero import Model
import dezero.layers as L
```

```

import dezero.functions as F

# 数据集的生成
np.random.seed(0)
x = np.random.rand(100, 1)
y = np.sin(2 * np.pi * x) + np.random.rand(100, 1)

lr = 0.2
iters = 10000

class TwoLayerNet(Model):
    def __init__(self, hidden_size, out_size):
        super().__init__()
        self.l1 = L.Linear(hidden_size)
        self.l2 = L.Linear(out_size)

    def forward(self, x):
        y = F.sigmoid(self.l1(x))
        y = self.l2(y)
        return y

model = TwoLayerNet(10, 1)

for i in range(iters):
    y_pred = model.forward(x) # 或者使用 model(x), 效果相同
    loss = F.mean_squared_error(y, y_pred)

    model.cleargrads()
    loss.backward()

    for p in model.params():
        p.data -= lr * p.grad.data

    if i % 1000 == 0:
        print(loss)

```

结果与之前相同。不过，这次神经网络是作为单个类实现的，因此更新参数和重置梯度的代码更加简洁。

7.3.5 优化器（最优化方法）

最后介绍用于更新模型参数的优化器类。我们先对之前实现的代码应用优化器。

ch07/dezero4.py

```

import numpy as np
from dezero import Model
from dezero import optimizers
import dezero.layers as L
import dezero.functions as F

# 生成数据集
np.random.seed(0)
x = np.random.rand(100, 1)
y = np.sin(2 * np.pi * x) + np.random.rand(100, 1)

lr = 0.2
iters = 10000

class TwoLayerNet(Model):
    def __init__(self, hidden_size, out_size):
        super().__init__()
        self.l1 = L.Linear(hidden_size)
        self.l2 = L.Linear(out_size)

    def forward(self, x):
        y = F.sigmoid(self.l1(x))
        y = self.l2(y)
        return y

model = TwoLayerNet(10, 1)
optimizer = optimizers.SGD(lr) # 生成优化器
optimizer.setup(model)

for i in range(iters):
    y_pred = model(x)
    loss = F.mean_squared_error(y, y_pred)

    model.cleargrads()
    loss.backward()

    optimizer.update() # 通过优化器更新
    if i % 1000 == 0:
        print(loss)

```

这里只介绍与之前代码不同的地方。首先，代码使用 `from dezero import optimizers` 导入了 `optimizers` 包。这个 `optimizers` 包中有多种最优化方法。这里通过 `optimizer = optimizers.SGD(lr)` 生成了一种名为 SGD 的最优化方法。和此前所做的事情一样，SGD 将参数沿梯度方向更新了 `lr` 倍。



SGD 是 Stochastic Gradient Descent 的缩写，意为“随机梯度下降法”。这里的“随机”（stochastic）是指从对象数据中随机（概率性）选取数据，然后对选取的数据应用梯度下降法。在深度学习领域，通常会从原始数据中随机选择数据，然后对这些数据应用梯度下降法。

然后，生成 optimizers 后可以通过 optimizer.setup(model) 来注册模型。这样，优化器就能更新参数了。参数的更新是通过每次调用 optimizer.update() 来进行的。通过这种使用优化器的方式，就可以将更新参数的工作交给优化器来完成。

使用梯度的最优化方法还有很多。典型的方法包括 Momentum、AdaGrad^[6]、AdaDelta^[7]、Adam^[8] 等。在 dezero.optimizers 包中，这些典型的最优化方法都已实现，并且可以轻松切换。如果想在上面的代码中使用 Adam 方法，那么只需替换以下位置的代码即可。

```
# optimizer = optimizers.SGD(lr)
optimizer = optimizers.Adam(lr)
```

通过这样使用优化器，可以轻松实现最优化方法之间的切换。以上就是对 DeZero 和神经网络的介绍。

7.4 Q 学习与神经网络

在第 6 章中，我们学习了 TD 方法，尤其是学习了强化学习中最著名的 Q 学习算法。本节的主题是 Q 学习与神经网络的“融合”。强化学习和深度学习的结合带来了许多创新，而我们终于进入了强化学习和深度学习融合的世界。首先从神经网络的预处理开始介绍。

7.4.1 神经网络的预处理

在神经网络中处理类别数据时，通常的做法是将其转换为独热（one-hot）向量。类别数据是诸如表示衣服尺寸的“S/M/L”和表示血型的“A/B/O/AB”的数据。对于这样的类别数据，在预处理阶段要把它们转换为独热向量。

独热向量是指只有一个元素为 1，其他元素为 0 的向量。例如，我们可以将类别数据转换为表示 L 的 (0, 0, 1) 和表示 M 的 (0, 1, 0) 这样的独热向量。

在 3×4 网格世界问题中，状态被表示为 (0, 0) 和 (0, 1) 这样的数据。状态将智能代理的位置以 (y, x) 这样的数据格式表示，对应于总共 12 个可能的位置中的一个，这种数据可以被视为类别数据。于是，作为预处理步骤，我们要将 3×4 网格世界中的状态转换为独热向量。代码如下所示。

```
import numpy as np

def one_hot(state):
    HEIGHT, WIDTH = 3, 4
    vec = np.zeros(HEIGHT * WIDTH, dtype=np.float32)
    y, x = state
    idx = WIDTH * y + x
    vec[idx] = 1.0
    return vec[np.newaxis, :] # 由于是批量操作，因此添加了一个新轴

state = (2, 0)
x = one_hot(state)

print(x.shape) # (1, 12)
print(x) # [[0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0.]]
```

one_hot 函数接收 state 并将其转换为了独热向量。one_hot 函数首先准备了包含 $3 \times 4 = 12$ 个元素（所有值均为 0）的向量，然后根据 state 将相应元素设置为 1.0。另外，考虑到要进行批量操作，所以代码中通过 vec[np.newaxis, :] 添加了一个新轴。这使得 one_hot 函数返回的张量形状变成了 (1, 12)（原来的 vec 的形状为 (12,)）。



神经网络通常批量处理数据。例如，要一次性处理 100 个数据，就需要输入形状为 (100, 12) 的数据。

7.4.2 表示 Q 函数的神经网络

回顾之前的内容，我们一直是将 Q 函数作为表格来实现的，在 Python 代码中，我们会将其作为字典 (defaultdict) 来实现。例如，考虑以下代码。

```
from collections import defaultdict

Q = defaultdict(lambda: 0)
state = (2, 0)
action = 0

print(Q[state, action]) # 0.0
```

在上面的代码中，Q 会接收 (state, action) 数据对作为输入，并输出 Q 函数的值。也就是说，对于每个 (state, action) 数据对，Q 函数的值都是单独存储的。

下面把表格所代表的 Q 函数“转换”成神经网络。为此，首先需要明确神经网络的输入和输出。这有几种可能的候选结构。可以考虑的典型结构是图 7-13 所示的两种神经网络结构。

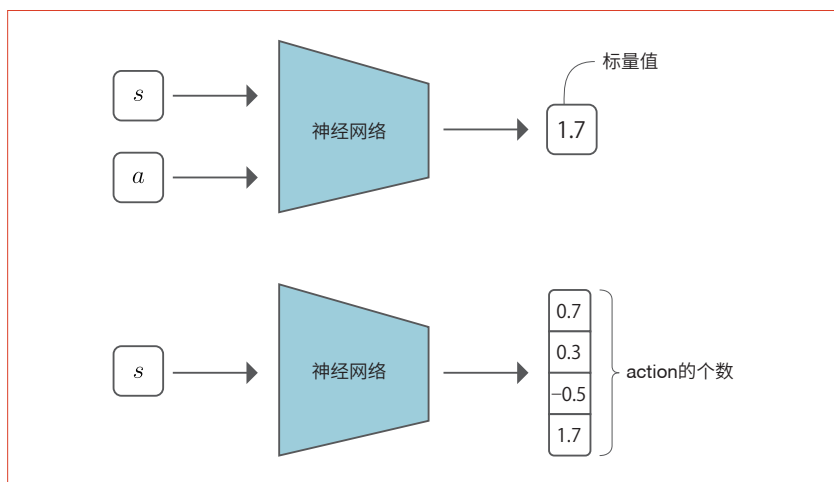


图 7-13 两种神经网络结构

一种结构是有“状态”和“行动”这两个输入的网络（图 7-13 的上图）。在这种情况下，输出只有一个，也就是 Q 函数的值（这里不考虑批量，只考虑仅输入一个数据的情况）。另一种结构是只将“状态”作为输入，输出多个（数量为候选行动个数）Q 函数值的网络（图 7-13 的下图）。如果有 4 个候选行动，那么网络就会输出一个包含 4 个元素的向量。

前面提到了两种候选网络结构，但第一种网络结构在计算成本方面存在问题。具体来说，在某个状态下求 Q 函数最大值（数学式为 $\max_a Q(s, a)$ ）的计算成本很高。



Q 学习需要计算 $\max_a Q(s, a)$ 。这个计算要找到在状态 s 下 Q 函数值最大的行动。在第一种网络结构中，Q 函数的值必须通过神经网络的前向传播求出，前向传播的次数与候选行动的数量相同。如果行动的数量是 4，那么就必须进行 4 次前向传播，并对每个行动求 Q 函数。而第二种网络结构的计算效率更高，因为只需进行一次前向传播即可得到所有行动的 Q 函数。

现在实现第二种网络结构（仅将状态作为输入的网络结构）。这里实现由两个全连接层组成的神经网络。代码如下所示。

```
from dezero import Model
import dezero.functions as F
import dezero.layers as L

class QNet(Model):
    def __init__(self):
        super().__init__()
        self.l1 = L.Linear(100) # 中间层的大小
        self.l2 = L.Linear(4) # 行动层的大小

    def forward(self, x):
        x = F.relu(self.l1(x))
        x = self.l2(x)
        return x

qnet = QNet()

state = (2, 0)
state = one_hot(state) # 转换为独热向量

qs = qnet(state)
print(qs.shape) # (1, 4)
```

上面的代码会按照 DeZero 的方法，通过继承 Model 类来实现神经网络的模型。初始化过程会生成所需的层。在 DeZero 中，只需指定输出的大小即可生成层。上面的代码生成了输出大小分别为 100 和 4 的两个进行线性变换的层。然后，我们可以在进行前向传播的 forward 方法中编写代码。神经网络的主要处理都可以在 forward 方法中进行。

现在我们已经将 Q 函数的实现替换为神经网络了。接下来，让我们使用这里实现的神经网络来实现 Q 学习的算法。

7.4.3 神经网络和 Q 学习

在使用神经网络实现 Q 学习之前，先回顾一下 Q 学习。我们在第 6 章学过，Q 学习通过以下式子更新 Q 函数。

$$Q'(S_t, A_t) = Q(S_t, A_t) + \alpha \left\{ R_t + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right\} \quad (7.3)$$

这个式子沿着目标 $R_t + \gamma \max_a Q(S_{t+1}, a)$ 的方向更新 $Q(S_t, A_t)$ 的值。此时可以通过 α 调整在目标方向的前进距离。

这里将目标 $R_t + \gamma \max_a Q(S_{t+1}, a)$ 表示为 T 。因此式 7.3 可以变化为以下式子。

$$Q'(S_t, A_t) = Q(S_t, A_t) + \alpha \{ T - Q(S_t, A_t) \} \quad (7.4)$$

式 7.4 可以被解释为更新 Q 函数，使其在输入为 S_t 和 A_t 时，输出为 T 。在神经网络的语境中，这等同于训练神经网络，使其在输入为 S_t 和 A_t 时，输出为 T 。换句话说， T 可以被视为正确答案的标签。此外，由于 T 是标量值，因此可以将问题视为回归问题。

基于上述探讨，下面我们来实现进行 Q 学习的智能代理。这里将代码分为两部分。首先是前半部分的代码。

ch07/q_learning_nn.py

```
class QLearningAgent:
    def __init__(self):
        self.gamma = 0.9
        self.lr = 0.01
        self.epsilon = 0.1
        self.action_size = 4

        self.qnet = QNet()
        self.optimizer = optimizers.SGD(self.lr)
        self.optimizer.setup(self.qnet)
```

```
def get_action(self, state):
    if np.random.rand() < self.epsilon:
        return np.random.choice(self.action_size)
    else:
        qs = self.qnet(state)
        return qs.data.argmax()
```

在 QLearningAgent 类的初始化过程中，神经网络和优化器会被初始化。然后我们可以在优化器中设置神经网络。

get_action 方法基于 ϵ -greedy 选择行动。也就是说，它会以 ϵ 的概率选择随机的行动，否则就选择 Q 函数值最大的行动。另外，get_action(self, state) 接收的 state 输入是独热向量化的状态。下面是 QLearningAgent 类代码的后半部分。

ch07/q_learning_nn.py

```
class QLearningAgent:
    ...

    def update(self, state, action, reward, next_state, done):
        if done:
            next_q = np.zeros(1) # [0.]
        else:
            next_qs = self.qnet(next_state)
            next_q = next_qs.max(axis=1)
            next_q.unchain()

        target = self.gamma * next_q + reward
        qs = self.qnet(state)
        q = qs[:, action]
        loss = F.mean_squared_error(target, q)

        self.qnet.cleargrads()
        loss.backward()
        self.optimizer.update()

        return loss.data
```

上面的代码会在 update 方法中更新 Q 函数。首先，求下一个状态中的 Q 函数 (next_q) 的最大值。但是，如果 done 为 True（在 next_state 是目标的情况下），那么 next_state 中的 Q 函数值将始终为 0，因此 next_q 会被设置为 0（具体的代码是 np.zeros(1)）。



`next_q` 用于创建正确答案的标签。监督学习不需要与正确答案标签相关的梯度，因此我们使用 `next_q.unchain()` 将 `next_q` 排除在反向传播之外（`unchain` 的意思是“解链”）。这样，`next_q` 就成了一个单纯的数值，即使后续进行反向传播，也不会进行涉及 `next_q` 的梯度计算，从而避免了不必要的计算。

然后，求 `target`，计算当前状态下的 Q 函数 (`q`)。接下来，计算 `target` 和 `q` 之间的均方差以作为损失函数。最后，按照 DeZero 的使用方法进行反向传播并更新参数。

另外，上面的代码使用了 `if` 语句来根据 `done` 标志位切换 `target` 的计算。也可以不使用 `if` 语句，而是按照如下方式实现。

```
class QLearningAgent:
    ...

    def update(self, state, action, reward, next_state, done):
        done = int(done) # 0 或 1
        next_qs = self.qnet(next_state)
        next_q = next_qs.max(axis=1)
        next_q.unchain()
        target = reward + (1 - done) * self.gamma * next_q

        ...
```

Python 语言在将 `bool` 型转换为 `int` 型时，`True` 会被转换为 1，`False` 会被转换为 0。通过使用 `done = int(done)` 和 `(1 - done)`，上面的代码能够得到与之前相同的结果。这段代码也能用于第 8 章将介绍的小批量学习。

以上就是 `QLearningAgent` 类的代码。下面是运行智能代理的代码。

ch07/q_learning_nn.py

```
env = GridWorld()
agent = QLearningAgent()

episodes = 1000 # 回合数量
loss_history = []

for episode in range(episodes):
    state = env.reset()
    state = one_hot(state)
```

```
total_loss, cnt = 0, 0
done = False

while not done:
    action = agent.get_action(state)
    next_state, reward, done = env.step(action)
    next_state = one_hot(next_state)

    loss = agent.update(state, action, reward, next_state, done)
    total_loss += loss
    cnt += 1
    state = next_state

average_loss = total_loss / cnt
loss_history.append(average_loss)
```

上面的代码通过将回合数量设为 1000，记录了每个回合的平均损失。结果如图 7-14 所示。

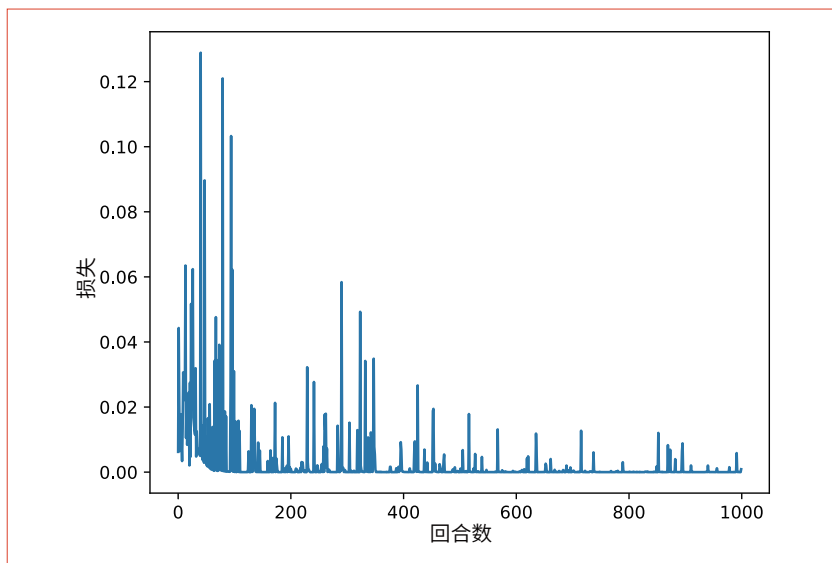


图 7-14 每个回合的损失的变化

通过绘制损失图可以发现，在使用神经网络进行强化学习的训练阶段，通常不会得到稳定的结果。虽然图 7-14 显示的变化幅度较大，但从更大的视角来看，随着回合的进行，损失在不断减小。图 7-15 展示了我们用上面

的代码最终得到的 Q 函数及其贪婪化策略。

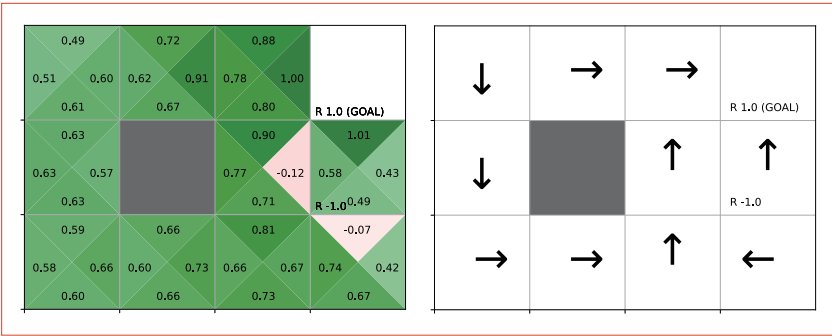


图 7-15 利用神经网络进行 Q 学习所得到的 Q 函数及其贪婪化策略

每次的结果都不相同，但总体来看效果不错。图 7-15 虽然不是完美的最优策略，但通过增加回合数量能够稳定地获得接近最优策略的策略。以上就是对使用神经网络实现 Q 学习的所有介绍。

7.5 小结

本章学习了如何使用 DeZero 框架来实现神经网络。DeZero 是一个设计思想与 PyTorch 等现代深度学习框架相同的框架。使用 DeZero，我们不仅可以像处理 NumPy 的 ndarray 一样处理数据，还可以通过反向传播求导。这样就能很容易实现利用导数更新参数的梯度下降法等方法。

本章不仅使用 DeZero 解决了线性回归等机器学习的基础问题，还对非线性数据集实现了神经网络，并确认了网络可以正确训练。DeZero 提供了层、优化器等便于神经网络实现的工具类。通过使用这些类，我们可以轻松地实现神经网络。

本章在最后使用神经网络实现了 Q 学习，基本的原理与第 6 章介绍的 Q 学习相同。只要理解了 Q 学习的工作原理，将神经网络融入其中就不难。我们已经进入了强化学习和深度学习相结合的深度强化学习的世界。现在，增加状态和行动的规模已不再是问题。

第 8 章

DQN

本章的主题是 DQN。DQN 是基于 Q 学习和神经网络的算法。第 7 章介绍了 Q 学习和神经网络相融合的方法。在此基础上，DQN 引入了新的技术，即“经验回放”和“目标网络”。在本章中，我们将学习并实现这些技术以验证其效果。此外，本章还将介绍 DQN 的扩展算法（Double DQN、优先级经验回放和 Dueling DQN）。

对于电视游戏这样复杂的任务，DQN 也实现了完美通关，并由此引发了现在的深度强化学习的热潮。从这一点来看，可以说 DQN 是深度强化学习中里程碑式的研究。DQN 于 2013 年被公布，虽然有些年头了，但现在仍然有许多基于 DQN 的算法不断涌现。DQN 依旧是重要的算法之一。

另外，从本章开始，我们将从之前的网格世界的问题“毕业”，致力于解决更具实践性的问题。具体来说，我们将使用 OpenAI Gym 这个工具解决倒立摆（CartPole）问题。下面先来了解一下 OpenAI Gym 的使用方法。

8.1 OpenAI Gym

OpenAI Gym 是一个开源库，它提供了各种强化学习任务（环境），如图 8-1 所示。

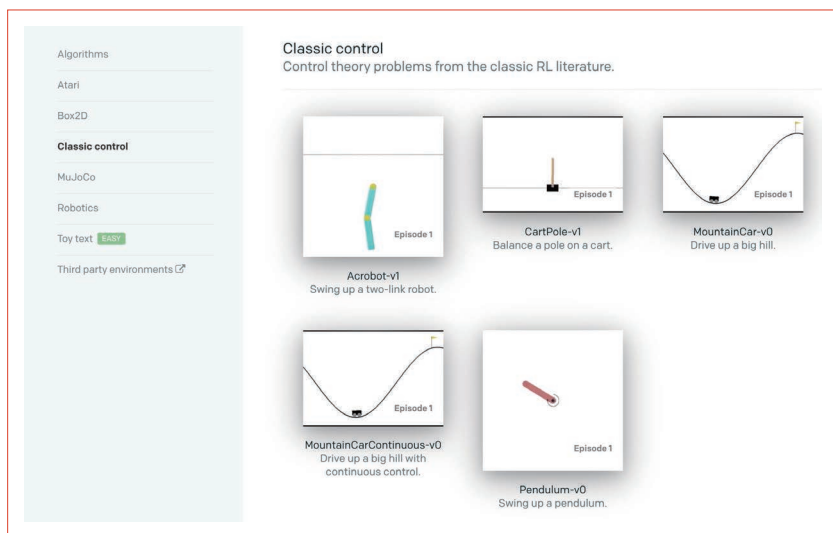


图 8-1 OpenAI Gym 的任务一览界面

OpenAI Gym 的许多任务的接口是通用的。因此，我们可以轻松地切换强化学习任务。另外，在强化学习相关论文中，OpenAI Gym（尤其是 OpenAI Gym 的 Atari 游戏）经常被用作基准。下面介绍 OpenAI Gym 的基本使用方法。

8.1.1 OpenAI Gym 的基础知识

首先安装 OpenAI Gym 的 gym 模块。可以通过 pip 进行安装，如下所示。

```
$ pip install gym
```

只要在终端上运行上面这行命令，即可完成安装。接下来使用 gym 模块。OpenAI Gym 提供了各种各样的环境，这里指定 CartPole-v0 环境。先从以下代码开始。

```
import gym

env = gym.make('CartPole-v0')
```

这样就生成了倒立摆的环境。倒立摆是图 8-2 所示的调整状态使杆子不倒的平衡游戏。

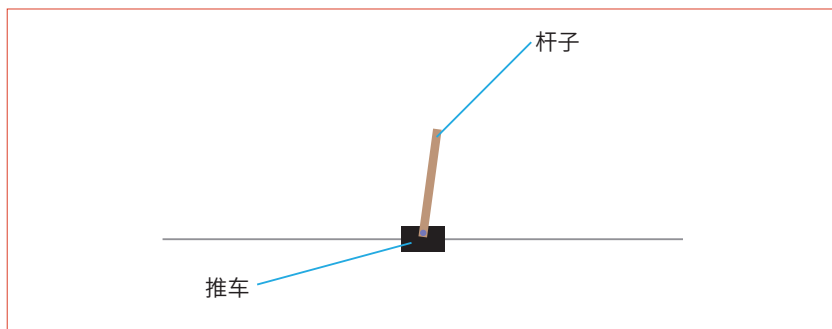


图 8-2 倒立摆

如图 8-2 所示，将推车向右或向左移动，以保持杆子的平衡。倒立摆的结束条件是杆子的平衡被打破（杆子超过一定的角度），或者推车的移动位置超出了某个范围。



倒立摆环境包含版本 0 (CartPole-v0) 和版本 1 (CartPole-v1)。版本 0 的上限为 200 步。如果能在 200 步之内保持平衡，那么游戏就结束了。版本 1 的上限为 500 步。

下面继续执行以下代码。

```
state = env.reset()
print(state) # 初始状态

action_space = env.action_space
print(action_space) # 行动的维度
```

输出结果

```
[ 0.03454657 -0.01361909 -0.02143636  0.02152179]
Discrete(2)
```

上面的代码通过 `state = env.reset()` 获得了初始状态。观察它的输出，你会发现它是拥有 4 个元素的数组。作为参考，下面依次列出这 4 个元素。

- 推车的位置
- 推车的速度
- 杆子的角度
- 杆子的角速度

另外，我们可以通过 `env.action_space` 获得行动的维度（可采取的行动数）。它的输出是一个名为 `Discrete(2)` 的类实例。这意味着有两个候选行动。具体来说，0 对应的是向左移动推车的行动，1 对应的是向右移动推车的行动。下面实际地采取行动，向前推进一个时间步。

```
action = 0 # 或者 1
next_state, reward, done, info = env.step(action)
print(next_state)
```

输出结果

```
[ 0.03454657 -0.01361909 -0.02143636 0.02152179]
```

上面的代码通过 `env.step(action)` 采取行动。作为结果，我们得到了以下 4 个信息。

- 下一个状态 (`next_state`)
- 奖励 (`reward`)
- 是否结束的标志位 (`done`)
- 附加信息 (`info`)

`reward` 是标量值 (`float`)。这次的任务在保持平衡的时候总是会得到奖励 1。`info` 包含有助于调试的信息（如环境模型）。但在实现和评估强化学习的算法时，基本不会用到 `info`。

8.1.2 随机智能代理

8.1.1 节所介绍的内容是在使用 OpenAI Gym 时需要掌握的知识。下面把代码汇总起来运行。这里假设有一个随机智能代理（随机采取行动的智能代理），我们让它运行一个回合。代码如下所示。

ch08/gym_play.py

```
import numpy as np
import gym

env = gym.make('CartPole-v0')
state = env.reset()
done = False

while not done:
    env.render()
    action = np.random.choice([0, 1])
    next_state, reward, done, info = env.step(action)
env.close()
```

上面的代码会使用 while 语句重复采取行动，直到回合结束。具体的行动是对 0 或 1 进行随机采样。另外，在 OpenAI Gym 中，可以通过 `env.render()` 进行任务的可视化。这会显示图 8-3 所示的窗口。

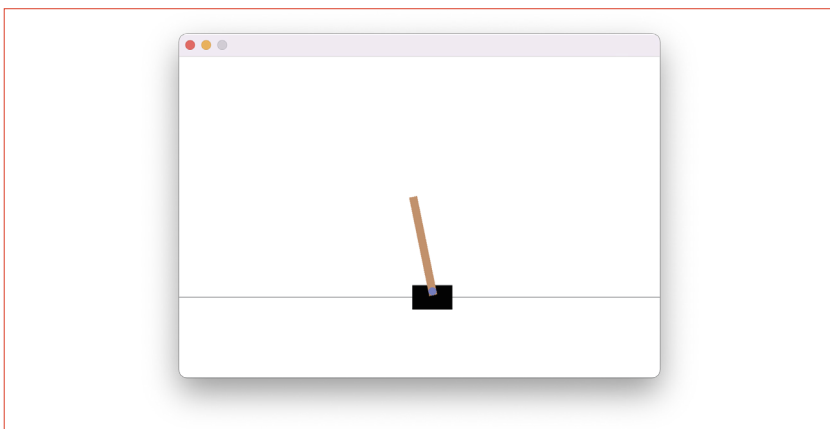


图 8-3 倒立摆的渲染（绘制）

由于本次采取的是随机的行动，因此我们马上就得到了平衡崩坏的结果。8.2 节将使用 DQN 挑战倒立摆问题。

【补充】关于状态和观察

OpenAI Gym 的文档使用术语“观察”（observation）来代替“状态”（state）。OpenAI Gym 的 API 也是以“观察”命名的（如 `env.observation_space` 等）。

状态和观察是不同的。状态是关于环境的“完整描述（信息）”。如果知道状态，那么马尔可夫决策过程将完全决定下一个状态和奖励的概率分布。而观察是状态的“部分描述”。想象一下从智能代理那里只能看到问题（世界）的一部分的场景（扑克牌、麻将等），就容易理解观察了。

在有些任务中，状态和观察也可以是相同的，但是考虑到各种强化学习的任务，比起“状态”，使用“观察”这个用语更加合适。因此，OpenAI Gym 使用了“观察”这个用语。由于本书只处理状态和观察相同的问题，因此将继续使用“状态”这个用语。

8.2 DQN 的核心技术

Q 学习通常是使用估计值来更新估计值（这个原理叫作“自举”）。此外，由于是使用尚不正确的估计值来更新现在的估计值，因此 Q 学习（更广泛地说是 TD 方法）有易于变得不稳定的趋势。如果此时再加入神经网络这样的表现力强的函数近似方法，则结果会变得更加不稳定。



表现力强虽然是神经网络的优点，但也可以成为其缺点。一个原因是，它有可能导致过度拟合训练数据的现象。这种现象叫作过拟合（overfitting）。

DQN 是 Q 学习和神经网络相结合的方法。它的特点是为了稳定神经网络的学习，使用了经验回放（experience replay）和目标网络（target network）这两项技术（除此之外还采用了其他技术，后面将进行说明）。通过这些技术，DQN 首次挑战成功了电视游戏这样复杂的任务。下面我们将依次介绍 DQN 的这两项核心技术。首先从经验回放开始。

8.2.1 经验回放

使用神经网络成功解决监督学习问题的案例有很多。但在 2013 年 DQN 发表之前，几乎没有使用神经网络成功解决强化学习问题的案例（唯一的成功案例是使用神经网络解决了西洋双陆棋的学习^[9]）。为什么很难对强化

学习的算法（尤其是 Q 学习）应用神经网络呢？如何才能成功地融合 Q 学习和神经网络呢？解决之道在监督学习和 Q 学习的区别之中。

首先回顾一下监督学习。这里将结合手写数字图像 MNIST 的例子进行说明。MNIST 主要用于分类问题。数据集是由包含图像数据和正确答案标签的一对数据构成。在神经网络中使用 MNIST 进行训练的一般流程如图 8-4 所示。

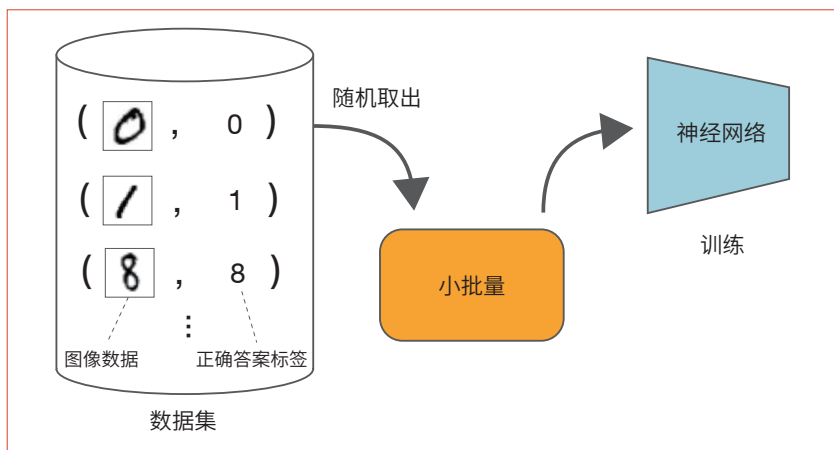


图 8-4 监督学习的流程

如图 8-4 所示，算法可以从训练用的数据集中随机取出一部分数据。取出的这部分数据叫作小批量（mini-batch）。算法使用小批量来更新神经网络的参数。在创建小批量时，需要注意的是其中的数据不要有偏差（比如防止创建只包含“2”的图像的小批量等）。在神经网络的训练中，为了防止数据偏差，通常可以从数据集中随机取出数据。

接下来是 Q 学习。在 Q 学习中，每当智能代理对环境采取行动时都会产生数据。具体来说，使用在某个时刻 t 得到的 $E_t = (S_t, A_t, R_t, S_{t+1})$ 更新 Q 函数。这里将 E_t 称为“经验数据”。这个经验数据虽然是随着时间 t 的推移而获得的，但经验数据之间有很强的相关性（例如， E_t 和 E_{t+1} 之间有很强的相关性）。也就是说，Q 学习使用强相关（有偏差）的数据进行训练。这就是监督学习和 Q 学习的第一个区别。弥补这个差异的技术包括经验回放。

如图 8-5 所示，使用经验回放进行训练的流程非常简单。首先，将智能代理的经验数据 $E_t = (S_t, A_t, R_t, S_{t+1})$ 保存到“缓冲区”（缓冲区是指暂时存储数据的存储装置）中。然后，在更新 Q 函数时，从缓冲区中随机取出经验数据并使用。

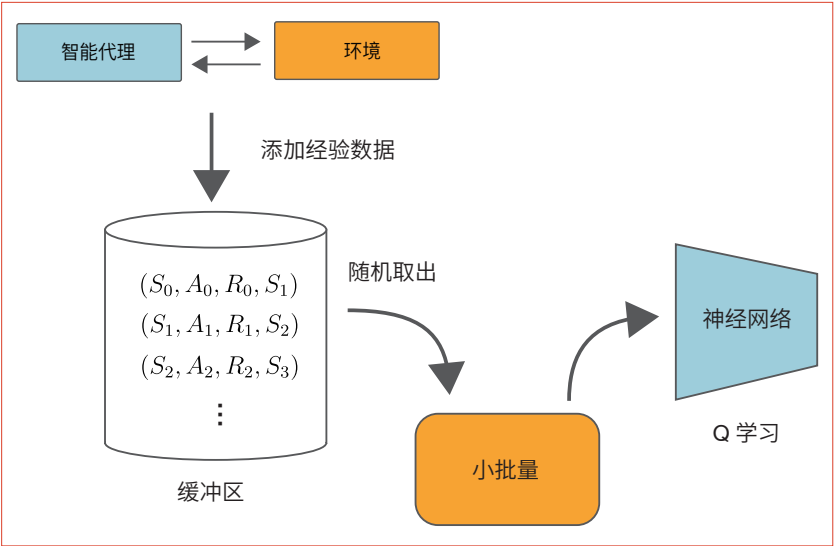


图 8-5 使用经验回放进行训练的流程

通过经验回放，经验数据间的相关性会减弱，进而得到偏差小的数据。而且由于可以重复使用经验数据，因此数据效率会更高。



经验回放不仅可以用于 Q 学习，也可以用于其他强化学习的算法。但是，能够使用经验回放的仅限于异策略型的算法。同策略型的算法只能使用从现在的策略得到的数据。也就是说，由于不能使用过去收集的经验数据，因此就不能使用经验回放。

下面介绍经验回放的实现方法。

8.2.2 经验回放的实现

经验回放的缓冲区不能存储无限多的数据。所以，我们需要事先决定

它的最大容量。例如，可以将缓冲区的最大容量设置为至多可存储 50 000 个经验数据。之后如果添加了超过最大容量的数据，则按照从旧到新的顺序依次删除数据。这样最新的数据就能存储在缓冲区中了。Python 标准库中的 `collections.deque` 非常适合以这种“First in First out”（先进先出）规则存储数据。

下面使用一个名为 `ReplayBuffer` 的类来实现经验回放机制。代码如下所示。

ch08/replay_buffer.py

```
from collections import deque
import random
import numpy as np

class ReplayBuffer:
    def __init__(self, buffer_size, batch_size):
        self.buffer = deque(maxlen=buffer_size)
        self.batch_size = batch_size

    def add(self, state, action, reward, next_state, done):
        data = (state, action, reward, next_state, done)
        self.buffer.append(data)

    def __len__(self):
        return len(self.buffer)

    def get_batch(self):
        data = random.sample(self.buffer, self.batch_size)

        state = np.stack([x[0] for x in data])
        action = np.array([x[1] for x in data])
        reward = np.array([x[2] for x in data])
        next_state = np.stack([x[3] for x in data])
        done = np.array([x[4] for x in data]).astype(np.int32)
        return state, action, reward, next_state, done
```

首先在初始化阶段接收 `buffer_size` 和 `batch_size` 作为参数。`buffer_size` 表示缓冲区的大小，`batch_size` 表示小批量的大小。缓冲区通过 `self.buffer = deque(maxlen=buffer_size)` 进行初始化。通过使用 `deque`，就可以像列表一样添加数据了。此外，如果添加超过最大容量的数据，则按照从旧到新的顺序依次删除数据。

添加经验数据的方法是 `add`。代码会将 `(state, action, reward, next_state, done)` 作为一个整体向缓冲区中添加数据。然后在 `__len__` 方法中，可以通过 `len` 函数来了解缓冲区的大小。例如，对于 `replay_buffer = ReplayBuffer(50000, 32)`，通过 `len(replay_buffer)` 就能够得出当前添加到缓冲区中的数据大小。

最后是 `get_batch` 方法。它是从缓冲区中取出数据以创建小批量的方法。具体来说，先从 `self.buffer` 中随机取出数据，然后为了便于神经网络的处理，再将数据转换成 `np.ndarray` 实例。内部的代码处理流程可以参考图 8-6。

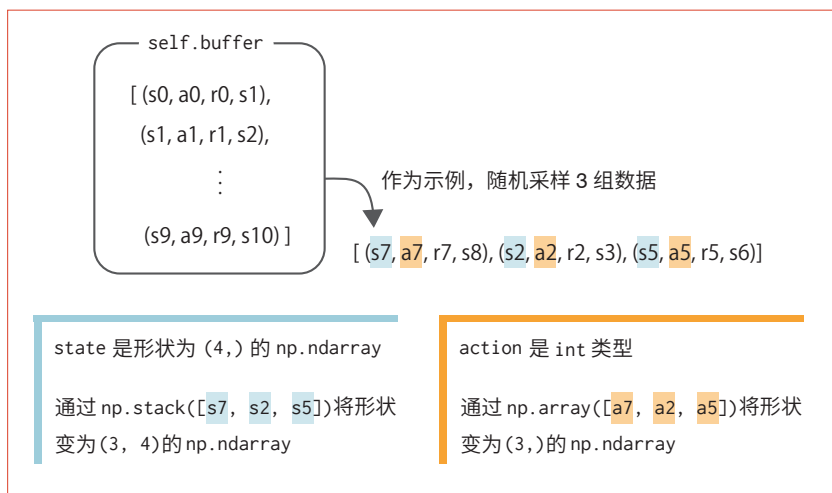


图 8-6 将小批量的元素转换为 `np.ndarray` 的代码示例

下面在倒立摆环境中使用经验回放。代码如下所示。

```
ch08/replay_buffer.py

import gym

env = gym.make('CartPole-v0')
replay_buffer = ReplayBuffer(buffer_size=10000, batch_size=32)

for episode in range(10):
    state = env.reset()
    done = False
```

```

while not done:
    action = 0
    next_state, reward, done, info = env.step(action)
    replay_buffer.add(state, action, reward, next_state, done)
    state = next_state

state, action, reward, next_state, done = replay_buffer.get_batch()
print(state.shape) # (32, 4)
print(action.shape) # (32,)
print(reward.shape) # (32,)
print(next_state.shape) # (32, 4)
print(done.shape) # (32,)

```

上面的代码运行了 10 个回合。在每个回合中，代码总是只采取第 0 个行动，并将得到的数据添加到 `replay_buffer` 中。最后再通过 `replay_buffer.get_batch()` 取出小批量。从输出结果可以看出，小批量（代码中的大小是 32）的数据是作为 `np.ndarray` 实例取出的。

以上就是经验回放的实现。下面介绍 Q 学习中使用的另一项核心技术——目标网络。

8.2.3 目标网络

这里对比来看一下监督学习与 Q 学习。在监督学习中，我们要为训练数据添加正确答案标签。在这种情况下，输入的正确答案标签不变。以 MNIST 数据集为例，如果输入图像的正确答案标签为“7”，那么该标签将总是为“7”。在神经网络的训练过程中，它的标签当然不会从“7”变成“4”。

那么 Q 学习是怎样的情况呢？Q 学习会将 $Q(S_t, A_t)$ 的值更新为 $R_t + \gamma \max_a Q(S_{t+1}, a)$ ，即它会朝着 TD 目标更新 Q 函数。TD 目标相当于监督学习中的正确答案标签。但是，TD 目标的值会随着 Q 函数的更新而变动。这就是监督学习和 Q 学习的区别。为了弥补这种差异，我们使用一种固定 TD 目标的技术，这种技术叫作“目标网络”。

下面具体说明一下目标网络的实现方法。首先，准备一个表示 Q 函数的原始网络（这个网络叫作 `qnet`）。然后，再准备了一个具有相同结构的网络（这个网络叫作 `qnet_target`）。`qnet` 通过正常的 Q 学习进行更新。`qnet_target`

则定期与 `qnet` 的权重同步，在其余的时间里保持权重参数固定。接下来使用 `qnet_target` 计算 TD 目标的值，就能抑制作为监督标签的 TD 目标的变动了。作为监督标签的 TD 目标不会一直变动，所以我们可以期待神经网络的训练能够保持稳定。



目标网络是一种固定 TD 目标值的技术。但如果 TD 目标完全不更新，那么 Q 函数的训练就无法进行，所以要定期（比如每 100 个回合）更新目标网络。

8.2.4 目标网络的实现

下面来看一下目标网络的代码。8.3 节将全面实现 DQN，这里先介绍一下类名为 `DQNAgent` 的智能代理的部分代码。

ch08/dqn.py

```
import copy
from dezero import Model
from dezero import optimizers
import dezero.functions as F
import dezero.layers as L

class QNet(Model):
    def __init__(self, action_size):
        super().__init__()
        self.l1 = L.Linear(128)
        self.l2 = L.Linear(128)
        self.l3 = L.Linear(action_size)

    def forward(self, x):
        x = F.relu(self.l1(x))
        x = F.relu(self.l2(x))
        x = self.l3(x)
        return x

class DQNAgent:
    def __init__(self):
        self.gamma = 0.98
        self.lr = 0.0005
        self.epsilon = 0.1
        self.buffer_size = 10000
        self.batch_size = 32
        self.action_size = 2
```

```

self.replay_buffer = ReplayBuffer(self.buffer_size, self.batch_size)
self.qnet = QNet(self.action_size)
self.qnet_target = QNet(self.action_size)
self.optimizer = optimizers.Adam(self.lr)
self.optimizer.setup(self.qnet) # 设置 qnet

def sync_qnet(self):
    self.qnet_target = copy.deepcopy(self.qnet)

def get_action(self, state):
    if np.random.rand() < self.epsilon:
        return np.random.choice(self.action_size)
    else:
        state = state[np.newaxis, :] # 增加小批量的维度
        qs = self.qnet(state)
        return qs.data.argmax()

```

首先实现作为神经网络的 `QNet` 类。之后是作为智能代理的 `DQNAgent` 类，它拥有 `self.qnet` 和 `self.qnet_target` 这两个神经网络（两者是具有相同结构的网络）。然后在优化器中只注册 `self.qnet`。这样权重参数的更新就只在 `self.qnet` 中进行（`self.qnet_target` 的权重参数不由优化器更新）。

接下来是 `sync_qnet` 方法。它是用于同步神经网络的方法。该方法使用了 Python 标准库中的 `copy.deepcopy` 方法。`deepcopy` 是“深复制”的意思，用于想要完全复制所有数据的情况。我们在这里创建 `self.qnet` 的完整副本，并将其作为 `self.qnet_target`。



`copy` 模块包含浅复制（`copy.copy`）和深复制（`copy.deepcopy`）。浅复制是考虑了数据效率的复制方法，仅复制对象中包含的数据的引用。如果在这里使用浅复制，那么两个神经网络将共享相同的权重参数。

最后是 `DQNAgent` 类中用于更新权重参数的方法。代码如下所示。

ch08/dqn.py

```

class DQNAgent:
    ...

    def update(self, state, action, reward, next_state, done):
        self.replay_buffer.add(state, action, reward, next_state, done)
        if len(self.replay_buffer) < self.batch_size:
            return

```

```

state, action, reward, next_state, done = self.replay_buffer.get_batch()
qs = self.qnet(state) # ①
q = qs[np.arange(self.batch_size), action] # ②

next_qs = self.qnet_target(next_state) # ③
next_q = next_qs.max(axis=1)
next_q.unchain()
target = reward + (1 - done) * self.gamma * next_q # ④

loss = F.mean_squared_error(q, target)

self.qnet.cleargrads()
loss.backward()
self.optimizer.update()

```

update 方法被调用后，首先会将经验数据添加到缓冲区（self.replay_buffer）中。此时如果缓冲区中存储了超过小批量大小的经验数据，就会将数据作为小批量取出。以下依次对代码的①～④处进行了说明。

- ① state 是形状为 (32, 4) 的 np.ndarray（小批量的大小为 32，状态的大小为 4）。32 个数据被汇总提供给神经网络（self.qnet）。这样，输出 qs 就会变成 (32, 2) 的形状。在倒立摆任务中，行动的大小为 2，因此会输出每个行动的 Q 函数。
- ② action 是形状为 (32,) 的 np.ndarray。这个 action 存储了智能代理采取的行动，比如 [0, 1, 0, 0, 0, ..., 1] 等数据。这里像图 8-7 那样从 qs 中取出 action 的相应元素，这可以通过 qs[np.arange(32), action] 来处理。

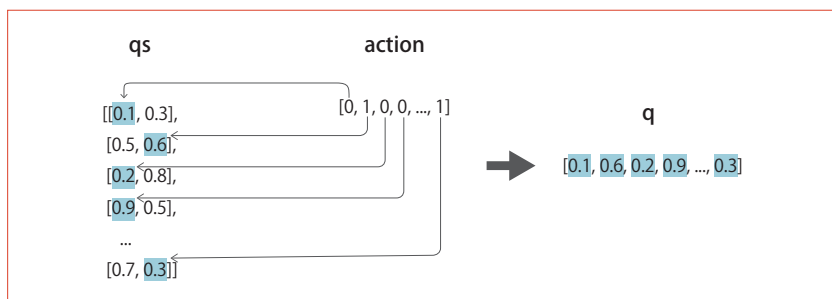


图 8-7 qs[np.arange(32), action] 进行的处理

- ③ 通过 `next_qs = self.qnet_target(next_state)` 求下一个状态下的 Q 函数的值。这里需要注意的是，计算是通过 `self.qnet_target` 而不是 `self.qnet` 进行的。然后，通过 `next_q = next_qs.max(axis=1)` 取出下一个状态下的 Q 函数。通过设置 `axis=1` 来取出每个小批量数据的最大值。
- ④ `done` 是表示是否已完成的标志位。不过 `1 - done` 这个计算会导致数据从 `bool` 型隐式地转换为 `int` 型。因此 `1 - done` 会作为“掩码”使用以计算 TD 目标。

以上就是 `DQNAgent` 类的代码。至此，DQN 中使用的核心技术就全部介绍完了。下面在倒立摆问题中实际地运行一下 DQN。

8.2.5 运行 DQN

下面使用 `DQNAgent` 类来解决倒立摆问题。代码如下所示。

ch08/dqn.py

```
episodes = 300
sync_interval = 20
env = gym.make('CartPole-v0')
agent = DQNAgent()
reward_history = []

for episode in range(episodes):
    state = env.reset()
    done = False
    total_reward = 0

    while not done:
        action = agent.get_action(state)
        next_state, reward, done, info = env.step(action)

        agent.update(state, action, reward, next_state, done)
        state = next_state
        total_reward += reward

    if episode % sync_interval == 0:
        agent.sync_qnet()

    reward_history.append(total_reward)
```

上面的代码一共玩了 300 个回合。另外，每隔 20 个回合调用一次 `agent.sync_qnet()` 以同步目标网络。其余代码与我们之前见过的代码基本相同。

接下来运行上面的代码（到运行结束为止，大约花费几分钟）。`reward_history` 记录了每个回合得到的奖励的总和，我们将数据绘制为图形，结果如图 8-8 所示（每次运行的结果都不同）。

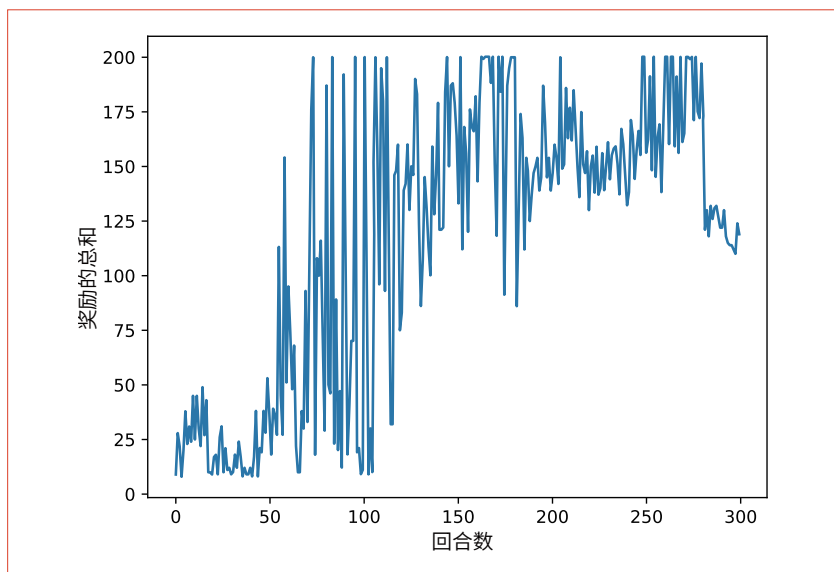


图 8-8 倒立摆游戏中每个回合获得的奖励总和的变化情况

图 8-8 的横轴是回合数，纵轴是奖励的总和。在本次任务中，奖励的总和是保持杆子平衡的时间（时间步）。从图 8-8 中可以看出，每经历一个回合，奖励的总和都会增加。但是图表的变动很大，只靠这点儿信息很难做出判断。在评估强化学习的算法时，仅用一次实验结果来判断是很危险的。更好的做法是重复进行同样的实验，对得到的结果取平均值后再进行评估。图 8-9 是进行 100 次相同的实验，对结果取平均值的结果。

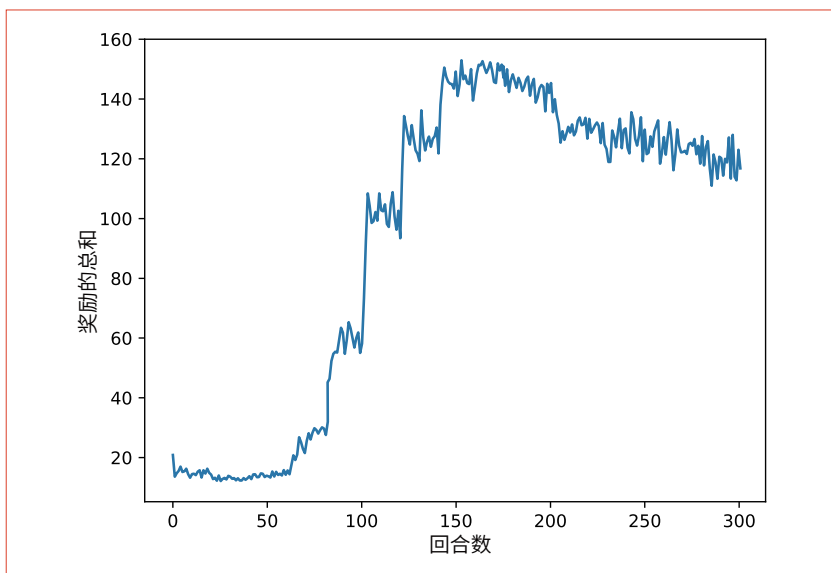


图 8-9 对 100 次实验取平均值的结果

根据图 8-9，一开始平衡马上就被打破了，不过在经历了 50 个回合之后，智能代理开始掌握了诀窍。在经历了 150 个回合之后，训练顺利进行，不过在此之后奖励又有所下降。但从整体来看，训练似乎在往好的方向发展。

另外，训练过程中的智能代理会基于 ϵ -greedy 采取行动。也就是说，它会以 ϵ 的概率采取随机行动。那就让训练后的智能代理采取贪婪行动吧。代码如下所示。

ch08/dqn.py

```
agent.epsilon = 0 # 贪婪策略
state = env.reset()
done = False
total_reward = 0

while not done:
    action = agent.get_action(state)
    next_state, reward, done, info = env.step(action)
    state = next_state
    total_reward += reward
    env.render()
print('Total Reward:', total_reward)
```

输出结果

Total Reward: 116

根据上面的结果，训练后的智能代理通过采取贪婪行动，一直到 116 步为止都保持了平衡。虽然结果每次都会发生变化（倒立摆的初始状态每次多少都有些不同），但基本上结果都超过了 100 次。虽然不能完全保持平衡，但智能代理还是沿着正确的方向完成了训练。另外，通过对超参数的调整，尤其是增加回合数，有望取得更好的结果。



超参数是由人预先设定的值。在我们的代码中，以下参数相当于超参数。

- 折现率 γ (`gamma = 0.98`)
- 学习率 (`lr = 0.005`)
- ϵ -greedy 算法的 ϵ (`epsilon = 0.05`)
- 经验回放的缓冲区的大小 (`buffer_size = 100000`)
- 小批量的大小 (`batch_size = 32`)
- 回合数 (`episodes = 300`)
- 同步间隔 (`sync_interval = 20`)
- 神经网络的结构（神经网络的层数、Linear 层的节点数量等）

8.3 DQN 与 Atari

DQN 是由论文“Playing Atari with Deep Reinforcement Learning”^[10]提出的算法。标题中的 Atari 是一家游戏开发商，其业务涉及街机游戏、游戏机和个人电脑行业。在强化学习领域，Atari 制作的怀旧游戏软件被称为“Atari”。



DQN 于 2013 年发表后，2015 年科学杂志 *Nature* 刊登了 DQN 的论文（标题为“Human-level control through deep reinforcement learning”^[11]）。这篇论文提到 DQN 可以达到人类的游戏水平。

我们可以训练 DQN 来玩倒立摆。那么，它能玩更难的 Atari 之类的游戏吗？这里有一个好消息和一个坏消息。好消息是，通过对前面实现的

DQN 的代码进行一些修改，DQN 也能很好地玩 Atari。坏消息是对 Atari 游戏的训练需要很长时间（使用 GPU 可以减少训练时间，但也需要整整一天）。因此，这里不提供用于 Atari 的 DQN 的代码，而是重点对与 8.2 节的 DQN 的区别进行说明。

8.3.1 Atari 的游戏环境

OpenAI Gym 还提供了 Atari 的游戏环境。要使用 Atari，需要额外进行安装（有关安装方法的详细信息，请参考 OpenAI Gym 的文档等资料）。另外，Atari 中还包含了各种各样的游戏。这里以图 8-10 所示的 Pong 游戏为对象进行说明。

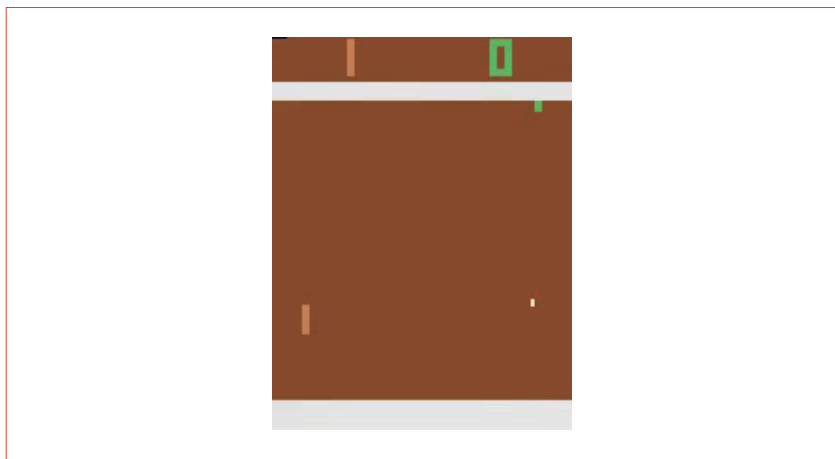


图 8-10 Pong 游戏的界面

如图 8-10 所示，Pong 是相互打球的游戏。一方上下移动左侧的板，与此同时，另一方上下移动右侧的板。如果球超出对方阵地，那么分数就会增加。向智能代理提供的数据是作为状态的游戏图像（具体来说是宽 210、高 160、有 3 个 RGB 通道的图像）。

我们同样可以使用 8.2 节的 DQN 来通关这款名为 Pong 的游戏。但由于它比倒立摆更复杂，因此需要下点儿功夫。下面介绍要做的事情。

8.3.2 预处理

本书前面介绍的强化学习理论是基于 MDP（马尔可夫决策过程）的。在 MDP 中，决定最优行动所需的信息包含在当前状态中。遗憾的是 Pong 不满足 MDP 的要求。这是因为只凭图 8-10 那样的图像，无法明确球朝哪个方向前进。因而，在球的前进方向不明的状态下无法采取最合适的行动。这样的问题叫作 POMDP（Partially Observable Markov Decision Process，部分可观测马尔可夫决策过程）。

对于 Pong 这样的电视游戏，将 POMDP 转换为 MDP 是很简单的。方法就是使用连续的帧。DQN 的论文^[1]中重叠了 4 帧连续的图像，以将其视为一个状态。通过使用连续的图像，就能知道状态的迁移（也能知道球的运动轨迹）。这样就能像以前一样将问题作为 MDP 来处理了。



对 POMDP 的强化学习的研究也很活跃。在 POMDP 中，仅靠当前的观测是不够的，所以需要结合过去的观测来决定行动。解决 POMDP 的有力方法是使用 RNN（循环神经网络）的方法。使用 RNN，可以结合过去输入的数据进行计算。

另外，DQN 的论文中还在重叠帧之前进行了一些例行处理。具体包括以下预处理。

- 对图像周围进行修剪（因为图像周围有无用的元素）。
- 转换为灰度（黑白）图像。
- 调整图像的大小。
- 正则化（将图像的元素转换为 0.0 和 1.0 之间的值）。

完成这些预处理之后，进行重叠 4 个帧的操作。

8.3.3 CNN

在 8.2 节的倒立摆问题中，我们使用了由全连接层构成的神经网络。而在处理 Atari 这样的图像数据时，CNN（卷积神经网络）是更高效的。CNN 是使用了卷积运算（卷积层）的神经网络。DQN 的论文中使用了图 8-11 所

示结构的 CNN。

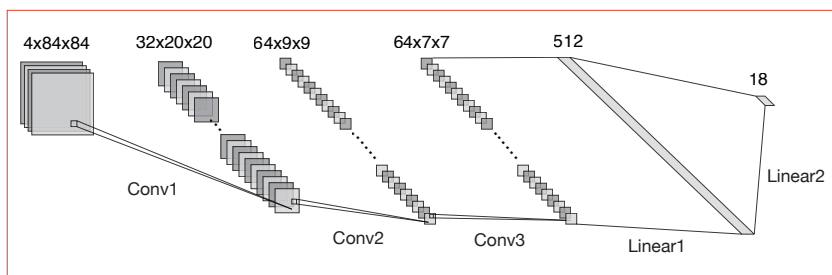


图 8-11 DQN 中使用的 CNN

如图 8-11 所示，DQN 在输入附近的层中使用的是卷积层，而在输出附近的层中使用的是全连接层。最后的输出层的神经元数量等于任务的候选行动的数量。另外，网络使用的激活函数是 ReLU 函数。

8.3.4 其他技巧

除了之前说明的方法，DQN 的论文中还使用了以下技巧。这些技巧不仅对 DQN 有效，对其他强化学习的算法也有效。

GPU 的使用

对于 Atari 这样的游戏，由于处理的是数据很大的图像数据，因此训练所需的计算量也很大。为了加快计算速度，使用 GPU（或 TPU）等进行并行计算是更高效的。顺带一提，DeZero 也具有使用 GPU 进行计算的功能。

ϵ 的调整

在强化学习中，“利用”和“探索”的平衡很重要。考虑到随着智能代理经验的增加，价值函数的可靠性也会提高，因此与回合数成比例地减少探索的占比是合理的。也就是说，在初期阶段让智能代理进行大量的探索，随着训练的进行则减少探索（增加利用）。要用 ϵ -greedy 算法实现这个想法，可以考虑随着智能代理行动的推进来减少 ϵ 。DQN 的论文中采用了在最初的 100 万步中将 ϵ 从 1.0 线性减少到 0.1，然后固定为 $\epsilon = 0.1$ 的做法，如图 8-12 所示。

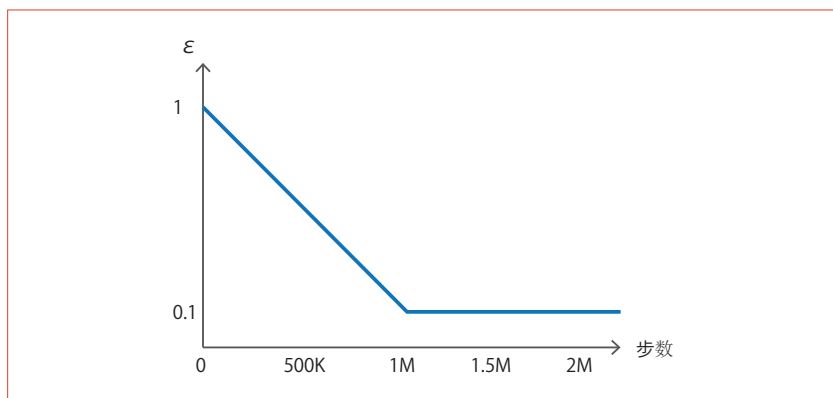


图 8-12 DQN 中的 ϵ 的变化情况

奖励裁剪 (reward clipping)

DQN 的论文中通过将奖励统一缩放到 $-1.0 \sim 1.0$ 的范围的做法，使训练得以顺利进行。而 Pong 游戏的奖励是 -1 、 0 、 1 三者之一。因此，奖励的范围是适当的，即使不进行奖励裁剪也不会影响性能。

借助上述技巧，DQN 可以很好地玩 Pong。在论文中，智能代理已进化到轻松战胜电脑对手的程度。另外，除了 Pong，对于其他的游戏（打砖块、吃豆人等），代码几乎无须修改即可运行，并已进化到轻松取胜的程度。从这一点也可以看出 DQN 的通用性很高。

8.4 DQN 的扩展

DQN 是深度强化学习中最著名的算法之一。自从 DQN 发表以来，人们提出了许多扩展 DQN 的算法。这里介绍其中最著名的 3 种算法。

8.4.1 Double DQN

第一种算法是 Double DQN^[12]。在开始介绍之前，先来回顾一下 DQN。DQN 使用了目标网络这项技术。这是一种在主网络之外，再使用另一个具

有不同参数的网络（目标网络）的算法。这里将两个网络的参数分别表示为 θ 和 θ' ，将两个网络的 Q 函数分别表示为 $Q_\theta(s, a)$ 和 $Q_{\theta'}(s, a)$ 。此时，将用于更新 Q 函数的目标表示为以下式子。

$$R_t + \gamma \max_a Q_{\theta'}(S_{t+1}, a)$$

DQN 会进行训练，以使 $Q_\theta(s, a)$ 的值逐渐接近 TD 目标，也就是上式的值。这里的问题是 $\max_a Q_{\theta'}(S_{t+1}, a)$ 。具体的问题是，如果对包含误差的估计值 ($Q_{\theta'}$) 使用 max 算子，那么与使用真正的 Q 函数进行计算的情况相比估计的结果就会过大。解决这个问题的办法是 Double DQN。Double DQN 会将下面的式子作为 TD 目标。

$$R_t + \gamma Q_{\theta'}\left(S_{t+1}, \underset{a}{\operatorname{argmax}} Q_\theta(S_{t+1}, a)\right)$$

要点是选择 $Q_\theta(s, a)$ 的值最大的行动，而实际值从 $Q_{\theta'}(s, a)$ 中取得。像这样分别使用两个 Q 函数，可以避免过估计，使训练更稳定。附录 C 会介绍什么是过估计以及避免它的原理是什么。如果你对此感兴趣，可以参考一下。

8.4.2 优先级经验回放

DQN 中使用的经验回放会将经验 $E_t = (S_t, A_t, R_t, S_{t+1})$ 保存在缓冲区中，在训练时从缓冲区随机取出经验数据进行使用。将这项技术进一步改进后的技术是优先级经验回放（prioritized experience replay, PER）^[13]。顾名思义，经验数据的选择不是随机的，而是根据优先级进行的。

那么，经验数据的优先级是如何确定的呢？我们很自然地会想到以下式子。

$$\delta_t = \left| R_t + \gamma \max_a Q_{\theta'}(S_{t+1}, a) - Q_\theta(S_t, A_t) \right|$$

上面的式子计算了作为 TD 目标的 $R_t + \gamma \max_a Q_{\theta'}(S_{t+1}, a)$ 和 $Q_\theta(S_t, A_t)$ 的差值，然后取绝对值 δ_t 。 δ_t 越大，要修正的值就越大，也就意味着要从数据中学习的东西就越多。反之， δ_t 越小，说明它已经是很好的参数了，要从中学习的东西就越少。

优先级经验回放在将经验数据保存到缓冲区中时还要计算 δ_t ，然后再将 δ_t 也附在经验数据中，将 $(S_t, A_t, R_t, S_{t+1}, \delta_t)$ 添加到缓冲区中。在从缓冲区中取出经验数据时，使用 δ_t 计算各经验数据被选择的概率。假设缓冲区中包含 N 个经验数据，那么第 i 个经验数据被选择的概率如下式所示。

$$p_i = \frac{\delta_i}{\sum_{k=0}^N \delta_k}$$

根据这个概率 p_i ，我们可以从缓冲区中选出经验数据。通过使用优先级经验回放，那些要学习的东西越多的数据越会被优先使用，所以训练有望更快地进行。

8.4.3 Dueling DQN

最后介绍一种叫作 Dueling DQN^[14] 的方法。Dueling DQN 是一种在神经网络结构上下功夫的方法。此方法的关键是优势函数（advantage function）。优势函数是 Q 函数和价值函数之间的差。其数学式如下所示。

$$A_{\pi}(s, a) = Q_{\pi}(s, a) - V_{\pi}(s) \quad (8.1)$$

式 8.1 的优势函数表示行动 a 比策略 π 好多少（或差多少）。考虑到以下两点就不难理解这么定义的原因了。

- $Q_{\pi}(s, a)$ 是在状态 s 下采取“特定的行动 a ”，之后基于 π 采取行动时得到的收益的期望值。
- $V_{\pi}(s)$ 是在状态 s 下完全基于策略 π 采取行动时得到的收益的期望值。

也就是说 $Q_{\pi}(s, a)$ 和 $V_{\pi}(s)$ 的区别在于，是在状态 s 下采取行动 a ，还是基于策略 π 采取行动。因此，优势函数可以被解释为行动 a 比基于策略 π 选择的行动好多少的指标。

另外，我们还可以根据式 8.1 的优势函数来计算 Q 函数。为此，可以将式 8.1 的优势函数变为如下形式。

$$Q_{\pi}(s, a) = A_{\pi}(s, a) + V_{\pi}(s) \quad (8.2)$$

Dueling DQN 是式 8.2 的神经网络表示，其结构如图 8-13 所示。

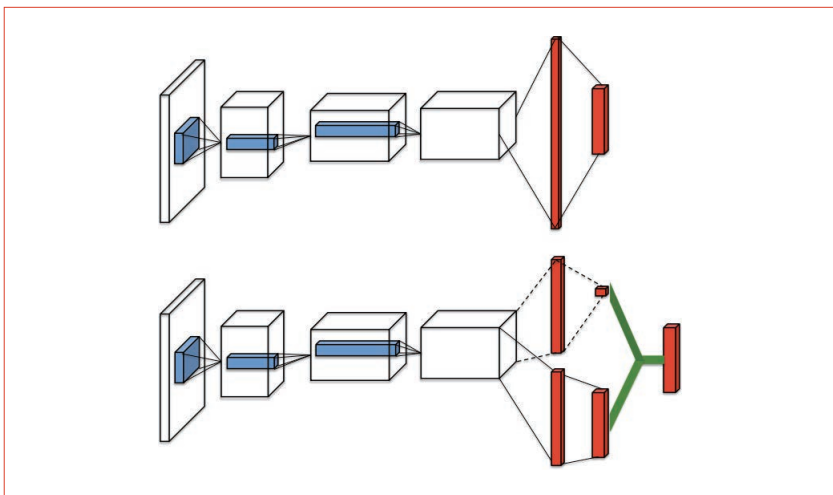


图 8-13 DQN（上）与 Dueling DQN（下）的比较（此图引用自论文^[14]）

如图 8-13 所示，到网络的中间为止，计算都是共享的，随后分离为了优势函数 $A(s,a)$ 和价值函数 $V(s)$ 。最后将这两个计算相加，输出 $Q(s,a)$ 。这个结构就是 Dueling DQN 的特征。

那么，分别训练优势函数和价值函数的优点在哪里呢？这主要是在无论采取什么样的行动，结果都几乎没有变化的状态下有优势。下面以 Pong 游戏在图 8-14 所示的情况为例进行说明。

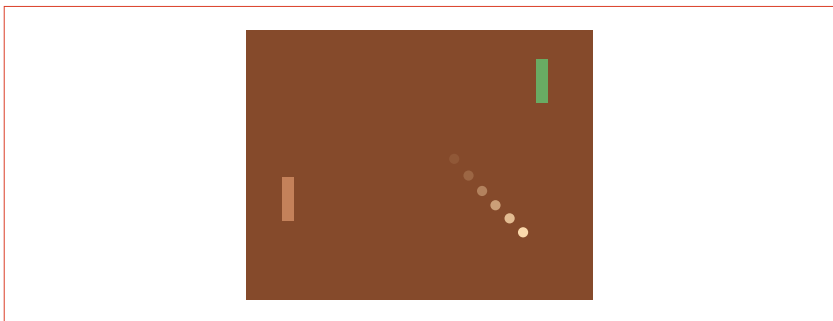


图 8-14 球进入自己阵地之前的界面

在图 8-14 的情况下，无论采取什么样的行动，结果都是输（负的奖励）。对于在某个状态 s 下实际采取的行动 a ，DQN 会训练 $Q(s,a)$ 。即使在图 8-14 所示的结果与行动无关的情况下，如果不尝试所有行动，也不会训练 $Q(s,a)$ 。而 Dueling DQN 用到了价值函数 $V(s)$ 。价值函数是某个状态 s 的价值（不考虑行动）。因此，如果经历图 8-14 所示的状态，那么就会训练 $V(s)$ ，这样就算不尝试其他行动， $Q(s,a)$ 的近似性能也会得到改善。这有助于提高训练效率。

以上就是对 DQN 的 3 种扩展算法的介绍。10.3 节会介绍基于 DQN 的进一步做了改进的算法。

8.5 小结

本章介绍了 DQN。DQN 的关键技术是经验回放和目标网络。经验回放是为了使用重复的经验数据的结构。经验回放可以提高数据的效率，并且减少样本间的相关性。目标网络是从另一个网络计算 TD 目标的技术。通过使用另一个网络，TD 目标会被固定，神经网络的训练将变得更加稳定。

虽然 DQN 的发表已经过了一段时间，但 DQN 仍然是一个重要的算法。至今仍有许多基于 DQN 的算法被提出。本章介绍了 3 种扩展 DQN 的算法：Double DQN、优先级经验回放和 Dueling DQN。

本章还介绍了 OpenAI Gym。Gym 提供了各种任务，由于它们的接口是共通的，因此用户可以轻松地各种任务间切换。本章使用 DQN 解决了 Gym 的“倒立摆”任务。此外，本章也对玩 Atari 游戏所需的技巧进行了说明。

第 9 章

策略梯度法

到目前为止，我们学习了 Q 学习、SARSA、蒙特卡洛方法等算法。这些算法大致可以被归类为基于价值的方法（value-based method）。这里所说的“价值”是指行动价值函数（Q 函数）和状态价值函数。基于价值的方法会对价值函数进行建模并训练价值函数。之后会“借助”价值函数得到策略。



基于价值的方法通常会基于**通用策略迭代**的思想找到最优策略。具体来说，就是通过重复进行价值函数的评估和改进策略的过程，逐渐接近最优策略。

除了基于价值的方法，还可以考虑不借助价值函数直接表示策略的方法。这就是基于策略的方法（policy-based method），其中通过神经网络等方法将策略模型化，并使用梯度来优化策略的方法叫作策略梯度法（policy gradient method）。

研究者们提出了各种基于策略梯度法的算法。本章首先介绍最简单的策略梯度法。然后，在改进这个简单的梯度法的过程中，我们推导出了被称为 REINFORCE 的算法。接下来，在进一步改进 REINFORCE 的过程中，我们又推导出了带基线的 REINFORCE 方法和 Actor-Critic 方法。

9.1 最简单的策略梯度法

策略梯度法是使用梯度来更新策略的方法的总称。策略梯度法的算法

有很多，这里推导最简单的策略梯度法。从 9.2 节开始，我们会在这里学到的方法的基础上进行改进，同时介绍新的方法。

9.1.1 策略梯度法的推导

随机性策略用数学式可以表示为 $\pi(a|s)$ 。 $\pi(a|s)$ 是在状态 s 下采取行动 a 的概率。这里采用神经网络对策略进行建模。此时用符号 θ 来汇总表示神经网络的所有权重参数（ θ 是将所有参数的元素排成一列的向量）。另外，可以将基于神经网络的策略表示为 $\pi_\theta(a|s)$ 。

接下来使用策略 π_θ 来设置目标函数。设置了目标函数之后，就要找到使目标函数值最大的参数 θ 。这个过程叫作“最优化”，是一个常见的神经网络训练过程。



对于最优化问题，这里设置的是目标函数，而不是常用的损失函数。对于损失函数的情况，我们通过梯度下降法寻找最小值；对于目标函数的情况，则通过梯度上升法寻找最大值。梯度下降法会沿着梯度的反方向更新参数，梯度上升法则会沿着梯度的方向更新参数。不过，只要将目标函数乘以负数，就可以作为损失函数来处理（反之亦然），所以损失函数和目标函数本质上作用相同。

下面使用策略 π_θ 设置目标函数。首先明确问题的设定。这里考虑的是回合制任务，并基于策略 π_θ 选择行动的情况。在这种情况下，假定得到了以下由“状态、行动、奖励”构成的时间序列数据。

$$\tau = (S_0, A_0, R_0, S_1, A_1, R_1, \dots, S_{T+1})$$

这个 τ 也叫作轨迹（trajectory）。此时可以使用折现率 γ 对收益（return）作如下定义。

$$G(\tau) = R_0 + \gamma R_1 + \gamma^2 R_2 + \dots + \gamma^T R_T$$

为了表明收益可以由 τ 计算出来，上面的式子将其表示为了 $G(\tau)$ 。此时，目标函数 $J(\theta)$ 可以表示为以下式子。

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [G(\tau)]$$

收益 $G(\tau)$ 是随机变动的，所以它的期望值是目标函数。上式中期望值 $\mathbb{E}$ 的下标为 $\tau \sim \pi_\theta$ ，这个下标表示 τ 是基于 π_θ 生成的。



τ 的生成过程除了与智能代理的策略有关，还与环境 $p(s' | s, a)$ 和 $r(s, a, s')$ 有关。不过我们能够控制的只有智能代理的策略，因此在 $\mathbb{E}_{\tau \sim \pi_\theta} [\dots]$ 中只将下标表示为 $\tau \sim \pi_\theta$ 。

确定了目标函数后，下一步是计算它的梯度。这里将参数 θ 的梯度表示为 ∇_θ 。我们的目标是求 $\nabla_\theta J(\theta)$ 。本章省略了它的推导过程，只给出了作为结果的式 9.1。D.1 节详细介绍了推导过程，如果你对此感兴趣，可以参考一下。

$$\begin{aligned} \nabla_\theta J(\theta) &= \nabla_\theta \mathbb{E}_{\tau \sim \pi_\theta} [G(\tau)] \\ &= \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T G(\tau) \nabla_\theta \log \pi_\theta(A_t | S_t) \right] \end{aligned} \quad (9.1)$$

式 9.1 中值得注意的是， ∇_θ 在 $\mathbb{E}$ 中（梯度计算的部分是 $\nabla_\theta \log \pi_\theta(A_t | S_t)$ ）。后面会对此做详细介绍。求出 $\nabla_\theta J(\theta)$ 之后，接下来更新神经网络的参数。最优化方法多种多样，下面的式子表示的是一种简单的方法。

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$$

上面的式子朝着梯度的方向更新参数 θ ，更新的值与 α 相关。这里的 α 表示学习率。这是属于梯度上升法的算法。

9.1.2 策略梯度法的算法

如式 9.1 所示， $\nabla_\theta J(\theta)$ 表示期望值。这个期望值可以通过蒙特卡洛方法求得。蒙特卡洛方法会进行多次采样，然后取其平均值。这里，我们令策略 π_θ 的智能代理实际采取行动，得到 n 个轨迹 τ 。此时，通过对每个 τ 计算式 9.1 的期望值内部的式子（ $\sum_{t=0}^T G(\tau) \nabla_\theta \log \pi_\theta(A_t | S_t)$ ），并求出其平均值，从而近似得到 $\nabla_\theta J(\theta)$ 。数学式如下所示。

$$\begin{aligned} \text{sampling: } \tau^{(i)} &\sim \pi_{\theta} \quad (i=1,2,\dots,n) \\ x^{(i)} &= \sum_{t=0}^T G(\tau^{(i)}) \nabla_{\theta} \log \pi_{\theta}(A_t^{(i)} | S_t^{(i)}) \\ \nabla_{\theta} J(\theta) &\simeq \frac{x^{(1)} + x^{(2)} + \dots + x^{(n)}}{n} \end{aligned}$$

上面式子中的 $\tau^{(i)}$ 表示在第 i 回合得到的轨迹， $A_t^{(i)}$ 表示在第 i 回合的时刻 t 的行动， $S_t^{(i)}$ 表示在第 i 回合的时刻 t 的状态。

另外，再思考一下蒙特卡洛方法的样本数为 1，即上式中 $n=1$ 的情况。在这种情况下，数学式可以简化为如下形式。

$$\begin{aligned} \text{sampling: } \tau &\sim \pi_{\theta} \\ \nabla_{\theta} J(\theta) &= \sum_{t=0}^T G(\tau) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \end{aligned} \quad (9.2)$$

为了简单起见，本章将使用以式 9.2 为对象的策略梯度法。式 9.2 的计算就是对所有时刻 ($t=0 \sim T$) 求 $\nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$ ，然后将各梯度乘以作为权重的收益 $G(\tau)$ ，最后求它们的和。这个计算过程如图 9-1 所示。

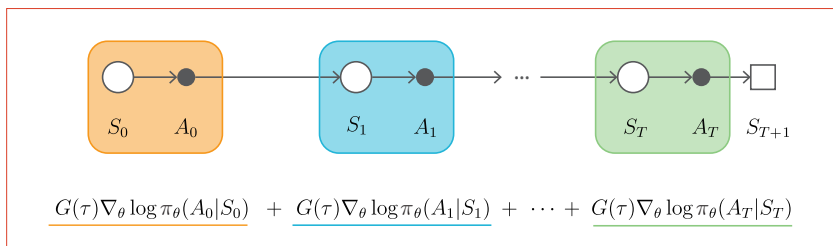


图 9-1 用策略梯度法进行的计算

下面来思考一下图 9-1 所做的计算的意义。首先，根据 $\log$ 的微分，有以下式子成立。

$$\nabla_{\theta} \log \pi_{\theta}(A_t | S_t) = \frac{\nabla_{\theta} \pi_{\theta}(A_t | S_t)}{\pi_{\theta}(A_t | S_t)}$$

如上式所示, $\nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$ 是梯度 $\nabla_{\theta} \pi_{\theta}(A_t | S_t)$ (向量) 的 $\frac{1}{\pi_{\theta}(A_t | S_t)}$ 倍。因此 $\nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$ 和 $\nabla_{\theta} \pi_{\theta}(A_t | S_t)$ 会指向相同的方向。也就是说, $\nabla_{\theta} \pi_{\theta}(A_t | S_t)$ 和 $\nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$ 都指向在状态 S_t 下采取行动 A_t 的概率增加最快的方向。根据 $G(\tau) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$, 需要在该方向上乘以权重 $G(\tau)$ 。

假设智能代理获得的收益 $G(\tau)$ 是 100。那么, 在这种情况下, 在这期间采取的行动将更容易被选择, 权重会被增强 100 倍。也就是说, 在顺利的情况下, 在此之前采取的行动会相应地增强。反之, 在不顺利的情况下, 在这期间采取的行动会相应地减弱。

9.1.3 策略梯度法的实现

下面让我们将话题转到最简单的策略梯度法的实现。首先是 `import` 语句和表示策略的神经网络的代码。

ch09/simple_pg.py

```
import numpy as np
import gym
from dezero import Model
from dezero import optimizers
import dezero.functions as F
import dezero.layers as L

class Policy(Model):
    def __init__(self, action_size):
        super().__init__()
        self.l1 = L.Linear(128)
        self.l2 = L.Linear(action_size)

    def forward(self, x):
        x = F.relu(self.l1(x))
        x = F.softmax(self.l2(x))
        return x
```

这里实现的神经网络模型由两层全连接层构成。最终输出的元素数是行动的数量 (`action_size`)。由于这个最终输出是 `Softmax` 函数的输出, 因此可以得到每个行动的概率。



如果将具有 n 个元素的向量输入到 Softmax 函数中，那么输出的同样也是具有 n 个元素的向量。此时，第 i 个输出 y_i 的式子如下所示。

$$y_i = \frac{e^{x_i}}{\sum_{k=1}^n e^{x_k}}$$

这里的 e 是自然常数（值为 2.718 28... 的无限小数）。Softmax 函数的输出值全部为 0 以上 1 以下的实数，它们的合计值为 1（ $\sum_{i=1}^n y_i = 1$ ）。因此，Softmax 函数的输出可以作为概率使用。

下面是 Agent 类的代码。首先显示初始化和 get_action 方法。

ch09/simple_pg.py

```
class Agent:
    def __init__(self):
        self.gamma = 0.98
        self.lr = 0.0002
        self.action_size = 2

        self.memory = []
        self.pi = Policy(self.action_size)
        self.optimizer = optimizers.Adam(self.lr)
        self.optimizer.setup(self.pi)

    def get_action(self, state):
        state = state[np.newaxis, :] # 增加小批量的轴
        probs = self.pi(state)
        probs = probs[0]
        action = np.random.choice(len(probs), p=probs.data)
        return action, probs[action]
```

get_action 方法决定了在 state 状态下采取的行动。为此，可以通过 self.pi(state) 进行神经网络的前向传播，得到概率分布 probs。然后，基于该概率分布，进行一次行动的采样。该方法还返回了所选行动的概率（上面代码中的 probs[action]）。

下面来试用一下 get_action 方法。代码如下所示。

```
env = gym.make('CartPole-v0')
state = env.reset()
agent = Agent()
```



```

action, prob = agent.get_action(state)
print('action:', action)
print('prob:', prob)

G = 100.0 # 虚拟权重
J = G * F.log(prob)
print('J:', J)

# 求梯度
J.backward()

```

输出结果

```

action: 1
prob: variable(0.49956715)
J: variable(69.4013237953186)

```

上面的代码取出了初始状态下的行动及其概率。另外，它还显示了使用虚拟的权重来计算由下式表示的梯度的代码（这是从式 9.2 取出的 $t = 0$ 的相关项的式子）。

$$G(\tau) \nabla_{\theta} \log \pi_{\theta}(A_0 | S_0)$$

作为参考，下面对照列出了上面的代码中出现的变量与相应的数学式。

- prob (Dezero.Variable): $\pi_{\theta}(A_0 | S_0)$
- G (float): $G(\tau)$
- J (Dezero.Variable): $G(\tau) \log \pi_{\theta}(A_0 | S_0)$

求出 J 之后，通过 J.backward() 求 $G(\tau) \nabla_{\theta} \log \pi_{\theta}(A_0 | S_0)$ 。下面是 Agent 类剩下的代码。

ch09/simple_pg.py

```

class Agent:
    ...

    def add(self, reward, prob):
        data = (reward, prob)
        self.memory.append(data)

    def update(self):
        self.pi.cleargrads()

```

```

G, loss = 0, 0
for reward, prob in reversed(self.memory):
    G = reward + self.gamma * G

for reward, prob in self.memory:
    loss += -F.log(prob) * G

loss.backward()
self.optimizer.update()
self.memory = [] # 重置内存

```

add 方法是智能代理每次采取行动并获得奖励时被调用的方法。该方法将奖励 (reward) 和智能代理采取的行动的概率 (prob) 存储在内存 (self.memory) 中。update 方法是智能代理到达目标时被调用的方法。该方法首先会求收益 G 。收益可以通过反向计算获得的奖励的做法来高效地计算 (5.2.3 节介绍过其原理)。然后该方法会计算损失函数。具体做法是对每个时刻求 $-F \cdot \log(\text{prob})$ ，再将其乘以作为权重的 G ，之后求它们的和。剩下的就是一直以来的神经网络的训练代码。



在训练神经网络时，通常要设置损失函数。对于这个例子，我们可以将目标函数 $J(\theta)$ 乘以 -1 所得到的 $-J(\theta)$ 作为损失函数，此时可以通过梯度下降法的最优化方法 (SGD、Adam 等) 更新参数。

最后在倒立摆环境中运行智能代理。代码如下所示。

```

episodes = 3000
env = gym.make('CartPole-v0')
agent = Agent()
reward_history = []

for episode in range(episodes):
    state = env.reset()
    done = False
    total_reward = 0

    while not done:
        action, prob = agent.get_action(state)
        next_state, reward, done, info = env.step(action)

        agent.add(reward, prob)
        state = next_state
        total_reward += reward

```

```
agent.update()  
reward_history.append(total_reward)
```

上面这段代码我们非常熟悉。首先，在 `while` 语句中，增加智能代理获得的奖励（`reward`）和行动的概率（`prob`）。然后在离开 `while` 语句后（回合结束时），通过 `agent.update()` 更新策略。

运行此代码，随着回合的推进，获得的奖励也会增加。图 9-2 是结果的示意图。

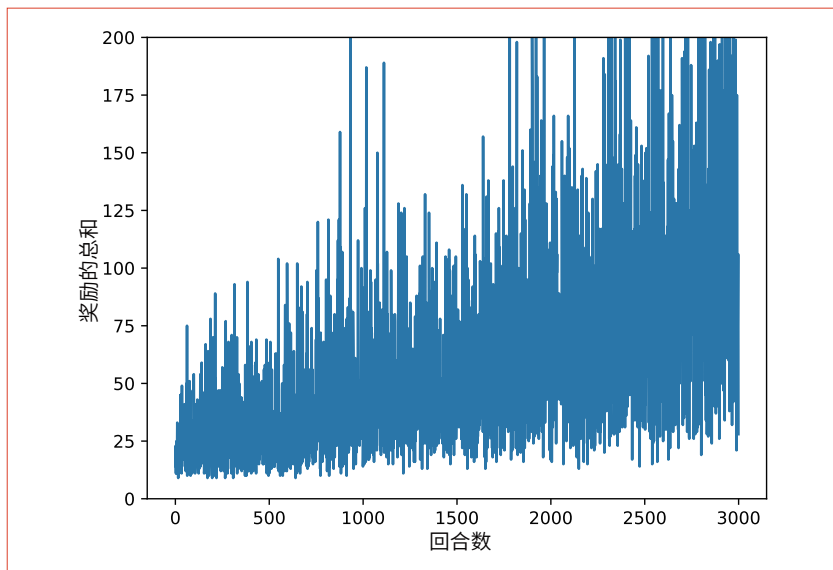


图 9-2 每个回合获得的奖励总和的变化情况

如图 9-2 所示，虽然变动很剧烈，但随着回合的推进，结果越变越好。不过，由于图 9-2 只是一次实验的结果，因此结果可能不可靠。于是，我们进行了 100 次实验，然后取平均值，结果如图 9-3 所示。

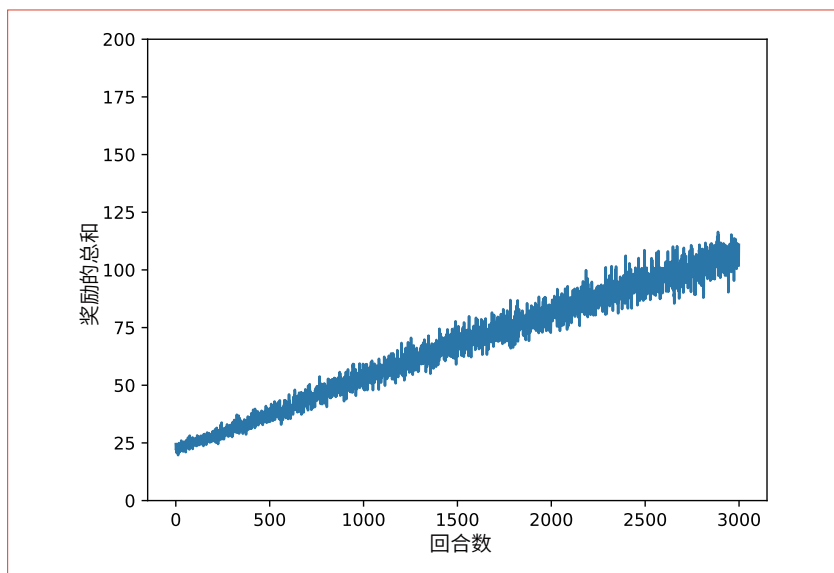


图 9-3 100 次实验的平均结果

从图 9-3 中可以看出，随着回合的推进，奖励的总和会逐渐增加。但即使经历了 3000 个回合，依然没有达到这次任务的上限值 200，似乎还有改进的余地。下面让我们来改进一下这里推导的最简单的策略梯度法。这个改进算法就是著名的 REINFORCE 算法。

9.2 REINFORCE

REINFORCE^[15] 是对 9.1 节的策略梯度法的改进算法。本节首先会基于数学式推导 REINFORCE 算法，然后会通过修改之前的部分代码的做法来实现 REINFORCE。



REINFORCE 这个名字是 “**RE**ward **I**ncrement = **N**onnegative **F**actor × **O**ffset **R**einforcement × **C**haracteristic **E**ligibility”（奖励增量 = 非负因子 × 偏移强化 × 特征资格）的首字母缩写。

9.2.1 REINFORCE 算法

先来复习一下 9.1 节。最简单的梯度策略法是基于式 9.1 实现的。

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T G(\tau) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right] \quad (9.1)$$

式 9.1 中的 $G(\tau)$ 是目前为止获得的所有奖励的总和（准确地说是“带折现率”的奖励的总和）。这里要思考的问题是，无论在哪个时刻 t ，式子中都是 $G(\tau) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$ ，我们始终会使用固定不变的权重 $G(\tau)$ 来增加（或减少）采取行动 A_t 的概率。

智能代理行动的好坏是根据行动之后获得的奖励总和来评估的（回顾一下价值函数的定义）。反过来说，采取某个行动之前获得的奖励与该行动的好坏无关。如果要评估在某个时刻 t 采取的行动 A_t ，那么在此之前做了什么以及获得了多少奖励都无所谓。我们是根据采取行动 A_t 之后的结果（在时刻 t 以后获得的奖励的总和）来判断行动 A_t 的好坏的。

式 9.1 中行动 A_t 的权重是 $G(\tau)$ 。这个权重 $G(\tau)$ 包括在时刻 t 之前的奖励。也就是说，原本不相关的奖励作为噪声数据包含在内了。为了改进这一点（去除噪声数据），可以对权重 $G(\tau)$ 作如下修改。

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T G_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right] \\ G_t &= R_t + \gamma R_{t+1} + \dots + \gamma^{T-t} R_T \end{aligned} \quad (9.3)$$

如式 9.3 所示，权重变成了 G_t 。权重 G_t 是在时刻 $t \sim T$ 获得的奖励的总和。因此，选择行动 A_t 的概率将由不包含时刻 t 之前的奖励的权重 G_t 增强。这就是改进 9.1 节的策略梯度法的思路。基于式 9.3 的算法叫作 REINFORCE。另外，本书不会进行式 9.3 成立的证明。如果你对证明感兴趣，请参考参考文献 [16] 和参考文献 [17]。



基于式 9.3 的 REINFORCE 算法优于最简单的策略梯度法（基于式 9.1 的算法）。通过无限增加的样本数，式 9.1 和式 9.3 都会收敛到正确的 $\nabla_{\theta} J(\theta)$ （可以说是无偏差的）。但式 9.1 的方差更大，因为式 9.1 中的权重包含了无关的数据（噪声）。

9.2.2 REINFORCE 的实现

由于 REINFORCE 的方差小，因此即使数据样本少，也能高精度地近似数据。下面我们来实现 REINFORCE 以验证其精度。REINFORCE 的代码与 9.1 节中的代码基本相同，不同之处只有 Agent 类的 update 方法。下面仅列出了不同部分的代码。

ch09/reinforce.py

```
class Agent:
    ...

    def update(self):
        self.pi.cleargrads()

        G, loss = 0, 0
        for reward, prob in reversed(self.memory):
            G = reward + self.gamma * G
            loss += -F.log(prob) * G

        loss.backward()
        self.optimizer.update()
        self.memory = []
```

`self.memory` 是按先后顺序保存智能代理获得的奖励（reward）和行动概率（prob）的列表。上面的代码按从后向前的顺序依次访问了 `self.memory` 的元素，并计算了每个时刻的 G 。

下面运行 REINFORCE 的代码。图 9-4 显示了只运行 1 次代码得到的结果和运行 100 次代码得到的平均结果。

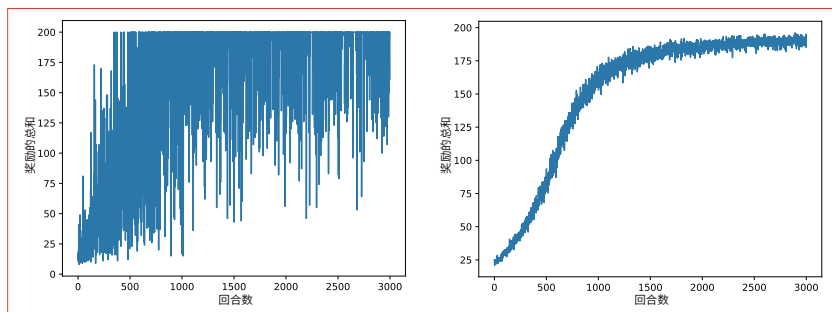


图 9-4 每个回合获得的奖励总和的变化情况（左图是运行 1 次的结果，右图是运行 100 次的平均结果）

从图 9-4 中可以看出，随着回合的推进，奖励的总和会逐渐增加。此外，图 9-4 的右图（平均结果）已接近上限 200。与上一次的结果相比，不但训练稳定了，训练速度也提高了。

9.3 基线

下面介绍一种叫作基线（baseline）的技术，该技术可以改进 REINFORCE。让我们先通过一个简单的例子来了解一下基线的思路，然后再将基线应用于 REINFORCE。

9.3.1 基线的思路

下面是一个简单的例子。假设现在有 A、B、C 3 人参加了考试，分别得了 90 分、40 分、50 分，如图 9-5 所示。

A	90
B	40
C	50

图 9-5 3 人的考试结果

我们对这个结果求方差。使用 NumPy 编写的代码如下所示。

```
import numpy as np

x = np.array([90, 40, 50])
print(np.var(x))
```

输出结果

466.6666666666667

如上所述，考试结果的方差为 466.6666666666667，这是一个很大的值。由于方差表示数据的离散程度，因此该结果表明考试结果的离散程度很大。我们要考虑的是如何减小方差。

这里使用 3 人之前的考试结果。假设我们获得了这些结果，如图 9-6 所示。

	第 1 次考试	第 2 次考试	...	第 10 次考试
A	92	80	...	74
B	32	51	...	56
C	45	53	...	49

图 9-6 3 人之前的考试结果

有了图 9-6 所示的之前的考试结果，我们就可以预测下一次考试的分数了。一种预测方法是对之前的分数取平均，然后将下一次考试结果作为与之前的平均分的差值进行预测。

对图 9-6 的结果分别取平均，最终 A 为 82 分、B 为 46 分、C 为 49 分。接下来，将它们作为预测值，计算其与分析对象的考试结果的差值，如图 9-7 所示。

A	90	-	A	82	=	A	8
B	40		B	46		B	-6
C	50		C	49		C	1
结果			之前分数的平均值（预测值）			差值	

图 9-7 计算实际的结果与预测值之间的差值

下面计算图 9-7 的差值的方差。代码如下所示。

```
x = np.array([90, 40, 50])

avg = np.array([82, 46, 49])
diff = x - avg # [8, -6, 1]

print(np.var(diff))
```

输出结果

32.666666666666664

如上所述，方差为 32.666666666666664。与之前相比，方差大幅减小了。如本例所示，我们可以通过对某个结果减去预测值的方法来减小方差。预测值的精度越高，方差就越小。这就是基线这种方法的思路。下面将基线应用于 REINFORCE。

9.3.2 带基线的策略梯度法

式 9.3 是 REINFORCE 的数学式。将基线应用于这个 REINFORCE 的数学式如式 9.4 所示。

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T G_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right] \quad (9.3)$$

$$= \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T (G_t - b(S_t)) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right] \quad (9.4)$$

式 9.4 用 $G_t - b(S_t)$ 代替了 G_t 。这里 $b(S_t)$ 可以是任何函数。也就是说，只要输入是 S_t ， $b(S_t)$ 是什么函数都行。这个 $b(S_t)$ 就是基线。



D.2 节进行了从式 9.3 到式 9.4 的变形成立（等式成立）的证明。如果你对此感兴趣，可以参考一下。

式 9.4 中的 $b(S_t)$ 可以是任何函数。例如，在状态 S_t 下，可以考虑使用之前获得的奖励的平均值作为 $b(S_t)$ 。实践中经常使用的是价值函数，数学式为 $b(S_t) = V_{\pi_{\theta}}(S_t)$ 。如果能够使用基线减小方差，那么就可以进行样本效率更高的训练。另外，将价值函数作为基线使用时，我们是不知道真正的价值函数 $v_{\pi_{\theta}}(S_t)$ 的。这种情况下还需要训练价值函数。

最后，我们再通过直观介绍补充说明一下为什么使用基线更好。这里以倒立摆为例，思考图 9-8 所示的状态。

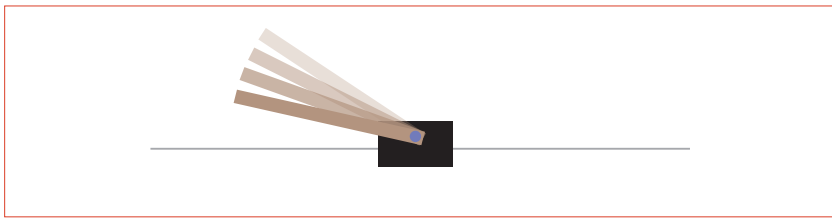


图 9-8 杆子失去平衡的状态

图 9-8 表示游戏结束之前的杆子失去平衡的状态¹。在这种状态下，无论采取什么样的行动，在几个时间步之后游戏都将结束。

设图 9-8 的状态为 s 、在此状态下采取的行动为 a 。假定从状态 s 开始几个时间步（比如 3 个时间步）后游戏一定会结束。在这种情况下，状态 s 的收益 G 为 3（这里设折现率 γ 为 1）。如果使用的是没有基线的 REINFORCE，那么状态 s 下的行动 a 就会因为权重 3 而被增强（状态 s 下选择行动 a 的概率会变高）。但无论采取什么样的行动，3 个时间步之后游戏一定会结束，所以可以说这种提高行动 a 被选择的概率的工作是无意义的。

此时就要用到基线了。这里使用价值函数作为基线，假设我们已经知道图 9-8 的例子中的 $V_{\pi_\theta}(S_t) = 3$ （在实际任务中，需要通过蒙特卡洛方法或 TD 方法等进行训练和预测）。此时的权重为 $G_t - V_{\pi_\theta}$ ，所以是 0。由于权重是 0，因此无论选择什么行动，采取那个行动的概率都不会变大，也不会变小。像这样使用基线，有望减少无谓的训练。

9.4 Actor-Critic

强化学习的算法大体可分为基于价值的方法和基于策略的方法。本章前面出现的都是基于策略的方法，而 DQN 和 SARSA 是基于价值的方法。因此，我们可以考虑一个使用了二者的方法（基于价值且基于策略的方法），如图 9-9 所示。

注 1：在 OpenAI Gym 的倒立摆环境中，如果杆子相对于垂直线超过了 15° ，则游戏结束。这里为了便于说明，我们假定杆子达到平行线后游戏结束。

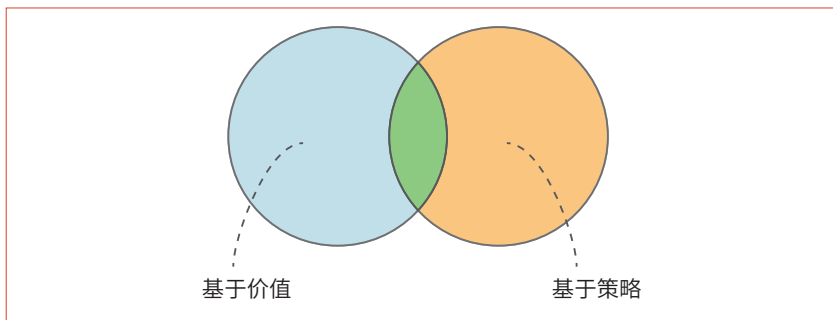


图 9-9 基于价值的方法和基于策略的方法

如果在 9.3 节介绍的带基线的 REINFORCE 中使用价值函数作为基线，那么就可以将其视为基于价值且基于策略的方法。本节将进一步改进带基线的 REINFORCE，推导一个叫作 Actor-Critic 的算法。Actor-Critic 也是基于价值且基于策略的方法。

9.4.1 Actor-Critic 的推导

首先从复习带基线的 REINFORCE 开始。带基线的 REINFORCE 的目标函数的梯度的数学式如下所示。

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T (G_t - b(S_t)) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right] \quad (9.4)$$

式 9.4 中的 G_t 表示收益、 $b(S_t)$ 表示基线。可以使用任何函数作为基线。这里我们使用基于神经网络建模的价值函数作为基线。因此，我们要用到以下这些新的记号。

- ω ：表示价值函数的神经网络的所有权重参数。
- $V_{\omega}(S_t)$ ：将价值函数模型化的神经网络。

此时目标函数的梯度的数学式如下所示。

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T (G_t - V_{\omega}(S_t)) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right] \quad (9.5)$$

式 9.5 中存在一个问题，即只要没抵达目标，就无法确定收益 G_t 的值。也就是说，在抵达目标之前，无法更新策略和价值函数。如果这是基于蒙特卡洛方法的算法，那么二者无法更新就是它的缺点。消除这个缺点的方法是第 6 章介绍的 TD 方法。使用 TD 方法训练价值函数时，使用 1 个时间步（或 n 个时间步）后的结果就能进行更新，如图 9-10 所示。

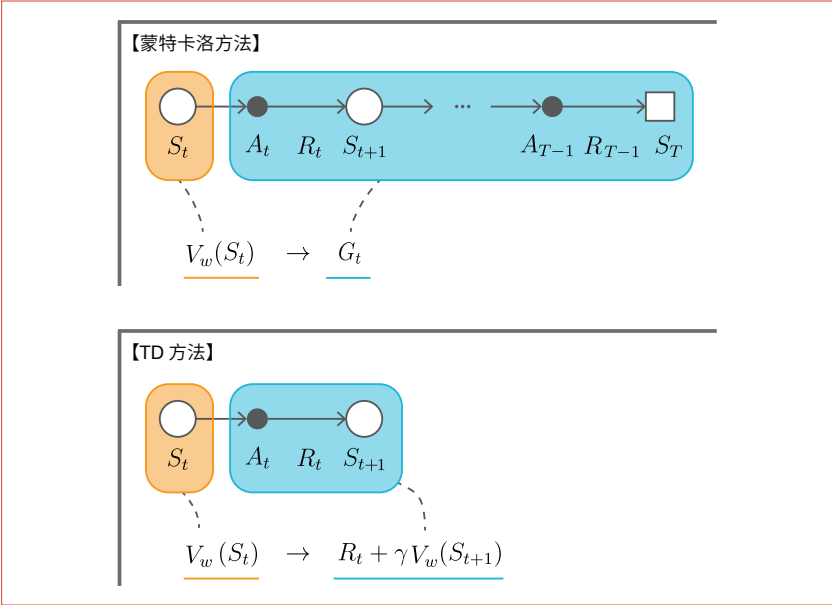


图 9-10 蒙特卡洛方法与 TD 方法

如图 9-10 所示，在训练价值函数 $V_w(S_t)$ 时，蒙特卡洛方法使用的是收益 G_t ，而 TD 方法使用的是 $R_t + \gamma V_w(S_{t+1})$ 。



使用神经网络对价值函数建模时，我们以接近 $R_t + \gamma V_w(S_{t+1})$ 为目标训练 $V_w(S_t)$ 的值。具体来说就是将 $V_w(S_t)$ 和 $R_t + \gamma V_w(S_{t+1})$ 的均方差作为损失函数，通过梯度下降法更新神经网络的权重。

下面将基于蒙特卡洛方法的式 9.5 切换为 TD 方法，其中，代替 G_t 的是 $R_t + \gamma V_w(S_{t+1})$ 。此时得到的式子如下所示。

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T (R_t + \gamma V_{\omega}(S_{t+1}) - V_{\omega}(S_t)) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right] \quad (9.6)$$

基于式 9.6 的算法就是 Actor-Critic。策略 π_{θ} 和价值函数 V_{ω} 是神经网络，我们要同时训练这两个神经网络。具体来说，对于策略 π_{θ} ，要基于式 9.6 进行训练；而对于价值函数 V_{ω} ，则通过 TD 方法，以接近 $R_t + \gamma V_{\omega}(S_{t+1})$ 为目标训练 $V_{\omega}(S_t)$ 的值。



Actor-Critic 中的 Actor 是“行动者”的意思，也就是采取行动的人，相当于策略 π_{θ} 。而 Critic 是“评论者”的意思，相当于价值函数 V_{ω} 。因此，Actor-Critic 的意思是“使用 V_{ω} 来评论基于策略 π_{θ} 采取的行动的好坏”。

9.4.2 Actor-Critic 的实现

下面实现 Actor-Critic。策略和价值函数这两个神经网络的代码如下所示。

ch09/actor_critic.py

```
import numpy as np
import gym
from dezero import Model
from dezero import optimizers
import dezero.functions as F
import dezero.layers as L

class PolicyNet(Model):
    def __init__(self, action_size=2):
        super().__init__()
        self.l1 = L.Linear(128)
        self.l2 = L.Linear(action_size)

    def forward(self, x):
        x = F.relu(self.l1(x))
        x = self.l2(x)
        x = F.softmax(x)
        return x

class ValueNet(Model):
    def __init__(self):
        super().__init__()
        self.l1 = L.Linear(128)
        self.l2 = L.Linear(1)
```

```
def forward(self, x):
    x = F.relu(self.l1(x))
    x = self.l2(x)
    return x
```

上面的代码将策略的网络实现为了 PolicyNet 类，将价值函数的网络实现为了 ValueNet 类。策略的最终输出是 Softmax 函数的输出，所以输出的是概率。接下来是 Agent 类的代码。

ch09/actor_critic.py

```
class Agent:
    def __init__(self):
        self.gamma = 0.98
        self.lr_pi = 0.0002
        self.lr_v = 0.0005
        self.action_size = 2

        self.pi = PolicyNet()
        self.v = ValueNet()
        self.optimizer_pi = optimizers.Adam(self.lr_pi).setup(self.pi)
        self.optimizer_v = optimizers.Adam(self.lr_v).setup(self.v)

    def get_action(self, state):
        state = state[np.newaxis, :] # 增加小批量的轴
        probs = self.pi(state)
        probs = probs[0]
        action = np.random.choice(len(probs), p=probs.data)
        return action, probs[action]

    def update(self, state, action_prob, reward, next_state, done):
        # 增加小批量的轴
        state = state[np.newaxis, :]
        next_state = next_state[np.newaxis, :]

        # ① self.v 的损失
        target = reward + self.gamma * self.v(next_state) * (1 - done)
        target.unchain()
        v = self.v(state)
        loss_v = F.mean_squared_error(v, target)

        # ② self.pi 的损失
        delta = target - v
        delta.unchain()
        loss_pi = -F.log(action_prob) * delta
```

```
self.v.cleargrads()
self.pi.cleargrads()
loss_v.backward()
loss_pi.backward()
self.optimizer_v.update()
self.optimizer_pi.update()
```

`get_action` 方法可以基于策略取出行动。需要注意的是, 由于输入到神经网络中的数据将作为小批量进行处理, 因此在处理其中一个数据 (状态) 时需要小批量的轴。另外, `get_action` 方法返回了两个值, 即选择的行动及其概率。选择行动的概率将在稍后的损失函数计算中使用。

`update` 方法可以训练价值函数和策略。在代码①处为价值函数 (`self.v`) 计算损失。为此要计算 TD 目标 (`target`), 求出当前状态下其与价值函数 (`v`) 的均方差。然后, 在代码②处为策略 (`self.pi`) 计算损失。根据式 9.6, 需要将其乘以 -1 的值作为损失。剩下的就是一直以来的神经网络的训练代码。

运行智能代理的代码与之前相同, 不再赘述。得到的结果如图 9-11 所示。

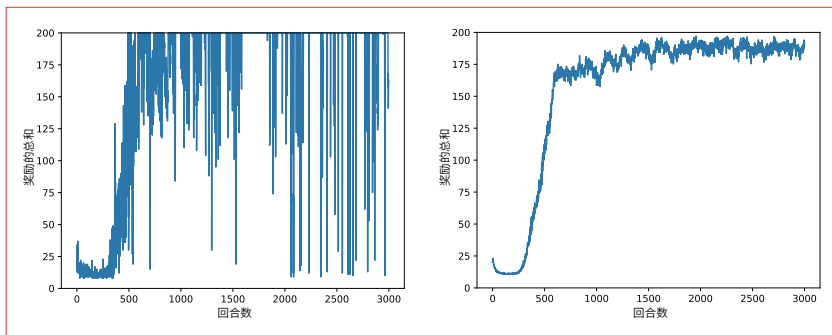


图 9-11 每个回合获得的奖励总和的变化情况 (左图是运行 1 次的结果, 右图是运行 100 次的平均结果)

如图 9-11 所示, 我们顺利地进行了训练。这样就完成了 Actor-Critic 的实现。

9.5 基于策略的方法的优点

基于策略的方法的优点有3个，下面我们分别来介绍一下。

(1) 策略直接模型化，更高效

我们最终想得到的是最优策略。基于价值的方法预测的是价值函数，并以其为基础确定策略。而基于策略的方法可以直接预测策略。有些问题的价值函数的形状可能很复杂，但其最优策略可能很简单。对于这样的情况，基于策略的方法有望更快地训练。

(2) 在连续的行动空间中也能使用

前面介绍的强化学习的问题全部是离散的行动空间。例如，对于倒立摆问题，要选择的是向右或向左这两个行动。在这样的离散行动空间中，行动是从多个非连续的候选中选择一个。但也有连续的行动空间的情况。例如，OpenAI Gym 的 Pendulum 是在摆杆的中心施加扭矩，抬起摆杆的任务（参见图 9-12）。在这种情况下，要施加的扭矩（如 2.05 或 -0.24 等）的连续值构成了行动空间。

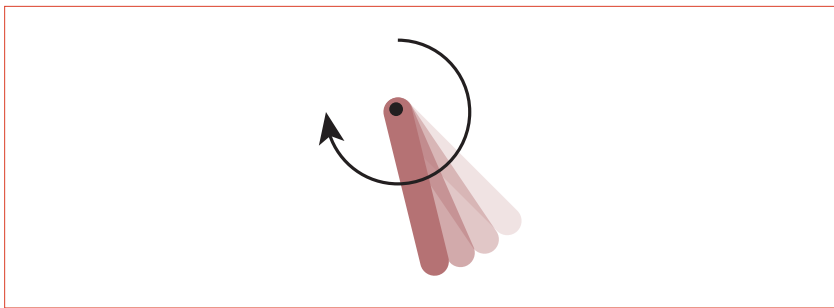


图 9-12 OpenAI Gym 的 Pendulum

基于价值的方法难以用于连续的行动空间的情况。可以考虑的解决方法有几种。一种方法是将连续的行动空间离散化。但是，如何离散化[这个过程叫作“量子化”(quantize)]是一个很难的问题。任务不同，合适的方法也不同。大多数情况下，好的离散化的方法需要不断尝试和寻找。

而基于策略的方法对于连续的行动空间的情况也能轻松应对。例如，对于神经网络的输出呈正态分布的情况，可以考虑让神经网络输出正态分布的均值和方差。根据其均值和方差进行采样，即可得到连续值，如图 9-13 所示。

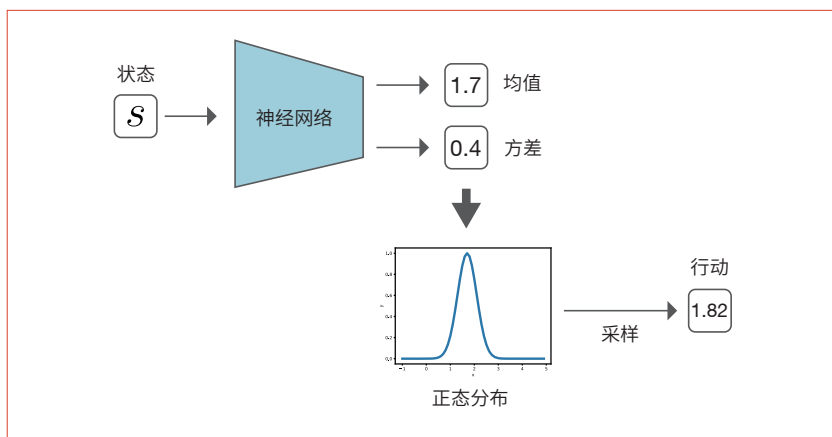


图 9-13 输出连续值的神经网络的例子

(3) 行动的选择概率平滑地变化

在基于价值的方法中，智能代理的行动通常通过 ϵ -greedy 算法来选择。在这种情况下，被选中的基本是 Q 函数值最大的行动。此时，如果 Q 函数的更新导致值最大的行动发生了改变，那么采取的行动也会突然改变。而基于策略的方法是通过 Softmax 函数确定各行动的概率。因此，在更新策略参数的过程中，各行动的概率会平滑地变化。这使得策略梯度法的训练更趋于稳定。

以上就是基于策略的方法的 3 个优点。需要注意的是，基于策略的方法并非总是优于基于价值的方法，它也有适合的任务和不适合的任务。另外，无论是基于策略的方法还是基于价值的方法，实现它们的算法都有各种不同的创新。因此，在选择算法时，我们需要考虑到这些问题。

9.6 小结

在本章中，我们学习了基于策略的方法——策略梯度法。具体来说，我们学习了 4 种策略梯度法的算法。它们的统一的数学式如下所示。

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \Phi_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right]$$

- (1) $\Phi_t = G(\tau)$ (最简单的策略梯度法)
- (2) $\Phi_t = G_t$ (REINFORCE)
- (3) $\Phi_t = G_t - b(S_t)$ (带基线的 REINFORCE)
- (4) $\Phi_t = R_t + \gamma V(S_{t+1}) - V(S_t)$ (Actor-Critic)

上面 4 种方法的权重 Φ_t 各不相同。最简单的策略梯度法在所有时刻的权重都是 $G(\tau)$ 。对它进行改进，将时刻 t 的收益 G_t 作为权重进行评估的是 REINFORCE。另外，通过加入“基线”这一方法，减小了方差的方法是带基线的 REINFORCE。Actor-Critic 是一种除了策略之外，对价值函数也用神经网络建模的方法。期待 (1) $\rightarrow$ (2) $\rightarrow$ (3) $\rightarrow$ (4) 这样的编号一直延续下去，从而有更高级的方法出现，产生更好的结果。

除了上面的 4 个选择，我们也可以像下面这样使用 Q 函数（使用 Q 函数也能使等式成立的证明请参考参考文献 [18]）。

$$\Phi_t = Q(S_t, A_t)$$

此外，利用基线的价值函数，可以将 Φ_t 设置为如下形式。

$$\begin{aligned} \Phi_t &= Q(S_t, A_t) - V(S_t) \\ &= A(S_t, A_t) \end{aligned}$$

如上式所示，我们可以从 Q 函数中减去价值函数的值。这就是优势函数，可以表示为 $A(s, a)$ 。

第 10 章

进一步学习

前面我们已经学习了许多关于强化学习的知识。本书的前半部分介绍了强化学习的基础知识，后半部分介绍了使用强化学习的重要方法。本章介绍的是更先进的现代深度强化学习的算法。本章首先展示深度强化学习的全貌图，然后介绍策略梯度法系列的算法，接下来介绍 DQN 系列的算法。

另外，本章还将介绍深度强化学习的重要研究案例，既包括围棋、将棋等棋盘游戏的案例，也包括机器人控制、半导体设计等在社会上取得巨大成果的案例。最后，本章将探讨深度强化学习的可能性和存在的课题。

10.1 深度强化学习算法的分类

如图 10-1 所示，深度强化学习的算法可以分为不同的类型。分类方法参考了参考文献 [19]。

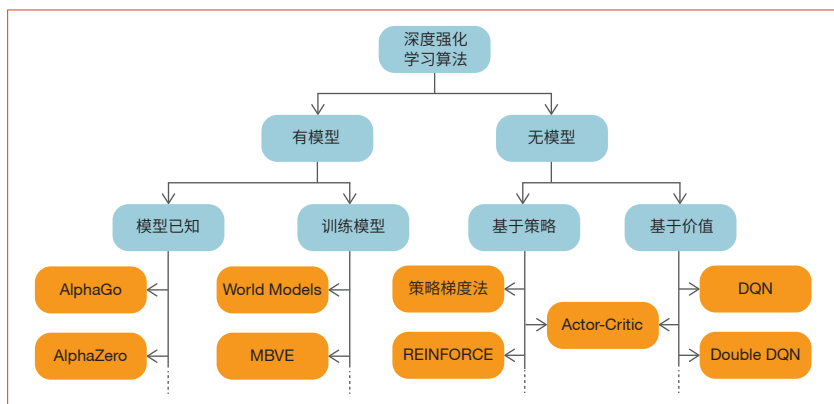


图 10-1 深度强化学习算法的分类图

下面我们基于图 10-1 来介绍一下深度强化学习算法的分类方法。首先是一个很大的分类条件：是否使用环境模型（状态迁移函数 $p(s'|s, a)$ 和奖励函数 $r(s, a, s')$ ）。不使用环境模型的方法叫作无模型（model-free）的方法，使用环境模型的方法叫作有模型（model-based）的方法。

有模型的方法可以分为两种，一种是提供了环境模型的方法，另一种是训练环境模型的方法。如果提供了环境模型，那么智能代理可以通过规划（planning）来解决问题，而不采取任何行动。在第 4 章中，我们访问了环境模型，使用动态规划法解决了问题。另外，在围棋、将棋等棋盘游戏中，环境模型可以作为已知的问题设定来处理。对于这种问题，也可以使用提供了环境模型的方法。著名的算法有 AlphaGo、AlphaZero 等（参见 10.4.1 节）。

如果没有提供环境模型，则可以考虑根据从环境中获得的经验来训练环境模型。训练的环境模型除了可以用于规划之外，还可以用于评估和改进策略。属于这个领域的方法包括 World Models^[20]、MBVE（Model-Bas Value Estimation）^[21] 等。目前这个领域的研究非常活跃，今后有望用于实现人类所拥有的通用智能。



训练环境模型的方法有几个问题。最大的问题是智能代理只能获得环境模型生成的一部分样本数据。这可能会导致训练的环境模型与实际环境之间出现偏差。即使智能代理在训练的模型中能够采取良好的行动，也有可能在实际环境中采取不好的行动。

训练环境模型的方法非常有潜力，不过，目前无模型的方法取得了很大的成果。本书的主要对象也是无模型的方法。无模型方法可以分为基于策略的方法和基于价值的方法，以及两者兼而有之的方法。

基于策略的方法包括策略梯度法和 REINFORCE。Actor-Critic 是基于策略且基于价值的方法（参见第 9 章）。10.2 节介绍了策略梯度法的改进算法。基于价值的方法是对价值函数进行建模和训练的方法，其中最重要的方法是 DQN。10.3 节介绍了 DQN 的改进算法。

10.2 策略梯度法的改进算法

第 9 章介绍过 REINFORCE、Actor-Critic 等策略梯度法的算法。本章将介绍它们的更先进的算法，具体如下。

- A3C 和 A2C（进行分散学习的算法）
- DDPG（具有确定性策略的算法）
- TRPO 和 PPO（在目标函数中增加约束的算法）

根据算法的特征，我们将算法分为了以上 3 组。首先来介绍一下 A3C 和 A2C。

10.2.1 A3C 和 A2C

A3C^[22] 是 “Asynchronous Advantage Actor-Critic” 的缩写。因为名字中有 3 个 “A” 和一个 “C”，所以被称为 A3C。A3C 的特点是 asynchronous，即 “异步”。这里所说的 “异步” 是指多个智能代理并行采取行动，异步更新参数。



在计算机科学领域,“并发”(concurrent)和“并行”(parallel)的含义不同。“并发”是一种在某一时刻只进行一项工作(处理),但通过多个处理之间的高速切换,使多个处理看起来同时进行的技术。而“并行”是指在物理上不同的场所同时进行多个处理。

A3C 训练使用的是神经网络建模的 Actor-Critic 学习策略。它需要使用一个全局网络和多个本地网络,如图 10-2 所示。

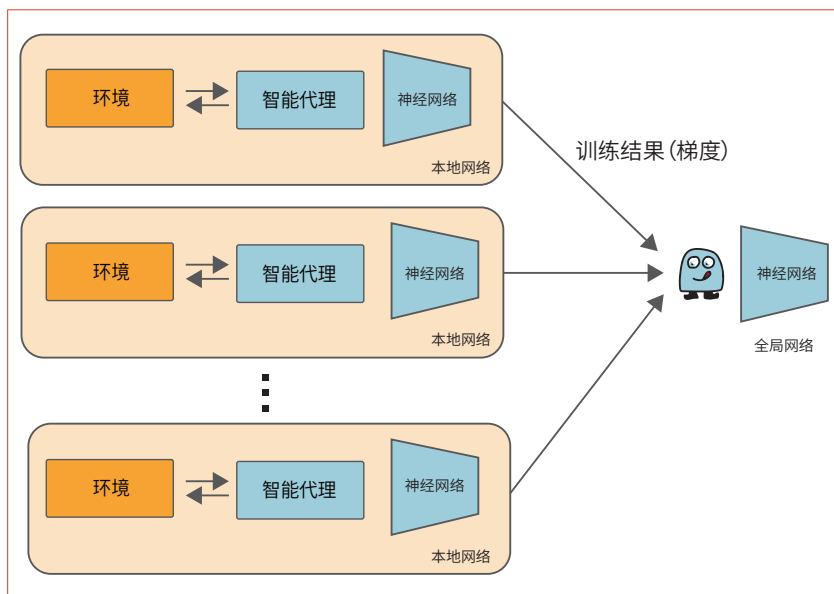


图 10-2 A3C 的结构

本地网络在各自的环境中独立进行训练。然后它们会将作为训练结果的梯度发送到全局网络。全局网络使用来自多个本地网络的梯度异步更新权重参数。这样,在更新全局网络的权重参数的同时,可以定期同步全局网络和本地网络的权重参数。

A3C 的优点是,多个智能代理并行运行可以加快训练速度。另外,由于智能代理并行独立地行动(探索),因此我们可以得到多样化的数据。这样就可以减少整体获得的训练数据之间的相关性,从而使训练更加稳定。



DQN 使用了一种叫作“经验回放”（experience replay）的技术来减少训练数据的相关性。这是一种将经验数据保存在缓冲区中，从中随机选择多个数据的方法。这样就能减少训练数据之间的相关性。但是经验回放技术不能用于异策略型的方法。另外，经验回放还有使得内存使用量和计算量增大的缺点。

并行处理是在异策略型的方法中也可以使用的通用的做法。多个智能代理的并行运行，可以不依赖于经验回放而减少数据的相关性。自 A3C 的论文发表以来，并行处理在许多研究中被使用，已成为一种趋势。

另外，A3C 的 Actor-Critic 将共享神经网络的权重。如图 10-3 所示，可以在一个神经网络中共享权重，并输出策略和价值函数。无论是策略还是价值函数，接近输入的层都会有相似的权重，所以这样的参数共享型网络结构能够有效地发挥作用。

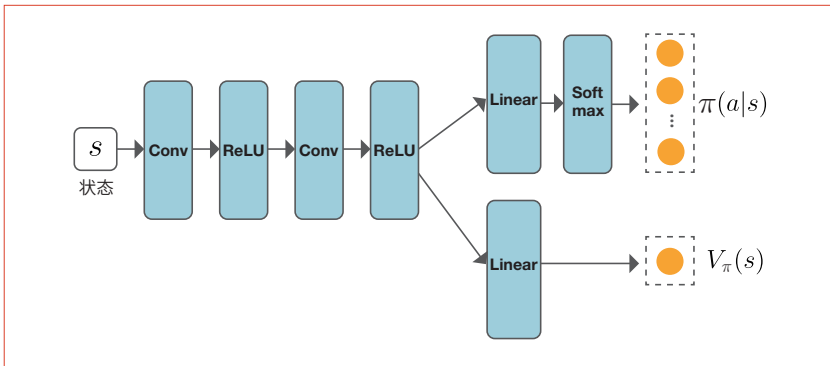


图 10-3 A3C 的网络组成

接下来是 A2C^[22]。A2C 是同步更新参数的方法，它不采取异步更新参数的方式。A2C 因从 A3C 中去掉第一个 A 而得名。A2C 的结构如图 10-4 所示。

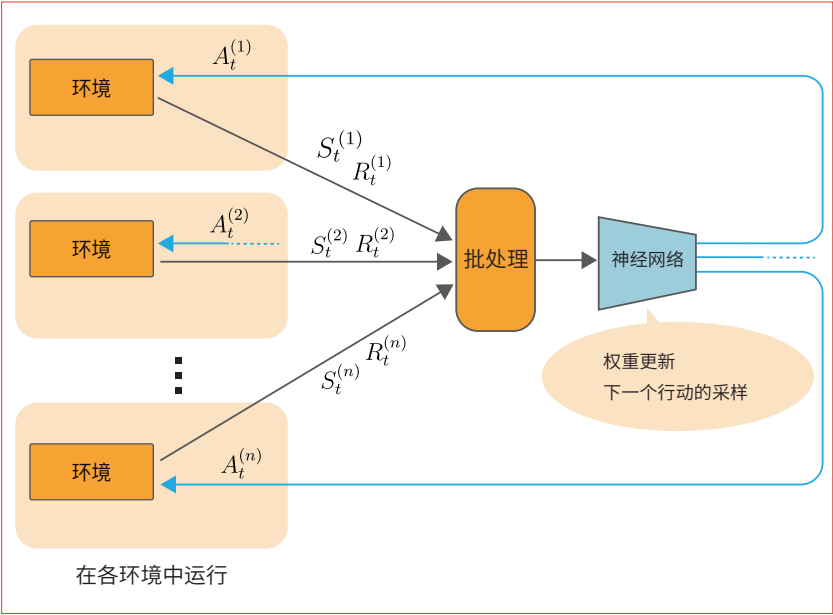


图 10-4 A2C 的结构

如图 10-4 所示，智能代理在各自的环境中独立运行。因此，时刻 t 的状态在各个环境中不同。于是我们将时刻 t 在各个环境中的状态同步地作为批处理汇总，进行神经网络的训练。此时，从神经网络的输出（策略的概率分布）中对下一个行动进行采样，并将采样后的行动传给各个环境。

实验表明，与 A3C 相比，即使 A2C 采取了同步更新，其性能也不会下降。另外，A2C 更易于实现，它可以高效地利用 GPU 等计算资源。因此，实践中 A2C 使用得更多。



A3C 需要在各个环境中运行神经网络。因此，如果准备了 N 个环境，那么理想情况下就需要准备 N 个 GPU。而 A2C 只在一个环境中运行神经网络，因此，即使只有一个 GPU 也可以计算。

10.2.2 DDPG

诸如直接对策略建模的策略梯度法之类的方法也可以用于连续的行动

空间问题。例如，表示策略的神经网络可以设计为输出“正态分布的平均值”（参见图 10-5）。实际行动是从它的正态分布中采样得到的。

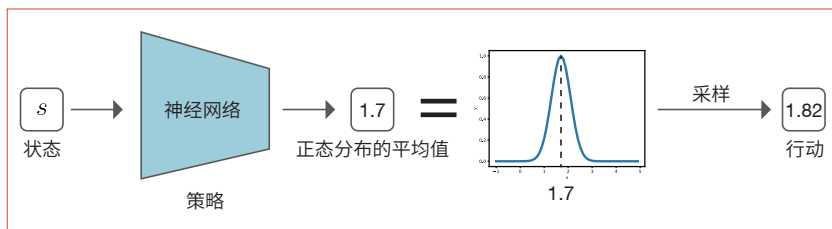


图 10-5 表示策略的神经网络输出连续值的概率分布示例

DDPG^[23] 是“Deep Deterministic Policy Gradient”（深度确定性策略梯度法）的缩写。从名字就能想到它的含义，它是针对连续行动空间问题而设计的算法。神经网络可以将行动作为连续值直接输出，如图 10-6 所示。

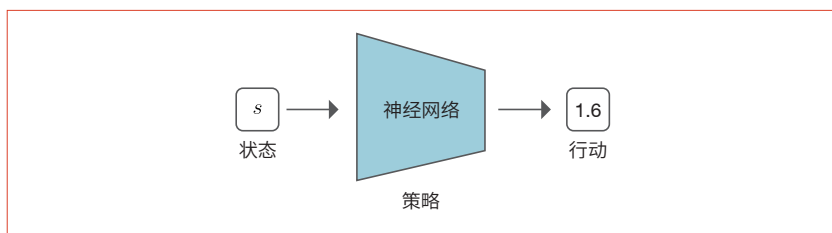


图 10-6 将行动作为连续值直接输出的神经网络

DDPG 的策略是在输入某个状态 s 时确定唯一的行动 a 的确定性的策略。DDPG 的这个确定性的策略是包含在 DQN 中的。这里将表示策略的神经网络设为 $\mu_\theta(s)$ ，将表示 DQN 的 Q 函数的神经网络设为 $Q_\phi(s, a)$ 。 θ 和 ϕ 分别作为每个神经网络的参数。此时，DDPG 通过以下两个过程更新参数。

- (1) 朝着使 Q 函数的输出变大的方向更新策略 $\mu_\theta(s)$ 的参数 θ 。
- (2) 通过 DQN 进行的 Q 学习来更新 Q 函数 $Q_\phi(s, a)$ 的参数 ϕ 。

首先介绍第一个训练过程。这是将两个神经网络组合使用，朝着使 Q 函数的输出变大的方向更新确定性策略 $\mu_\theta(s)$ 的参数 θ 的过程，如图 10-7 所示。

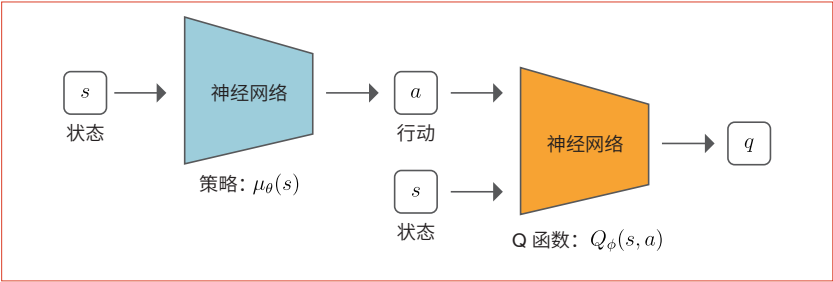


图 10-7 基于两个神经网络的计算流程

图 10-7 中的重点是 $\mu_\theta(s)$ 输出的行动 a 是连续值，该值可直接作为 $Q_\phi(s, a)$ 的输入。这样就能通过两个神经网络进行反向传播了。通过反向传播可以求梯度 $\nabla_\theta q$ （这里的 q 是 Q 函数的输出）。这样就能使用梯度 $\nabla_\theta q$ 更新参数 θ 了。



在图 10-7 中，假设根据随机性策略对行动进行采样，那么反向传播就会在进行采样的地方停止（在此之后，传播的梯度都是 0）。在这种情况下，无法更新策略的参数。

第二个训练是在 DQN 中进行的 Q 学习。第 8 章介绍过这个方法。但这里可以使用确定性策略 $\mu_\theta(s)$ 进行更高效的计算。

在 DQN 中进行的 Q 函数的更新是使 $Q_\phi(S_t, A_t)$ 的值为 $R_t + \gamma \max_a Q_\phi(S_{t+1}, a)$ （或接近这个值）。这里通过第一个训练，策略 $\mu_\theta(s)$ 输出了使 Q 函数变大的行动。因此，可以使用以下近似值。

$$\max_a Q_\phi(s, a) \approx Q_\phi(s, \mu_\theta(s))$$

求最大值 $\max_a$ 通常需要大量的计算。尤其是在行动空间连续的情况下，我们还得解决复杂的优化问题。DDPG 将 $\max_a Q_\phi(s, a)$ 替换成了 $Q_\phi(s, \mu_\theta(s))$ 的两个神经网络的前向传播（参见图 10-7）。这样可以简化计算，有望提高训练效率。



DDPG 还采用了“软目标”“探索噪声”等办法。“软目标”是使 DQN 的目标网络“缓慢化”的方法。这样，目标网络中的参数将不再定期与训练中的参数同步，而是每次都朝着训练中的参数的方向靠近。“探索噪声”是向确定性策略中加入噪声，在行动中引入随机性的方法。详细介绍请参考参考文献 [23]。

10.2.3 TRPO 和 PPO

策略梯度法使用神经网络对策略进行建模，并使用梯度法对参数进行更新。策略梯度法的问题在于，虽然根据梯度可以知道参数更新的“方向”，但不知道“步幅”，也就是前进多少合适。步幅过大会使策略变差，步幅过小则会导致训练几乎没有进展。解决此问题的方法是 TRPO (Trust Region Policy Optimization) [24]。这个名字的意思是“信赖领域策略优化”。顾名思义，我们可以在信赖的领域中以合适的步幅优化策略。

评测两个概率分布相似程度的指标是 KL 散度 (Kullback-Leibler divergence)。TRPO 以更新策略前后的 KL 散度为指标，增加了该值不超过阈值的约束。也就是说，这是一个在 KL 散度的约束下，求目标函数最大化的问题。通过施加此约束，可以实现合适的步幅。



TRPO 的详细内容（尤其是数学推导）比较复杂，本书将不再赘述其技术内容。它的重点是，为了在梯度更新时使步幅合适，在每次更新中施加约束，以避免策略发生太大变化（在该约束下进行训练）。这就将问题转化为了有约束的最优化问题，进而可以运用数学工具来解决。

在使用 TRPO 求解有约束的最优化问题时，需要计算海塞矩阵这种二阶微分。海塞矩阵需要大量的计算，因此计算量大是它的瓶颈。改进此问题的方法是 PPO (Proximal Policy Optimization, 近端策略优化) [25]。PPO 是使 TRPO 更加简单的方法，它在削减计算量的同时，保持了与 TRPO 相同的性能，因此我们在实践中经常使用它。

10.3 DQN 的改进算法

DQN 是深度强化学习中最重要算法。现在研究者们仍在提出许多基于 DQN 的扩展方法。本节介绍的是起源于 DQN 的改进算法中特别重要的算法，如图 10-8 所示。

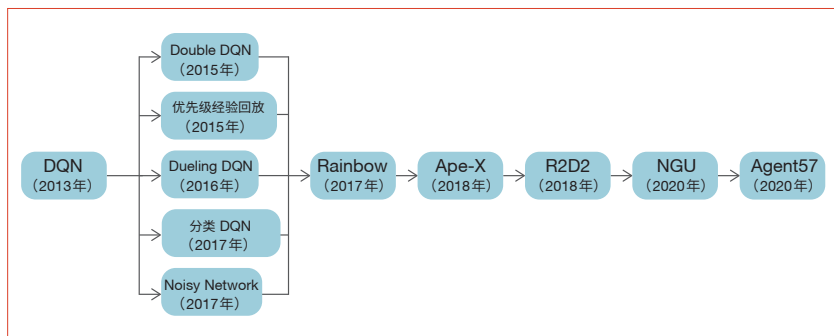


图 10-8 起源于 DQN 的改进算法

如图 10-8 所示，以 DQN 作为起源并对其加以改进的方式，现代的深度强化学习得到了发展。以下是 8.4 节中介绍过的 3 种算法。

- Double DQN
- 优先级经验回放
- Dueling DQN

本节将依次介绍除了上面 3 种算法之外的几个方法。

10.3.1 分类 DQN

首先介绍分类 DQN^[26] 方法。下面我们来回顾一下 Q 函数，其数学式如下所示。

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a]$$

Q 函数的特点是用期望值这一个值来表示收益 G_t 这一概率事件，如图 10-9 所示。

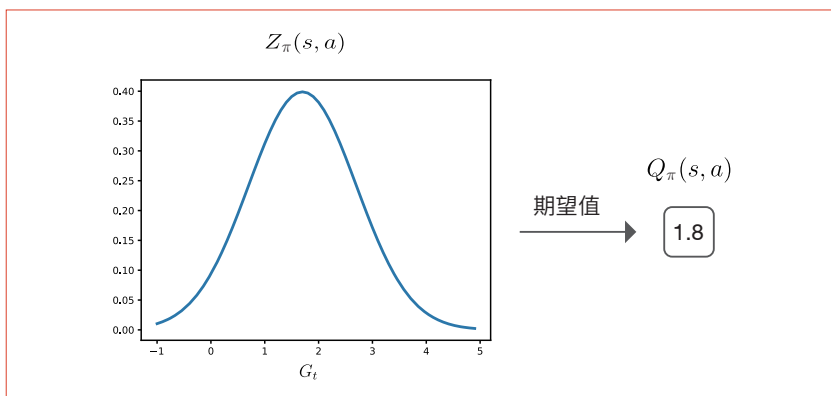


图 10-9 收益的概率分布与 Q 函数之间的关系（这里用 $Z_\pi(s, a)$ 表示收益的概率分布）

更进一步说，DQN 在 Q 学习中要训练的是由 Q 函数这个期望值所表示的值。进一步扩展这个思路，不要训练 Q 函数这个期望值，而要训练“分布”。这个思路叫作分布强化学习（distributional reinforcement learning）。分布强化学习将训练收益 $Z_\pi(s, a)$ 的概率分布。分类 DQN 是基于分布强化学习的方法。分类是指将图 10-10 所示的分类分布模型化。

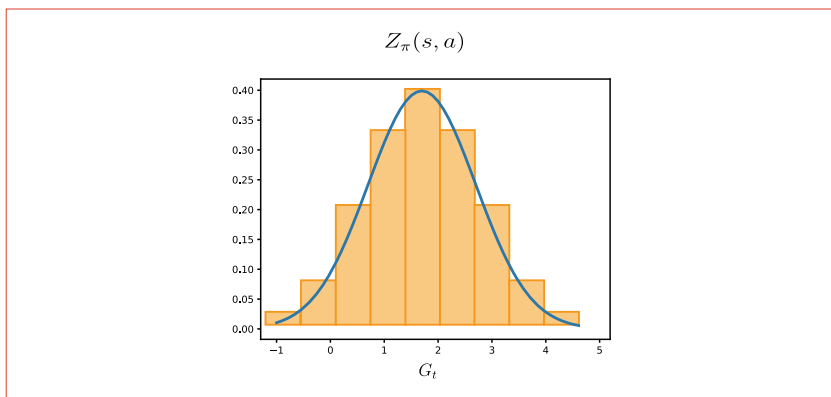


图 10-10 基于分类分布的模型化（蓝线是真正的分布）

分类分布是属于多个类别（离散的值）中某一个类别的概率分布。如图 10-10 所示，获得的收益值被划分在几个区域（该区域叫作“格子”）中，进入每个格子的概率被建模为分类分布。

分类 DQN 会将收益建模为分类分布，并训练该分布的形状。因此需要推导分类分布版的贝尔曼方程，并基于它来更新分布。顺带一提，当分类分布的格子的数量为 51 时，Atari 任务的性能最好，所以这种方法有时候也被称为“C51”。

10.3.2 Noisy Network

DQN 根据 ϵ -greedy 算法选择行动。也就是说，DQN 会以 ϵ 的概率随机选择行动，以 $1 - \epsilon$ 的概率选择贪婪的行动（函数值最大的行动）。在实践中，我们常常要进行“调度设置”（schedule setting），即随着回合的发展，逐渐降低 ϵ 的值。这里的问题是 ϵ 的值。 ϵ 是超参数，它的设置方法对最终精度有很大影响。但是 ϵ 的值有多种候选选项。

为了解决 ϵ 的设置问题，人们提出了 Noisy Network^[27]。Noisy Network 在神经网络中加入了随机性。因此，无须借助 ϵ -greedy 算法，使用贪婪算法就能选择行动。具体来说，就是在输出侧的全连接层中使用有噪声的全连接层。在有噪声的全连接层中，权重会被建模为正态分布的均值和方差，并在每次前向传播时从正态分布对权重进行采样。这样，每个前向传播都会增加随机性，最终的输出也会发生变化。

10.3.3 Rainbow

到目前为止，我们已经了解了 DQN 的多种扩展算法。将这些全部组合起来的算法是 Rainbow^[28]。在原始的 DQN 的基础上，Rainbow 应用了下列所有方法。

- Double DQN
- 优先级经验回放
- Dueling DQN
- 分类 DQN
- Noisy Network

使用 Atari 进行的实验的结果如图 10-11 所示。

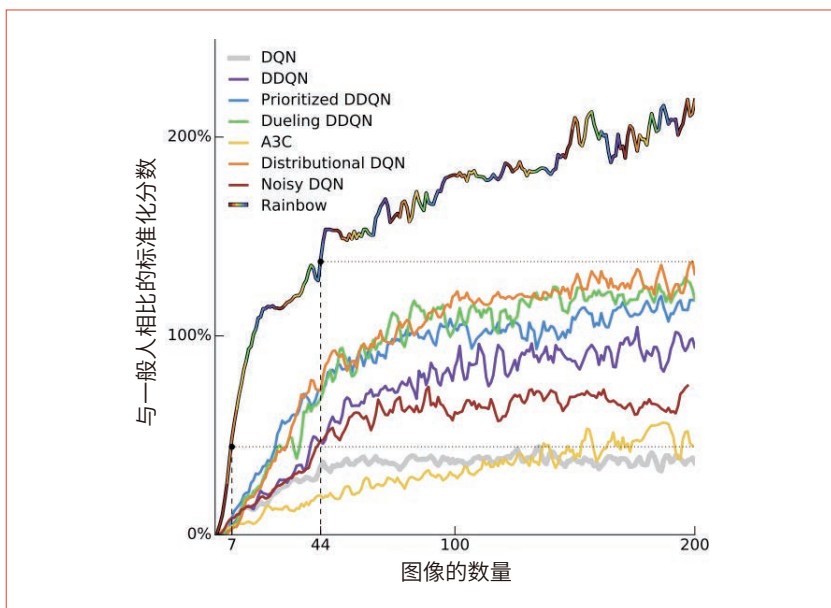


图 10-11 Rainbow 与其他方法的精度比较（图片出处见参考文献 [28]）

图 10-11 的横轴是在训练中使用的图像的数量（其实实际使用的数量是横轴数值的 100 万倍）。纵轴是与一般人相比的标准化分数，值越大性能越好。如图 10-11 所示，与其他方法相比，Rainbow 的性能有了飞跃性的提升。

10.3.4 在 Rainbow 以后提出的改进算法

在 Rainbow 被提出之后，使用多个 CPU/GPU 的分布式并行化训练取得了丰硕的成果。这种在多个运行环境中进行训练的做法也被称为“分布式强化学习”。分布式强化学习的著名方法是 Ape-X^[29]。通过以 Rainbow 为基础，Ape-X 让更多的智能代理在不同的 CPU 上独立行动，一边收集经验数据一边进行训练。所有的分布式并行智能代理都被设置了不同的探索率 ϵ ，以收集不同的经验数据。分布式并行化不仅减少了训练时间，还通过多样的经验数据实现了性能的提升。

进一步改进了 Ape-X 的方法是 R2D2^[30]。R2D2 在 Ape-X 的基础上使用了处理时间序列数据的 RNN（实际上使用的是 LSTM）。虽然这是一个简单的思路，但该方法在 RNN 的训练上想了很多办法，成功地进一步提升了 Ape-X 的性能。顺带一提，R2D2 这个名字由 Recurrent 和 Replay（经验回放）的两个“R”，以及 Distributed 和 Deep Q-Network 的两个“D”构成（当然，这个名字也是对电影《星球大战》中的“R2D2”机器人的致敬）。

接下来是进一步改进了 R2D2 的 NGU^[31]。NGU 是“Never Give Up”（永不放弃）的简称。顾名思义，即使是困难的任务，尤其是奖励稀疏的任务，该方法也会毫不犹豫地进行探索，永不放弃。NGU 在 R2D2 的基础上增加了内部奖励（intrinsic reward）的机制。内部奖励是在状态的迁移与预想的不一样（根据差异的“吃惊”程度）时给自己加上的奖励。在奖励几乎为 0 的稀疏任务的情况下，内部奖励机制能够促进基于“好奇心”的行动。在这个过程中，如果顺利的话，可以找到将通常的奖励最大化的策略。



孩子（尤其是婴儿）经常通过玩耍来寻求某种新奇的惊喜体验。与我说他们是为了特定的目的而学习，不如说是出于好奇心而行动。内部奖励的目的是将这种好奇心引起的行动引入智能代理中。

最后介绍 Agent57^[32] 方法。Agent57 是改进了 NGU 的方法。它重点改进了内部奖励机制，并使用了一种叫作“元控制器”（meta-controller）的机制，为智能代理的决策分配提供了灵活性。Atari（准确地说是“Atari 2600”）共有 57 个游戏。Agent57 成功地取得了在所有的游戏中都比人玩得好的成就。这是第一个取得如此成就的强化学习算法。

10.4 案例研究

深度强化学习最有名的案例是电视游戏和围棋等棋盘游戏。除此之外，它在机器人、自动驾驶、医疗、金融、生物等各个领域都取得了成果。本节将介绍几个有名的案例，以展示深度强化学习的用途。

10.4.1 棋盘游戏

围棋、将棋、黑白棋等棋盘游戏具有以下特点。

- 棋盘上的所有信息都一目了然（所有信息都已知的情况叫作“完全信息”）。
- 一方赢，一方输（二者的收益之和为0，这个特性因此被称为“零和性”）。
- 状态迁移没有偶然的要素（由确定性决定）。

在具有这些特点的游戏里，重点在于“预判”：如果我下了这步棋，对方会下哪步棋，那样的话我再下哪步棋……考虑将来各种各样的可能性，就能找到好的下法。棋盘游戏的“预判”可以表示为树结构，如图10-12所示。

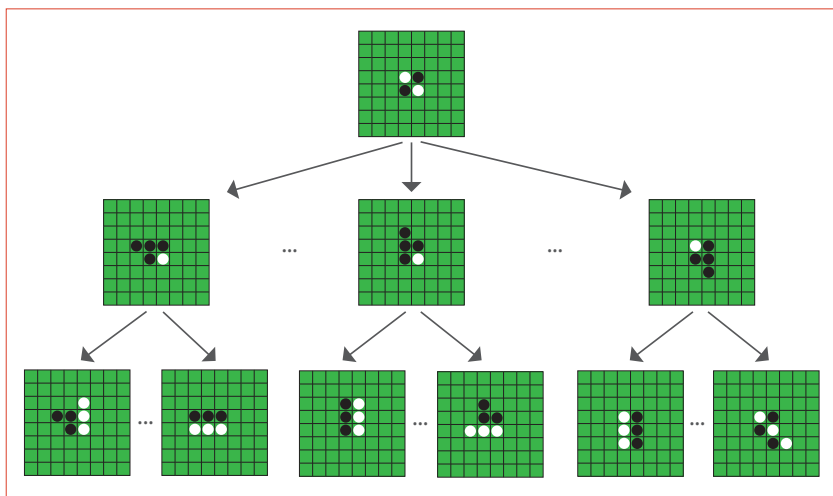


图 10-12 以黑白棋为例的游戏树

图10-12所示的游戏树（game tree）用图的节点表示棋面，用边（箭头）表示棋步。如果将游戏树全部展开，那么我们就知道所有的结果，因此便能找到最好的一手棋。但围棋和将棋的候选状态（棋子等放置模式的模式）的数量是天文数字，把游戏树全部展开是不现实的。因此，我们

需要高效地探索游戏树。



不同棋盘游戏的游戏树的所有状态（节点）是不同的。黑白棋是 10 的 58 次方，国际象棋是 10 的 123 次方，将棋是 10 的 226 次方，围棋是 10 的 400 次方。

蒙特卡洛树搜索（Monte Carlo tree search, MCTS）是一种通过蒙特卡洛方法近似地展开树的方法。在评估棋面好坏的时候，让两个玩家随机出手，直至决出胜负。反复进行这样的尝试（从特定的棋面开始的游戏），统计胜败的结果，看赢了多少。这样就能近似地表示棋面的良好程度了。另外，决出胜负之前的随机出手叫作“模拟对局”（playout）或“轨迹模拟”（rollout）。

蒙特卡洛树搜索的游戏流程是，评估当前盘面上所有可能的棋步，并根据结果决定下一步棋。这就是蒙特卡洛树搜索的基本思路。除了上述思路，蒙特卡洛树搜索通常还会采用展开看起来有希望的盘面，然后对其进行评估并反馈结果的办法。有关蒙特卡洛树搜索的更多信息，请参考参考文献 [33]。

AlphaGo

AlphaGo^[34]是结合了蒙特卡洛树搜索和深度强化学习的方法。AlphaGo 在 2016 年战胜了世界围棋顶尖棋手，这震惊了全世界。

AlphaGo 使用了两个神经网络：一个是评估当前盘面获胜概率的 Value 网络，另一个是表示策略的 Policy 网络。Policy 网络用于输出下一步落子的概率（例如，在 (1, 1) 落子的概率为 2.4%，在 (1, 2) 落子的概率为 0.2%，等等）。这两个神经网络使蒙特卡洛树搜索更精确。

另外，AlphaGo 在使用人类的棋谱数据训练这两个网络之后，会以自我对弈（self play）的形式反复进行对战，并使用在这个过程中收集的经验数据进行进一步的训练。



自我对弈可以用强化学习的框架来解释，即有一个下围棋的“智能代理”存在，智能代理的副本则作为对手来扮演“环境”的角色。这二者交互对战，最终获得胜利 / 失败的奖励。在自我对弈的环境中相互作用并收集数据这一点上，可以说 AlphaGo 使用的方法是强化学习。

AlphaGo Zero

AlphaGo 使用了人的棋谱数据作为训练数据。AlphaGo Zero^[35] 则不使用这样的训练数据，它只通过自我对弈的强化学习进行训练。另外，不使用 AlphaGo 所使用的领域知识（对象领域的专业知识）也是 AlphaGo Zero 的特征。以下是 AlphaGo Zero 的其他改进。

- 用一个神经网络表示 Policy 网络和 Value 网络。
- 不在蒙特卡洛树搜索中模拟对局，只使用神经网络的输出来评估各节点（状态）。

通过这些改进，AlphaGo Zero 比以往的 AlphaGo 更简单、更通用了。而且，在没有人类知识的情况下，AlphaGo Zero 完全可以只通过强化学习进行训练。令人惊讶的是，不利用人类知识的 AlphaGo Zero 已经超过了以往的 AlphaGo。在实际进行的比赛中，AlphaGo Zero 以 100 比 0 的成绩大胜 AlphaGo。



在使用人类数据的监督学习中，模型所能达到的水平是训练数据中包含的人类的水平。通过强化学习，从自身经验中学习的方法则没有这样的约束，理论上可以超越人类。事实上，AlphaGo Zero 的实力已经远远超过了人类。这也是显示了强化学习的潜力的重要研究。

AlphaZero

与只能用于围棋的 AlphaGo Zero 相比，AlphaZero^[36] 还支持国际象棋和将棋。AlphaZero 的算法只是对 AlphaGo Zero 进行了微调，因此可以将二者视为基本相同的算法。也就是说，AlphaZero 是一种与棋盘游戏的种类无关的通用算法。

10.4.2 机器人控制

深度强化学习也可用于机器人控制等现实世界的系统。Google AI 的研究^[37] 成功地训练了能够抓住各种物体的机器人操作。具体来说，在机器人的上部安装摄像头，机器人根据从摄像头中得到的图像采取行动，如

图 10-13 所示。最终根据行动的结果进行奖励：如果能抓住物体就作为成功给予奖励；如果不能抓住就作为失败给予奖励。训练是在这个强化学习的框架中进行的。

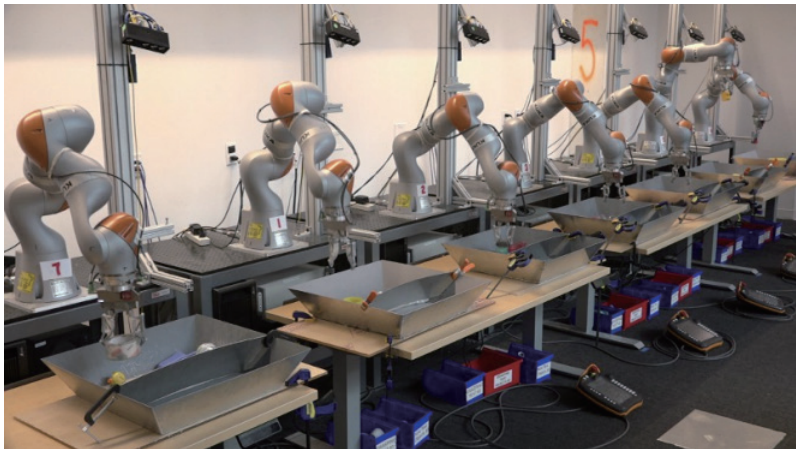


图 10-13 用 7 台机器人收集经验数据（图片出处见参考文献 [37]）

如图 10-13 所示，数据收集是由 7 台机器人在几个月的实际运行中进行的。这个过程中使用的强化学习算法是基于 Q 学习的 QT-Opt 方法。由于 QT-Opt 基于 Q 学习，因此它是异策略型的方法。也就是说，过去得到的经验数据也可以使用。使用现实世界中的机器人时，由于收集成本变高，因此能够利用过去的数据这一点很重要。

实验表明，QT-Opt 对未见过的物体的抓取成功率是 96%，与 Google AI 以前开发的监督学习算法相比，QT-Opt 可以将失败率减少到 20% 以下。

10.4.3 NAS

深度学习的架构（网络结构）通常由人来决定。设计一个好的架构需要经验，并且要进行多次尝试。最近，自动设计最佳架构的研究非常盛行^[38]。这就是被称为 NAS（Neural Architecture Search，神经架构搜索）的领域。

贝叶斯优化、遗传编程等都是进行 NAS 的方法，其中最具有前景的候选方法是强化学习。强化学习的 NAS 热潮是由一篇题为“Neural Architecture Search with Reinforcement Learning”^[39] 的论文引发的。该论文通过强化学习自动优化网络结构，成功地发现了与人的设计同等优秀甚至更好的架构。下面简要介绍一下该论文的方法。

该论文的核心思路是，神经网络的架构可以通过“文本”指定。下面是一个将神经网络的架构指定为文本数据的示例。不过这是一个虚构的格式。

```
graph {
  node {
    input: "Input1"
    output: "Result ReLU"
    op_type: "Relu"
  }
  node {
    input: "Result ReLU"
    input: "Input2"
    output: "Output"
    op_type: "Add"
  }
  ...}
```

另外，使用 RNN 可以生成可变长度的文本。理论上，也可以考虑使用输出上面这样文本的 RNN。综上所述，可以考虑使用 RNN 作为智能代理，在强化学习的框架下寻找最佳架构，如图 10-14 所示。

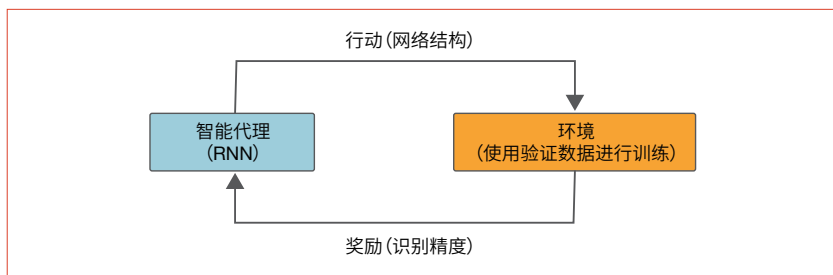


图 10-14 探索神经网络架构的框架

在图 10-14 中，RNN 以文本形式生成了神经网络的架构。那就是智能代理的行动。然后，生成的架构（神经网络）使用验证数据进行训练，最

后评测了识别精度。这个识别精度就是奖励。论文“Neural Architecture Search with Reinforcement Learning”中使用 REINFORCE 算法更新了 RNN 的参数。这样 RNN 就能输出可以提高识别精度的架构了。



论文“Neural Architecture Search with Reinforcement Learning”中列举了一些办法来限制要探索的架构的范围，其中一个示例是将参数限定于卷积层的过滤器大小、步幅等，RNN 按顺序输出了这些参数。

10.4.4 其他案例

深度强化学习的应用案例还有很多，下面我们再介绍 3 个案例。

自动驾驶

目前，为了实现自动驾驶汽车，全球的研究者们都在进行研究开发，其中许多研究采用了基于深度学习的监督学习。与此同时，我们也观察到一些研究倾向于采用强化学习的方式^[40]。由于自动驾驶问题需要观察环境，因此选择最适合环境的行动（踩刹车、转动方向盘等）的一系列决策自然适用于强化学习的框架。

这里，AWS DeepRacer 是学习自动驾驶和强化学习的环境。DeepRacer 是一款自主驾驶的赛车，它的控制采用了基于深度强化学习的方法。用户可以实现自己的算法，并自行设置奖励函数等。然后可以使用模拟器在 AWS 上进行训练。最终创建的模型可以在虚拟赛车和实际赛车中使用。另外，还有定期举办的“冠军杯”等赛事，参赛者之间通过竞争来赢取奖金。

建筑物的能源管理

建筑物内部每天要消耗大量电力。降低能耗是一个重要的挑战，但建筑物内部环境复杂，通常难以高效运营。针对这个问题，最近人们提出了很多深度强化学习的方法，并取得了重大成果^[41]。

例如，在办公楼空调设备的运营中，有人提出了基于 DQN 的方法。据报道，这项研究在保持工作人员舒适性的同时，成功实现了 30% 以上的成本削减^[42]。另外，谷歌的数据中心通过基于机器学习的方法对冷却系统进

行控制，成功地削减了 40% 的电力消耗。这个案例也部分采用了强化学习的方法^[43]。

半导体芯片的设计

在半导体芯片的设计中，需要将数十亿个晶体管配置在小型基板上。这是一个复杂的问题，通常伴随着很多困难。过去，半导体芯片的设计需要熟练的技术人员花费大量时间来操作。但谷歌提出了基于深度强化学习的半导体芯片设计方法^[44]。

谷歌的这项研究将半导体芯片的设计视为强化学习问题，智能代理为了找到最佳的设计，不断尝试各种半导体的布局，反复试错。最终，这项研究取得了显著成果，在不到 6 小时的时间内完成了设计，并在所有主要指标上实现了优于或等同于专家设计的方案的性能。这项技术已经在谷歌开发的 TPU（Tensor Processing Unit）的设计中得到应用。

10.5 深度强化学习的挑战和可能性

本节首先探讨将深度强化学习应用于现实世界的系统时面临的挑战和解决方案，然后探讨将强化学习问题建模为 MDP 时需要考虑的要点，最后探讨强化学习的可能性。

10.5.1 应用于实际系统

深度强化学习已被应用于机器人控制等现实世界的系统。但不同于虚拟世界，现实世界有许多的限制。例如，机器人价格昂贵，难以大量准备，因此收集机器人的经验数据非常困难。此外，在强化学习中，我们通过试错的方式收集经验数据，但在此过程中，必须避免机器人出现故障或对周围环境造成危险。下面我们来介绍一下针对这些问题的一些有效的解决方案。

使用模拟器

现实世界中的许多限制可以通过模拟器来解决。在模拟器上，智能代

理与环境的交互可以快速重复。另外，即使智能代理有危险的行动，在模拟器上也不会产生任何问题。但是，现实世界和模拟器之间有差距。由于模拟器不能完全模仿现实世界的环境，因此在模拟器上训练的模型（智能代理）在现实世界的环境中往往不能按照预期工作。解决这个问题的研究领域被称为“Sim2Real”。

Sim2Real 拥有一些有效的方法，下面我们介绍一种叫作领域随机化（domain randomization）的方法。该方法通过向模拟器添加随机元素来创建多样化的环境，并在这些多样化的环境中让智能代理采取行动以进行学习。这样做可以提高泛化性能，进而使智能代理能够很好地应对现实世界中的任务。例如，针对用拥有五根手指的机械手解开魔方的任务，OpenAI 公开了一项使用领域随机化的研究，如图 10-15 所示。

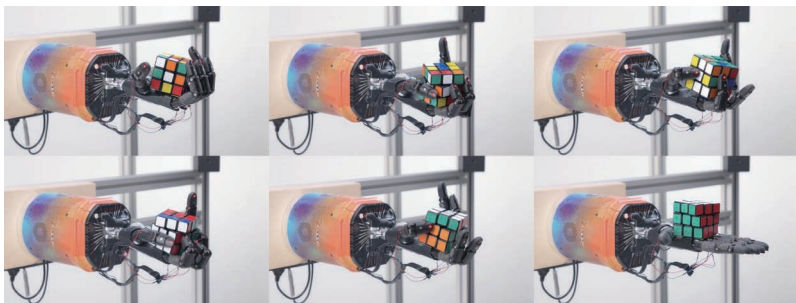


图 10-15 使用强化学习和领域随机化，使得拥有五根手指的机械手解开了魔方（图片出处见参考文献 [45]）

通过向光源、纹理等渲染图像的参数以及摩擦等力学参数引入随机值，这项研究创建了多样化的环境，并对智能代理进行了训练。结果表明，即使在现实世界中，智能代理也能成功解决任务，而无须进行额外的训练。

离线强化学习

有一种方法能够巧妙地利用过去收集到的经验数据。例如，自动驾驶、机器人控制等领域可能已经积累了人类操作过的许多经验数据。此外，在对话系统等领域，我们也许很容易就能收集到过去的人际对话历史

记录（有许多研究利用增强学习实现了聊天机器人这样的对话系统）。我们可以利用这些过去获取的经验数据来进行智能代理的训练，这样的经验数据也叫作“离线数据”。此外，我们还可以仅使用这些离线数据，在不与环境进行交互的情况下推测出最优策略。这种做法叫作离线强化学习（offline reinforcement learning）（参见图 10-16）。

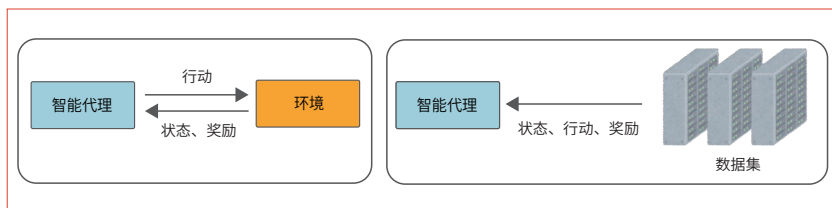


图 10-16 常规强化学习（左）与离线强化学习（右）

如图 10-16 的左图所示，常规的强化学习会发生智能代理与环境的交互（这种学习也可以叫作“在线”的强化学习）。而图 10-16 的右图的离线强化学习只使用之前收集的经验数据（离线的数据集）进行训练。离线强化学习的特点是不与现实环境发生交互。



离线强化学习与异策略型的方法相关。异策略型通过行为策略收集经验数据，然后再使用这些经验数据更新目标策略。因此，我们可以使用异策略型的方法实现离线强化学习。但是，一般的异策略型的方法要创建符合目标策略的行为策略，在此基础上边与环境交互一边推进智能代理的训练（请回顾一下 Q 学习）。而离线强化学习完全不与环境交互，只使用离线经验数据进行训练。

离线强化学习方面的技术课题很多。例如，如何仅使用离线的数据集评估策略，以及如何仅使用离线的数据集改进策略。对于这样的课题，我们可以使用之前在介绍异策略型的算法时提到的方法（重要性采样等）。另外，研究者们还提出了其他各种方法，目前正在积极地研究中^[46]。

模仿学习

人往往可以从熟练的人（专家）那里学到很多东西。例如，棒球少年可以模仿职业棒球选手的击球动作来进行练习。与此类似，我们也可以考虑模

仿专家的动作来训练策略。这就是模仿学习 (imitation learning)。在模仿学习中, 我们通过参考专家的示范, 以模仿其动作为目标进行策略的训练。

研究者们提出了许多模仿学习的方法。例如, 论文“Deep Q-learning from Demonstrations”^[47]中就使用了专家的 Atari 游戏演示来执行 DQN。具体来说, 就是将专家在游戏时获得的“状态、行动、奖励”的时间序列数据 (以下称为“专家数据”) 添加到 DQN 的经验回放缓冲区中, 并在此基础上进行 DQN 的训练。在此过程中调整 DQN 的更新式, 使得从经验回放缓冲区中更容易选择专家数据, 并使 DQN 逐渐接近专家数据。通过这种方式就可以实现更加重视专家数据的 DQN 模型。

10.5.2 将问题表示为 MDP 形式时的建议

强化学习的很多理论是以 MDP 为前提的。本书以强化学习的问题可以被表示为 MDP 形式为前提, 在此基础上探索了各种算法。因此, 在通过强化学习来解决现实问题时, 首先要将问题表示为 MDP 形式。此时, 如何将 MDP 形式化会对最终结果产生很大影响。本节将讨论在将强化学习问题表示为 MDP 形式时应考虑的问题。

MDP 的灵活性

并非现实世界中的所有问题都可以表示为 MDP 这个问题形式。但出乎意料的是, 大多数问题可以表示为 MDP 形式。在 MDP 中, 环境和智能代理相互“抛出”状态、行动和奖励这 3 个信息。这时, 使用什么传感器、如何控制行动以及如何设定奖励等细节都可以根据问题的具体情况灵活决定。这种灵活性使得 MDP 的适用范围比想象中广泛。

另外, 智能代理的行动可以按照从高到低的不同层级分开考虑。例如, 在控制机器人时, 高层级的意识决策可以是扔掉垃圾、充电等; 低层级的意识决策可以是向电机输入 x 伏的电流这样的行动。同样, 状态也可以按照从高到低的不同层级分开考虑。

时间单位 (时间步) 也可以根据智能代理做出决定的时间来确定。例如, 一个时间步既可以是一毫秒, 也可以是一分钟、一天或一个月。这种

根据问题灵活改变状态、行动和时间单位的特性，使得 MDP 的适用范围非常广泛。

表示为 MDP 时需要确定的事项

在尝试解决现实问题时，如何将问题表示为 MDP 是关键。要将新问题表示为 MDP，必须确定以下事项。

- 要解决的问题是回合制任务还是连续性任务？
- 奖励的值是什么？（奖励函数的设定）
- 智能代理可以采取的行动是什么？
- 将什么作为环境的状态？
- 收益的折现率 γ 的值是什么？
- 环境和智能代理的范围各是什么？

这些事项既有能够根据问题自然而然确定的，也有难以确定的。例如，对于围棋这样的棋盘游戏，事项的确定就很简单。围棋是一个回合制任务，如果最终赢了，那么奖励就为 1；如果输了，那么奖励就为 -1（除此之外的每一手棋的奖励为 0）。另外，智能代理的行动是在哪里下棋子。棋盘上的棋子布局可以被视为环境的状态。

在将问题表示为 MDP 形式时，界定哪些部分属于智能代理，哪些部分属于环境的工作有时候是相当困难的。通常的规则是将可以自由控制的部分视为智能代理。这里以操作机器人手臂为例。对于机器人手臂，每个关节都有电机，我们可以自由控制流向电机的电流（或电压）。但要完全控制机械臂或机械手等物理结构（硬件）是困难的。因此，对于硬件部分，将其作为智能代理的外部（也就是环境）更加合适。就像这个例子所探讨的那样，一般的规则是将可以完全控制的部分作为智能代理。



当我们试图使用强化学习来解决现实问题时，奖励函数的设计往往是关键。因此，在实践中，在设计奖励函数和查看训练后的策略的结果的同时，需要反复对奖励函数进行调整。

好的 MDP 形式化的建议

至于 MDP 形式化的好方法，与其说是科学方法，不如说是更接近于艺术的一面。经验和直觉比理论更有用。学习 MDP 形式化的一种方法是参考强化学习领域的著名研究。参考时要关注上面提到的在进行 MDP 形式化时需要确定的几个事项，其中包含了 MDP 形式化的建议。

10.5.3 通用人工智能系统

如今，人工智能领域的发展日新月异。人们夜以继日地开发 AI 系统，并取得了重大成果。但所有这些系统基本局限于特定任务也是不争的事实。识别图像、下棋或驾驶汽车的系统都是专门为这些任务而开发的。而人类拥有各种各样的技能。我们可以识别周围环境、理解语言、驾驶汽车、下棋，等等。通用人工智能（artificial general intelligence, AGI）一词就是用来描述实现人类具有的各种技能的能力。

一些研究人员认为，强化学习是实现或至少接近 AGI 的一个关键领域。例如，DeepMind 的研究人员发表了一篇题为“Reward is enough”的论文^[48]。该论文讨论了“可以通过最大化奖励的总和来充分理解智能和相关能力”这一假设。在此基础上，论文指出了通过以实现奖励总和的最大化为目标的强化学习技术实现通用人工智能的可能性。

在本书中，我们一直在学习强化学习。强化学习是通过与环境的交互，让智能代理以实现奖励总和最大化为目标来学习策略。奖励总和最大化的目标可能足以展现与智能相关的能力，而强化学习方法将来可能会实现通用人工智能。这些只是假设，能否实现，现在还不得而知。但了解到这些观点本身就足以丰富我们的视野。

10.6 小结

本章从深度强化学习的算法分类讲起，然后介绍了一些著名的深度强化学习的改进算法，包括策略梯度法的改进算法和 DQN 的改进算法。另

外，本章还进行了深度强化学习的案例研究，介绍了几个重要的研究案例。毫不夸张，光看那些例子，你就能切身感受到深度强化学习的巨大影响。

至此，我们已经学到了很多关于强化学习的知识。本书的前半部分介绍了强化学习的基础知识，后半部分介绍了目前流行的与深度强化学习相关的技术。即使是最先进的深度强化学习技术，也是建立在我们此前所掌握的强化学习基础之上的，希望你能够透过本书理解这一点。只要掌握了这些知识，我们就能以坚实的步伐迈向更深层次的深度强化学习的世界。

作为强化学习基础的技术不会有太大的变化。在不断变化的 AI 世界中，如果你能学到这种“不变的技术”，那么作为作者，我会感到非常幸福。感谢你读到了本书的最后，谢谢你。

附录 A

异策略型的蒙特卡洛方法

本附录将介绍异策略型的蒙特卡洛方法。首先，我们会介绍异策略型的蒙特卡洛方法的理论知识。然后，我们会针对 3×4 网格世界任务，实现异策略型的蒙特卡洛方法。另外，本附录是第 5 章的扩展阅读内容。

A.1 异策略型的蒙特卡洛方法的理论知识

这里对强化学习的问题应用重要性采样。首先回顾一下同策略型的蒙特卡洛方法。我们的目标是用蒙特卡洛方法近似下面的式子所定义的 Q 函数。

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G | s, a]$$

$q_{\pi}(s, a)$ 表示从状态 s 、行动 a 开始，之后按照策略 π 行动时得到的收益 G 的期望值。要使用蒙特卡洛方法来近似 Q 函数，需要对基于策略 π 行动时所得到的收益取平均值。如果得到了 n 个收益的样本数据，那么就可以做如下近似。

$$\begin{aligned} \text{sampling: } G^{(i)} &\sim \pi \quad (i = 1, 2, \dots, n) \\ Q_{\pi}(s, a) &= \frac{G^{(1)} + G^{(2)} + \dots + G^{(n)}}{n} \end{aligned}$$

接下来思考异策略型的蒙特卡洛方法。使用重要性采样将 Q 函数表示为以下式子。

$$q_\pi(s, a) = \mathbb{E}_b[\rho G | s, a] \quad (\text{A.1})$$

式 A.1 中有以下两个重点。

- 式子表示的是对策略 b 的期望值 ($\mathbb{E}_b[\dots]$)。
- 为了填补两个策略（概率分布）之间的差距，式子中增加了权重 ρ 。

权重 ρ 是“策略为 π 时获得收益 G 的概率”和“策略为 b 时获得收益 G 的概率”的比率。接下来用蒙特卡洛方法近似式 A.1，如下所示。

$$\begin{aligned} \text{sampling: } G^{(i)} &\sim b \quad (i=1, 2, \dots, n) \\ Q_\pi(s, a) &= \frac{\rho^{(1)}G^{(1)} + \rho^{(2)}G^{(2)} + \dots + \rho^{(n)}G^{(n)}}{n} \end{aligned}$$

这里用 $\rho^{(i)}$ 表示第 i 个收益 $G^{(i)}$ 的权重。如上式所示，智能代理会基于行为策略 b 采取行动，使用得到的样本数据求权重为 ρ 的加权平均。

接下来是权重 ρ 的计算方法，我们具体来看一下。这里考虑基于策略 b 得到了图 A-1 所示的时间序列数据的情况。

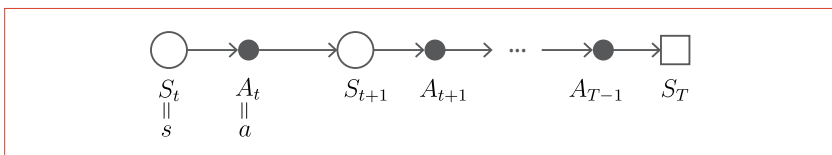


图 A-1 以状态 $S_t = s$ 、行动 $A_t = a$ 为起点，基于策略 b 得到的时间序列数据的示例

这里将图 A-1 的时间序列数据称为“trajectory”，数学式如下所示。

$$\text{trajectory} = S_t, A_t, S_{t+1}, A_{t+1}, \dots, A_{T-1}, S_T$$

此时权重 ρ 可以用如下数学式表示。

$$\rho = \frac{\Pr(\text{trajectory} | \pi)}{\Pr(\text{trajectory} | b)}$$

上面式子中的 $\Pr$ 表示概率。 $\Pr(\text{trajectory} | \pi)$ 表示策略为 π 时获得 trajectory 的概率， $\Pr(\text{trajectory} | b)$ 表示策略为 b 时获得 trajectory 的概率。另外，考虑到 $\Pr(\text{trajectory} | b)$ 是马尔可夫过程，因此式子可以进一步简化。马尔可

夫过程是指环境的状态迁移和智能代理的行动只取决于前一个状态和行动。这个过程如图 A-2 所示。

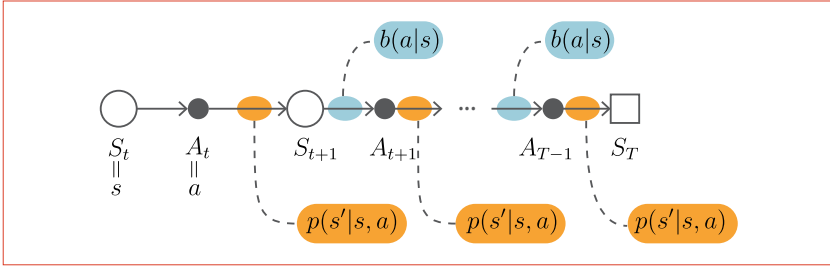


图 A-2 状态迁移概率 $p(s'|s, a)$ 与策略 $b(a|s)$ 的关系

如图 A-2 所示，当给定状态 s 和行动 a 时，可以根据环境的状态迁移概率 $p(s'|s, a)$ 决定下一个状态。如果给定了状态 s ，则由智能代理的策略 $b(a|s)$ 决定下一个行动 a 。因此， $\Pr(\text{trajectory} | b)$ 可以用如下数字式表示。

$$\Pr(\text{trajectory} | b) = p(S_{t+1} | S_t, A_t) b(A_{t+1} | S_{t+1}) \cdots p(S_T | S_{T-1}, A_{T-1}) \quad (\text{A.2})$$

式 A.2 是当策略为 b 时，得到样本数据 trajectory 的概率。同样，当策略为 π 时，可以得到以下式子。

$$\Pr(\text{trajectory} | \pi) = p(S_{t+1} | S_t, A_t) \pi(A_{t+1} | S_{t+1}) \cdots p(S_T | S_{T-1}, A_{T-1}) \quad (\text{A.3})$$

权重 ρ 是式 A.2 与式 A.3 的比率。两个式子环境的状态迁移概率 $p(s'|s, a)$ 是共通的。因此，分母和分子可以互相抵消。最终得到了以下式子。

$$\rho = \frac{\Pr(\text{trajectory} | \pi)}{\Pr(\text{trajectory} | b)} = \frac{\pi(A_{t+1} | S_{t+1}) \cdots \pi(A_{T-1} | S_{T-1})}{b(A_{t+1} | S_{t+1}) \cdots b(A_{T-1} | S_{T-1})} \quad (\text{A.4})$$

如式 A.4 所示，权重 ρ 可以只通过策略的比率求出。以上就是异策略型的蒙特卡洛方法的介绍。下面将算法的步骤总结如下。

- (1) 用行为策略 b 进行采样（得到时间序列数据 trajectory）。
- (2) 根据得到的 trajectory 计算收益 G 。
- (3) 根据式 A.4 计算权重 ρ 。
- (4) 反复重复 (1)~(3) 步，求 ρG 的平均值。

A.2 异策略型的蒙特卡洛方法的实现

下面进入实现的环节。首先介绍权重 ρ 的高效实现方法。这与 5.2.3 节介绍的方法相同。先来看一下图 A-3。

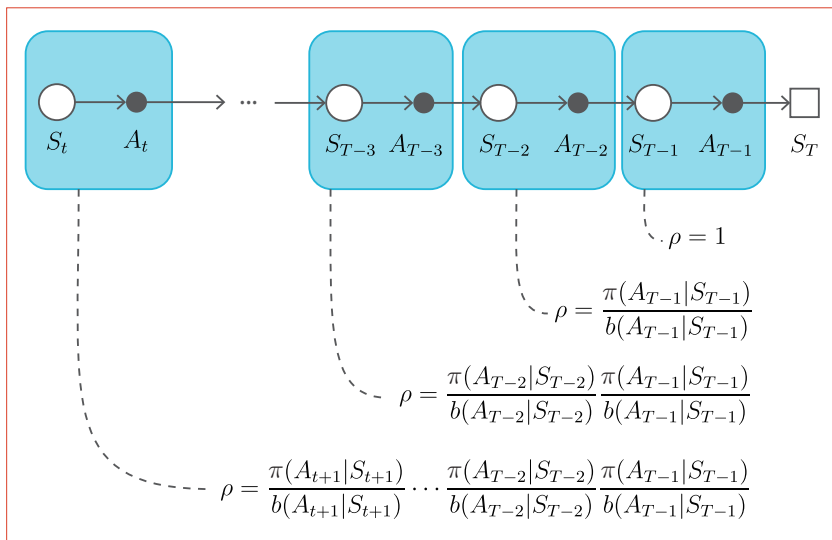


图 A-3 将所有“状态和行动对”视为开始位置

图 A-3 是将“状态和行动对” S_t 和 A_t 作为开始位置时得到的结果。此时，对于中途的状态行动对的数据，我们也可以将它们视为在开始位置得到的样本数据。对于这种情况，从终点开始反向更新权重 ρ 的效率更高。具体来说，首先将 ρ 初始化为 1。这样一来， $Q_\pi(S_{T-1}, A_{T-1})$ 的权重 ρ 为 1。然后将 $Q_\pi(S_{T-2}, A_{T-2})$ 的权重 ρ 更新为如下形式。

$$\rho \leftarrow \frac{\pi(A_{T-1}|S_{T-1})}{b(A_{T-1}|S_{T-1})} \times \rho$$

同样，将 $Q_\pi(S_{T-3}, A_{T-3})$ 的权重 ρ 更新为如下形式。

$$\rho \leftarrow \frac{\pi(A_{T-2}|S_{T-2})}{b(A_{T-2}|S_{T-2})} \times \rho$$

这样就通过从终点开始反向更新权重的做法高效地完成了计算。下面实现使用异策略型的蒙特卡洛方法进行策略控制的智能代理。代码如下所示。

ch05/mc_control_offpolicy.py

```
import numpy as np
from common.gridworld import GridWorld
from common.utils import greedy_probs

class McOffPolicyAgent:
    def __init__(self):
        self.gamma = 0.9
        self.epsilon = 0.1
        self.alpha = 0.2
        self.action_size = 4

        random_actions = {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25}
        self.pi = defaultdict(lambda: random_actions)
        self.b = defaultdict(lambda: random_actions) # ①行为策略
        self.Q = defaultdict(lambda: 0)
        self.memory = []

    def get_action(self, state):
        action_probs = self.b[state] # ②基于行为策略采取行动
        actions = list(action_probs.keys())
        probs = list(action_probs.values())
        return np.random.choice(actions, p=probs)

    def add(self, state, action, reward):
        data = (state, action, reward)
        self.memory.append(data)

    def reset(self):
        self.memory.clear()

    def update(self):
        G = 0
        rho = 1

        for data in reversed(self.memory):
            state, action, reward = data
            key = (state, action)

            # ③使用样本数据更新 Q 函数
            G = self.gamma * rho * G + reward
            self.Q[key] += (G - self.Q[key]) * self.alpha
            rho *= self.pi[state][action] / self.b[state][action]
```

```
# ④ pi 是贪婪的, b 是  $\epsilon$ -greedy 的
self.pi[state] = greedy_probs(self.Q, state, epsilon=0)
self.b[state] = greedy_probs(self.Q, state, self.epsilon)
```

上面的代码与 5.4 节实现的 McAgent 类有很多共同之处。这里主要介绍不同点。首先, 代码①处用 b 这个名字初始化行为策略。这个行为策略可以被初始化为随机性策略。然后, 代码②处的 get_action 方法从策略 b 获取行动。接下来, 代码③处使用重要性采样的权重 rho 进行更新。

这段代码可能让人觉得很复杂, 其实与 5.4 节中实现的 McAgent 类相比, 变更的只有以下代码。

```
# 同策略型 (之前的代码)
G = self.gamma * G + reward
self.Q[key] += (G - self.Q[key]) * self.alpha

# 异策略型 (本次的代码)
G = self.gamma * rho * G + reward
self.Q[key] += (G - self.Q[key]) * self.alpha
rho *= self.pi[state][action] / self.b[state][action]
```

作为样本数据得到的收益必须通过权重 rho 进行校正。因此, 如上面的代码所示, 我们将 rho 用于收益 G 的更新。最后在代码④处改进策略。具体来说, 使用 ϵ -greedy 算法 ($\epsilon = 0.1$) 更新行为策略 b, 使用 ϵ -greedy 算法 ($\epsilon = 0$) 更新目标策略 pi。由于更新目标策略 pi 时 $\epsilon = 0$, 因此这是完全的贪婪化。



行为策略的目的是“探索”, 所以使用均等选择所有行动的方法 (随机性策略) 也没问题。但为了减小收益的方差, 这里通过 ϵ -greedy 算法进行更新。 ϵ -greedy 算法使行为策略 b 更接近目标策略 pi 的概率分布, 同时也可以进行探索。另外, 5.5.3 节介绍过通过使两个概率分布接近来减小方差的做法。

下面使用 McOffPolicyAgent 类来解决问题。结果如图 A-4 所示。

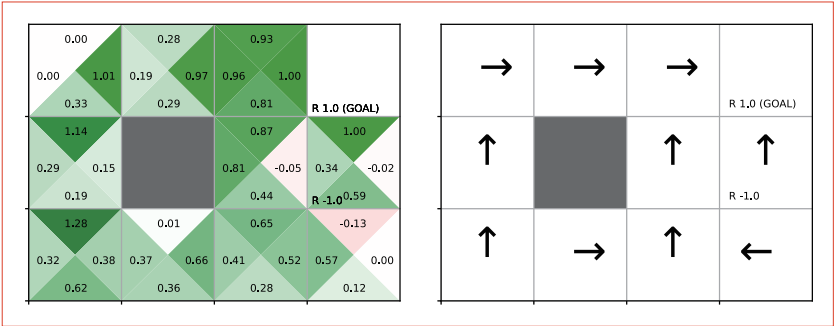


图 A-4 使用异策略型的智能代理得到的结果

每次运行的结果都不同，但基本上是良好的结果。顺带一提，图 A-4 的策略与最优策略相同。



在这次的小型问题上，异策略型的蒙特卡洛方法效果很好。但当问题规模变大后，就难以获得好的结果了。原因是样本数据的方差变大了。具体来说，随着问题规模的扩大，智能代理在到达终点之前会经历很多状态和行动。在这种情况下，基于重要性采样的蒙特卡洛方法的权重 ρ 的方差会随之变大。这样的话，异策略型的蒙特卡洛方法需要大量的回合来进行策略改进，这会导致计算非常耗时。

附录 B

n -step TD 方法

第 6 章介绍的 TD 方法的 TD 目标只使用了下一步的信息。这个 TD 目标可以扩展为使用 2 步、3 步甚至 n 步之后的信息。这就是 n -step TD 方法的思路。这个方法的价值函数的更新式如下所示。

$$V'_\pi(S_t) = V_\pi(S_t) + \alpha \{G_t^{(n)} - V_\pi(S_t)\}$$

此时， n 取不同的值时的 $G_t^{(n)}$ 如下所示。

$$\begin{aligned} n = 1 \text{ 时, } G_t^{(1)} &= R_t + \gamma V_\pi(S_{t+1}) \\ n = 2 \text{ 时, } G_t^{(2)} &= R_t + \gamma R_{t+1} + \gamma^2 V_\pi(S_{t+2}) \\ &\vdots \\ n = \infty \text{ 时, } G_t^{(\infty)} &= R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots \end{aligned}$$

如上面的式子所示，当 n 取不同的值时，TD 目标 $G_t^{(n)}$ 也不同。当 $n = 1$ 时为 TD 方法；当 $n = \infty$ 时，到达终点，此时为 MC 方法。在 n -step TD 方法中， $n = 1$ 和 $n = \infty$ 的两极分别对应于 TD 方法和 MC 方法，二者之间可以解释为渐变。

那么，怎样选择 n 的值才好呢？当然，不同的任务的最合适的 n 值也是不同的。聪明的做法是使用所有的 n 个 TD 目标，而不是试图选择一个最佳的 n 。此时可以将数学式表示为每个 TD 目标的加权和。这就是 TD(λ) 方法。TD(λ) 的 TD 目标如下式所示。

$$G_t^\lambda = (1-\lambda)G_t^{(1)} + (1-\lambda)\lambda G_t^{(2)} + \cdots + (1-\lambda)\lambda^{n-1}G_t^{(n)}$$

上面的式子使用了从 $G_t^{(1)}$ 到 $G_t^{(n)}$ 的所有的 n 个 TD 目标。而且，随着 n 的增加，每项的权重都是前一项的 λ 倍（ λ 是介于 0 和 1 之间的实数）。顺便一提，当 $n = \infty$ 时，所有的权重之和为 1，如下面的式子所示。

$$\begin{aligned} (1-\lambda) + (1-\lambda)\lambda + (1-\lambda)\lambda^2 + \cdots &= (1-\lambda)(1 + \lambda + \lambda^2 + \cdots) \\ &= (1-\lambda)\frac{1}{1-\lambda} \\ &= 1 \end{aligned}$$

以上是对 n -step TD 方法和 $\text{TD}(\lambda)$ 的说明。

附录 C

Double DQN 的理解

8.4.1 节指出，DQN 的 TD 目标存在问题（还有改进的余地）。具体来说，作为 TD 目标的 $R_t + \gamma \max_a Q_\theta(S_{t+1}, a)$ 中的 $\max_a Q_\theta(S_{t+1}, a)$ 计算存在过估计的问题。本附录将介绍什么是过估计，以及如何在 Double DQN 中加以改善（这里的内容参考了博客文章^[49]）。

C.1 什么是过估计

假定当前可以采取的候选行动有 4 个任务，状态 s 下的 Q 函数的值全部相同，也就是说 $q(s, a_0) = q(s, a_1) = q(s, a_2) = q(s, a_3) = 0$ 。在这种情况下，有以下数学式成立。

$$\mathbb{E} \left[\max_a q(s, a) \right] = 0$$

由于 Q 函数的值都为 0，因此如上面的式子所示，期望值中的 max 算子的计算也为 0。

接下来考虑使用正在估计的 Q 函数的情况。这里用 Q 表示正在估计的 Q 函数，该值包含作为误差的基于正态分布生成的随机数。在这种情况下，有以下数学式成立。

$$\mathbb{E} \left[\max_a Q(s, a) \right] > 0$$

也就是说，估计值大于真正的值（0）。这就是过估计的意思。

下面我们用实际的代码来确认一下这个过估计的现象。

```
import numpy as np
import matplotlib.pyplot as plt

samples = 1000
action_size = 4
Qs = []

for _ in range(samples):
    Q = np.random.randn(action_size)
    Qs.append(Q.max())

plt.hist(Qs, bins=16)
plt.axvline(x=0, color="red")
plt.axvline(x=np.array(Qs).mean(), color="cyan")
plt.show()
```

上面代码的运行结果如图 C-1 所示。

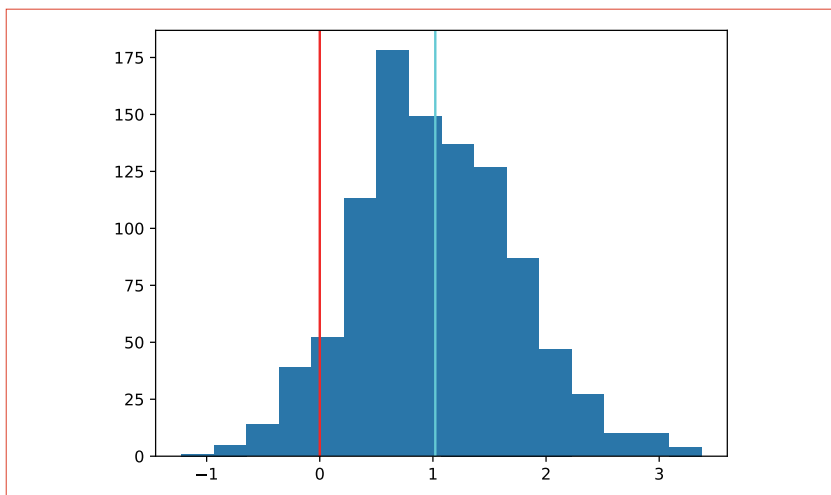


图 C-1 `Q.max()` 的数据分布（红线是真正的期望值，青色线是用 `Q` 求出的期望值）

`Q` 函数的值 0 是真正的值。此次由于是估计值，因此 `Q` 的值被加上了作为噪声的基于正态分布（均值 0，标准差 1）生成的随机数。代码中通过 `Q.max()` 进行了 100 次选择最大值的尝试，将其分布绘制为直方图。从图 C-1 可以看出，求出的期望值偏离在真正的值（0）的右侧，也就是过估计了。

C.2 过估计的解决方法

下面介绍防止过估计的“Double DQN”。从“Double”一词可以看出，这个方法使用了两个 Q 函数。首先来看一下代码。

```
import numpy as np
import matplotlib.pyplot as plt

samples = 1000
action_size = 4
Qs = []

for _ in range(samples):
    Q = np.random.randn(action_size)
    Q_prime = np.random.randn(action_size) # 另一个 Q 函数
    idx = np.argmax(Q)
    Qs.append(Q_prime[idx])

plt.hist(Qs, bins=16)
plt.axvline(x=0, color="red")
plt.axvline(x=np.array(Qs).mean(), color="cyan")
plt.show()
```

上面代码的运行结果如图 C-2 所示。

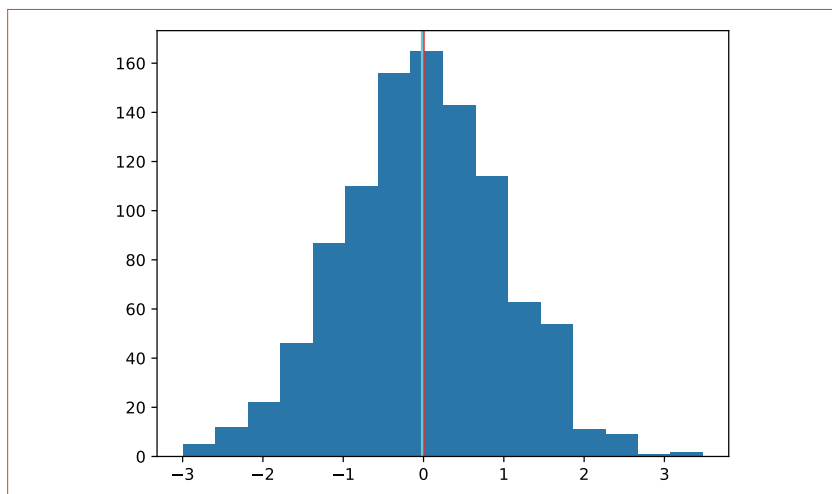


图 C-2 使用 Double DQN 时的数据分布（红线是真正的期望值，青色线是使用 Q 和 Q_prime 求出的期望值）

从图 C-2 可以看出，直方图是以 0 为中心分布的。图 C-2 的红线和青色线重叠在了一起，这说明消除了过估计现象。与之前的代码不同，这段代码使用了两个 Q 函数。两个 Q 函数都是估计值，且包含了误差。但误差是相互独立的。代码中在进行 Q 函数的 max 计算时，使用两个 Q 函数做了区分。具体使用方法如下。

- 在选择 max 的行动时使用 Q。
- 从另一个 Q 函数 `Q_prime` 获取所选行动的值。

通过这种使用不同的 Q 函数进行的“行动的选择”和“值的获取”，就可以防止过估计。以上就是对 DQN 的过估计问题以及 Double DQN 的解决方法的介绍。

附录 D

策略梯度法的证明

在本附录中，我们将对第 9 章中使用的数学式进行证明。

D.1 策略梯度法的推导

9.1.1 节介绍过，当 $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [G(\tau)]$ 时，其梯度如下面的式子所示。

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T G(\tau) \nabla_\theta \log \pi_\theta(A_t | S_t) \right] \quad (9.1)$$

下面进行式 9.1 的推导。

首先梳理一下记号。这里用 π_θ 表示策略，用 $\Pr(\tau | \theta)$ 表示得到轨迹 τ 的概率。接下来使用数学式逐步展开 $\nabla_\theta J(\theta)$ 。

$$\begin{aligned} \nabla_\theta J(\theta) &= \nabla_\theta \mathbb{E}_{\tau \sim \pi_\theta} [G(\tau)] \\ &= \nabla_\theta \sum_{\tau} \Pr(\tau | \theta) G(\tau) \quad (\text{展开期望值}) \\ &= \sum_{\tau} \nabla_\theta (\Pr(\tau | \theta) G(\tau)) \quad (\text{将 } \nabla_\theta \text{ 移动到 } \sum \text{ 中}) \\ &= \sum_{\tau} \{G(\tau) \nabla_\theta \Pr(\tau | \theta) + \Pr(\tau | \theta) \nabla_\theta G(\tau)\} \quad (\text{积的微分}) \\ &= \sum_{\tau} G(\tau) \nabla_\theta \Pr(\tau | \theta) \quad (\nabla_\theta G(\tau) \text{ 永远为 } 0) \end{aligned}$$

$$\begin{aligned}
&= \sum_{\tau} G(\tau) \Pr(\tau | \theta) \frac{\nabla_{\theta} \Pr(\tau | \theta)}{\Pr(\tau | \theta)} \left(\text{乘以} \frac{\Pr(\tau | \theta)}{\Pr(\tau | \theta)} \right) \\
&= \sum_{\tau} G(\tau) \Pr(\tau | \theta) \nabla_{\theta} \log \Pr(\tau | \theta) \quad (\text{log 梯度的技巧}) \quad (\text{D.1}) \\
&= \mathbb{E}_{\tau \sim \pi_{\theta}} [G(\tau) \nabla_{\theta} \log \Pr(\tau | \theta)]
\end{aligned}$$

结合数学式右侧的注释，我们逐行来看一下。只要掌握了导数相关的知识，应该就没有特别难的地方。这里只对“log 梯度的技巧”进行补充说明。这个技巧利用了以下等式。

$$\nabla_{\theta} \log \Pr(\tau | \theta) = \frac{\nabla_{\theta} \Pr(\tau | \theta)}{\Pr(\tau | \theta)}$$

虽然上面的式子只求了 log 的梯度，但由这个式子可知， $\nabla_{\theta} \Pr(\tau | \theta)$ 和 $\Pr(\tau | \theta) \nabla_{\theta} \log \Pr(\tau | \theta)$ 是可以互换的。这就是著名的 log 梯度的技巧 (log-derivative trick)，是机器学习领域常用的数学式的变形形式。

接下来，我们将利用以下等式进一步展开式 D.1。

$$\begin{aligned}
\Pr(\tau | \theta) &= p(S_0) \pi_{\theta}(A_0 | S_0) p(S_1 | S_0, A_0) \cdots \pi_{\theta}(A_T | S_T) p(S_{T+1} | S_T, A_T) \\
&= p(S_0) \prod_{t=0}^T \pi_{\theta}(A_t | S_t) p(S_{t+1} | S_t, A_t)
\end{aligned}$$

这里， $p(S_0)$ 表示初始状态 S_0 的概率。上面的式子表明，得到轨迹 τ 的概率可以用初始状态的概率、策略以及下一个状态的迁移概率的乘积来表示。另外，我们可以用下面的式子来表示 $\log \Pr(\tau | \theta)$ 。

$$\log \Pr(\tau | \theta) = \log p(S_0) + \sum_{t=0}^T \log p(S_{t+1} | S_t, A_t) + \sum_{t=0}^T \log \pi_{\theta}(A_t | S_t)$$

由于 $\log xy = \log x + \log y$ ，因此可以像上面的式子那样表示为和的形式。基于上面的式子，可以将 $\nabla_{\theta} \log \Pr(\tau | \theta)$ 展开为如下形式。

$$\begin{aligned}
\nabla_{\theta} \log \Pr(\tau | \theta) &= \nabla_{\theta} \left\{ \log p(S_0) + \sum_{t=0}^T \log p(S_{t+1} | S_t, A_t) + \sum_{t=0}^T \log \pi_{\theta}(A_t | S_t) \right\} \\
&= \nabla_{\theta} \sum_{t=0}^T \log \pi_{\theta}(A_t | S_t)
\end{aligned}$$

∇_{θ} 是对 θ 的梯度。与 θ 无关的元素的梯度 $\nabla_{\theta} \log p(S_0)$ 和 $\nabla_{\theta} \sum_{t=0}^T \log p(S_{t+1} | S_t, A_t)$ 为 0。因此，从上面的式子可以得到下列式子。

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [G(\tau) \nabla_{\theta} \log \Pr(\tau | \theta)] \quad (\text{D.1})$$

$$= \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T G(\tau) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right] \quad (\text{9.1})$$

这样我们就完成了 $\nabla_{\theta} J(\theta)$ 的推导。

D.2 基线的推导

9.3.2 节介绍了式 9.3 的变形形式，即式 9.4。

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T G_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right] \quad (\text{9.3})$$

$$= \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T (G_t - b(S_t)) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right] \quad (\text{9.4})$$

如式 9.4 所示，我们可以使用 $G_t - b(S_t)$ 代替 G_t 。 $b(S_t)$ 是任何函数，我们称之为“基线”。下面进行式 9.4 的推导。

首先，证明以下式子成立。

$$\mathbb{E}_{x \sim P_{\theta}} [\nabla_{\theta} \log P_{\theta}(x)] = 0 \quad (\text{D.2})$$

这里假设随机变量 x 是基于概率分布 $P_{\theta}(x)$ 生成的。 $P_{\theta}(x)$ 会根据参数 θ 改变概率分布的形状。此时有以下式子成立。

$$\sum_x P_{\theta}(x) = 1$$

由于 $P_{\theta}(x)$ 是概率分布，因此所有 x 的值的和为 1。然后，求这个式子的梯度。

$$\nabla_{\theta} \sum_x P_{\theta}(x) = \nabla_{\theta} 1 = 0$$

接下来，使用 $\log$ 梯度的技巧将式子展开，过程如下所示。

$$\begin{aligned}
 0 &= \nabla_{\theta} \sum_x P_{\theta}(x) \\
 &= \sum_x \nabla_{\theta} P_{\theta}(x) \\
 &= \sum_x P_{\theta}(x) \nabla_{\theta} \log P_{\theta}(x) \\
 &= \mathbb{E}_{x \sim P_{\theta}} [\nabla_{\theta} \log P_{\theta}(x)]
 \end{aligned}$$

这样就完成了式 D.2 的证明。

下面将式 D.2 应用于我们的问题。具体来说，用 A_t 代替式 D.2 中的 x ，然后使用 $\pi_{\theta}(\cdot | S_t)$ 代替 $P_{\theta}(\cdot)$ 。这样就可以得到以下式子。

$$\mathbb{E}_{A_t \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(A_t | S_t)] = 0 \quad (\text{D.3})$$

式 D.3 是对 A_t 的期望值。因此，我们可以像下面的式子那样，将任何函数 $b(S_t)$ 放入期望值中。

$$\mathbb{E}_{A_t \sim \pi_{\theta}} [b(S_t) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)] = 0 \quad (\text{D.4})$$

$b(S_t)$ 是以 S_t 为参数的函数，即使 A_t 发生变化，它的值也不会改变。由于式 D.4 是对 A_t 的期望值，因此即使在期望值中加入函数 $b(S_t)$ ，等式也成立。



行动 A_t 的变化会导致收益 G_t 的变化，因此以下式子不成立。

$$\mathbb{E}_{A_t \sim \pi_{\theta}} [G_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)] = 0$$

式 D.4 在整个 $t = 0 \sim T$ 的范围内均成立，因此可以得到以下式子。

$$\mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T b(S_t) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right] = 0$$

由此可知式 9.4 成立。

后记

愿原力与你同在。

—— 电影《星球大战》

当我在 2016 年写完“深度学习入门 & 进阶”系列的第一本书《深度学习入门：基于 Python 的理论与实现》的时候，就想过这一系列的下一本书（如果有机会写的话）将会以强化学习为主题。当时，能够玩电子游戏的 DQN 和击败围棋世界冠军的 AlphaGo 备受关注，我也对这些技术（深度强化学习）非常着迷。那时虽然是第一次认真研究强化学习，但我立即感受到了它的巨大潜力。原因在于强化学习的核心是从与环境的交互试错中学习，我认为它是更具通用性的技术。

在对强化学习认真研究了一段时间之后，我意识到要达到写一本书的水平还需要一些时间。思前想后，我决定下一本书要在上一本书的基础上更进一步，朝着能够处理时间序列数据的方向编写。具体来说，我将这一系列的第二本书（《深度学习进阶：自然语言处理》）的写作目标定为能够理解 RNN、LSTM，以及当时开始引发关注的 Attention。顺带一提，其实一开始写作该书的时候我并没有把自然语言处理作为主题。直到有一天，在该书写到一半时，我突然想到如果以自然语言处理为主题，那么全书就能够讲好一个故事，而且能够使得读者连贯地学到重要的技术。现在我还记得那天晚上的情景：我兴奋地给编辑宫川先生发了一封邮件，说我接下来想写深度学习的自然语言处理部分。

在即将告别 2018 年，顺利完成了《深度学习进阶：自然语言处理》的写作后，我想这次该写强化学习了，于是开始做编写的准备。但人生并不总是朝着规划的方向发展。一次偶然的机会（缘于与 Chainer 框架开发者

得居先生和 Preferred Networks 公司的团队成员的交谈)，我改变了想法，觉得在下一本书中创建一个更纯粹的深度学习框架也不错。因此，我编写了这一系列的第三本书（《深度学习入门 2：自制框架》）。再啰唆几句，Chainer 框架在当时是超前的。它的核心代码简单、优美且实用。而且，从教育的角度来看，它也是非常出色的。例如，在学习算法、数据结构和编程的测试技术时，它是非常好的学习参考。遗憾的是，Chainer 框架已经停止开发了。但能将 Chainer 的思想作为一本书流传下来这件事让我感到了小小的自豪和骄傲。

自从我决定写一本关于强化学习的书以来，已经过去很长时间了。与那时相比，我掌握了更多的知识，现在可以用自己的语言来介绍强化学习了（尽管我还有很多东西要学，并将继续努力）。强化学习领域的理论非常艰深，有很多地方我一开始并不理解。现在想起来，当时我绕了一大圈，花了很长时间才弄明白。但这也是很难得的经历。最重要的是，我很高兴能在过去几年中亲身体验到强化学习的乐趣。我希望能与尽可能多的读者分享这种乐趣。

本书是“深度学习入门 & 进阶”系列的第四本书（4th）。毫无疑问，能够将技术图书作为一个系列继续出版要归功于许多人的支持。没有开辟这一领域的研究者和开发者，就没有本书。没有参与本书创作的人士和阅读本书的读者，本书也不可能问世。

感谢大家给了我到达 4th（fourth）的机会。愿原力（force）与你同在。

2022 年 1 月 1 日

斋藤康毅

参考文献

第 1 章 老虎机问题

- [1] Laurens van der Maaten, Geoffrey Hinton. “Visualizing data using t-SNE.” Journal of machine learning research 9.11 (2008).
- [2] Peter Auer, Nicolo Cesa-Bianchi, Paul Fischer. “Finite-time analysis of the multiarmed bandit problem.” Machine learning 47.2 (2002): 235-256.
- [3] Ronald J. Williams. “Simple statistical gradient-following algorithms for connectionist reinforcement learning.” Machine learning 8.3 (1992): 229-256.

第 2 章 马尔可夫决策过程

- [4] Csaba Szepesvari. “Algorithms for Reinforcement Learning.” Springer, 2010.

第 4 章 动态规划法

- [5] Richard S. Sutton, Andrew G. Barto. “Reinforcement learning: An introduction.” MIT press, 2018.

第 7 章 神经网络和 Q 学习

- [6] John Duchi, Elad Hazan, Yoram Singer. “Adaptive subgradient methods for online learning and stochastic optimization.” Journal of machine learning research 12.7 (2011).
- [7] Matthew D. Zeiler. “Adadelata: an adaptive learning rate method.” arXiv preprint arXiv:1212.5701 (2012).
- [8] Diederik P. Kingma, Jimmy Ba. “Adam: A method for stochastic optimization.” arXiv preprint arXiv:1412.6980 (2014).

第 8 章 DQN

- [9] Gerald Tesauro. “Practical issues in temporal difference learning.” Machine learning 8.3 (1992): 257-277.

- [10] Volodymyr Mnih, et al. “Playing Atari with Deep Reinforcement Learning.” arXiv preprint arXiv:1312.5602 (2013).
- [11] Volodymyr Mnih, et al. “Human-level control through deep reinforcement learning.” *Nature* 518.7540 (2015): 529-533.
- [12] Hado Van Hasselt, Arthur Guez, David Silver. “Deep reinforcement learning with double q-learning.” *Proceedings of the AAAI conference on artificial intelligence*. Vol. 30. No. 1. 2016.
- [13] Tom Schaul, et al. “Prioritized experience replay.” arXiv preprint arXiv:1511.05952 (2015).
- [14] Wang Ziyu, et al. “Dueling network architectures for deep reinforcement learning.” *International conference on machine learning*. PMLR, 2016.

第 9 章 策略梯度法

- [15] Ronald J. Williams. “Simple statistical gradient-following algorithms for connectionist reinforcement learning.” *Machine learning* 8.3 (1992): 229-256.
- [16] OpenAI Spining Up “Part 3: Intro to Policy Optimization”.
- [17] OpenAI Spining Up “Extra Material: Proof for Don’t Let the Past Distract You”.
- [18] OpenAI Spining Up “Extra Material: Proof for Using Q-Function in Policy Gradient Formula”.

第 10 章 进一步学习

- [19] OpenAI Spining Up “Part 2: Kinds of RL Algorithms”.
- [20] David Ha, Jürgen Schmidhuber. “World models.” arXiv preprint arXiv:1803.10122 (2018).
- [21] Vladimir Feinberg, et al. “Model-based value estimation for efficient model-free reinforcement learning.” arXiv preprint arXiv:1803.00101 (2018).
- [22] Volodymyr Mnih, et al. “Asynchronous methods for deep reinforcement learning.” *International conference on machine learning*. PMLR, 2016.
- [23] Timothy P. Lillicrap, et al. “Continuous control with deep reinforcement learning.” arXiv preprint arXiv:1509.02971 (2015).
- [24] John Schulman, et al. “Trust region policy optimization.” *International conference on machine learning*. PMLR, 2015.
- [25] John Schulman, et al. “Proximal policy optimization algorithms.” arXiv preprint arXiv:1707.06347 (2017).
- [26] Marc G. Bellemare, Will Dabney, Rémi Munos. “A distributional perspective on reinforcement learning.” *International Conference on Machine Learning*. PMLR, 2017.

- [27] Meire Fortunato, et al. “Noisy networks for exploration.” arXiv preprint arXiv:1706.10295 (2017).
- [28] Matteo Hessel, et al. “Rainbow: Combining improvements in deep reinforcement learning.” Thirty-second AAAI conference on artificial intelligence. 2018.
- [29] Dan Horgan, et al. “Distributed prioritized experience replay.” arXiv preprint arXiv:1803.00933 (2018).
- [30] Steven Kapturowski, et al. “Recurrent experience replay in distributed reinforcement learning.” International conference on learning representations. 2018.
- [31] Adrià Puigdomènech Badia, et al. “Never give up: Learning directed exploration strategies.” arXiv preprint arXiv:2002.06038 (2020).
- [32] Adrià Puigdomènech Badia, et al. “Agent57: Outperforming the atari human benchmark.” International Conference on Machine Learning. PMLR, 2020.
- [33] Rémi Coulom. “Efficient selectivity and backup operators in Monte Carlo tree search.” International conference on computers and games. Springer, Berlin, Heidelberg, 2006.
- [34] David Silver, et al. “Mastering the game of Go with deep neural networks and tree search.” *Nature* 529.7587 (2016): 484-489.
- [35] David Silver, et al. “Mastering the game of Go without human knowledge.” *Nature* 550.7676 (2017): 354-359.
- [36] David Silver, et al. “Mastering chess and shogi by self-play with a general reinforcement learning algorithm.” arXiv preprint arXiv:1712.01815 (2017).
- [37] Dmitry Kalashnikov, et al. “Qt-opt: Scalable deep reinforcement learning for vision-based robotic manipulation.” arXiv preprint arXiv:1806.10293 (2018).
- [38] Thomas Elsken, Jan Hendrik Metzen, Frank Hutter. “Neural architecture search: A survey.” *The Journal of Machine Learning Research* 20.1 (2019): 1997-2017.
- [39] Barret Zoph, Quoc V. Le. “Neural Architecture Search with Reinforcement Learning.” arXiv preprint arXiv:1611.01578 (2016).
- [40] B. Ravi Kiran, et al. “Deep reinforcement learning for autonomous driving: A survey.” *IEEE Transactions on Intelligent Transportation Systems* (2021).
- [41] Xu Xu, Jia Youwei, Xu Yan, et al. “A multi-agent reinforcement learning-based data-driven method for home energy management.” *IEEE Transactions on Smart Grid* 11.4 (2020): 3201-3211.
- [42] Wei Tianshu, Wang Yanzhi, Zhu Qi. “Deep reinforcement learning for building HVAC control.” *Proceedings of the 54th annual design automation conference* 2017. 2017.
- [43] DeepMind blog “DeepMind AI Reduces Google Data Centre Cooling Bill by 40%”.

- [44] Azalia Mirhoseini, et al. “Chip placement with deep reinforcement learning.” arXiv preprint arXiv:2004.10746 (2020).
- [45] Ilge Akkaya, et al. “Solving rubik’s cube with a robot hand.” arXiv preprint arXiv:1910.07113 (2019).
- [46] Sergey Levine, et al. “Offline reinforcement learning: Tutorial, review, and perspectives on open problems.” arXiv preprint arXiv:2005.01643 (2020).
- [47] Todd Hester, et al. “Deep Q-learning from Demonstrations.” Proceedings of the AAAI Conference on Artificial Intelligence. Vol. 32. No. 1. 2018.
- [48] David Silver, et al. “Reward is enough.” Artificial Intelligence 299 (2021): 103535.

附录 C

- [49] horomary 「DQN の進化史② Double-DQN, Dueling-network, Noisy network」.

深度学习入门④：强化学习

沿袭“鱼书”系列风格，提供实际代码，边实践边学习，无须依赖外部库，从零开始实现支撑强化学习的基础技术。

本书有什么特点？

■ 把握潮流中的变与不变

在快速发展变化的深度学习领域，有变化的事物，有不变的事物。有些事物会随潮流而消逝，有些则会被传承下去。本书从马尔可夫决策过程、贝尔曼方程、蒙特卡洛方法、时间差分法等强化学习基础方法，自然而然地过渡到神经网络和深度学习，从前沿视角为读者遴选出最值得关注的强化学习理论和方法。

■ 内容丰富，讲解简明易懂

作为超高人气“鱼书”系列第四部作品，本书延续了这一系列的写作和讲解风格，搭配丰富的图、表、代码示例，加上轻松、简明的讲解，能够让人非常信服地理解强化学习中各种方法之间的关系，于不知不觉中登堂入室。

■ 原理与实践并重

本书旨在确保读者能够牢固掌握强化学习的独特理论，奉行“只有做出来才能真正理解”的理念，将这一主题的每个构成要素都从“理论”和“实践”两个方面进行详尽解释，并鼓励读者动手尝试。与仅通过数学公式解释理论的图书不同，读者将通过实际运行本书代码获得许多令人惊叹的领悟。

图灵社区：iTuring.cn

分类建议 计算机 / 人工智能

人民邮电出版社网址：www.ptpress.com.cn

O'Reilly Japan, Inc. 授权人民邮电出版社有限公司出版

此简体中文版仅限于在中华人民共和国境内（不包括香港特别行政区、澳门特别行政区及台湾地区）销售发行

This Authorized Edition for sale only in the People's Republic of China (excluding Hong Kong SAR, Macao SAR and Taiwan)



扫码领取
随书代码资料

ISBN 978-7-115-64917-1



9 787115 649171 >

定价：109.80元